DataAnalyticsLang

alex

October 28, 2026

Contents

1 Typing terms that obey lattice subtyping **5**

2 Typing judgments **6**

3 Additional definitions **16**
3.1 Facts about *table-cols-of* as an automorphism of set algebra . 17

4 Language algebra. Initial algebra is defined as term*etyp*, and a generic algebra is defined as term*interp*. **19**

5 Stable deflations **21**

6 Stable subdomains **22**
6.1 Stable deflations and stable subdomains 27
6.2 Semi-factor deflations 29

7 Stable subdomains as a lattice of sets **32**
7.1 Meets of stable deflations 33
7.2 Arbitrary meets and directed joins 35

8 The domain of stable subdomains **37**
8.1 Images as sets 40

9 Distributive domains: joins of stable subdomains **41**
9.1 Joins of stable deflations 41
9.2 Joins in the domain of stable subdomains 45
9.3 Semi-factors 47
9.4 Meets, distributivity and the frame law 50

10 The type of semi-factors **53**

11 Semi-factors over dI-domains **57**
11.1 Meets of semi-factors . . . 59
11.2 Algebraicity . . . 60
11.3 Componentwise reasoning on tuples . . . 71
11.4 Semi-factors of the tuple domain . . . 72
11.5 Dimension sets as semi-factors of the tuple domain . . . 76
11.6 Compact tuples . . . 80

12 Continuous congruences (functional-dependence relations) **81**
12.1 Continuous congruences (admissible refining preorders) . . . 81
12.2 The generated congruence (least cong above a relation) . . . 83
12.3 Flat LODs (the generators) . . . 83

13 Finitary semi-factors and locally finitely detailed domains **85**
13.1 Levels through the constructions . . . 87
13.2 Levels through isomorphisms . . . 89
13.3 Discriminator for separated sums . . . 90
13.4 Finitary and faithful deflations under joins . . . 92
13.5 Local finite detail as a deflation predicate . . . 93
13.6 Tuples . . . 98

14 Projections onto semi-factors **100**
14.1 Compact semi-factors project to compact keys . . . 100
14.2 Where a key lives . . . 103
14.3 Keys maximal at an LOD . . . 103
14.4 Masks . . . 106
14.5 Masks acting on keys . . . 107
14.6 Compact maps . . . 108
14.7 Levels of detail . . . 109

15 Levels of detail as syntactic dimension sets **110**
15.1 Kernel orders and functional dependence . . . 110

16 Filters **111**

17 Datasets **112**

18 Slicers **113**
18.1 Selections . . . 113
18.2 Semilattice homomorphisms . . . 114
18.3 Slice evaluation . . . 114
18.4 Coarsening . . . 116
18.5 Pins . . . 118
18.6 Rows versus keys . . . 118

19 Relational vocabulary over a set of rows **120**
19.1 Functional dependence . . . 120
19.2 Splitting (unconditional independence) . . . 120
19.3 Divisibility at a grain . . . 121
19.4 Equivalence up to a grain . . . 121
19.5 Equivalence modulo a grain . . . 122

20 Schemas: presence rules and chase rules **124**
20.1 Antecedents and consequents . . . 125
20.2 Guards . . . 126
20.3 Functional dependencies and key joins as chase rules . . . 127
20.4 The combined chase step . . . 129
20.5 The chase on the Hoare powerdomain . . . 131
20.6 Lawful datasets . . . 133
20.7 The chase as the least conforming dataset above . . . 135
20.8 The laws on maximal members . . . 137

21 The schema locale **140**

22 Views and functional dependencies **142**
22.1 Dependencies on a set of rows . . . 143
22.2 The key part, the section, and the determination order . . . 146
22.3 Weak dependencies . . . 151

23 Dependencies over all conforming datasets **152**
23.1 Strong keys . . . 152
23.2 Supports of content functions . . . 153

24 Frames **155**

25 Frame transformers **155**

26 Frame semantics **156**

27 Key images **157**

28 Transferring the classes along domain isomorphisms **158**

29 Example: a non-recursive domain **159**

30 Example: a recursive domain **159**

31 Values **160**
31.1 Scalars . . . 161
31.2 Measures: values with nulls . . . 161
31.3 Keys: values with NA . . . 162

31.4 Data 163

32 Coercions between the levels **163**

33 Record-level operators **165**
33.1 Equality 167
33.2 Truth 167

34 The not-null level of keys **169**

35 The content law **170**
35.1 Observations 171
35.2 Groups and contents 175
35.3 Supports of guards 176
35.4 Column supports of frames and sections of datasets 180
35.5 The laws 183
35.6 Lemmas for the combinators 189

36 Combinators **190**
36.1 Tuples 190
36.2 Keys 190
36.3 Table columns 191
36.4 Maps 193
36.5 *RESTRICT* 194
36.6 *FOR* and *FIXED* 196
36.7 Including dimensions 198
36.8 The gate 198
36.9 Collect 200

37 Evaluating components at their own keys **202**

38 Aggregation **205**

39 Expression syntax **217**

40 The known function symbols **217**

41 Dimensions and the key reading **219**
41.1 The key reading 221
41.2 Coarsening conditions 222

42 Conditions as slices **225**
42.1 The slicer laws 227
42.2 Pins 229

43 Levels of expressions **231**

44 Semantics **232**

45 Outer joins of bags **235**
45.1 Sides and indices 236
45.2 Partner masses 238
45.3 The join on the basis 241
45.4 Monotonicity 242
45.5 The join of bags into pairs 247
45.6 Joins on one row domain 248

theory *Typing* **imports** *Main* **begin**

unbundle *lattice-syntax*

1 Typing terms that obey lattice subtyping

locale *typing* =
  fixes *of-type* :: *'val* ⇒ *'typ*::{*complete-lattice*} ⇒ *bool*
  assumes *of-type-mono*: $t \leq t' \Longrightarrow$ *of-type v t* $\Longrightarrow$ *of-type v t'*
    and *of-type-Inf*: $(\bigwedge t.\ t \in ts \Longrightarrow$ *of-type v t*$) \Longrightarrow$ *of-type v* $(\bigsqcap ts)$

begin

definition *principal-type* :: *'val* ⇒ *'typ* **where** *principal-type v* = $\bigsqcap$ {*t*. *of-type v t*}

theorem *principal-type-is-principal*: *of-type v t* ⟷ *principal-type v* ≤ *t*
proof
  fix *v t*
  assume *of-type v t*
  then show *principal-type v* ≤ *t* **by** (*simp add*: *Inf-lower principal-type-def*)
next
  fix *v t*
  assume *0*: *principal-type v* ≤ *t*
  then show *of-type v t* **using** *of-type-Inf of-type-mono principal-type-def 0* **by** *fastforce*
qed

lemma *principal-type-elim*[*elim*]: *principal-type v* ≤ *t* ⟹ *of-type v t*
  using *principal-type-is-principal* **by** *fast*

theorem *Sup-principal-type*: $\bigsqcup$(*principal-type* ‘ *V*) = $\bigsqcap$ {*t*. ∀ *v* ∈ *V*. *of-type v t*}
proof −
  have $\bigsqcup$(*principal-type* ‘ *V*) = $\bigsqcap$ {*t*. ∀ *v* ∈ *V*. *principal-type v* ≤ *t*} **by** (*simp add*: *Sup-eq-Inf*)
  also have ... = $\bigsqcap$ {*t*. ∀ *v* ∈ *V*. *of-type v t*} **using** *principal-type-is-principal* **by** *simp*
  finally show *?thesis* **.**
qed

interpretation *typing-self*: *typing*
  where *of-type* = ‹(≤)::('*typ*::{*complete-lattice*} ⇒ '*typ* ⇒ *bool*)›
  by (*auto intro*!: *typing.intro simp*: *Inf-greatest*)
end

2 Typing judgments

datatype '*t judg* = *Of* '*t* | *Empty*

instantiation *judg* :: (*ord*) *ord* **begin**

definition *less-eq-judg* **where**
x ≤ *y* ≡ *case x of Empty* ⇒ *True* | *Of x* ⇒ (*case y of Empty* ⇒ *False* | *Of y* ⇒ *x* ≤ *y*)

definition
  (*x*::'*a judg*) < *y* ≡ *x* ≤ *y* ∧ *x* ≠ *y*

lemma *less-eq-empty*: *Empty* ≤ *x* **by** (*auto simp add*: *less-eq-judg-def*)
lemma *less-eq-judg-Of*: *x* ≤ *y* ⟷ *Of x* ≤ *Of y* **by** (*auto simp add*: *less-eq-judg-def*)

instance ..

end

instantiation *judg* :: (*order*) *order* **begin**

instance
  by *standard* (*auto simp add*: *less-eq-judg-def less-judg-def intro*: *order-trans split*: *judg.splits*)

end

instantiation *judg* :: (*bounded-lattice*) *bounded-lattice* **begin**
definition *sup-judg* **where**
a ⊔ *b* ≡ *case a of Empty* ⇒ *b*
  | *Of x* ⇒ (*case b of Empty* ⇒ *Of x* | *Of y* ⇒ *Of* (*x* ⊔ *y*))
definition *inf-judg* **where**
a ⊓ *b* ≡ *case a of Empty* ⇒ *Empty*
  | *Of x* ⇒ (*case b of Empty* ⇒ *Empty* | *Of y* ⇒ *Of* (*x* ⊓ *y*))
definition *top-judg* **where** ⊤ = *Of* ⊤
definition *bot-judg* **where** ⊥ = *Empty*

instance
  by *standard*
    (*auto simp add*:
        sup-judg-def inf-judg-def top-judg-def bot-judg-def less-eq-judg-def
    split: *judg.splits*)

end

type-synonym *'v model-function* = *'v list* ⇒ *'v option*

type-synonym *'t judgment-function* = *'t list* ⇒ *'t judg*

context *typing* **begin**

primrec *judges-type* **where**
  judges-type v (*Of t*) = *of-type v t*
| *judges-type v Empty* = *False*

context fixes *mf* :: *'val model-function* **begin**
context fixes *ts* :: *'typ list* **begin**

abbreviation *of-typeAll* **where** *of-typeAll* ≡ *list-all2 of-type*

definition *fn-value-set* :: *'val set* **where**
  fn-value-set ≡ { *v* | *vs v*. *of-typeAll vs ts* ∧ *mf vs* = *Some v* }

abbreviation *sup-type* **where** *sup-type* ≡ ⨆(*principal-type* ‘ *fn-value-set*)
abbreviation *min-common-type* **where** *min-common-type* ≡ ⨅{*t*. ∀ *v* ∈ *fn-value-set*. *of-type v t*}

lemma *sup-type-eq-min-common-type*: *sup-type* = *min-common-type*
  by (*rule Sup-principal-type*)

definition *fn-type-judgment* :: *'typ judg* **where**
  fn-type-judgment ≡ *if* (*fn-value-set* = {}) *then Empty else Of min-common-type*

lemma *fn-type-judgment-Empty*: *fn-type-judgment* = *Empty* ⟷ *fn-value-set* = {}
  using *fn-type-judgment-def*
  by (*cases fn-value-set* = {}) *auto*

lemma *fn-type-judgment-Of*: *fn-value-set* ≠ {} ⟹
  fn-type-judgment = *Of min-common-type*
  using *fn-type-judgment-def*
  by *auto*

definition *sound-judgment* :: *'typ judg* ⇒ *bool*
  where *sound-judgment t* ≡ ∀ *v* ∈ *fn-value-set*. *judges-type v t*

lemma *min-common-type-is-common*: **assumes** *v* ∈ *fn-value-set*
  shows *of-type v min-common-type*
  using *of-type-Inf assms* **by** *fastforce*

theorem *fn-type-judgment-sound*: *sound-judgment fn-type-judgment*

unfolding *sound-judgment-def*
using *fn-type-judgment-Of min-common-type-is-common* **by** *fastforce*

lemma *sound-nonempty*: **assumes** *fn-value-set* $\neq \{\}$ **and** *sound-judgment t*
obtains *t0* **where** $t = Of\ t0$
using *assms* **by** (*cases t*) (*simp-all add*: *sound-judgment-def*)

theorem *fn-type-judgment-minimal*: *sound-judgment t* $\Longrightarrow$ *fn-type-judgment* $\leq t$
proof (*cases fn-value-set* $= \{\}$)
case *True*
then show *?thesis* **using** *fn-type-judgment-Empty less-eq-empty* **by** *auto*
next
case *nonempty*: *False*
with *nonempty* **have** *judged*: *fn-type-judgment* $= Of$ *min-common-type*
using *fn-type-judgment-Of* **by** *fast*
assume *sound*: *sound-judgment t*
with *nonempty* **obtain** *t0* **where** *t0-def*: $t = Of\ t0$ **using** *sound-nonempty* **by** *blast*
from *sound t0-def* **have** $\forall\ v \in$ *fn-value-set*. *of-type v t0*
using *sound-judgment-def judges-type-def* **by** *simp*
then have *t0* $\in \{t.\ \forall\ v \in$ *fn-value-set*. *of-type v t*$\}$ **by** *blast*
then have *min-common-type* $\leq$ *t0* **by** (*simp add*: *Inf-lower*)
with *judged t0-def* **show** *?thesis* **using** *less-eq-judg-def* **by** *fastforce*
qed

abbreviation *leqAll* **where** *leqAll xs ys* $\equiv$ *list-all2* $(\leq)$ *xs ys*

lemma *of-types-mono*: ‹*of-typeAll vs ts* $\Longrightarrow$ *leqAll ts ts'* $\Longrightarrow$ *of-typeAll vs ts'*›
by (*rule list-all2-trans*[**where** *?bs* $=$ *ts* **and** *?P1.0*=*of-type* **and** *?P2.0*=*less-eq*])
(*auto intro*: *of-type-mono*)

end

context
fixes *ts* :: *'typ list* **and** *ts'*
assumes *ts-leq*: *leqAll ts ts'*
begin

lemma *fn-value-set-mono*: *fn-value-set ts* $\subseteq$ *fn-value-set ts'*
unfolding *fn-value-set-def* **using** *ts-leq of-types-mono* **by** *auto*

lemma *sound-judgment-mono1*: *sound-judgment ts' t* $\Longrightarrow$ *sound-judgment ts t*
unfolding *sound-judgment-def* **using** *fn-value-set-mono* **by** *fast*

theorem *type-judgment-mono*: (*fn-type-judgment ts*) $\leq$ (*fn-type-judgment ts'*)
using *fn-type-judgment-sound fn-type-judgment-minimal sound-judgment-mono1*
by *auto*

end

definition *arity* **where** *arity n* ≡ ∃ *vs*. *length vs* = *n* ∧ *mf vs* ≠ *None*

definition *max-arity* **where** *max-arity* ≡ *Max* {*n*. *arity n*}

definition *signature-ith* **where**
‹*signature-ith i* ≡ ∏{*t*. ∀ *vs*. *length vs* ≥ *Suc i* ∧ *mf vs* ≠ *None* ⟶ *of-type* (*vs* ! *i*) *t*}›

definition *signature* **where**
‹*signature* = *map signature-ith* [*0*..<*max-arity*]›

end — mf
end — typing
end — theory
theory *Domain* **imports** *Main Typing*
HOL−Library.Disjoint-Sets
abbrevs .> = ⇀ **and** >< = ⋈
begin

lemma *map-restrict-self-dom*: *m* |‘ *dom m* = *m*
by (*simp add*: *ext domIff restrict-map-def*)

lemma *map-restrict-dom-inter*: *m* |‘ *A* = *m* |‘ (*A* ∩ *dom m*)
by (*simp add*: *ext domIff restrict-map-def*)

lemma *map-restrict-dom-subset*:
fixes *m*:: *'col* ⇀ *'val*
fixes *s*:: *'col set*
assumes *dom m* ⊆ *s*
shows *m* |‘ *s* = *m*
using *assms map-restrict-self-dom map-restrict-dom-inter*[*of m s*]
by (*metis inf.absorb-iff2*)

lemma *map-restrict-le*: *m* |‘ *A* ⊆$_m$ *m* **by** (*metis restrict-in domIff restrict-out map-le-def*)

lemma *map-concat-restrict-dom*:
fixes *m*:: *'col* ⇀ *'val*
and *n*:: *'col* ⇀ *'val*
and *h*:: *'col set*
assumes *dom m* ∩ *h* = {}
shows (*m* ++ *n*) |‘ *h* = *n* |‘ *h*
using *assms* **unfolding** *restrict-map-def map-add-def fun-eq-iff* **by** (*auto split*: *option.split*)

lemma *map-restrict-union*:
fixes *m*:: *'col* ⇀ *'val*
and *s1*:: *'col set*

```
  and s2:: 'col set
shows m |` (s1 ∪ s2) = m |` s1 ++ m |` s2
proof
  let ?m1 = m |` s1
  let ?m2 = m |` s2
  fix x
  show (m |` (s1 ∪ s2)) x = (?m1 ++ ?m2) x
  proof (cases x ∈ dom ?m2)
    case True
    then show ?thesis by (simp add: Map.map-add-dom-app-simps)
  next
    case not-in: False
   then have (?m1 ++ ?m2) x = ?m1 x by (simp add: Map.map-add-dom-app-simps)
     also have ... = (m |` (s1 ∪ s2)) x  by (metis not-in restrict-out restrict-in
Un-iff domIff)
    finally show ?thesis ..
  qed
qed

abbreviation nil ≡ Map.empty

lemma disjoint-family-on-subset: fixes A B S shows
  disjoint-family-on A S ⟹ ∀ x ∈ S. B x ⊆ A x ⟹ disjoint-family-on B S
  unfolding disjoint-family-on-def by blast

locale unrestricted-rows =
  fixes
  table-of :: 'col ⇒ 'tab
  and table-columns :: 'tab ⇒ 'col set
  and data :: 'tab ⇒ ('col ⇀ 'val) set
defines
  table-columns-def: table-columns t ≡ {c. table-of c = t}
assumes
  data-dom: ⋀t. ∀ m ∈ data t. dom m ⊆ table-columns t
begin

lemma disjoint-tables: disjoint-family table-columns
proof −
  have table-columns t1 ∩ table-columns t2 = {} if t1 ≠ t2 for t1 t2
    unfolding table-columns-def
    using that by auto
  then show ?thesis unfolding disjoint-family-on-def by auto
qed

abbreviation data' where data' x ≡ insert nil (data x)
definition choice-space where choice-space ≡ (Π x ∈ UNIV. data' x)

theorem choice-space-def': choice-space = {g. ∀ x. g x = nil ∨ g x ∈ data x}
  unfolding choice-space-def
```

by *blast*

lemma *choice-space-nil*: $(\lambda\text{-}.nil) \in choice\text{-}space$
unfolding *choice-space-def'* **by** *simp*

abbreviation *factorized* :: $('tab \Rightarrow ('col \rightharpoonup 'val)) \Rightarrow bool$
where $factorized\ g \equiv \forall\ t.\ dom\ (g\ t) \subseteq table\text{-}columns\ t$

lemma *choice-space-factorized*[*dest*]: $g \in choice\text{-}space \Longrightarrow factorized\ g$
unfolding *choice-space-def' mem-Collect-eq* **by** (*metis data-dom dom-empty empty-subsetI*)

definition *product* :: $('tab \Rightarrow ('col \rightharpoonup 'val)) \Rightarrow ('col \rightharpoonup 'val)$
where $product\ g\ c \equiv g\ (table\text{-}of\ c)\ c$

definition *factor* :: $('col \rightharpoonup 'val) \Rightarrow ('tab \Rightarrow ('col \rightharpoonup 'val))$
where $factor\ m\ t\ c \equiv if\ table\text{-}of\ c = t\ then\ m\ c\ else\ None$

lemma *prod-inv-factor*: $product\ (factor\ m) = m$ **for** m
unfolding *product-def factor-def* **by** *auto*

lemma *factor-inv-prod*: **assumes** *factorized g* **shows** $factor\ (product\ g) = g$
proof (*rule*, *rule*)
fix $t :: 'tab$ **and** c
have $dom\ (g\ t) \subseteq table\text{-}columns\ t$ **using** *assms* **by** *simp*
also have $... = \{c.\ table\text{-}of\ c = t\}$ **using** *table-columns-def* **by** *simp*
finally have $table\text{-}of\ c \neq t \Longrightarrow g\ t\ c = None$ **using** *table-columns-def* **by** *blast*
then have $g\ t\ c = (if\ table\text{-}of\ c = t\ then\ g\ t\ c\ else\ None)$ **by** *simp*
also have $... = (if\ table\text{-}of\ c = t\ then\ g\ (table\text{-}of\ c)\ c\ else\ None)$ **by** *simp*
also have $... = (if\ table\text{-}of\ c = t\ then\ product\ g\ c\ else\ None)$
using *product-def* **by** *simp*
also have $... = factor\ (product\ g)\ t\ c$ **using** *factor-def* **by** *simp*
finally show $factor\ (product\ g)\ t\ c = g\ t\ c$ **..**
qed

theorem *prod-inj*: **assumes** $\bigwedge x.\ x \in S \Longrightarrow factorized\ x$ **shows** *inj-on product S*
by (*rule inj-on-inverseI*[*of - factor*]) (*simp add*: *assms factor-inv-prod*)

corollary *prod-inj-choice-space*: *inj-on product choice-space*
using *prod-inj choice-space-factorized* **by** *blast*

theorem *factor-inj*: *inj factor*
by (*rule inj-on-inverseI*[*of - product*])
(*simp add*: *pointfree-idE prod-inv-factor*)

abbreviation *lim-table* **where** $lim\text{-}table\ t\ m \equiv m\ |`\ table\text{-}columns\ t$

lemma *factor-lim-table*: $lim\text{-}table\ t\ m = factor\ m\ t$
using *factor-def table-columns-def* **by** *auto*

corollary *factorized-factor*: *factorized* (*factor m*)
by (*metis factor-lim-table dom-restrict IntE subset-iff*)

corollary *factorized-factor-set*: $g \in$ *factor* ' $S \Longrightarrow$ *factorized g* **for** *g*
using *factorized-factor* **by** *auto*

corollary *lim-table-prod*: **assumes** *factorized g* **shows** *lim-table t* (*product g*) = *g t*
using *assms factor-lim-table factor-inv-prod* **by** *simp*

definition *support* :: ('*a* ⇒ ('*col* ⇀ '*val*)) ⇒ '*a set*
where *support g* ≡ {*t*. *g t* ≠ *nil*}

definition *limit* :: '*a set* ⇒ ('*a* ⇒ ('*col* ⇀ '*val*)) ⇒ ('*a* ⇒ ('*col* ⇀ '*val*))
where *limit T g t* ≡ *if t* ∈ *T then g t else nil*

lemma *support-limit*: *support* (*limit T g*) = *support g* ∩ *T* **for** *T g*
unfolding *support-def limit-def*
by (*rule set-eqI*) *simp*

lemma *limit-support*: *support g* ⊆ *T* ⟹ *limit T g* = *g* **for** *T g*
unfolding *support-def limit-def*
by *force*

lemma *limit-choice-space*: *g* ∈ *choice-space* ⟹ *limit T g* ∈ *choice-space* **for** *T g*
unfolding *choice-space-def' limit-def*
by *simp*

lemma *limit-limit*: *limit T* ∘ *limit S* = *limit* (*T* ∩ *S*) **for** *T S*
unfolding *limit-def*
by *fastforce*

lemma *support-factor*: *t* ∈ *support* (*factor m*) ⟷ *dom m* ∩ *table-columns t* ≠ {} **for** *t*
proof
assume *t* ∈ *support* (*factor m*)
then have *factor m t* ≠ *nil* **using** *support-def* **by** *fast*
then have *lim-table t m* ≠ *nil* **using** *factor-lim-table* **by** *simp*
then have {} ≠ *dom* (*lim-table t m*) **by** *blast*
also have *dom* (*lim-table t m*) = *dom m* ∩ *table-columns t* **using** *Map.dom-restrict* **by** *simp*
finally show *dom m* ∩ *table-columns t* ≠ {} **..**
next
assume *dom m* ∩ *table-columns t* ≠ {}
then obtain *c* **where** *c* ∈ *dom m* ∩ *table-columns t* **by** *blast*
then have *m c* ≠ *None* **and** *table-of c* = *t* **using** *table-columns-def* **by** *auto*
then have *factor m t c* ≠ *None* **using** *factor-def* **by** *simp*
then have *factor m t* ≠ *nil* **by** *auto*

then show $t \in$ *support* (*factor* m) **using** *support-def* **by** *fast*
qed

definition *subspace* **where** *subspace* $U \equiv \{g \in$ *choice-space*. *support* $g \subseteq U\}$

definition *restricted-universe* **where**
restricted-universe $U =$ *product* ‘ *subspace* U

lemma *choice-space-subspace*: *choice-space* = *subspace UNIV*
unfolding *subspace-def* **by** *simp*

theorem *subspace-eq-factor*: *subspace* $U =$ *factor* ‘ *restricted-universe* U **for** U
proof −
have *ss-f*: $g \in$ *subspace* $U \Longrightarrow$ *factorized* g **for** g
using *subspace-def choice-space-factorized* **by** *simp*
have *ru-f*: $g \in$ *factor* ‘ *restricted-universe* $U \Longrightarrow$ *factorized* g **for** g
using *factorized-factor-set* **by** *simp*
from *ss-f ru-f* **have** *inj-on-Un*: *inj-on product* (*subspace* $U \cup$ (*factor* ‘ *restricted-universe* U))
using *prod-inj* **by** *fastforce*
thm *inj-on-Un-image-eq-iff*

have *product* ‘ *subspace* $U =$ *restricted-universe* U
unfolding *subspace-def restricted-universe-def* **by** *presburger*
also have ... = *product* ‘ *factor* ‘ *restricted-universe* U
by (*subst image-comp*) (*simp add*: *prod-inv-factor*)
finally show *subspace* $U =$ *factor* ‘ *restricted-universe* U
using *inj-on-Un-image-eq-iff inj-on-Un* **by** *metis*
qed

lemma *subspace-limit-def*: *subspace* $S =$ *limit* S ‘ *choice-space*
proof (*rule*; *rule subsetI*)
fix x **assume** $x \in$ *subspace* S
then have *cs*: $x \in$ *choice-space* **and** *supp*: *support* $x \subseteq S$
using *subspace-def* **by** *simp-all*
from *supp* **have** *limit* S $x = x$ **by** (*rule limit-support*)
with *cs* **show** $x \in$ *limit* S ‘ *choice-space* **by** *force*
next
fix x **assume** $x \in$ *limit* S ‘ *choice-space*
then obtain y **where** *y-def*:$y \in$ *choice-space* $x =$ *limit* S y **by** *blast*
then have *1*: *support* $x \subseteq S$ **using** *support-limit* **by** *fast*
from *y-def* **have** *2*: $x \in$ *choice-space* **using** *limit-choice-space* **by** *simp*
from *1 2* **show** $x \in$ *subspace* S **unfolding** *subspace-def* **by** *simp*
qed

lemma *limit-subspace*: *limit* S ‘ *subspace* $T =$ *subspace* $(S \cap T)$
using *subspace-limit-def image-comp limit-limit* **by** *metis*

lemma *restricted-universe-factor-support*:

assumes $m \in$ *restricted-universe* T
shows *support* (*factor* m) $\subseteq T$
proof −
from *assms* **obtain** g **where** $m =$ *product* g **and** $g \in$ *choice-space*
and *sub-t*: *support* $g \subseteq T$
using *restricted-universe-def subspace-def* **by** *auto*
then have *factor* $m = g$ **using** *factor-inv-prod* **by** *auto*
with *sub-t* **show** *?thesis* **by** *simp*
qed

definition *limit-tables* $T\ m \equiv m\ |`\ (table\text{-}of\ -`\ T)$

lemma *limit-tables-factor*: *limit-tables* $T =$ *product* $\circ$ (*limit* T) $\circ$ *factor*
unfolding *limit-tables-def factor-def product-def limit-def*
by (*intro ext*) *simp*

lemma *limit-tables-restricted*:
limit-tables S ` *restricted-universe* $T =$ *restricted-universe* $(S \cap T)$
proof −
have *limit-tables* S ` (*restricted-universe* T) =
(*product* $\circ$ (*limit* S) $\circ$ *factor*) ` *restricted-universe* T
using *limit-tables-factor* **by** *simp*
also have ... = *product* ` *limit* $S$ ` *factor* ` *restricted-universe* T
using *image-comp* **by** *metis*
also have *factor* ` *restricted-universe* $T =$ *subspace* T **by** (*fact subspace-eq-factor*[*symmetric*])
also have *limit* S ` *subspace* $T =$ *subspace* $(S \cap T)$ **by** (*fact limit-subspace*)
also have *product* ` *subspace* $(S \cap T) =$ *restricted-universe* $(S \cap T)$
using *restricted-universe-def* **by** *presburger*
finally show *?thesis* **.**
qed

lemma *subspace-mono*: $S \subseteq T \Longrightarrow$ *subspace* $S \subseteq$ *subspace* T **unfolding** *subspace-def* **by** *auto*

lemma *restricted-universe-mono*: $S \subseteq T \Longrightarrow$ *restricted-universe* $S \subseteq$ *restricted-universe* T
unfolding *restricted-universe-def* **using** *subspace-mono* **by** *blast*

lemma *factor-subspace-mem*: *factor* $m \in$ *subspace* $T \Longrightarrow m \in$ *restricted-universe* T
unfolding *subspace-eq-factor*
by (*simp add*: *factor-inj inj-image-mem-iff*)

definition *universe* **where** *universe* $\equiv$ *product* ` *choice-space*

lemma *universe-restricted*: *universe* = *restricted-universe UNIV*
unfolding *universe-def restricted-universe-def*
using *choice-space-subspace* **by** *simp*

corollary *restricted-universe-limit*: *limit-tables S ‘ universe = restricted-universe S*
using *limit-tables-restricted universe-restricted* **by** *simp*

corollary *factor-universe*: *factor ‘ universe = choice-space*
using *universe-restricted subspace-eq-factor choice-space-subspace* **by** *simp*

corollary *subset-universe*: *restricted-universe S ⊆ universe* **for** *S*
unfolding *universe-restricted* **using** *restricted-universe-mono* **by** *simp*

lemma *universe-nil*: *nil ∈ universe* **proof** −
have *nil = product* (*λ-. nil*) **using** *product-def* **by** *auto*
also have *... ∈ universe* **using** *choice-space-nil universe-def* **by** *auto*
finally show *?thesis*.
qed

definition *tables-of* :: (*'col ⇀ 'val*) *⇒ 'tab set*
where *tables-of m = table-of ‘* (*dom m*)

lemma *tables-of-support*: *tables-of m = support* (*factor m*)
proof −
have *tables-of m = table-of ‘* (*dom m*) **using** *tables-of-def* **by** *simp*
also have *... =* {*table-of c | c. m c ≠ None*} **by** *blast*
also have *... =* {*table-of c | c. factor m* (*table-of c*) *c ≠ None*} **using** *factor-def* **by** *simp*
also have *... =* {*t. ∃ c. factor m t c ≠ None*} **using** *factor-def* **by** *auto*
also have *... =* {*t. factor m t ≠ nil*} **by** *auto*
also have *... = support* (*factor m*) **using** *support-def* **by** *metis*
finally show *?thesis* .
qed

lemma *limit-tables-subset*: **assumes** *tables-of m ⊆ S* **shows** *limit-tables S m = m*
proof −
have *limit-tables S m = product* (*limit S* (*factor m*))
using *limit-tables-factor* **by** *auto*
also have *limit S* (*factor m*) = *factor m* **using** *limit-support tables-of-support assms* **by** *auto*
also have *product ... = m* **by** (*rule prod-inv-factor*)
finally show *?thesis* .
qed

lemma *support-restricted-universe-mem*: *g ∈ universe ⟹ g ∈ restricted-universe* (*tables-of g*)
proof −
define *T* **where** *T = tables-of g*
then have *T-eq*: *T = support* (*factor g*) **using** *tables-of-support* **by** *simp*

assume *g ∈ universe*
then have *factor g ∈ choice-space* **using** *factor-universe* **by** *auto*

with *T-eq* **have** *factor g ∈ subspace T* **using** *subspace-def* **by** *simp*
then show *g ∈ restricted-universe T* **by** (*rule factor-subspace-mem*)
qed

lemma *restricted-universe-tables-of*: *m ∈ restricted-universe S ⟹ tables-of m ⊆ S* **for** *m S*
using *restricted-universe-factor-support tables-of-support* **by** *simp*

lemma *restricted-universe-nil*: *nil ∈ restricted-universe T* **for** *T*
proof
have *tables-of nil = {}* **using** *tables-of-def* **by** *fast*
then show *nil ∈ restricted-universe {}*
using *support-restricted-universe-mem universe-nil* **by** *metis*
show *restricted-universe {} ⊆ restricted-universe T*
using *restricted-universe-mono* **by** *simp*
qed

definition *extended-universe* **where**
extended-universe ≡ {m′. ∃ m ∈ universe. m′ ⊆m m}

abbreviation *project* **where** *project C m ≡ m |` C*

theorem *project-project*: *project A* (*project B m*) = *project* (*A ∩ B*) *m* **for** *A B m*
unfolding *Map.restrict-map-def* **by** *fastforce*

lemma *extended-universe-mem*: *universe ⊆ extended-universe*
unfolding *extended-universe-def* **by** *force*

lemma *extended-universe-mono*:
m ∈ extended-universe ⟹ m′ ⊆m m ⟹ m′ ∈ extended-universe
unfolding *extended-universe-def* **using** *map-le-trans* **by** *blast*

3 Additional definitions

abbreviation *universe-of cols ≡ restricted-universe* (*table-of ` cols*)

abbreviation *is-key* :: *′col set ⇒ bool*
where *is-key cols ≡ inj-on* (*project cols*) (*universe-of cols*)

abbreviation *table-cols-of cols ≡ table-of − ` table-of ` cols*

abbreviation *key-limit cols ≡ project* (*table-cols-of cols*)

lemma *key-limit-tables*: *key-limit cols m = limit-tables* (*table-of ` cols*) *m*
by (*simp add*: *limit-tables-def*)

corollary *key-limit-universe-of*: *m ∈ universe ⟹ key-limit cols m ∈ universe-of cols*
unfolding *key-limit-tables*

using *restricted-universe-limit*
by *blast*

3.1 Facts about *table-cols-of* as an automorphism of set algebra

lemma *table-cols-tables*: *table-of* ‘ (*table-cols-of cols*) = *table-of* ‘ *cols*
by *blast*

corollary *table-cols-of-idem*: *table-cols-of* (*table-cols-of cols*) = *table-cols-of cols*
using *table-cols-tables* **by** *blast*

lemma *table-cols-of-union*: *table-cols-of A* ∪ *table-cols-of B* = *table-cols-of* (*A* ∪ *B*)
by *auto*

lemma *table-cols-of-superset*: *cols* ⊆ *table-cols-of cols* **by** *blast*

abbreviation *agrees A x y* ≡ *x* |‘ *A* = *y* |‘ *A*

lemma *agrees-iff*: *agrees A x y* ⟷ (∀ *a* ∈ *A*. *x a* = *y a*)
proof −
have *agrees A x y* ⟷ (∀ *a*. (*x* |‘ *A*) *a* = (*y* |‘ *A*) *a*) **by** *auto*
also have ... = (∀ *a* ∈ *A*. (*x* |‘ *A*) *a* = (*y* |‘ *A*) *a*) ∧ (∀ *a*. *a* ∉ *A* ⟶ (*x* |‘ *A*) *a* = (*y* |‘ *A*) *a*)
by *fastforce*
also have (∀ *a*. *a* ∉ *A* ⟶ (*x* |‘ *A*) *a* = (*y* |‘ *A*) *a*) ⟷ *True* **by** *simp*
also have (∀ *a* ∈ *A*. (*x* |‘ *A*) *a* = (*y* |‘ *A*) *a*) ⟷ (∀ *a* ∈ *A*. *x a* = *y a*) **by** *simp*
finally show *?thesis* **by** *simp*
qed

lemma *agrees-union-iff*[*iff*]: *agrees A x y* ∧ *agrees B x y* ⟷ *agrees* (*A* ∪ *B*) *x y*
unfolding *agrees-iff* **by** *blast*

lemma *agrees-mono*: *A* ⊆ *B* ⟹ *agrees B x y* ⟹ *agrees A x y*
unfolding *agrees-iff* **by** *blast*

abbreviation *is-key-universal* :: *'col set* ⇒ *prop*
where *is-key-universal cols* ≡ (⋀*x y*. *x* ∈ *universe* ⟹ *y* ∈ *universe* ⟹
agrees cols x y ⟹
agrees (*table-cols-of cols*) *x y*)

lemma *key-limit-project-project*: *project cols* (*key-limit cols m*) = *project cols m*
using *project-project table-cols-of-superset* **by** (*simp add*: *inf.absorb-iff2*)

theorem *is-key-universalD*[*dest?*]: *is-key cols* ⟹ *PROP is-key-universal cols* **proof** −
assume *inj-1*: *is-key cols*

fix *x y*
assume *x*: *x* ∈ *universe* **and** *y*: *y* ∈ *universe*
assume *proj-eq*: *project cols x* = *project cols y*
let *?x1* = *key-limit cols x* **and** *?y1* = *key-limit cols y*
from *x* **have** *x1*: *?x1* ∈ *universe-of cols* **by** (*rule key-limit-universe-of*)
from *y* **have** *y1*: *?y1* ∈ *universe-of cols* **by** (*rule key-limit-universe-of*)
have *project cols ?x1* = *project cols x* **by** (*rule key-limit-project-project*)
also have ... = *project cols y* **by** (*fact proj-eq*)
also have ... = *project cols ?y1* **by** (*rule key-limit-project-project*[*symmetric*])
finally show *?x1* = *?y1* **using** *x1 y1 inj-1 inj-onD* **by** *metis*
qed

lemma *is-key-universalI*[*intro?*]: *PROP is-key-universal cols* ⟹ *is-key cols* **proof** (*rule inj-onI*)
assume *inj-2*: *x* ∈ *universe* ⟹ *y* ∈ *universe* ⟹ *project cols x* = *project cols y* ⟹
key-limit cols x = *key-limit cols y* **for** *x y*
fix *x y*
assume *x*: *x* ∈ *universe-of cols* **and** *y*: *y* ∈ *universe-of cols*
have *conv*: *z* ∈ *universe* ∧ *key-limit cols z* = *z* (**is** *?prop z*) **if** *z* ∈ *universe-of cols* **for** *z*
using *subset-universe limit-tables-subset restricted-universe-tables-of that key-limit-tables subsetD* **by** *metis*

from *x* **have** *x1*: *?prop x* **by** (*rule conv*)
from *y* **have** *y1*: *?prop y* **by** (*rule conv*)

assume *project cols x* = *project cols y*
with *inj-2 x1 y1* **show** *x* = *y* **by** *metis*
qed

thm *agrees-union-iff*

theorem *is-key-union*: *is-key A* ⟹ *is-key B* ⟹ *is-key* (*A* ∪ *B*)
proof (*rule is-key-universalI*)
assume *is-key A* **then have** *aprop*: *PROP is-key-universal A* **by** (*rule is-key-universalD*)
assume *is-key B* **then have** *bprop*: *PROP is-key-universal B* **by** (*rule is-key-universalD*)
fix *x y* **assume** *x*: *x* ∈ *universe* **and** *y*: *y* ∈ *universe*
assume *congrU*: *agrees* (*A* ∪ *B*) *x y*
then have *agrees A x y* ∧ *agrees B x y* **by** *simp*
then have *congrA*: *agrees A x y* **and** *congrB*: *agrees B x y*
by (*satx*+)
from *x y congrA* **have** *klA*: *agrees* (*table-cols-of A*) *x y* **by** (*rule aprop*)
from *x y congrB* **have** *klB*: *agrees* (*table-cols-of B*) *x y* **by** (*rule bprop*)
from *klA klB* **have** *agrees* (*table-cols-of A* ∪ *table-cols-of B*) *x y* **using** *agrees-union-iff* **by** *fast*
then show *agrees* (*table-cols-of* (*A* ∪ *B*)) *x y* **using** *table-cols-of-union* **by** *simp*
qed

end
end
theory *Interp* **imports** *Complex-Main* **begin**

4 Language algebra. Initial algebra is defined as term*etyp*, and a generic algebra is defined as term*interp*.

datatype *etyp* =
  int | *float* | *numeric* | *string* | *date* | *datetime* | *interval*

type-synonym *fid* = *string*

datatype *esyntax* =
  Int-lit int
  | *Bool-lit bool*
  | *Numeric-lit rat*
  | *Lit etyp string*
  | *Call fid esyntax list*

record *'a interp* =
  int-lit :: *int* ⇒ *'a*
  bool-lit :: *bool* ⇒ *'a*
  numeric-lit :: *rat* ⇒ *'a*
  lit :: *etyp* ⇒ *string* ⇒ *'a*
  call :: *fid* ⇒ *'a list* ⇒ *'a*

Catamorphism esyntax => interp.

fun *efold* :: *'a interp* ⇒ *esyntax* ⇒ *'a* **where**
  efold-int-lit: *efold R* (*Int-lit i*) = *int-lit R i*
| *efold-bool-lit*: *efold R* (*Bool-lit b*) = *bool-lit R b*
| *efold-numeric-lit*: *efold R* (*Numeric-lit r*) = *numeric-lit R r*
| *efold-lit*: *efold R* (*Lit t s*) = *lit R t s*
| *efold-call*: *efold R* (*Call f args*) = *call R f* (*map* (*efold R*) *args*)

definition *Syntax-interp* :: *esyntax interp* **where**
‹*Syntax-interp* = (|
  int-lit = *Int-lit*,
  bool-lit = *Bool-lit*,
  numeric-lit = *Numeric-lit*,
  lit = *Lit*,
  call = *Call*
|)›

inductive-set *generated*:: *'a interp* ⇒ *'a set* **for** *i* **where**
  int-lit: *int-lit i x* ∈ *generated i*
| *bool-lit*: *bool-lit i x* ∈ *generated i*
| *numeric-lit*: *numeric-lit i x* ∈ *generated i*
| *lit*: *lit i t x* ∈ *generated i*
| *call*: (⋀ *x*::*'a*. *x* ∈ *set args* ⟹ *x* ∈ *generated i*) ⟹ *call i fn args* ∈ *generated i*

abbreviation *Generated* :: *'a interp* ⇒ *'a set* **where** *Generated i* ≡ *range* (*efold i*)

lemma *efold-generated*: *efold i e* ∈ *generated i*
  by (*induction e*, *simp-all add*: *generated.intros*, *rule*, *auto*)

lemma *generated-efold*: *x* ∈ *generated i* ⟹ ∃ *e*. *efold i e* = *x*
proof (*induction x rule*: *generated.induct*, *auto intro*!: *efold.simps*)
  case (*call args fn*)
  assume *arg-exist*: ⋀ *arg*. (*arg* ∈ *set args* ⟹ (∃ *e*. *efold i e* = *arg*))
  have ∃ *args0*. *map* (*efold i*) *args0* = *args* **by** (*metis arg-exist ex-map-conv*)
  then obtain *args0* **where** *args0-map*: *map* (*efold i*) *args0* = *args* **..**
  define *e* **where** *e* = *Call fn args0*
  then have *efold i e* = *call i fn args* **by** (*simp add*: *args0-map*)
  then show ∃ *e*. *efold i e* = *call i fn args* **..**
  qed

theorem *generated-equiv*: *generated i* = *Generated i*
  using *generated-efold efold-generated* **by** *fast*

definition *is-hom* :: *'a interp* ⇒ *'b interp* ⇒ (*'a* ⇒ *'b*) ⇒ *bool* **where**
‹*is-hom A B f* ≡ ∀ *x*. *f* (*efold A x*) = *efold B x*›

lemma *Syntax-idem* : *efold Syntax-interp x* = *x*
  by (*induction x*) (*auto simp*: *Syntax-interp-def list.map-ident-strong*)

lemma *cata1* : *is-hom Syntax-interp i* (*efold i*)
  by (*simp add*: *is-hom-def Syntax-idem*)

lemma *cata2* : *is-hom Syntax-interp i f* ⟹ *f x* = *efold i x*
  by (*simp add*: *is-hom-def Syntax-idem*)

theorem *is-cata* : *is-hom Syntax-interp i f* = (*f* = *efold i*)
  using *cata1 cata2* **by** *blast*

type-synonym *'a esyntax'* = *'a interp* ⇒ *'a*

Functions which are "morally an AST"

definition *syntax-uniform* **where** *syntax-uniform i f g* ≡ *f i* = *efold i g*

theorem *tst*: **assumes** *f i* = *efold i* (*f Syntax-interp*) **shows** *True*
  by *simp*

end
theory *Subdomain*
imports *HOLCF HOLCF−extra.Directed HOLCF−extra.Lattice-Complete*
abbrevs ,, = ⌊
begin

default-sort {*bifinite*, *bounded-complete*}

[1]

5 Stable deflations

A deflation is *stable* when its image is downward closed. The pointwise facts about stable subdomains are stated here for deflations; the subdomains below only add the construction of the projection.

locale *stable-defl* = *deflation d* **for** *d* :: 'a::{*bifinite*, *bounded-complete*} → 'a +
assumes *downclosed*: $d{\cdot}y = y \Longrightarrow x \sqsubseteq y \Longrightarrow d{\cdot}x = x$
begin

lemma *deflation*: *deflation d* **by** (*fact deflation-axioms*)

Every image element below x is below $d{\cdot}x$; every lower bound of an image element lies in the image.

lemma *below-image*: $d{\cdot}a = a \Longrightarrow a \sqsubseteq x \Longrightarrow a \sqsubseteq d{\cdot}x$
by (*metis monofun-cfun-arg*)

lemma *lower-fixed*: $l \sqsubseteq d{\cdot}x \Longrightarrow d{\cdot}l = l$
by (*rule downclosed*[*OF idem*])

Hence the deflation commutes with existing binary meets.

lemma *glb-fixed*:
assumes *m*: *is-glb* {*x*, *y*} *m*
shows *is-glb* {$d{\cdot}x$, $d{\cdot}y$} ($d{\cdot}m$)
proof (*rule is-glbI*)
fix *z* **assume** $z \in \{d{\cdot}x, d{\cdot}y\}$
moreover have $m \sqsubseteq x$ $m \sqsubseteq y$ **using** *m* **by** (*auto intro*: *is-glb-below*)
ultimately show $d{\cdot}m \sqsubseteq z$ **by** (*auto intro*: *monofun-cfun-arg*)
next
fix *l* **assume** $\forall z{\in}\{d{\cdot}x, d{\cdot}y\}.\ l \sqsubseteq z$
then have *lx*: $l \sqsubseteq d{\cdot}x$ **and** *ly*: $l \sqsubseteq d{\cdot}y$ **by** *simp-all*
have *fix-l*: $d{\cdot}l = l$ **using** *lx* **by** (*rule lower-fixed*)
have $l \sqsubseteq x$ **using** *lx below* **by** (*rule below-trans*)
moreover have $l \sqsubseteq y$ **using** *ly below* **by** (*rule below-trans*)
ultimately have $l \sqsubseteq m$ **using** *m* **by** (*auto intro*: *is-glb-greatest*)
then have $d{\cdot}l \sqsubseteq d{\cdot}m$ **by** (*rule monofun-cfun-arg*)
with *fix-l* **show** $l \sqsubseteq d{\cdot}m$ **by** *simp*

qed

Below an image element, meeting with z or with its image is the same.

lemma *glb-relative*:
assumes *m*: *is-glb* $\{d{\cdot}c,\ d{\cdot}z\}\ m$
shows *is-glb* $\{d{\cdot}c,\ z\}\ m$
proof (*rule is-glbI*)
fix w **assume** $w \in \{d{\cdot}c,\ z\}$
moreover have $m \sqsubseteq d{\cdot}c$ $m \sqsubseteq d{\cdot}z$ **using** m **by** (*auto intro*: *is-glb-below*)
ultimately show $m \sqsubseteq w$ **using** *below*[*of z*] **by** (*auto intro*: *below-trans*)
next
fix l **assume** $\forall\, w{\in}\{d{\cdot}c,\ z\}.\ l \sqsubseteq w$
then have *lc*: $l \sqsubseteq d{\cdot}c$ **and** *lz*: $l \sqsubseteq z$ **by** *simp-all*
have $d{\cdot}l = l$ **using** *lc* **by** (*rule lower-fixed*)
moreover from *lz* **have** $d{\cdot}l \sqsubseteq d{\cdot}z$ **by** (*rule monofun-cfun-arg*)
ultimately have $l \sqsubseteq d{\cdot}z$ **by** *simp*
with *lc* **show** $l \sqsubseteq m$ **using** m **by** (*auto intro*: *is-glb-greatest*)
qed

end

lemma *stable-deflI*:
deflation $d \Longrightarrow (\bigwedge x\ y.\ d{\cdot}y = y \Longrightarrow x \sqsubseteq y \Longrightarrow d{\cdot}x = x) \Longrightarrow$ *stable-defl* d
by (*simp add*: *stable-defl-def stable-defl-axioms-def*)

Jung, Lemma 4.1.6(i): the image of x below some image element y is the meet $x \sqcap y$. Binary meets need bounded completeness.

lemma *stable-defl-image-meet*:
assumes *d*: *stable-defl* d **and** *y*: $d{\cdot}y = y$ **and** *le*: $d{\cdot}x \sqsubseteq y$
shows $d{\cdot}x = x \sqcap y$
proof (*rule below-antisym*)
interpret *stable-defl* d **by** (*rule d*)
show $d{\cdot}x \sqsubseteq x \sqcap y$ **using** *below le* **by** (*rule meet-greatest*)
have $d{\cdot}(x \sqcap y) = x \sqcap y$ **using** *y meet-below2* **by** (*rule downclosed*)
then show $x \sqcap y \sqsubseteq d{\cdot}x$ **using** *meet-below1* **by** (*rule below-image*)
qed

6 Stable subdomains

locale *stable-subdomain* =
fixes $S :: 'a\ set$ (**structure**)
assumes *lub-closed*: $T \subseteq S \Longrightarrow T <<|\ t \Longrightarrow t \in S$
assumes *downward*: $y \in S \Longrightarrow x \sqsubseteq y \Longrightarrow x \in S$
begin
definition *proj* a = *lub* $\{x \in S.\ x \sqsubseteq a\}$

lemma *proj-set-directed*: *directed* $\{x \in S.\ x \sqsubseteq a\}$
proof (*rule directedI*, *safe*)

fix x y **assume** S: $x \in S$ $y \in S$
assume a: $x \sqsubseteq a$ $y \sqsubseteq a$
then have *lub*: $\{x, y\} <<| (x \sqcup y)$ **by** (*rule join-is-lub*)
from *lub-closed*[*OF - this*] S **have** *lubS*: $x \sqcup y \in S$
by *blast*
from *lub* **have** *le*: $x \sqsubseteq x \sqcup y$ $y \sqsubseteq x \sqcup y$ **by** (*auto dest*: *is-lubD1 is-ubD*)
moreover from a **have** $x \sqcup y \sqsubseteq a$
using *is-lubD2*[*OF lub*, *of a*] **by** *auto*
ultimately show $\exists z \in \{x \in S.\ x \sqsubseteq a\}.\ x \sqsubseteq z \wedge y \sqsubseteq z$
using *lubS* **by** *blast*
qed

lemma *bot-in-S*: $\bot \in S$
using *lub-closed*[*of* $\{\}$ $\bot$] **by** *simp*

lemma *proj-set-ne*: $\{x \in S.\ x \sqsubseteq a\} \neq \{\}$
proof −
have $\bot \in \{x \in S.\ x \sqsubseteq a\}$ **using** *bot-in-S* **by** *simp*
then show *?thesis* **by** *auto*
qed

corollary *proj-is-lub*: $\{x \in S.\ x \sqsubseteq a\} <<|$ *proj a*
unfolding *proj-def* **using** *proj-set-directed* **by** (*rule directed-lub*)

corollary *proj-in-S*: *proj a* $\in S$
using *lub-closed*[*OF - proj-is-lub*] **by** *blast*

lemma *proj-below*: *proj a* $\sqsubseteq a$
by (*blast intro*: *proj-is-lub*[*THEN is-lubD2*, *of a a*] *is-ubI*)

corollary *proj-fixed*: $a \in S \Longrightarrow$ *proj a* $= a$
proof (*rule below-antisym*)
show *proj a* $\sqsubseteq a$ **by** (*fact proj-below*)
assume $a \in S$
then have $a \in \{x \in S.\ x \sqsubseteq a\}$ **by** *simp*
with *proj-is-lub*[*THEN is-lubD1*, *THEN is-ubD*] **show** $a \sqsubseteq$ *proj a* **.**
qed

corollary *proj-fixed-iff*: *proj a* $= a \longleftrightarrow a \in S$
using *proj-in-S proj-fixed* **by** *metis*

corollary *below-set-is-down-set*:
$\{x \in S.\ x \sqsubseteq a\} = \{x.\ x \sqsubseteq$ *proj a*$\}$
proof (*intro subset-antisym subsetI*; *safe*)
fix x **assume** $x \in S$ $x \sqsubseteq a$
with *proj-is-lub*[*THEN is-lubD1*, *THEN is-ubD*]
show $x \sqsubseteq$ *proj a* **by** *blast*
next
fix x **assume** $*$: $x \sqsubseteq$ *proj a*

from *below-trans*[*OF* ∗ *proj-below*]
show $x \sqsubseteq a$ **.**
from *downward*[*OF proj-in-S* ∗]
show $x \in S$ **.**
qed

lemma *approximants-proj*:
approximants (*proj a*) = {$x \in$ *approximants a*. *Rep-compact-basis* $x \in S$}
unfolding *approximants-def*
using *below-set-is-down-set* **by** *blast*

lemma *proj-mono*: *monofun proj*
proof (*rule monofunI*)
fix *x y* :: *'a* **assume** $x \sqsubseteq y$
then have {$z \in S$. $z \sqsubseteq x$} ⊆ {$z \in S$. $z \sqsubseteq y$}
using *below-trans* **by** *blast*
with *proj-is-lub proj-is-lub* **show** *proj x* ⊑ *proj y*
by (*rule lub-subset-mono*)
qed

corollary *below-proj-of*: $a \in S \Longrightarrow a \sqsubseteq x \Longrightarrow a \sqsubseteq$ *proj x*
using *proj-is-lub*[*THEN is-lubD1*, *THEN is-ubD*] **by** *blast*

lemma *proj-monoD*: $x \sqsubseteq y \Longrightarrow$ *proj x* ⊑ *proj y*
by (*rule monofunE*[*OF proj-mono*])

lemma *proj-cont*: *cont proj*
proof (*rule contI2*)
show *monofun proj* **by** (*fact proj-mono*)
fix *Y* :: *nat* ⇒ *'a* **assume** *Y*: *chain Y*
assume *pY*: *chain* (λ*i*. *proj* (*Y i*))
from *Y* **have** *1*: *approximants* (⨆ *i*. *Y i*) = (⋃ *i*. *approximants* (*Y i*))
by (*rule compact-basis.rep-lub*)
from *pY* **have** *2*: *approximants* (⨆ *i*. *proj*(*Y i*)) = (⋃ *i*. *approximants* (*proj* (*Y i*)))
by (*rule compact-basis.rep-lub*)
from *1 2 approximants-proj*
have *approximants* (*proj* (⨆ *i*. *Y i*)) ⊆ *approximants* (⨆ *i*. *proj*(*Y i*))
by *auto*
then show *proj* (⨆ *i*. *Y i*) ⊑ (⨆ *i*. *proj*(*Y i*))
by (*rule compact-basis.belowI*)
qed

definition *p* **where** *p* ≡ Λ *a*. *proj a*

lemma *p-beta*: *p*·*x* = *proj x*
unfolding *p-def*
by (*simp add*: *proj-cont*)

lemma *deflation-p*: *deflation p*
proof
  show $p{\cdot}(p{\cdot}x) = p{\cdot}x$ **for** x
    using *proj-in-S proj-fixed*
    by (*simp add*: *p-beta*)
  show $p{\cdot}x \sqsubseteq x$ **for** x
    using *proj-below* **by** (*simp add*: *p-beta*)
qed

p-form duplicates of the projection lemmas. Since *p-beta* ($p{\cdot}x = proj\ x$) is deliberately not a simp rule it rewrites $p{\cdot}x$ against *deflation.below* etc. we restate the projection facts directly for $p{\cdot}x$ via $proj = Rep\text{-}cfun\ p$ and use those.

lemma *proj-Rep-p*: *proj* = *Rep-cfun p*
  by (*rule ext*) (*simp add*: *p-beta*)

lemmas *p-in-S*[*simp*] = *proj-in-S*[*unfolded proj-Rep-p*]
lemmas *p-below* = *proj-below*[*unfolded proj-Rep-p*]
lemmas *p-fixed* = *proj-fixed*[*unfolded proj-Rep-p*]
lemmas *p-fixed-iff* = *proj-fixed-iff*[*unfolded proj-Rep-p*]

The projection is a stable deflation; the deflation lemmas are inherited, and restated with membership in S for the fixed points.

sublocale *stable-defl p*
  by (*rule stable-deflI*[*OF deflation-p*]) (*use downward p-fixed-iff* **in** *simp*)

lemmas *below-p-of* = *below-image*[*unfolded p-fixed-iff*]
lemmas *p-meet* = *stable-defl-image-meet*[*OF stable-defl-axioms*, *unfolded p-fixed-iff*]

lemma *p-fixes*: $\{x.\ p{\cdot}x = x\} = S$
  by (*simp add*: *proj-fixed-iff p-beta*)

lemma *p-range*: *range* $(\lambda x.\ p{\cdot}x) = S$
  by (*simp only*: *p-fixes range-eq-fixes*)

end

locale *semi-factor-subdomain* = *stable-subdomain* +
  assumes *slide-rule*: $y \in S \Longrightarrow p{\cdot}x \sqsubseteq y \Longrightarrow x \Uparrow y$

lemma *stable-subdomain-deflation*:
  fixes *p* **assumes** *p*: *deflation p* **and** *ss*: *stable-subdomain* $\{x.\ p{\cdot}x = x\}$
  shows *p* = *stable-subdomain.p* $\{x.\ p{\cdot}x = x\}$
proof −
  interpret *p*: *deflation p* **by** (*rule p*)
  interpret *S*: *stable-subdomain* $\{x.\ p{\cdot}x = x\}$ **by** (*rule ss*)
  show *?thesis*

```
  proof (intro below-antisym)
    show p ⊑ S.p
    proof (rule p.belowI)
      fix x assume p·x = x
      then show S.p·x = x
        by (metis (mono-tags, lifting) S.p-fixes mem-Collect-eq)
    qed
    show S.p ⊑ p
    proof (rule S.belowI)
      fix x assume S.p·x = x
      then show p·x = x
        by (metis (mono-tags, lifting) S.p-fixes mem-Collect-eq)
    qed
  qed
qed
```

The range of any deflation is automatically closed under lubs of bounded subsets: only downward closure distinguishes a stable-subdomain projection from a general deflation.

```
lemma deflation-lub-closed:
  assumes d: deflation d and T: T ⊆ {x. d·x = x} and lub: T <<| t
  shows d·t = t
proof −
  interpret deflation d by (rule d)
  have T <| d·t
  proof (rule is-ubI)
    fix x assume x: x ∈ T
    with T have d·x = x by auto
    moreover from is-lubD1[OF lub] x have x ⊑ t by (rule is-ubD)
    ultimately show x ⊑ d·t by (metis monofun-cfun-arg)
  qed
  from is-lubD2[OF lub this] have t ⊑ d·t .
  with below show d·t = t by (rule below-antisym)
qed

lemma deflation-stable-subdomainI:
  assumes d: deflation d
    and dc: ⋀x y. d·y = y ⟹ x ⊑ y ⟹ d·x = x
  shows stable-subdomain {x. d·x = x}
proof
  fix y x assume y ∈ {x. d·x = x} x ⊑ y
  with dc show x ∈ {x. d·x = x} by simp
next
  fix T t assume T ⊆ {x. d·x = x} T <<| t
  with deflation-lub-closed[OF d] show t ∈ {x. d·x = x} by simp
qed
```

6.1 Stable deflations and stable subdomains

The image of a stable deflation is a stable subdomain (*deflation-stable-subdomainI*). Unlike the order predicates on deflations, stability is not antitone, but it is admissible on algebraic types. It is closed under the domain constructions, so e.g. the *take* functions of a domain are stable.

lemma *stable-defl-stable-subdomain*:
assumes *stable-defl d*
shows *stable-subdomain* {*x*. *d*·*x* = *x*}
by (*rule deflation-stable-subdomainI*[*OF stable-defl.deflation*[*OF assms*] *stable-defl.downclosed*[*OF assms*]])

lemma *stable-defl-bottom*: *stable-defl* (⊥ :: *'a* → *'a*)
by (*rule stable-deflI*[*OF deflation-bottom*]) *simp*

lemma *stable-defl-ID*: *stable-defl* (*ID* :: *'a* → *'a*)
by (*rule stable-deflI*[*OF deflation-ID*]) *simp*

lemma *adm-stable-defl*: *adm* (*stable-defl* :: (*'a* → *'a*) ⇒ *bool*)
proof (*rule admI*)
fix *D* :: *nat* ⇒ *'a* → *'a*
assume *chain*: *chain D* **and** *H*: ∀ *i*. *stable-defl* (*D i*)
have *Hd*: *deflation* (*D i*) **for** *i* **by** (*rule stable-defl.axioms*(*1*)[*OF H*[*rule-format*]])
have *defl*: *deflation* (⨆ *i*. *D i*) **by** (*rule admD*[*OF adm-deflation chain Hd*])
show *stable-defl* (⨆ *i*. *D i*)
proof (*rule stable-deflI*[*OF defl*])
fix *x y* **assume** *fy*: (⨆ *i*. *D i*)·*y* = *y* **and** *xy*: *x* ⊑ *y*
obtain *C* **where** *C*: *chain C* ∀ *i*. *compact* (*C i*) *x* = (⨆ *i*. *C i*)
by (*rule obtain-compact-chain*)
have *fixed*: (⨆ *i*. *D i*)·(*C i*) = *C i* **for** *i*
proof −
have *C i* ⊑ *x* **using** *is-ub-thelub*[*OF C*(*1*)] *C*(*3*) **by** *simp*
then have *C i* ⊑ *y* **using** *xy* **by** (*rule below-trans*)
then have *C i* ⊑ (⨆ *i*. *D i*)·*y* **using** *fy* **by** *simp*
also have ... = (⨆ *j*. *D j*·*y*) **by** (*rule contlub-cfun-fun*[*OF chain*])
finally have *Cilub*: *C i* ⊑ (⨆ *j*. *D j*·*y*) **.**
have *chy*: *chain* (λ*j*. *D j*·*y*) **by** (*rule ch2ch-Rep-cfunL*[*OF chain*])
from *compactD2*[*OF C*(*2*)[*rule-format*] *chy Cilub*] **obtain** *j* **where** *j*: *C i* ⊑ *D j*·*y* **by** *blast*
have *idem*: *D j*·(*D j*·*y*) = *D j*·*y* **by** (*rule deflation.idem*[*OF Hd*])
have *CiFix*: *D j*·(*C i*) = *C i*
by (*rule stable-defl.downclosed*[*OF H*[*rule-format*] *idem j*])
have *D j* ⊑ (⨆ *i*. *D i*) **by** (*rule is-ub-thelub*[*OF chain*])
then have *D j*·(*C i*) ⊑ (⨆ *i*. *D i*)·(*C i*) **by** (*rule monofun-cfun-fun*)
with *CiFix* **have** *C i* ⊑ (⨆ *i*. *D i*)·(*C i*) **by** *simp*
then show *?thesis* **by** (*rule below-antisym*[*OF deflation.below*[*OF defl*]])
qed
have *rcx*: *range C* <<| *x* **using** *cpo-lubI*[*OF C*(*1*)] *C*(*3*) **by** *simp*
have *range C* ⊆ {*z*. (⨆ *i*. *D i*)·*z* = *z*} **using** *fixed* **by** *auto*

from *deflation-lub-closed*[*OF defl this rcx*] **show** $(\bigsqcup i.\ D\ i)\cdot x = x$ **.**
qed
qed

lemma *stable-defl-u-map*:
assumes *f*: *stable-defl* $(f :: 'a \to 'a)$
shows *stable-defl* $(u\text{-}map\cdot f)$
proof (*rule stable-deflI*)
have *fd*: *deflation f* **by** (*rule stable-defl.axioms*(*1*)[*OF f*])
then show *deflation* $(u\text{-}map\cdot f)$ **by** (*rule deflation-u-map*)
fix *x y* **assume** *fy*: $u\text{-}map\cdot f\cdot y = y$ **and** *xy*: $x \sqsubseteq y$
show $u\text{-}map\cdot f\cdot x = x$
proof (*cases x*)
case *bottom* **then show** *?thesis* **by** *simp*
next
case (*up x'*)
with *xy* **obtain** y' **where** *y*: $y = up\cdot y'$ $x' \sqsubseteq y'$ **by** (*cases y*) *auto*
with *fy* **have** $f\cdot y' = y'$ **by** *simp*
from *stable-defl.downclosed*[*OF f this* ‹$x' \sqsubseteq y'$›] *up* **show** *?thesis* **by** *simp*
qed
qed

lemma *stable-defl-ssum-map*:
assumes *f*: *stable-defl f* **and** *g*: *stable-defl g*
shows *stable-defl* $(ssum\text{-}map\cdot f\cdot g)$
proof (*rule stable-deflI*)
show *deflation* $(ssum\text{-}map\cdot f\cdot g)$
by (*rule deflation-ssum-map*[*OF stable-defl.axioms*(*1*)[*OF f*] *stable-defl.axioms*(*1*)[*OF g*]])
fix *x y* **assume** *fy*: $ssum\text{-}map\cdot f\cdot g\cdot y = y$ **and** *xy*: $x \sqsubseteq y$
show $ssum\text{-}map\cdot f\cdot g\cdot x = x$
proof (*cases x*)
case *bottom* **then show** *?thesis* **by** *simp*
next
case (*sinl x'*)
with *xy* **obtain** y' **where** *y*: $y = sinl\cdot y'$ $y' \neq \bot$ $x' \sqsubseteq y'$ **by** (*cases y*) *auto*
with *fy* **have** $f\cdot y' = y'$ **by** *simp*
from *stable-defl.downclosed*[*OF f this* ‹$x' \sqsubseteq y'$›] *sinl* **show** *?thesis* **by** *simp*
next
case (*sinr x'*)
with *xy* **obtain** y' **where** *y*: $y = sinr\cdot y'$ $y' \neq \bot$ $x' \sqsubseteq y'$ **by** (*cases y*) *auto*
with *fy* **have** $g\cdot y' = y'$ **by** *simp*
from *stable-defl.downclosed*[*OF g this* ‹$x' \sqsubseteq y'$›] *sinr* **show** *?thesis* **by** *simp*
qed
qed

lemma *stable-defl-sprod-map*:
assumes *f*: *stable-defl f* **and** *g*: *stable-defl g*
shows *stable-defl* $(sprod\text{-}map\cdot f\cdot g)$

proof (*rule stable-deflI*)
show *deflation* (*sprod-map·f·g*)
by (*rule deflation-sprod-map*[*OF stable-defl.axioms*(*1*)[*OF f*] *stable-defl.axioms*(*1*)[*OF g*]])
fix *x y* **assume** *fy*: *sprod-map·f·g·y* = *y* **and** *xy*: *x* ⊑ *y*
show *sprod-map·f·g·x* = *x*
proof (*cases x*)
case *bottom* **then show** *?thesis* **by** *simp*
next
case (*spair a b*)
with *xy* **obtain** *c e* **where** *y*: *y* = (:*c*, *e*:) *c* ≠ ⊥ *e* ≠ ⊥ *a* ⊑ *c b* ⊑ *e*
by (*cases y*) (*auto simp*: *spair-below-iff*)
with *fy* **have** *f·c* = *c g·e* = *e* **by** (*auto simp*: *spair-eq-iff*)
with *stable-defl.downclosed*[*OF f*] *stable-defl.downclosed*[*OF g*] *y spair*
have *f·a* = *a g·b* = *b* **by** *blast*+
with *spair* **show** *?thesis* **by** *simp*
qed
qed

lemma *stable-defl-iso*:
fixes *abs* :: *'r* → *'t* **and** *rep*
assumes *iso-abs*: ⋀*x*. *rep·*(*abs·x*) = *x* **and** *iso-rep*: ⋀*y*. *abs·*(*rep·y*) = *y*
and *f*: *stable-defl f*
shows *stable-defl* (*abs oo f oo rep*)
proof (*rule stable-deflI*)
interpret *iso abs rep* **by** (*rule iso.intro*[*OF iso-abs iso-rep*])
show *deflation* (*abs oo f oo rep*)
by (*rule deflation-abs-rep*[*OF abs-iso rep-iso stable-defl.axioms*(*1*)[*OF f*]])
fix *x y* **assume** *fy*: (*abs oo f oo rep*)·*y* = *y* **and** *xy*: *x* ⊑ *y*
from *fy* **have** *rep·*(*abs·*(*f·*(*rep·y*))) = *rep·y* **by** *simp*
then have *f·*(*rep·y*) = *rep·y* **by** *simp*
moreover from *xy* **have** *rep·x* ⊑ *rep·y* **by** (*rule monofun-cfun-arg*)
ultimately have *f·*(*rep·x*) = *rep·x* **by** (*rule stable-defl.downclosed*[*OF f*])
then show (*abs oo f oo rep*)·*x* = *x* **by** *simp*
qed

6.2 Semi-factor deflations

The deflation form of a semi-factor: a stable deflation satisfying the slide rule. Closure of the image under existing lubs need not be assumed: the image of any deflation has it (*deflation-lub-closed*). The deflation is then the projection onto its image, and conversely the projection of a semi-factor is one.

locale *semi-factor-defl* = *stable-defl d* **for** *d* :: *'a*::{*bifinite*, *distributive-cpo*} → *'a* +
assumes *slide*: *d·y* = *y* ⟹ *d·x* ⊑ *y* ⟹ *x* ⇑ *y*

lemma *semi-factor-deflI*:

assumes *stable-defl d* **and** $\bigwedge x\ y.\ d{\cdot}y = y \Longrightarrow d{\cdot}x \sqsubseteq y \Longrightarrow x \Uparrow y$
shows *semi-factor-defl d*
using *assms* **by** (*simp add*: *semi-factor-defl-def semi-factor-defl-axioms-def*)

lemma *semi-factor-defl-iff*:
semi-factor-defl d $\longleftrightarrow$ *deflation d* $\wedge$ *semi-factor-subdomain* $\{x.\ d{\cdot}x = x\}$
proof
assume *sd*: *semi-factor-defl d*
then have *st*: *stable-defl d* **by** (*rule semi-factor-defl.axioms(1)*)
have *d*: *deflation d* **by** (*rule stable-defl.deflation*[*OF st*])
have *ss*: *stable-subdomain* $\{x.\ d{\cdot}x = x\}$ **by** (*rule stable-defl-stable-subdomain*[*OF st*])
have *p*: *stable-subdomain.p* $\{x.\ d{\cdot}x = x\} = d$
by (*rule stable-subdomain-deflation*[*OF d ss*, *symmetric*])
have *semi-factor-subdomain* $\{x.\ d{\cdot}x = x\}$
by (*rule semi-factor-subdomain.intro*[*OF ss*], *unfold-locales*, *unfold p*)
(*auto intro*: *semi-factor-defl.slide*[*OF sd*])
with *d* **show** *deflation d* $\wedge$ *semi-factor-subdomain* $\{x.\ d{\cdot}x = x\}$ **by** *blast*
next
assume *deflation d* $\wedge$ *semi-factor-subdomain* $\{x.\ d{\cdot}x = x\}$
then have *d*: *deflation d* **and** *S*: *semi-factor-subdomain* $\{x.\ d{\cdot}x = x\}$ **by** *blast+*
have *ss*: *stable-subdomain* $\{x.\ d{\cdot}x = x\}$ **by** (*rule semi-factor-subdomain.axioms(1)*[*OF S*])
have *p*: *stable-subdomain.p* $\{x.\ d{\cdot}x = x\} = d$
by (*rule stable-subdomain-deflation*[*OF d ss*, *symmetric*])
show *semi-factor-defl d*
proof (*rule semi-factor-deflI*)
show *stable-defl d*
by (*rule stable-deflI*[*OF d*]) (*use stable-subdomain.downward*[*OF ss*] **in** *blast*)
fix *x y* **assume** $d{\cdot}y = y$ $d{\cdot}x \sqsubseteq y$
then show $x \Uparrow y$ **using** *semi-factor-subdomain.slide-rule*[*OF S*] *p* **by** *auto*
qed
qed

lemma *semi-factor-defl-bottom*: *semi-factor-defl* ($\bot$ :: *'a*::{*bifinite*, *distributive-cpo*} $\rightarrow$ *'a*)
proof (*rule semi-factor-deflI*[*OF stable-defl-bottom*])
fix *x y* :: *'a* **assume** $\bot{\cdot}y = y$
then have $y = \bot$ **by** *simp*
then show $x \Uparrow y$ **by** (*simp add*: *joinable-sym bottom-joinable*)
qed

A semi-factor is the image of its projection.

lemma *semi-factor-defl-p*:
assumes *semi-factor-subdomain S*
shows *semi-factor-defl* (*stable-subdomain.p S*)
proof −
note *ss* = *semi-factor-subdomain.axioms(1)*[*OF assms*]

have $\{x.\ \mathit{stable\text{-}subdomain.p}\ S{\cdot}x = x\} = S$ **by** (*rule stable-subdomain.p-fixes*[*OF ss*])
with *assms stable-subdomain.deflation-p*[*OF ss*] **show** *?thesis*
by (*simp add*: *semi-factor-defl-iff*)
qed

A pcpo carved from a bounded-complete type by a typedef whose set is closed under lubs of bounded subsets is again bounded complete. The order on the subtype must be the one induced by *Rep*, as supplied by **pcpodef**.

lemma *bounded-complete-subtypeI*:
fixes *Rep* :: *'b*::*pcpo* ⇒ *'a*::*bounded-complete*
assumes *type*: *type-definition Rep Abs Q*
and *below*: $\bigwedge x\ y.\ (x \sqsubseteq y) = (\mathit{Rep}\ x \sqsubseteq \mathit{Rep}\ y)$
and *Q-lub-closed*: $\bigwedge A\ a.\ A \subseteq Q \Longrightarrow A <<| a \Longrightarrow a \in Q$
shows *OFCLASS*(*'b*, *bounded-complete-class*)
proof
interpret *type-definition Rep Abs Q* **by** (*rule type*)
fix *A* :: *'b set* **and** *b* :: *'b*
assume $A \neq \{\}$ **and** *Ab*: $A <| b$
have *RepAb*: *Rep* ‘ $A <|$ *Rep b*
proof (*rule is-ubI*)
fix *y* **assume** $y \in$ *Rep* ‘ *A*
then obtain *x* **where** *x*: $x \in A\ y = \mathit{Rep}\ x$ **by** *auto*
from *is-ubD*[*OF Ab x*(*1*)] **have** *Rep x* ⊑ *Rep b* **using** *below* **by** *simp*
with *x*(*2*) **show** *y* ⊑ *Rep b* **by** *simp*
qed
then obtain *l* **where** *l*: *Rep* ‘ $A <<|$ *l* **using** *ex-bound-ex-lub* **by** *blast*
have *Rep* ‘ $A \subseteq Q$ **using** *Rep* **by** *auto*
from *Q-lub-closed*[*OF this l*] **have** *lQ*: $l \in Q$ **.**
have $A <<|$ *Abs l*
proof (*rule is-lubI*)
show $A <|$ *Abs l*
proof (*rule is-ubI*)
fix *x* **assume** $x \in A$
from *is-ubD*[*OF is-lubD1*[*OF l*]] *this* **have** *Rep x* ⊑ *l* **by** *auto*
then show *x* ⊑ *Abs l* **using** *below Abs-inverse*[*OF lQ*] **by** *simp*
qed
next
fix *u* **assume** $A <| u$
have *Rep* ‘ $A <|$ *Rep u*
proof (*rule is-ubI*)
fix *y* **assume** $y \in$ *Rep* ‘ *A*
then obtain *x* **where** *x*: $x \in A\ y = \mathit{Rep}\ x$ **by** *auto*
from *is-ubD*[*OF* ‹$A <| u$› *x*(*1*)] **have** *Rep x* ⊑ *Rep u* **using** *below* **by** *simp*
with *x*(*2*) **show** *y* ⊑ *Rep u* **by** *simp*
qed
with *l* **have** *l* ⊑ *Rep u* **by** (*rule is-lubD2*)
then show *Abs l* ⊑ *u* **using** *below Abs-inverse*[*OF lQ*] **by** *simp*
qed

then show $\exists z.\ A <<| z$ **..**
qed

Continuity of binary meet (*cont-meet-arg*) is in *Distributive*.

7 Stable subdomains as a lattice of sets

Stable subdomains form a closure system: they contain *UNIV* and are closed under arbitrary intersection. Hence they form a complete lattice under inclusion, with meet given by intersection.

lemma *stable-subdomain-UNIV*: *stable-subdomain UNIV*
by *unfold-locales auto*

lemma *p-UNIV*: *stable-subdomain.p UNIV*$\cdot x = x$
by (*simp add*: *stable-subdomain.p-fixed*[*OF stable-subdomain-UNIV*])

lemma *stable-subdomain-bot*: *stable-subdomain* $\{\bot::'a\}$
proof
fix $t :: 'a$ **and** $T :: 'a\ set$ **assume** $T \subseteq \{\bot\}$
then have $T = \{\} \vee T = \{\bot\}$ **by** *fast*
then have $T <<| \bot$ **by** *auto*
moreover assume $T <<| t$
ultimately have $t = \bot$ **by** (*rule is-lub-unique*[*symmetric*])
then show $t \in \{\bot\}$ **..**
qed *auto*

lemma *p-bot*: *stable-subdomain.p* $\{\bot\}\cdot x = \bot$
using *stable-subdomain.p-in-S*[*OF stable-subdomain-bot*] **by** *fast*

lemma *stable-subdomain-Inter*:
assumes $\bigwedge S.\ S \in \mathcal{S} \Longrightarrow$ *stable-subdomain S*
shows *stable-subdomain* $(\bigcap\ \mathcal{S})$
proof (*unfold-locales*)
fix $x\ y$ **assume** $y \in \bigcap\ \mathcal{S}$ **and** $x \sqsubseteq y$
with *assms* **show** $x \in \bigcap\ \mathcal{S}$
by (*auto intro*: *stable-subdomain.downward*)
next
fix $T\ t$ **assume** T: $T \subseteq \bigcap\ \mathcal{S}$ **and** *lub*: $T <<| t$
show $t \in \bigcap\ \mathcal{S}$
proof (*rule InterI*)
fix X **assume** $X \in \mathcal{S}$
with *assms* **have** *stable-subdomain X* **by** *simp*
moreover from T ‹$X \in \mathcal{S}$› **have** $T \subseteq X$ **by** *auto*
ultimately show $t \in X$ **using** *lub* **by** (*rule stable-subdomain.lub-closed*)
qed
qed

Jung, Lemma 4.1.5(ii) (one direction): a finer stable subdomain has a smaller

projection.

lemma *proj-mono-subdomain*:
assumes A: *stable-subdomain* A **and** S: *stable-subdomain* S **and** *sub*: $A \subseteq S$
shows *stable-subdomain.proj* $A\ x \sqsubseteq$ *stable-subdomain.proj* $S\ x$
proof −
interpret A: *stable-subdomain* A **by** (*rule* A)
interpret S: *stable-subdomain* S **by** (*rule* S)
have $\{a \in A.\ a \sqsubseteq x\} \subseteq \{s \in S.\ s \sqsubseteq x\}$ **using** *sub* **by** *blast*
with *A.proj-is-lub S.proj-is-lub* **show** *A.proj* $x \sqsubseteq$ *S.proj* x
by (*rule lub-subset-mono*)
qed

The projection-cfun version of ⟦*stable-subdomain ?A*; *stable-subdomain ?S*; *?A* ⊆ *?S*⟧ ⟹ *stable-subdomain.proj ?A ?x* ⊑ *stable-subdomain.proj ?S ?x*.

lemma *p-mono-subdomain*:
assumes A: *stable-subdomain* A **and** S: *stable-subdomain* S **and** *sub*: $A \subseteq S$
shows *stable-subdomain.p* $A \sqsubseteq$ *stable-subdomain.p* S
proof (*rule cfun-belowI*)
fix x
interpret A: *stable-subdomain* A **by** (*rule* A)
interpret S: *stable-subdomain* S **by** (*rule* S)
show *stable-subdomain.p* $A{\cdot}x \sqsubseteq$ *stable-subdomain.p* $S{\cdot}x$
using *proj-mono-subdomain*[*OF A S sub*] **by** (*simp add*: *A.p-beta S.p-beta*)
qed

lemma *stable-defl-subdomain-p*:
assumes *stable-subdomain* S **shows** *stable-defl* (*stable-subdomain.p* S)
proof −
interpret *stable-subdomain* S **by** (*rule assms*)
show *?thesis* **by** (*rule stable-defl-axioms*)
qed

7.1 Meets of stable deflations

The composition of stable deflations has the intersection of the images as its image, and is again stable. It is the pointwise meet, and the meet in the function space.

lemma *stable-defl-oo-fixed*:
assumes p: *stable-defl* p **and** q: *stable-defl* q
shows $(p\ oo\ q){\cdot}y = y \longleftrightarrow p{\cdot}y = y \wedge q{\cdot}y = y$
proof
interpret p: *stable-defl* p **by** (*rule* p)
interpret q: *stable-defl* q **by** (*rule* q)
assume y: $(p\ oo\ q){\cdot}y = y$
have $y \sqsubseteq q{\cdot}y$ **using** *p.below*[*of* $q{\cdot}y$] y **by** *simp*
then have *qy*: $q{\cdot}y = y$ **using** *q.below* **by** (*rule below-antisym*[*rotated*])
with y **show** $p{\cdot}y = y \wedge q{\cdot}y = y$ **by** *simp*
qed *simp*

lemma *stable-defl-oo*:
assumes *p*: *stable-defl p* **and** *q*: *stable-defl q*
shows *stable-defl* (*p oo q*)
proof −
interpret *p*: *stable-defl p* **by** (*rule p*)
interpret *q*: *stable-defl q* **by** (*rule q*)
show *?thesis*
proof *standard*
fix *x*
have $q{\cdot}(p{\cdot}(q{\cdot}x)) = p{\cdot}(q{\cdot}x)$ **using** *p.below* **by** (*rule q.lower-fixed*)
then show $(p \mathbin{oo} q){\cdot}((p \mathbin{oo} q){\cdot}x) = (p \mathbin{oo} q){\cdot}x$ **by** (*simp add*: *p.idem*)
next
fix *x*
show $(p \mathbin{oo} q){\cdot}x \sqsubseteq x$ **by** (*simp add*: *below-trans*[*OF p.below q.below*])
next
fix *x y* **assume** $(p \mathbin{oo} q){\cdot}y = y$ **and** *xy*: $x \sqsubseteq y$
then have $p{\cdot}y = y$ $q{\cdot}y = y$ **unfolding** *stable-defl-oo-fixed*[*OF p q*] **by** *simp-all*
with *xy* **have** $p{\cdot}x = x$ $q{\cdot}x = x$ **by** (*simp-all add*: *p.downclosed q.downclosed*)
then show $(p \mathbin{oo} q){\cdot}x = x$ **by** *simp*
qed
qed

lemma *stable-defl-oo-glb*:
assumes *p*: *stable-defl p* **and** *q*: *stable-defl q*
shows *is-glb* $\{p{\cdot}x,\ q{\cdot}x\}$ $(p{\cdot}(q{\cdot}x))$
proof (*rule is-glbI*)
interpret *p*: *stable-defl p* **by** (*rule p*)
interpret *q*: *stable-defl q* **by** (*rule q*)
fix *z* **assume** $z \in \{p{\cdot}x,\ q{\cdot}x\}$
moreover have $p{\cdot}(q{\cdot}x) \sqsubseteq p{\cdot}x$ **using** *q.below* **by** (*rule monofun-cfun-arg*)
ultimately show $p{\cdot}(q{\cdot}x) \sqsubseteq z$ **using** *p.below* **by** *auto*
next
interpret *p*: *stable-defl p* **by** (*rule p*)
interpret *q*: *stable-defl q* **by** (*rule q*)
fix *l* **assume** $\forall z{\in}\{p{\cdot}x,\ q{\cdot}x\}.\ l \sqsubseteq z$
then have *lp*: $l \sqsubseteq p{\cdot}x$ **and** *lq*: $l \sqsubseteq q{\cdot}x$ **by** *simp-all*
have $p{\cdot}l = l$ **using** *lp* **by** (*rule p.lower-fixed*)
moreover from *lq* **have** $p{\cdot}l \sqsubseteq p{\cdot}(q{\cdot}x)$ **by** (*rule monofun-cfun-arg*)
ultimately show $l \sqsubseteq p{\cdot}(q{\cdot}x)$ **by** *simp*
qed

lemma *stable-defl-meet-oo*:
assumes *p*: *stable-defl p* **and** *q*: *stable-defl q*
shows $p \sqcap q = p \mathbin{oo} q$
proof −
note *glb* = *stable-defl-oo-glb*[*OF p q*]
have *is-glb* $\{p,\ q\}$ $(p \mathbin{oo} q)$
proof (*rule is-glbI*)

```
    fix f assume f ∈ {p, q}
    then show p oo q ⊑ f
      by (auto intro!: cfun-belowI intro: is-glb-below[OF glb])
  next
    fix f assume ∀ g∈{p, q}. f ⊑ g
    then have f·x ⊑ p·x f·x ⊑ q·x for x by (auto intro: monofun-cfun-fun)
    then show f ⊑ p oo q
      by (auto intro!: cfun-belowI is-glb-greatest[OF glb])
  qed
  then show ?thesis unfolding meet-def by (rule glb-eqI)
qed
```

7.2 Arbitrary meets and directed joins

The glb in the function space of a nonempty family of stable deflations is again a stable deflation, with the intersection of the images as its image. Each value $g{\cdot}x$ lies below the image elements $d{\cdot}x$, hence in every image; conversely a compact k in every image yields the step function *stepfn k k*, a lower bound of the family, so $k \sqsubseteq g{\cdot}k$. Algebraicity extends this to all common fixed points.

```
lemma stable-defl-glb-fixes:
  assumes ne: 𝒟 ≠ {} and st: ⋀d. d ∈ 𝒟 ⟹ stable-defl d
  shows glb 𝒟·y = y ⟷ (∀ d∈𝒟. d·y = y)
proof −
  obtain g0 where is-glb 𝒟 g0 using ne by (rule bounded-complete-class.is-glb)
  then have g: is-glb 𝒟 (glb 𝒟) by (rule is-glb-glb)
  from ne obtain d0 where d0: d0 ∈ 𝒟 by blast
 have glb 𝒟·x ⊑ d0·x for x using is-glb-below[OF g d0] by (rule monofun-cfun-fun)
  then have below: glb 𝒟·x ⊑ x for x
     using stable-defl.deflation[OF st[OF d0]] by (blast intro: below-trans defla-
tion.below)
  show ?thesis
  proof
    assume fy: glb 𝒟·y = y
    show ∀ d∈𝒟. d·y = y
    proof
      fix d assume d: d ∈ 𝒟
      have glb 𝒟·y ⊑ d·y using is-glb-below[OF g d] by (rule monofun-cfun-fun)
      with fy show d·y = y by (simp add: stable-defl.lower-fixed[OF st[OF d]])
    qed
  next
    assume fix-all: ∀ d∈𝒟. d·y = y
    obtain C where C: chain C ∀ i. compact (C i) y = (⨆ i. C i)
      by (rule obtain-compact-chain)
    have C i ⊑ glb 𝒟·y for i
    proof −
      have Ci: C i ⊑ y using is-ub-thelub[OF C(1)] C(3) by simp
      have stepfn (C i) (C i) ⊑ d if d: d ∈ 𝒟 for d
```

```
    proof −
      have d·(C i) = C i
        using fix-all d Ci by (blast intro: stable-defl.downclosed[OF st[OF d]])
      then show ?thesis by (simp add: stepfn-below-iff[OF C(2)[rule-format]])
    qed
    then have stepfn (C i) (C i) ⊑ glb 𝒟 by (blast intro: is-glb-greatest[OF g])
    then have stepfn (C i) (C i)·y ⊑ glb 𝒟·y by (rule monofun-cfun-fun)
    with Ci show ?thesis by (simp add: stepfn-beta[OF C(2)[rule-format]])
  qed
  then have y ⊑ glb 𝒟·y unfolding C(3) by (rule lub-below[OF C(1)])
  then show glb 𝒟·y = y by (rule below-antisym[OF below])
 qed
qed

lemma stable-defl-glb:
 assumes ne: 𝒟 ≠ {} and st: ⋀d. d ∈ 𝒟 ⟹ stable-defl d
 shows stable-defl (glb 𝒟)
proof −
 note fx = stable-defl-glb-fixes[OF ne st]
 obtain g0 where is-glb 𝒟 g0 using ne by (rule bounded-complete-class.is-glb)
 then have g: is-glb 𝒟 (glb 𝒟) by (rule is-glb-glb)
 have fixed: d·(glb 𝒟·x) = glb 𝒟·x if d: d ∈ 𝒟 for d x
 proof −
   have glb 𝒟·x ⊑ d·x using is-glb-below[OF g d] by (rule monofun-cfun-fun)
   then show ?thesis by (rule stable-defl.lower-fixed[OF st[OF d]])
 qed
 from ne obtain d0 where d0: d0 ∈ 𝒟 by blast
 have below: glb 𝒟·x ⊑ x for x
    using monofun-cfun-fun[OF is-glb-below[OF g d0]] stable-defl.deflation[OF
st[OF d0]]
   by (blast intro: below-trans deflation.below)
 show ?thesis
 proof standard
   fix x show glb 𝒟·(glb 𝒟·x) = glb 𝒟·x using fixed by (simp add: fx)
 next
   fix x show glb 𝒟·x ⊑ x by (rule below)
 next
   fix x y assume glb 𝒟·y = y and xy: x ⊑ y
   then have ∀ d∈𝒟. d·y = y by (simp add: fx)
   with xy have ∀ d∈𝒟. d·x = x by (blast intro: stable-defl.downclosed[OF st])
   then show glb 𝒟·x = x by (simp add: fx)
 qed
qed
```

The lub of a directed family of stable deflations is a stable deflation: stable deflations are admissible (*adm-stable-defl*), and admissible predicates are closed under directed lubs.

```
lemma stable-defl-directed-lub:
 assumes dir: directed 𝒟 and ne: 𝒟 ≠ {} and st: ⋀d. d ∈ 𝒟 ⟹ stable-defl d
```

shows *stable-defl* (*lub* $\mathcal{D}$)
by (*rule directed-admD*[*OF adm-stable-defl dir ne*]) (*simp add*: *st*)

8 The domain of stable subdomains

Stable subdomains are represented by their projections, the stable deflations. These are closed under $\bot$ and admissible, so they form a pcpo under the pointwise order; the lattice order is the same order. Binary meets are compositions and arbitrary meets are glbs in the function space; binary and arbitrary joins are the meets of the upper bounds.

pcpodef *'a stable-subdom* = {*d* :: *'a* → *'a*. *stable-defl d*}
morphisms *stable-subdom-p subdom-of-defl*
by (*simp-all add*: *stable-defl-bottom adm-stable-defl*)

notation *stable-subdom-p* (‹p₁›)

setup-lifting *type-definition-stable-subdom*

lemma *stable-defl-stable-subdom-p* [*simp*]: *stable-defl* p_A
using *stable-subdom-p* **by** *simp*

lemma *stable-subdom-below-iff*: $A \sqsubseteq B \longleftrightarrow \mathrm{p}_A \sqsubseteq \mathrm{p}_B$
by (*simp add*: *below-stable-subdom-def*)

lemma *stable-subdom-proj-eqI*: $\mathrm{p}_A = \mathrm{p}_B \Longrightarrow A = B$
by (*simp add*: *stable-subdom-p-inject*)

lemma *stable-subdom-p-subdom-of-defl*: *stable-defl d* $\Longrightarrow \mathrm{p}_{\textit{subdom-of-defl } d} = d$
by (*simp add*: *subdom-of-defl-inverse*)

instantiation *stable-subdom* :: ({*bifinite*, *bounded-complete*}) *complete-lattice*
begin

lift-definition *less-eq-stable-subdom* :: *'a stable-subdom* ⇒ *'a stable-subdom* ⇒ *bool*
is ($\sqsubseteq$) **.**

lift-definition *less-stable-subdom* :: *'a stable-subdom* ⇒ *'a stable-subdom* ⇒ *bool*
is $\lambda p\ q.\ p \sqsubseteq q \wedge \neg\ q \sqsubseteq p$ **.**

lift-definition *inf-stable-subdom* :: *'a stable-subdom* ⇒ *'a stable-subdom* ⇒ *'a stable-subdom*
is $\lambda p\ q.\ p \sqcap q$ **by** (*simp add*: *stable-defl-meet-oo stable-defl-oo*)

lift-definition *top-stable-subdom* :: *'a stable-subdom*
is *ID* **by** (*rule stable-defl-ID*)

lift-definition *bot-stable-subdom* :: *'a stable-subdom*

is $\bot$ **by** (*rule stable-defl-bottom*)

lift-definition *Inf-stable-subdom* :: *'a stable-subdom set* ⇒ *'a stable-subdom*
is $\lambda\mathcal{D}$. *if* $\mathcal{D} = \{\}$ *then ID else glb* $\mathcal{D}$
by (*auto intro*: *stable-defl-ID stable-defl-glb*)

definition *sup-stable-subdom* :: *'a stable-subdom* ⇒ *'a stable-subdom* ⇒ *'a stable-subdom*
where *sup-stable-subdom A B* = *Inf* $\{C.\ A \leq C \wedge B \leq C\}$

definition *Sup-stable-subdom* :: *'a stable-subdom set* ⇒ *'a stable-subdom*
where *Sup-stable-subdom* $\mathcal{A}$ = *Inf* $\{C.\ \forall A{\in}\mathcal{A}.\ A \leq C\}$

lemma *Inf-lower-stable-subdom*: $A \in \mathcal{D} \Longrightarrow$ *Inf* $\mathcal{D} \leq A$ **for** A :: *'a stable-subdom*
proof *transfer*
fix $\mathcal{D}$ :: (*'a* → *'a*) *set* **and** *a*
assume $a \in \mathcal{D}$
then have $\mathcal{D} \neq \{\}$ **by** *blast*
then obtain *g* **where** *is-glb* $\mathcal{D}$ *g* **by** (*rule bounded-complete-class.is-glb*)
with ‹$a \in \mathcal{D}$› **show** (*if* $\mathcal{D} = \{\}$ *then ID else glb* $\mathcal{D}$) $\sqsubseteq a$
by (*auto intro*: *is-glb-below*[*OF is-glb-glb*])
qed

lemma *Inf-greatest-stable-subdom*: $(\bigwedge C.\ C \in \mathcal{D} \Longrightarrow A \leq C) \Longrightarrow A \leq$ *Inf* $\mathcal{D}$ **for** A :: *'a stable-subdom*
proof *transfer*
fix $\mathcal{D}$ :: (*'a* → *'a*) *set* **and** *a*
assume *D*: *Ball* $\mathcal{D}$ *stable-defl* **and** *st*: *stable-defl a*
and *lb*: $\bigwedge c.$ *stable-defl c* $\Longrightarrow c \in \mathcal{D} \Longrightarrow a \sqsubseteq c$
show $a \sqsubseteq$ (*if* $\mathcal{D} = \{\}$ *then ID else glb* $\mathcal{D}$)
proof (*cases* $\mathcal{D} = \{\}$)
case *True*
then show *?thesis* **using** *deflation.below-ID*[*OF stable-defl.deflation*[*OF st*]]
by *simp*
next
case *False*
then obtain *g* **where** *is-glb* $\mathcal{D}$ *g* **by** (*rule bounded-complete-class.is-glb*)
with *False D lb* **show** *?thesis* **by** (*auto intro*: *is-glb-greatest*[*OF is-glb-glb*])
qed
qed

lemma *bot-le-stable-subdom*: *bot* ≤ *A* **for** *A* :: *'a stable-subdom*
by *transfer simp*

lemma *le-antisym-stable-subdom*: $A \leq B \Longrightarrow B \leq A \Longrightarrow A = B$ **for** A :: *'a stable-subdom*
by *transfer* (*rule below-antisym*)

instance

proof
fix A B C :: *'a stable-subdom* **and** $\mathcal{D}$ :: *'a stable-subdom set*
show $(A < B) = (A \leq B \wedge \neg\ B \leq A)$ **by** *transfer simp*
show $A \leq A$ **by** *transfer simp*
show $A \leq B \Longrightarrow B \leq C \Longrightarrow A \leq C$ **by** *transfer* (*rule below-trans*)
show $A \leq B \Longrightarrow B \leq A \Longrightarrow A = B$ **by** *transfer* (*rule below-antisym*)
show *inf* A $B \leq A$ **by** *transfer* (*rule meet-below1*)
show *inf* A $B \leq B$ **by** *transfer* (*rule meet-below2*)
show $A \leq B \Longrightarrow A \leq C \Longrightarrow A \leq$ *inf* B C **by** *transfer* (*rule meet-greatest*)
show $A \leq$ *sup* A B $B \leq$ *sup* A B
unfolding *sup-stable-subdom-def* **by** (*auto intro*: *Inf-greatest-stable-subdom*)
show $A \leq C \Longrightarrow B \leq C \Longrightarrow$ *sup* A $B \leq C$
unfolding *sup-stable-subdom-def* **by** (*auto intro*: *Inf-lower-stable-subdom*)
show $A \in \mathcal{D} \Longrightarrow$ *Inf* $\mathcal{D} \leq A$ **by** (*rule Inf-lower-stable-subdom*)
show $(\bigwedge B.\ B \in \mathcal{D} \Longrightarrow A \leq B) \Longrightarrow A \leq$ *Inf* $\mathcal{D}$ **by** (*rule Inf-greatest-stable-subdom*)
show $A \in \mathcal{D} \Longrightarrow A \leq$ *Sup* $\mathcal{D}$
unfolding *Sup-stable-subdom-def* **by** (*auto intro*: *Inf-greatest-stable-subdom*)
show $(\bigwedge B.\ B \in \mathcal{D} \Longrightarrow B \leq A) \Longrightarrow$ *Sup* $\mathcal{D} \leq A$
unfolding *Sup-stable-subdom-def* **by** (*auto intro*: *Inf-lower-stable-subdom*)
show *Inf* $\{\}$ = (*top* :: *'a stable-subdom*) **by** *transfer simp*
show *Sup* $\{\}$ = (*bot* :: *'a stable-subdom*)
unfolding *Sup-stable-subdom-def*
by (*rule le-antisym-stable-subdom*[*OF Inf-lower-stable-subdom bot-le-stable-subdom*])
simp
qed

end

The domain order is the lattice order; hence the domain is bounded complete, indeed every set has a lub.

lemma *below-stable-subdom-iff-le*: $A \sqsubseteq B \longleftrightarrow A \leq B$ **for** A B :: *'a stable-subdom*
by (*simp add*: *stable-subdom-below-iff less-eq-stable-subdom.rep-eq*)

lemma *stable-subdom-is-lub*: $\mathcal{A} <<|$ *Sup* $\mathcal{A}$ **for** $\mathcal{A}$ :: *'a stable-subdom set*
by (*auto intro*!: *is-lubI is-ubI Sup-upper Sup-least*
simp: *below-stable-subdom-iff-le is-ub-def*)

instance *stable-subdom* :: ({*bifinite*, *bounded-complete*}) *bounded-complete*
by *standard* (*blast intro*: *stable-subdom-is-lub*)

instance *stable-subdom* :: ({*bifinite*, *bounded-complete*}) *lattice-complete*
by *intro-classes* (*rule exI*[*of - top*], *simp add*: *below-stable-subdom-iff-le*)

lemma *p-below-iff-le*: $\mathrm{p}_A \sqsubseteq \mathrm{p}_B \longleftrightarrow A \leq B$
by (*simp add*: *less-eq-stable-subdom.rep-eq*)

lemma *le-of-p-below*: $\mathrm{p}_A \sqsubseteq \mathrm{p}_B \Longrightarrow A \leq B$ **and** *p-below-of-le*: $A \leq B \Longrightarrow \mathrm{p}_A \sqsubseteq \mathrm{p}_B$
by (*simp-all add*: *p-below-iff-le*)

lemma *p-inf-meet-rep*: $\mathrm{p}_{inf\ A\ B} = \mathrm{p}_A \sqcap \mathrm{p}_B$
by (*rule inf-stable-subdom.rep-eq*)

The meet is composition (*stable-defl-meet-oo*).

lemma *p-inf*: $\mathrm{p}_{inf\ A\ B} = \mathrm{p}_A\ oo\ \mathrm{p}_B$
by (*simp add*: *p-inf-meet-rep stable-defl-meet-oo*)

lemma *p-bot-stable-subdom*: $\mathrm{p}_{bot} = \bot$ **and** *p-top-stable-subdom*: $\mathrm{p}_{top} = ID$
by (*simp-all add*: *bot-stable-subdom.rep-eq top-stable-subdom.rep-eq*)

8.1 Images as sets

The image of the projection is the stable subdomain; conversely a stable subdomain gives the element represented by its projection.

definition *set-of-subdom* :: *'a stable-subdom* ⇒ *'a set*
where *set-of-subdom A* = {*x*. $\mathrm{p}_A{\cdot}x = x$}

definition *subdom-of-set* :: *'a set* ⇒ *'a stable-subdom*
where *subdom-of-set S* = *subdom-of-defl* (*stable-subdomain.p S*)

lemma *p-fixes*: $\mathrm{p}_A{\cdot}x = x \longleftrightarrow x \in$ *set-of-subdom A*
by (*simp add*: *set-of-subdom-def*)

lemma *stable-subdomain-of-set-of-subdom*: *stable-subdomain* (*set-of-subdom A*)
unfolding *set-of-subdom-def* **by** (*rule stable-defl-stable-subdomain*) *simp*

interpretation *subdom*: *stable-subdomain set-of-subdom A*
rewrites *stable-subdomain.p* (*set-of-subdom A*) ≡ p_A
proof −
show *stable-subdomain* (*set-of-subdom A*) **by** (*rule stable-subdomain-of-set-of-subdom*)
show *stable-subdomain.p* (*set-of-subdom A*) ≡ p_A
unfolding *set-of-subdom-def*
by (*rule eq-reflection*, *rule stable-subdomain-deflation*[*symmetric*])
(*simp-all add*: *stable-defl.deflation stable-defl-stable-subdomain*)
qed

lemma *stable-subdom-p-subdom-of-set*:
stable-subdomain S ⟹ $\mathrm{p}_{subdom\text{-}of\text{-}set\ S}$ = *stable-subdomain.p S*
unfolding *subdom-of-set-def* **by** (*rule stable-subdom-p-subdom-of-defl*, *rule stable-defl-subdomain-p*)

lemma *set-of-subdom-of-set*: *stable-subdomain S* ⟹ *set-of-subdom* (*subdom-of-set S*) = *S*
by (*simp add*: *set-of-subdom-def stable-subdom-p-subdom-of-set stable-subdomain.p-fixes*)

lemma *set-of-subdom-inf*: *set-of-subdom* (*inf A B*) = *set-of-subdom A* ∩ *set-of-subdom B*

unfolding *set-of-subdom-def p-inf*
stable-defl-oo-fixed[*OF stable-defl-stable-subdom-p stable-defl-stable-subdom-p*]
by *blast*

lemma *stable-subdomain-Int*:
stable-subdomain $S \Longrightarrow$ *stable-subdomain* $T \Longrightarrow$ *stable-subdomain* $(S \cap T)$
using *stable-subdomain-Inter*[*of* $\{S, T\}$] **by** *auto*

9 Distributive domains: joins of stable subdomains

default-sort {*bifinite*, *distributive-cpo*}

Jung, Lemma 4.1.6(ii) (binary form): a stable deflation preserves existing joins. This is where distributivity of the domain is used.

lemma *stable-defl-image-join*:
fixes $d :: 'a \to 'a$
assumes d: *stable-defl* d **and** j: $x \Uparrow y$
shows $d{\cdot}(x \sqcup y) = d{\cdot}x \sqcup d{\cdot}y$
proof −
interpret *stable-defl* d **by** (*rule* d)
have *px*: $d{\cdot}x = x \sqcap d{\cdot}(x \sqcup y)$
by (*rule stable-defl-image-meet*[*OF d idem monofun-cfun-arg*[*OF joinableD*(*1*)[*OF j*]]])
have *py*: $d{\cdot}y = y \sqcap d{\cdot}(x \sqcup y)$
by (*rule stable-defl-image-meet*[*OF d idem monofun-cfun-arg*[*OF joinableD*(*2*)[*OF j*]]])
have $d{\cdot}x \sqcup d{\cdot}y = (x \sqcap d{\cdot}(x \sqcup y)) \sqcup (y \sqcap d{\cdot}(x \sqcup y))$
using *px py* **by** *simp*
also have $\ldots = (x \sqcup y) \sqcap d{\cdot}(x \sqcup y)$
by (*rule meet-join-distrib2*[*OF j*, *symmetric*])
also have $\ldots = d{\cdot}(x \sqcup y)$
by (*metis meet-sym meet-absorbI below*)
finally show *?thesis* **by** (*rule sym*)
qed

9.1 Joins of stable deflations

Jung, Lemma 4.1.9 in deflation form. Deflations lie below *ID*, so they are joinable, and their join is pointwise. It is idempotent because each deflation preserves the joins of the other's images, and its image is downward closed because, below an image element $y = p{\cdot}y \sqcup q{\cdot}y$, distributivity splits $x = (x \sqcap p{\cdot}y) \sqcup (x \sqcap q{\cdot}y) = p{\cdot}x \sqcup q{\cdot}x$.

lemma *stable-defl-join-app*:
fixes $p :: 'a \to 'a$
assumes p: *stable-defl* p **and** q: *stable-defl* q
shows $(p \sqcup q){\cdot}x = p{\cdot}x \sqcup q{\cdot}x$
by (*rule cfun-join*[*OF deflation-joinable*[*OF stable-defl.deflation*[*OF p*]

stable-defl.deflation[*OF q*]]])

lemma *stable-defl-join*:
fixes $p :: 'a \to 'a$
assumes *p*: *stable-defl p* **and** *q*: *stable-defl q*
shows *stable-defl* $(p \sqcup q)$
proof −
interpret *p*: *stable-defl p* **by** (*rule p*)
interpret *q*: *stable-defl q* **by** (*rule q*)
have *pq*: $p{\cdot}x \Uparrow q{\cdot}x$ **for** *x* **by** (*rule joinableI*[*OF p.below q.below*])
note *app* = *stable-defl-join-app*[*OF p q*]
show *?thesis*
proof *standard*
fix *x*
have $p{\cdot}(p{\cdot}x \sqcup q{\cdot}x) = p{\cdot}x \sqcup p{\cdot}(q{\cdot}x)$
by (*simp add*: *stable-defl-image-join*[*OF p pq*] *p.idem*)
also have $\ldots = p{\cdot}x$ **by** (*rule join-absorb2*, *rule monofun-cfun-arg*, *rule q.below*)
finally have *pj*: $p{\cdot}(p{\cdot}x \sqcup q{\cdot}x) = p{\cdot}x$ **.**
have $q{\cdot}(p{\cdot}x \sqcup q{\cdot}x) = q{\cdot}(p{\cdot}x) \sqcup q{\cdot}x$
by (*simp add*: *stable-defl-image-join*[*OF q pq*] *q.idem*)
also have $\ldots = q{\cdot}x$ **by** (*rule join-absorb*, *rule monofun-cfun-arg*, *rule p.below*)
finally have *qj*: $q{\cdot}(p{\cdot}x \sqcup q{\cdot}x) = q{\cdot}x$ **.**
show $(p \sqcup q){\cdot}((p \sqcup q){\cdot}x) = (p \sqcup q){\cdot}x$ **by** (*simp add*: *app pj qj*)
next
fix *x*
show $(p \sqcup q){\cdot}x \sqsubseteq x$ **unfolding** *app* **by** (*rule join-belowI*[*OF p.below q.below*])
next
fix *x y* **assume** *y*: $(p \sqcup q){\cdot}y = y$ **and** *xy*: $x \sqsubseteq y$
have *px*: $p{\cdot}x = x \sqcap p{\cdot}y$
by (*rule stable-defl-image-meet*[*OF p p.idem monofun-cfun-arg*[*OF xy*]])
have *qx*: $q{\cdot}x = x \sqcap q{\cdot}y$
by (*rule stable-defl-image-meet*[*OF q q.idem monofun-cfun-arg*[*OF xy*]])
have $x = x \sqcap y$ **using** *xy* **by** (*simp add*: *meet-absorbI*)
also have $\ldots = x \sqcap (p{\cdot}y \sqcup q{\cdot}y)$ **using** *y* **by** (*simp add*: *app*)
also have $\ldots = (x \sqcap p{\cdot}y) \sqcup (x \sqcap q{\cdot}y)$ **by** (*rule meet-join-distrib*[*OF pq*])
also have $\ldots = (p \sqcup q){\cdot}x$ **by** (*simp add*: *app px qx*)
finally show $(p \sqcup q){\cdot}x = x$ **by** (*rule sym*)
qed
qed

The slide rule: below $y = p{\cdot}y \sqcup q{\cdot}y$, slide x first through p onto $p{\cdot}y$, then $p{\cdot}y \sqcup x$ through q onto $q{\cdot}y$.

lemma *semi-factor-defl-join*:
assumes *p*: *semi-factor-defl p* **and** *q*: *semi-factor-defl q*
shows *semi-factor-defl* $(p \sqcup q)$
proof −
interpret *p*: *semi-factor-defl p* **by** (*rule p*)
interpret *q*: *semi-factor-defl q* **by** (*rule q*)
note *app* = *stable-defl-join-app*[*OF p.stable-defl-axioms q.stable-defl-axioms*]

```
  show ?thesis
  proof (rule semi-factor-deflI)
    show stable-defl (p ⊔ q)
      by (rule stable-defl-join[OF p.stable-defl-axioms q.stable-defl-axioms])
    fix x y assume y: (p ⊔ q)·y = y and le: (p ⊔ q)·x ⊑ y
    have pq: p·z ⇑ q·z for z by (rule joinableI[OF p.below q.below])
    have yab: y = p·y ⊔ q·y using y by (simp add: app)
    from le have pxy: p·x ⊑ y and qxy: q·x ⊑ y
      unfolding app by (auto intro: below-trans joinableD[OF pq])
    have p·x ⊑ p·y using p.idem pxy by (rule p.below-image)
    with p.idem have x-pa: x ⇑ p·y by (rule p.slide)
    then have pa-x: p·y ⇑ x by (simp add: joinable-sym)
    have q·(p·y ⊔ x) = q·(p·y) ⊔ q·x
      by (rule stable-defl-image-join[OF q.stable-defl-axioms pa-x])
    also have ... ⊑ q·y
        by (rule join-belowI[OF monofun-cfun-arg[OF p.below] q.below-image[OF
q.idem qxy]])
    finally have q·(p·y ⊔ x) ⊑ q·y .
    with q.idem have pax-pb: p·y ⊔ x ⇑ q·y by (rule q.slide)
    let ?w = (p·y ⊔ x) ⊔ q·y
    note w = joinableD[OF pax-pb]
    have x ⊑ ?w using joinableD(2)[OF pa-x] w(1) by (rule below-trans)
    moreover have p·y ⊔ q·y ⊑ ?w
      by (rule join-belowI[OF below-trans[OF joinableD(1)[OF pa-x] w(1)] w(2)])
    then have y ⊑ ?w by (simp only: yab[symmetric])
    ultimately show x ⇑ y by (rule joinableI)
  qed
qed
```

The image of the join is the set of joins of image elements (Jung, Lemma 4.1.9): every image element is $p{\cdot}x \sqcup q{\cdot}x$, and a joinable $a \sqcup b$ of image elements lies below $p{\cdot}(a \sqcup b) \sqcup q{\cdot}(a \sqcup b)$.

```
lemma stable-defl-join-fixes:
  fixes p :: 'a → 'a
  assumes p: stable-defl p and q: stable-defl q
  shows {x. (p ⊔ q)·x = x} = {a ⊔ b |a b. p·a = a ∧ q·b = b ∧ a ⇑ b}
proof
  interpret p: stable-defl p by (rule p)
  interpret q: stable-defl q by (rule q)
  note app = stable-defl-join-app[OF p q]
  have pq: p·x ⇑ q·x for x by (rule joinableI[OF p.below q.below])
  show {x. (p ⊔ q)·x = x} ⊆ {a ⊔ b |a b. p·a = a ∧ q·b = b ∧ a ⇑ b}
  proof
    fix x assume x ∈ {x. (p ⊔ q)·x = x}
    then have x = p·x ⊔ q·x by (simp add: app)
    with pq[of x] p.idem q.idem show x ∈ {a ⊔ b |a b. p·a = a ∧ q·b = b ∧ a ⇑
b} by blast
  qed
  show {a ⊔ b |a b. p·a = a ∧ q·b = b ∧ a ⇑ b} ⊆ {x. (p ⊔ q)·x = x}
```

proof
fix x **assume** $x \in \{a \sqcup b \mid a\ b.\ p{\cdot}a = a \wedge q{\cdot}b = b \wedge a \Uparrow b\}$
then obtain $a\ b$ **where** a: $p{\cdot}a = a$ **and** b: $q{\cdot}b = b$ **and** ab: $a \Uparrow b$ **and** x: $x = a \sqcup b$
by *blast*
have $a \sqsubseteq p{\cdot}(a \sqcup b)$ **using** a *joinableD*(*1*)[*OF ab*] **by** (*rule p.below-image*)
moreover have $b \sqsubseteq q{\cdot}(a \sqcup b)$ **using** b *joinableD*(*2*)[*OF ab*] **by** (*rule q.below-image*)
ultimately have $a \sqcup b \sqsubseteq p{\cdot}(a \sqcup b) \sqcup q{\cdot}(a \sqcup b)$
by (*blast intro*: *join-belowI below-trans joinableD*[*OF pq*])
moreover have $p{\cdot}(a \sqcup b) \sqcup q{\cdot}(a \sqcup b) \sqsubseteq a \sqcup b$ **by** (*rule join-belowI*[*OF p.below q.below*])
ultimately show $x \in \{x.\ (p \sqcup q){\cdot}x = x\}$ **unfolding** x **by** (*simp add*: *app below-antisym*)
qed
qed

The set form, for stable subdomains given as sets.

lemma *semi-factor-subdomain-join*:
fixes $A\ B :: {'a}\ set$
assumes A: *semi-factor-subdomain* A **and** B: *semi-factor-subdomain* B
shows *semi-factor-subdomain* $\{a \sqcup b \mid a\ b.\ a \in A \wedge b \in B \wedge a \Uparrow b\}$
proof −
note sA = *semi-factor-subdomain.axioms*(*1*)[*OF A*] **and** sB = *semi-factor-subdomain.axioms*(*1*)[*OF B*]
have p: *semi-factor-defl* (*stable-subdomain.p A*) **by** (*rule semi-factor-defl-p*[*OF A*])
have q: *semi-factor-defl* (*stable-subdomain.p B*) **by** (*rule semi-factor-defl-p*[*OF B*])
have $\{x.\ (\mathit{stable\text{-}subdomain.p}\ A \sqcup \mathit{stable\text{-}subdomain.p}\ B){\cdot}x = x\}$
$= \{a \sqcup b \mid a\ b.\ a \in A \wedge b \in B \wedge a \Uparrow b\}$
unfolding *stable-defl-join-fixes*[*OF p*[*THEN semi-factor-defl.axioms*(*1*)] *q*[*THEN semi-factor-defl.axioms*(*1*)]]
by (*simp add*: *stable-subdomain.p-fixed-iff*[*OF sA*] *stable-subdomain.p-fixed-iff*[*OF sB*])
with *semi-factor-defl-join*[*OF p q*] **show** *?thesis* **by** (*simp add*: *semi-factor-defl-iff*)
qed

The slide rule passes through composition as for the meet of semi-factors (Jung, Lemma 4.1.9(ii)): first through the outer, then through the inner deflation.

lemma *semi-factor-defl-oo*:
assumes p: *semi-factor-defl* p **and** q: *semi-factor-defl* q
shows *semi-factor-defl* $(p\ oo\ q)$
proof −
interpret p: *semi-factor-defl* p **by** (*rule p*)
interpret q: *semi-factor-defl* q **by** (*rule q*)
show *?thesis*
proof (*rule semi-factor-deflI*)
show *stable-defl* $(p\ oo\ q)$ **by** (*rule stable-defl-oo*[*OF p.stable-defl-axioms q.stable-defl-axioms*])

fix x y **assume** y: $(p \mathbin{oo} q)\cdot y = y$ **and** px: $(p \mathbin{oo} q)\cdot x \sqsubseteq y$
from y **have** py: $p\cdot y = y$ **and** qy: $q\cdot y = y$
unfolding *stable-defl-oo-fixed*[*OF p.stable-defl-axioms q.stable-defl-axioms*] **by** *simp-all*
from px **have** $p\cdot(q\cdot x) \sqsubseteq y$ **by** *simp*
with py **have** *step1*: $q\cdot x \Uparrow y$ **by** (*rule p.slide*)
have $\{q\cdot x,\ y\} \subseteq \{z.\ q\cdot z = z\}$ **by** (*simp add*: *qy q.idem*)
then have *memB*: $q\cdot(q\cdot x \sqcup y) = q\cdot x \sqcup y$
by (*rule deflation-lub-closed*[*OF q.deflation - joinable-lub*[*OF step1*]])
from *step1* **have** $q\cdot x \sqsubseteq q\cdot x \sqcup y$ **by** (*rule joinableD*(*1*))
with *memB* **have** *step2*: $x \Uparrow q\cdot x \sqcup y$ **by** (*rule q.slide*)
then have $x \sqsubseteq x \sqcup (q\cdot x \sqcup y)$ **by** (*rule joinableD*(*1*))
moreover from *step1 step2* **have** $y \sqsubseteq x \sqcup (q\cdot x \sqcup y)$
by (*blast dest*: *joinableD*(*2*) *below-trans*)
ultimately show $x \Uparrow y$ **by** (*rule joinableI*)
qed
qed

lemma *semi-factor-defl-ID*: *semi-factor-defl ID*
by (*rule semi-factor-deflI*[*OF stable-defl-ID*]) (*auto intro*: *joinableI*)

9.2 Joins in the domain of stable subdomains

Over a distributive domain the binary join is pointwise.

lemma *p-sup*: $\mathrm{p}_{sup\ A\ B} = \mathrm{p}_A \sqcup \mathrm{p}_B$ **for** $A\ B$ :: *'a stable-subdom*
proof −
let *?j* = $\mathrm{p}_A \sqcup \mathrm{p}_B$
have *st*: *stable-defl ?j* **by** (*rule stable-defl-join*) *simp-all*
have *jl*: $\{\mathrm{p}_A, \mathrm{p}_B\}$ <<| *?j*
by (*rule joinable-lub*, *rule deflation-joinable*) (*simp-all add*: *stable-defl.deflation*)
have *sup A B* ≤ *subdom-of-defl ?j*
using *is-lubD1*[*OF jl*]
by (*intro sup-least*; *simp add*: *p-below-iff-le*[*symmetric*] *stable-subdom-p-subdom-of-defl*[*OF st*])
then have $\mathrm{p}_{sup\ A\ B} \sqsubseteq$ *?j* **by** (*simp add*: *p-below-iff-le*[*symmetric*] *stable-subdom-p-subdom-of-defl*[*OF st*])
moreover have *?j* $\sqsubseteq \mathrm{p}_{sup\ A\ B}$
by (*rule is-lubD2*[*OF jl*]) (*simp add*: *p-below-iff-le*)
ultimately show *?thesis* **by** (*rule below-antisym*)
qed

lemma *p-sup-join*: $\mathrm{p}_{sup\ a\ b}\cdot x = \mathrm{p}_a\cdot x \sqcup \mathrm{p}_b\cdot x$ **for** $a\ b$ :: *'a stable-subdom*
by (*simp add*: *p-sup stable-defl-join-app*)

lemma *set-of-subdom-sup*:
fixes $A\ B$ :: *'a stable-subdom*
shows *set-of-subdom* (*sup A B*) =
$\{a \sqcup b \mid a\ b.\ a \in$ *set-of-subdom* $A \wedge b \in$ *set-of-subdom* $B \wedge a \Uparrow b\}$
unfolding *set-of-subdom-def p-sup stable-defl-join-fixes*[*OF stable-defl-stable-subdom-p*

stable-defl-stable-subdom-p]
by *simp*

Directed joins are pointwise lubs, already over bounded-complete domains (*stable-defl-directed-lub*).

lemma *p-Sup-directed*:
fixes $\mathcal{A}$ :: *'b*::{*bifinite*, *bounded-complete*} *stable-subdom set*
assumes *directed*: $\bigwedge X\ Y.\ X \in \mathcal{A} \Longrightarrow Y \in \mathcal{A} \Longrightarrow \exists\, Z \in \mathcal{A}.\ X \leq Z \wedge Y \leq Z$
and *nonempty*: $\mathcal{A} \neq \{\}$
shows $\mathrm{p}_{Sup\ \mathcal{A}} = lub\ ((\lambda A.\ \mathrm{p}_A)\ `\ \mathcal{A})$ (**is** - = *lub ?P*)
proof −
have *dir*: *directed ?P*
proof (*rule directedI*)

fix *u v* **assume** *u* ∈ *?P v* ∈ *?P*
then obtain *X Y* **where** $X \in \mathcal{A}$ $Y \in \mathcal{A}$ $u = \mathrm{p}_X$ $v = \mathrm{p}_Y$ **by** *blast*
with *directed* **show** ∃ *z*∈*?P*. *u* ⊑ *z* ∧ *v* ⊑ *z* **by** (*fastforce simp*: *p-below-iff-le*)
qed
have *ne*: *?P* ≠ {} **using** *nonempty* **by** *blast*
have *st*: *stable-defl* (*lub ?P*) **by** (*rule stable-defl-directed-lub*[*OF dir ne*]) *auto*
note *lubP* = *directed-lub*[*OF dir*]
have *Sup* $\mathcal{A} \leq$ *subdom-of-defl* (*lub ?P*)
proof (*rule Sup-least*)
fix *A* **assume** $A \in \mathcal{A}$
then have $\mathrm{p}_A \sqsubseteq$ *lub ?P* **by** (*blast intro*: *is-ubD*[*OF is-lubD1*[*OF lubP*]])
then show $A \leq$ *subdom-of-defl* (*lub ?P*)
by (*simp add*: *p-below-iff-le*[*symmetric*] *stable-subdom-p-subdom-of-defl*[*OF st*])
qed
then have $\mathrm{p}_{Sup\ \mathcal{A}} \sqsubseteq$ *lub ?P*
by (*simp add*: *p-below-iff-le*[*symmetric*] *stable-subdom-p-subdom-of-defl*[*OF st*])
moreover have *lub ?P* ⊑ $\mathrm{p}_{Sup\ \mathcal{A}}$
by (*rule is-lubD2*[*OF lubP*], *rule ub-imageI*) (*simp add*: *p-below-iff-le Sup-upper*)
ultimately show *?thesis* **by** (*rule below-antisym*)
qed

lemma *directed-stable-proj-sup*:
fixes $\mathcal{A}$ :: *'b*::{*bifinite*, *bounded-complete*} *stable-subdom set*
assumes *directed*: $\bigwedge X\ Y.\ X \in \mathcal{A} \Longrightarrow Y \in \mathcal{A} \Longrightarrow \exists\, Z \in \mathcal{A}.\ X \leq Z \wedge Y \leq Z$
and *nonempty*: $\mathcal{A} \neq \{\}$
shows $(\lambda A.\ \mathrm{p}_A{\cdot}x)\ `\ \mathcal{A} <<|\ \mathrm{p}_{Sup\ \mathcal{A}}{\cdot}x$
proof −
have *dir*: *directed* $((\lambda A.\ \mathrm{p}_A)\ `\ \mathcal{A})$
proof (*rule directedI*)

fix *u v* **assume** $u \in (\lambda A.\ \mathrm{p}_A)\ `\ \mathcal{A}$ $v \in (\lambda A.\ \mathrm{p}_A)\ `\ \mathcal{A}$
then obtain *X Y* **where** $X \in \mathcal{A}$ $Y \in \mathcal{A}$ $u = \mathrm{p}_X$ $v = \mathrm{p}_Y$ **by** *blast*
with *directed* **show** ∃ *z*∈$(\lambda A.\ \mathrm{p}_A)\ `\ \mathcal{A}$. *u* ⊑ *z* ∧ *v* ⊑ *z* **by** (*fastforce simp*: *p-below-iff-le*)

qed
have $(\lambda f.\ f{\cdot}x)$ ` $((\lambda A.\ \mathrm{p}_A)$ ` $\mathcal{A}) <\!\!<\!\!|$ *lub* $((\lambda A.\ \mathrm{p}_A)$ ` $\mathcal{A}){\cdot}x$
using *nonempty* **by** (*intro cont-directed-is-lub*[*OF cont-Rep-cfun1 dir*]) *blast*
then show *?thesis* **by** (*simp add*: *p-Sup-directed*[*OF directed nonempty*] *image-image*)
qed

9.3 Semi-factors

The bottom subdomain $\{\bot\}$ satisfies the slide condition trivially.

lemma *semi-factor-subdomain-bot*: *semi-factor-subdomain* $\{\bot\}$
proof (*intro semi-factor-subdomain.intro*[*OF stable-subdomain-bot*] *semi-factor-subdomain-axioms.intro*)
fix *x y* **assume** $y \in \{\bot\}$ **and** *stable-subdomain.p* $\{\bot\}{\cdot}x \sqsubseteq y$
then have $y = \bot$ **by** *simp*
then show $x \Uparrow y$ **by** *simp*
qed

definition *is-semi-factor* :: *'a stable-subdom* ⇒ *bool*
where *is-semi-factor A* ⟷ *semi-factor-defl* p_A

lemma *p-set-of-subdom*: *stable-subdomain.p* (*set-of-subdom A*) = p_A
unfolding *set-of-subdom-def*
by (*rule stable-subdomain-deflation*[*symmetric*])
(*simp-all add*: *stable-defl.deflation stable-defl-stable-subdomain*)

lemma *is-semi-factor-iff-subdomain*: *is-semi-factor A* ⟷ *semi-factor-subdomain* (*set-of-subdom A*)
unfolding *is-semi-factor-def semi-factor-defl-iff*
by (*simp add*: *set-of-subdom-def stable-defl.deflation*)

lemma *is-semi-factorI*:
assumes *slide*: $\bigwedge x\ y.\ y \in$ *set-of-subdom A* ⟹ $\mathrm{p}_A{\cdot}x \sqsubseteq y \Longrightarrow x \Uparrow y$
shows *is-semi-factor A*
unfolding *is-semi-factor-def*
by (*rule semi-factor-deflI*[*OF stable-defl-stable-subdom-p*]) (*simp add*: *slide p-fixes*)

lemma *is-semi-factor-bot*: *is-semi-factor bot*
by (*simp add*: *is-semi-factor-def p-bot-stable-subdom semi-factor-defl-bottom*)

lemma *is-semi-factor-top*: *is-semi-factor top*
by (*simp add*: *is-semi-factor-def p-top-stable-subdom semi-factor-defl-ID*)

Semi-factors are closed under binary joins (Jung 4.1.9(i)) and meets (4.1.9(ii)).

lemma *is-semi-factor-sup*:
is-semi-factor a ⟹ *is-semi-factor b* ⟹ *is-semi-factor* (*sup a b*)
by (*simp add*: *is-semi-factor-def p-sup semi-factor-defl-join*)

lemma *is-semi-factor-inf*:
is-semi-factor a ⟹ *is-semi-factor b* ⟹ *is-semi-factor* (*inf a b*)

by (*simp add*: *is-semi-factor-def p-inf semi-factor-defl-oo*)

... and hence, by induction, through finite suprema.

lemma *finite-semi-factor-Sup*:
fixes $\mathcal{A}$ :: *'a stable-subdom set*
assumes *finite* $\mathcal{A}$ **and** $\bigwedge A.\ A \in \mathcal{A} \Longrightarrow$ *is-semi-factor A*
shows *is-semi-factor* (*Sup* $\mathcal{A}$)
using *assms*
proof (*induction* $\mathcal{A}$ *rule*: *finite-induct*)
case *empty*
show *?case* **by** (*simp add*: *is-semi-factor-bot*)
next
case (*insert A* $\mathcal{A}$)
from *insert.prems* **have** *is-semi-factor A* **by** *blast*
moreover have *is-semi-factor* (*Sup* $\mathcal{A}$)
by (*rule insert.IH*) (*use insert.prems* **in** *blast*)
ultimately show *?case* **using** *is-semi-factor-sup*
by *simp*
qed

lemma *directed-semi-factor-Sup*:
fixes $\mathcal{A}$ :: *'a stable-subdom set*
assumes *directed*: $\bigwedge X\ Y.\ X \in \mathcal{A} \Longrightarrow Y \in \mathcal{A} \Longrightarrow \exists Z \in \mathcal{A}.\ X \leq Z \wedge Y \leq Z$
and *nonempty*: $\mathcal{A} \neq \{\}$
and *sf*: $\bigwedge A.\ A \in \mathcal{A} \Longrightarrow$ *is-semi-factor A*
shows *is-semi-factor* (*Sup* $\mathcal{A}$)
proof (*rule is-semi-factorI*)
fix *x y* **assume** *y*: $y \in$ *set-of-subdom* (*Sup* $\mathcal{A}$)
assume *px-below*: $\mathrm{p}_{Sup\ \mathcal{A}} \cdot x \sqsubseteq y$
from *directed-stable-proj-sup*[*OF directed nonempty*]
have *p-lub*: $(\lambda A.\ \mathrm{p}_A \cdot x)$ ` $\mathcal{A}$ <<| $\mathrm{p}_{Sup\ \mathcal{A}} \cdot x$ **for** *x* **.**
let *?p* = $\mathrm{p}_{Sup\ \mathcal{A}}$
have *xpAy*: $x \Uparrow \mathrm{p}_A \cdot y$ **if** $A \in \mathcal{A}$ **for** *A*
proof −
interpret *A*: *semi-factor-defl* p_A **using** *sf*[*OF that*] **by** (*simp add*: *is-semi-factor-def*)
from *that* **have** $\mathrm{p}_A \sqsubseteq$ *?p* **by** (*intro p-below-of-le Sup-upper*)
then have $\mathrm{p}_A \cdot x = \mathrm{p}_A \cdot (?p \cdot x)$
by (*simp add*: *deflation-below-comp1*[*OF subdom.deflation-p subdom.deflation-p*])
also have ... $\sqsubseteq \mathrm{p}_A \cdot y$ **using** *px-below* **by** (*rule monofun-cfun-arg*)
finally have $\mathrm{p}_A \cdot x \sqsubseteq \mathrm{p}_A \cdot y$ **.**
with *subdom.idem* **show** $x \Uparrow \mathrm{p}_A \cdot y$ **by** (*rule A.slide*)
qed
let *?W* = $(\lambda A.\ x \sqcup \mathrm{p}_A \cdot y)$ ` $\mathcal{A}$
have *dirW*: *directed ?W*
proof (*rule directedI*)
fix *u v* **assume** *u* ∈ *?W v* ∈ *?W*
then obtain *A B* **where** *A*: $A \in \mathcal{A}$ $u = x \sqcup \mathrm{p}_A \cdot y$
and *B*: $B \in \mathcal{A}$ $v = x \sqcup \mathrm{p}_B \cdot y$ **by** *blast*
from *directed*[*OF A*(*1*) *B*(*1*)] **obtain** *C* **where** *C*: $C \in \mathcal{A}$ $A \leq C$ $B \leq C$ **by**

blast
  from $C(2)$ **have** AC: $\mathrm{p}_A{\cdot}y \sqsubseteq \mathrm{p}_C{\cdot}y$
    by (*auto intro*: *p-below-of-le monofun-cfun-fun*)
  from $C(3)$ **have** BC: $\mathrm{p}_B{\cdot}y \sqsubseteq \mathrm{p}_C{\cdot}y$
    by (*auto intro*: *p-below-of-le monofun-cfun-fun*)
  have $u \sqsubseteq x \sqcup \mathrm{p}_C{\cdot}y$ **unfolding** $A(2)$
    by (*rule join-mono2*[*OF xpAy*[*OF C(1)*] *AC*])
  moreover have $v \sqsubseteq x \sqcup \mathrm{p}_C{\cdot}y$ **unfolding** $B(2)$
    by (*rule join-mono2*[*OF xpAy*[*OF C(1)*] *BC*])
  moreover have $x \sqcup \mathrm{p}_C{\cdot}y \in ?W$ **using** $C(1)$ **by** *blast*
  ultimately show $\exists z{\in}?W.\ u \sqsubseteq z \wedge v \sqsubseteq z$ **by** *blast*
 qed
 define w **where** $w = lub\ ?W$
 have *Wlub*: $?W <<|\ w$ **unfolding** *w-def* **using** *dirW* **by** (*rule directed-lub*)
 from *nonempty* **obtain** $A0$ **where** $A0$: $A0 \in \mathcal{A}$ **by** *blast*
 have *memA0*: $x \sqcup \mathrm{p}_{A0}{\cdot}y \in ?W$ **using** $A0$ **by** *blast*
 have $x \sqsubseteq x \sqcup \mathrm{p}_{A0}{\cdot}y$ **by** (*rule joinableD(1)*[*OF xpAy*[*OF A0*]])
 also have $\ldots \sqsubseteq w$ **using** *is-ubD*[*OF is-lubD1*[*OF Wlub*] *memA0*] **.**
 finally have *xw*: $x \sqsubseteq w$ **.**
 have *Py-y*: $(\lambda A.\ \mathrm{p}_A{\cdot}y)\ `\ \mathcal{A} <<|\ y$
   using *p-lub*[*of y*] *y* **by** (*simp add*: *p-fixes*[*symmetric*])
 have *Py-w*: $(\lambda A.\ \mathrm{p}_A{\cdot}y)\ `\ \mathcal{A} <|\ w$
 proof (*rule ub-imageI*)
   fix A **assume** A: $A \in \mathcal{A}$
   have *memA*: $x \sqcup \mathrm{p}_A{\cdot}y \in ?W$ **using** A **by** *blast*
   have $\mathrm{p}_A{\cdot}y \sqsubseteq x \sqcup\ \mathrm{p}_A{\cdot}y$ **by** (*rule joinableD(2)*[*OF xpAy*[*OF A*]])
   also have $\ldots \sqsubseteq w$ **using** *is-ubD*[*OF is-lubD1*[*OF Wlub*] *memA*] **.**
   finally show $\mathrm{p}_A{\cdot}y \sqsubseteq w$ **.**
 qed
 have *yw*: $y \sqsubseteq w$ **using** *Py-y Py-w* **by** (*rule is-lubD2*)
 show $x \Uparrow y$ **using** *xw yw* **by** (*rule joinableI*)
qed

lemma *Fpow-singleton*: $x \in A \Longrightarrow \{x\} \in Fpow\ A$
 by (*simp add*: *Fpow-def*)

lemma (**in** *complete-lattice*) *Sup-of-Fpow*: $Sup\ (Sup\ `\ Fpow(\mathcal{A})) = Sup\ \mathcal{A}$
proof (*rule order-antisym*)
 show $Sup\ \mathcal{A} \leq Sup\ (Sup\ `\ Fpow\ \mathcal{A})$
   by (*rule Sup-least*, *rule Sup-upper*, *drule Fpow-singleton*, *force*)
 show $Sup\ (Sup\ `\ Fpow\ \mathcal{A}) \leq Sup\ \mathcal{A}$
   by (*rule Sup-least*, *erule imageE*, *auto simp add*: *Sup-subset-mono Fpow-def*)
qed

lemma *is-semi-factor-Sup*:
 fixes $\mathcal{A}$ :: *'a stable-subdom set*
 assumes *sf*: $\bigwedge A.\ A \in \mathcal{A} \Longrightarrow is\text{-}semi\text{-}factor\ A$
 shows *is-semi-factor* (*Sup* $\mathcal{A}$)
proof −

have *directed*: $X \in Sup\ `\ Fpow(\mathcal{A}) \Longrightarrow Y \in Sup\ `\ Fpow(\mathcal{A}) \Longrightarrow \exists Z \in Sup\ `\ Fpow(\mathcal{A}).\ X \leq Z \wedge Y \leq Z$
for $X\ Y$
proof (*simp*, *safe*, *intro bexI*)
fix $x\ y :: 'a\ stable\text{-}subdom\ set$
show $Sup\ x \leq Sup\ (x \cup y) \wedge Sup\ y \leq Sup\ (x \cup y)$
by (*simp add*: *Sup-subset-mono*)
assume $x \in Fpow(\mathcal{A})\ y \in Fpow(\mathcal{A})$
then show $x \cup y \in Fpow(\mathcal{A})$ **by** (*simp add*: *Fpow-def*)
qed
moreover have $Sup\ `\ Fpow(\mathcal{A}) \neq \{\}$ **by** (*simp add*: *Fpow-not-empty*)
moreover have $B \in Sup\ `\ Fpow(\mathcal{A}) \Longrightarrow is\text{-}semi\text{-}factor\ (B)$ **for** B
using *finite-semi-factor-Sup sf* **unfolding** *Fpow-def* **by** *auto*
ultimately have *is-semi-factor* (*Sup* (*Sup* ` *Fpow*($\mathcal{A}$)))
by (*rule directed-semi-factor-Sup*)
then show *?thesis* **by** (*simp add*: *Sup-of-Fpow*)
qed

9.4 Meets, distributivity and the frame law

lemma *p-inf-meet*: $\mathrm{p}_{inf\ A\ B}{\cdot}x = \mathrm{p}_A{\cdot}x \sqcap \mathrm{p}_B{\cdot}x$
unfolding *meet-def p-inf*
by (*simp add*: *glb-eqI*[*OF stable-defl-oo-glb*[*OF stable-defl-stable-subdom-p stable-defl-stable-subdom-p*]])

lemma *p-joinable*[*simp*]: $\mathrm{p}_A{\cdot}x \Uparrow \mathrm{p}_B{\cdot}x$
by (*rule joinableI*; *rule subdom.below*)

instance *stable-subdom* :: ({*bifinite*,*distributive-cpo*}) *distrib-lattice*
by (*intro-classes*, *intro stable-subdom-proj-eqI cfun-eqI*)
(*simp add*: *p-inf-meet p-sup-join join-meet-distrib*)

Composition of projections is the projection onto the meet, and commutative.

lemma *p-comp-inf*: $\mathrm{p}_b{\cdot}(\mathrm{p}_a{\cdot}x) = \mathrm{p}_{inf\ a\ b}{\cdot}x$
using *p-inf-meet*[*of b a x*] *p-inf-meet*[*of a b x*] **by** (*simp add*: *p-inf meet-sym*)

lemma *p-app-commute*: $\mathrm{p}_a{\cdot}(\mathrm{p}_b{\cdot}x) = \mathrm{p}_b{\cdot}(\mathrm{p}_a{\cdot}x)$
by (*simp add*: *p-comp-inf inf-commute*)

lemma *p-comp-inf2*: $\mathrm{p}_a{\cdot}(\mathrm{p}_b{\cdot}x) = \mathrm{p}_{inf\ a\ b}{\cdot}x$
by (*simp add*: *p-comp-inf inf-commute*)

lemma (**in** *stable-subdomain*) *join-closed*:
$a \Uparrow b \Longrightarrow a \in S \Longrightarrow b \in S \Longrightarrow a \sqcup b \in S$
using *lub-closed*[*OF - joinable-lub*]
by *fast*

The projection of a finite (nonempty) sup is the lub of the pointwise projec-

tions.

lemma *finite-proj-sup*:
fixes $\mathcal{A}$ :: *'a stable-subdom set*
assumes *finite* $\mathcal{A}$ **and** $\mathcal{A} \neq \{\}$
shows $(\lambda A.\ \mathrm{p}_A{\cdot}x)\ `\ \mathcal{A} <<|\ \mathrm{p}_{Sup\ \mathcal{A}}{\cdot}x$
using *assms*
proof (*induction rule*: *finite-ne-induct*)
case (*singleton A*)
have $\{\mathrm{p}_A{\cdot}x\} <<|\ \mathrm{p}_A{\cdot}x$ **by** (*rule is-lub-singleton*)
then show *?case* **by** *simp*
next
case (*insert A* $\mathcal{A}$)
have *j*: $\mathrm{p}_A{\cdot}x \Uparrow \mathrm{p}_{Sup\ \mathcal{A}}{\cdot}x$ **by** (*rule joinableI*[*OF subdom.p-below subdom.p-below*])
have *insert* $(\mathrm{p}_A{\cdot}x)\ ((\lambda B.\ \mathrm{p}_B{\cdot}x)\ `\ \mathcal{A}) <<|\ \mathrm{p}_A{\cdot}x \sqcup \mathrm{p}_{Sup\ \mathcal{A}}{\cdot}x$
by (*rule insert-join-is-lub*[*OF insert.IH j*])
moreover have $(\lambda B.\ \mathrm{p}_B{\cdot}x)\ `\ insert\ A\ \mathcal{A} = insert\ (\mathrm{p}_A{\cdot}x)\ ((\lambda B.\ \mathrm{p}_B{\cdot}x)\ `\ \mathcal{A})$ **by** *simp*
moreover have $\mathrm{p}_A{\cdot}x \sqcup \mathrm{p}_{Sup\ \mathcal{A}}{\cdot}x = \mathrm{p}_{Sup\ (insert\ A\ \mathcal{A})}{\cdot}x$
by (*simp add*: *p-sup-join*)
ultimately show *?case* **by** *simp*
qed

The projection of an *arbitrary* sup is the lub of the pointwise projections (Jung's full $p_{\bigsqcup\mathcal{A}}{\cdot}x = \bigsqcup_{A\in\mathcal{A}} p_A{\cdot}x$, in $<<|$ form): combine the directed result over $Sup\ `\ Fpow\ \mathcal{A}$ (whose sup is $Sup\ \mathcal{A}$) with the finite case. An upper bound of the pointwise projections bounds every finite sub-join, hence the directed lub, hence $\mathrm{p}_{Sup\ \mathcal{A}}{\cdot}x$.

lemma *proj-Sup*:
fixes $\mathcal{A}$ :: *'a stable-subdom set*
shows $(\lambda A.\ \mathrm{p}_A{\cdot}x)\ `\ \mathcal{A} <<|\ \mathrm{p}_{Sup\ \mathcal{A}}{\cdot}x$
proof −
have *dir*: $X \in Sup\ `\ Fpow\ \mathcal{A} \Longrightarrow Y \in Sup\ `\ Fpow\ \mathcal{A} \Longrightarrow \exists\, Z \in Sup\ `\ Fpow\ \mathcal{A}.\ X \leq Z \wedge Y \leq Z$
for *X Y*
proof (*simp*, *safe*, *intro bexI*)
fix *x y* :: *'a stable-subdom set*
show $Sup\ x \leq Sup\ (x \cup y) \wedge Sup\ y \leq Sup\ (x \cup y)$ **by** (*simp add*: *Sup-subset-mono*)
assume $x \in Fpow\ \mathcal{A}\ y \in Fpow\ \mathcal{A}$
then show $x \cup y \in Fpow\ \mathcal{A}$ **by** (*simp add*: *Fpow-def*)
qed
have *neF*: $Sup\ `\ Fpow\ \mathcal{A} \neq \{\}$ **by** (*simp add*: *Fpow-not-empty*)
have *D*: $(\lambda F.\ \mathrm{p}_F{\cdot}x)\ `\ (Sup\ `\ Fpow\ \mathcal{A}) <<|\ \mathrm{p}_{Sup\ \mathcal{A}}{\cdot}x$
using *directed-stable-proj-sup*[*OF dir neF*] **by** (*simp add*: *Sup-of-Fpow*)
show *?thesis*
proof (*rule is-lubI*)
show $(\lambda A.\ \mathrm{p}_A{\cdot}x)\ `\ \mathcal{A} <|\ \mathrm{p}_{Sup\ \mathcal{A}}{\cdot}x$
proof (*rule ub-imageI*)
fix *A* **assume** $A \in \mathcal{A}$

```
    then have p_A ⊑ p_(Sup A) by (intro p-below-of-le Sup-upper)
    then show p_A·x ⊑ p_(Sup A)·x by (rule monofun-cfun-fun)
  qed
 next
  fix u assume u: (λA. p_A·x) ` A <| u
  have (λF. p_F·x) ` (Sup ` Fpow A) <| u
  proof (rule ub-imageI)
    fix F assume F ∈ Sup ` Fpow A
    then obtain G where G: G ∈ Fpow A and F: F = Sup G by auto
    from G have finG: finite G and GA: G ⊆ A by (auto simp: Fpow-def)
    show p_F·x ⊑ u
    proof (cases G = {})
      case True
      with F show ?thesis by (simp add: p-bot-stable-subdom)
    next
      case False
        have lub: (λA. p_A·x) ` G <<| p_(Sup G)·x using finG False by (rule fi-
nite-proj-sup)
      have (λA. p_A·x) ` G <| u
      proof (rule ub-imageI)
        fix A assume A ∈ G
        with GA have p_A·x ∈ (λA. p_A·x) ` A by auto
        with u show p_A·x ⊑ u by (rule is-ubD)
      qed
      with lub have p_(Sup G)·x ⊑ u by (rule is-lubD2)
      with F show ?thesis by simp
    qed
  qed
  with D show p_(Sup A)·x ⊑ u by (rule is-lubD2)
 qed
qed
```

The frame law: binary meet distributes over arbitrary joins. The image y of x under $A \sqcap \bigsqcup \mathcal{B}$ is the lub of its projections through the members of $\mathcal{B}$ (*proj-Sup*), and each of those is fixed by $A \sqcap B$, hence by the right-hand side.

```
lemma inf-Sup-stable-subdom:
 fixes A :: 'a stable-subdom
 shows inf A (Sup B) = (SUP B∈B. inf A B)
proof (rule order.antisym)
 show (SUP B∈B. inf A B) ≤ inf A (Sup B)
   by (rule SUP-least) (auto intro: inf-mono Sup-upper le-infI2)
 let ?C = SUP B∈B. inf A B
 show inf A (Sup B) ≤ ?C
 proof (rule le-of-p-below, rule cfun-belowI)
   fix x
   let ?y = p_(inf A (Sup B))·x
   have ?y ∈ set-of-subdom (inf A (Sup B)) by (rule subdom.p-in-S)
```

then have *yA*: *?y* ∈ *set-of-subdom A* **and** *yS*: *?y* ∈ *set-of-subdom* (*Sup* $\mathcal{B}$)
by (*simp-all add*: *set-of-subdom-inf*)
have *lub*: (λ*B*. p_{B}·*?y*) ‘ $\mathcal{B}$ <<| *?y*
using *proj-Sup*[**where** $\mathcal{A} = \mathcal{B}$ **and** *x* = *?y*] *yS* **by** (*simp add*: *p-fixes*[*symmetric*])
have (λ*B*. p_{B}·*?y*) ‘ $\mathcal{B}$ <| $\mathrm{p}_{?C}$·*?y*
proof (*rule ub-imageI*)
fix *B* **assume** *B*: *B* ∈ $\mathcal{B}$
have *zS*: p_{B}·*?y* ∈ *set-of-subdom* (*inf A B*)
using *subdom.downward*[*OF yA subdom.p-below*] *subdom.p-in-S* **by** (*simp add*: *set-of-subdom-inf*)
moreover have *le*: *inf A B* ≤ *?C* **using** *B* **by** (*rule SUP-upper*)
ultimately have $\mathrm{p}_{?C}(\mathrm{p}_{B}\cdot ?y) = \mathrm{p}_{B}\cdot ?y$
proof −
have $\mathrm{p}_{B}\cdot ?y = \mathrm{p}_{inf\ A\ B}(\mathrm{p}_{B}\cdot ?y)$ **using** *zS* **by** (*simp add*: *p-fixes*[*symmetric*])
also have ... ⊑ $\mathrm{p}_{?C}(\mathrm{p}_{B}\cdot ?y)$ **using** *le* **by** (*intro monofun-cfun-fun p-below-of-le*)
finally show *?thesis* **by** (*rule below-antisym*[*OF subdom.p-below*])
qed
moreover have $\mathrm{p}_{?C}(\mathrm{p}_{B}\cdot ?y) \sqsubseteq \mathrm{p}_{?C}\cdot ?y$ **by** (*rule monofun-cfun-arg*, *rule subdom.p-below*)
ultimately show $\mathrm{p}_{B}\cdot ?y \sqsubseteq \mathrm{p}_{?C}\cdot ?y$ **by** *simp*
qed
with *lub* **have** *?y* ⊑ $\mathrm{p}_{?C}\cdot ?y$ **by** (*rule is-lubD2*)
also have ... ⊑ $\mathrm{p}_{?C}\cdot x$ **by** (*rule monofun-cfun-arg*, *rule subdom.p-below*)
finally show *?y* ⊑ $\mathrm{p}_{?C}\cdot x$ **.**
qed
qed

lemma *Sup-contains-top*: *top* ∈ (*X*::'*a*::*complete-lattice set*) ⟹ *Sup X* = *top*
by (*rule order.antisym*) (*auto intro*: *Sup-upper*)

10 The type of semi-factors

Following Jung, the semi-factors (stable subdomains satisfying the slide condition) form a complete lattice. Sups, binary meets, top and bottom are inherited from the type *'a stable-subdom* (the semi-factors are closed under those operations), but an arbitrary intersection of semi-factors need not be one. The meet of a family is therefore recovered as the sup of its semi-factor lower bounds.

Semi-factors are closed under arbitrary sups, in particular under lubs of chains: they form a sub-pcpo of the stable subdomains.

pcpodef *'a semi-factor* = {*A* :: *'a stable-subdom. is-semi-factor A*}
morphisms *subdom-of-semi-factor semi-factor-of-subdom*
proof −
have *b*: *bot* = (⊥ :: *'a stable-subdom*) **by** (*rule bottomI*) (*simp add*: *below-stable-subdom-iff-le*)
show ⊥ ∈ {*A* :: *'a stable-subdom. is-semi-factor A*} **using** *is-semi-factor-bot* **by** (*simp add*: *b*[*symmetric*])

next
show *adm* (λ*A*. *A* ∈ {*A* :: *'a stable-subdom*. *is-semi-factor A*})
proof (*rule admI*, *simp*)
fix *Y* :: *nat* ⇒ *'a stable-subdom* **assume** *Y*: *chain Y* ∀ *i*. *is-semi-factor* (*Y i*)
have (⨆ *i*. *Y i*) = *Sup* (*range Y*) **by** (*rule lub-eqI*) (*rule stable-subdom-is-lub*)
then show *is-semi-factor* (⨆ *i*. *Y i*) **using** *Y*(*2*) **by** (*auto intro*: *is-semi-factor-Sup*)
qed
qed

setup-lifting *type-definition-semi-factor*

lemma *is-semi-factor-subdom-of-semi-factor*: *is-semi-factor* (*subdom-of-semi-factor F*)
using *subdom-of-semi-factor* **by** *blast*

instantiation *semi-factor* :: ({*bifinite*,*distributive-cpo*}) *complete-lattice*
begin

lift-definition *less-eq-semi-factor* :: *'a semi-factor* ⇒ *'a semi-factor* ⇒ *bool*
is (≤) **.**

lift-definition *less-semi-factor* :: *'a semi-factor* ⇒ *'a semi-factor* ⇒ *bool*
is (<) **.**

lift-definition *inf-semi-factor* :: *'a semi-factor* ⇒ *'a semi-factor* ⇒ *'a semi-factor*
is *inf* **by** (*rule is-semi-factor-inf*)

lift-definition *sup-semi-factor* :: *'a semi-factor* ⇒ *'a semi-factor* ⇒ *'a semi-factor*
is *sup* **by** (*rule is-semi-factor-sup*)

lift-definition *top-semi-factor* :: *'a semi-factor*
is *top* **by** (*rule is-semi-factor-top*)

lift-definition *bot-semi-factor* :: *'a semi-factor*
is *bot* **by** (*rule is-semi-factor-bot*)

lift-definition *Sup-semi-factor* :: *'a semi-factor set* ⇒ *'a semi-factor*
is *Sup* **by** (*rule is-semi-factor-Sup*)

Arbitrary meet = sup of the semi-factor lower bounds.

lift-definition *Inf-semi-factor* :: *'a semi-factor set* ⇒ *'a semi-factor*
is λ*S*. *Sup* {*b*. (∀ *a*∈*S*. *b* ≤ *a*) ∧ *is-semi-factor b*}
by (*auto intro*: *is-semi-factor-Sup*)

instance
by (*standard*; *transfer*) (*auto intro*: *Sup-least Sup-upper is-semi-factor-top Sup-contains-top*)

end

instance *semi-factor* :: ({*bifinite*,*distributive-cpo*}) *distrib-lattice*
by (*intro-classes*, *transfer'*, *rule sup-inf-distrib1*)

As for stable subdomains, the domain order is the lattice order.

lemma *below-semi-factor-iff-le*: $F \sqsubseteq G \longleftrightarrow F \leq G$ **for** F G :: *'a semi-factor*
by (*simp add*: *below-semi-factor-def less-eq-semi-factor.rep-eq below-stable-subdom-iff-le*)

lemma *semi-factor-is-lub*: $\mathcal{A}$ <<| *Sup* $\mathcal{A}$ **for** $\mathcal{A}$ :: *'a semi-factor set*
by (*auto intro*!: *is-lubI is-ubI Sup-upper Sup-least*
simp: *below-semi-factor-iff-le is-ub-def*)

instance *semi-factor* :: ({*bifinite*,*distributive-cpo*}) *bounded-complete*
by *standard* (*blast intro*: *semi-factor-is-lub*)

instance *semi-factor* :: ({*bifinite*,*distributive-cpo*}) *lattice-complete*
by *intro-classes* (*rule exI*[*of - top*], *simp add*: *below-semi-factor-iff-le*)

lemma *join-semi-factor*: $F \sqcup G$ = *sup* F G **for** F G :: *'a semi-factor*
unfolding *join-def* **by** (*rule lub-eqI*) (*simp add*: *semi-factor-is-lub*[*of* {F, G}, *simplified*])

lemma *meet-semi-factor*: $F \sqcap G$ = *inf* F G **for** F G :: *'a semi-factor*
unfolding *meet-def* **by** (*rule glb-eqI*) (*simp add*: *is-glb-def below-semi-factor-iff-le*)

instance *semi-factor* :: ({*bifinite*,*distributive-cpo*}) *distributive-cpo*
by *intro-classes* (*simp add*: *join-semi-factor meet-semi-factor inf-sup-distrib1*)

lift-definition *semi-factor-p* :: *'a semi-factor* ⇒ *'a* → *'a*
is *stable-subdom-p* **.**

lift-definition *set-of-semi-factor* :: *'a semi-factor* ⇒ *'a set*
is *set-of-subdom* **.**

interpretation *semi-factor*: *semi-factor-subdomain set-of-semi-factor A*
rewrites *semi-factor-p-eq*: *stable-subdomain.p* (*set-of-semi-factor A*) ≡ *semi-factor-p A*
proof −
show *semi-factor-subdomain* (*set-of-semi-factor A*)
by *transfer* (*simp add*: *is-semi-factor-iff-subdomain*)
show *stable-subdomain.p* (*set-of-semi-factor A*) ≡ *semi-factor-p A*
unfolding *set-of-semi-factor.rep-eq semi-factor-p.rep-eq*
by (*rule eq-reflection*, *rule p-set-of-subdom*)
qed

lemma *semi-factor-defl-semi-factor-p*: *semi-factor-defl* (*semi-factor-p F*)
using *is-semi-factor-subdom-of-semi-factor*[*of F*]
by (*simp add*: *is-semi-factor-def semi-factor-p.rep-eq*)

Morally semi-factors are deflations; *semi-factor-p* is the representation and

semi-factor-of-defl the abstraction.

definition *semi-factor-of-defl* :: $('a \to 'a) \Rightarrow 'a$ *semi-factor*
where *semi-factor-of-defl d* = *semi-factor-of-subdom* (*subdom-of-defl d*)

lemma *semi-factor-p-of-defl*:
assumes *d*: *semi-factor-defl d*
shows *semi-factor-p* (*semi-factor-of-defl d*) = *d*
proof −
have *st*: *stable-defl d* **by** (*rule semi-factor-defl.axioms*(*1*)[*OF d*])
have *is-semi-factor* (*subdom-of-defl d*)
by (*simp add*: *is-semi-factor-def stable-subdom-p-subdom-of-defl*[*OF st*] *d*)
then show *?thesis*
by (*simp add*: *semi-factor-of-defl-def semi-factor-p.rep-eq semi-factor-of-subdom-inverse*
stable-subdom-p-subdom-of-defl[*OF st*])
qed

lemma *semi-factor-below-iff-p*: $F \sqsubseteq G \longleftrightarrow$ *semi-factor-p F* $\sqsubseteq$ *semi-factor-p G*
by (*simp add*: *below-semi-factor-def stable-subdom-below-iff semi-factor-p.rep-eq*)

The bottom semi-factor projects to $\bot$, the top one is the identity, and joins project to joins.

lemma *semi-factor-bot-bottom*: (*bot* :: *'a semi-factor*) = $\bot$
by (*rule bottomI*) (*simp add*: *below-semi-factor-iff-le*)

lemma *semi-factor-top-lattice-top*: (*top* :: *'a semi-factor*) = *lattice-top*
by (*rule below-antisym*) (*rule below-lattice-top*, *simp add*: *below-semi-factor-iff-le*)

lemma *semi-factor-p-bot*: *semi-factor-p* $\bot = \bot$
by (*rule cfun-eqI*) (*simp add*: *semi-factor-bot-bottom*[*symmetric*] *semi-factor-p.rep-eq*
bot-semi-factor.rep-eq p-bot-stable-subdom)

lemma *semi-factor-p-top*: *semi-factor-p lattice-top* = *ID*
by (*simp add*: *semi-factor-top-lattice-top*[*symmetric*] *semi-factor-p.rep-eq top-semi-factor.rep-eq*
p-top-stable-subdom)

lemma *semi-factor-p-sup*: *semi-factor-p* $(F \sqcup G)\cdot x$ = *semi-factor-p* $F\cdot x \sqcup$ *semi-factor-p* $G\cdot x$
by (*simp add*: *join-semi-factor semi-factor-p.rep-eq sup-semi-factor.rep-eq p-sup-join*)

lemma *semi-factor-p-inject*: *semi-factor-p F* = *semi-factor-p G* $\longleftrightarrow$ *F* = *G*
by *transfer* (*auto intro*: *stable-subdom-proj-eqI*)

lemma *semi-factor-of-defl-p* [*simp*]: *semi-factor-of-defl* (*semi-factor-p F*) = *F*
by (*rule semi-factor-p-inject*[*THEN iffD1*])
(*simp add*: *semi-factor-p-of-defl semi-factor-defl-semi-factor-p*)

abbreviation *semi-factor-proj* (**infixl** ‹⌊› *200*)
where *semi-factor-proj x a* ≡ *semi-factor-p a*·*x*

lemma *semi-factor-proj-comp*:
$x \lfloor a \lfloor b = x \lfloor (a \sqcap b)$
unfolding *meet-semi-factor* **by** (*transfer'*, *rule p-comp-inf*)

lemma *semi-factor-proj-comm*:
$x \lfloor a \lfloor b = x \lfloor b \lfloor a$
by (*simp add*: *semi-factor-proj-comp lattice-meet-commute*)

corollary *semi-factor-comp-simp*[*simp*]:
$a \sqsubseteq b \Longrightarrow x \lfloor a \lfloor b = x \lfloor a$
$a \sqsubseteq b \Longrightarrow x \lfloor b \lfloor a = x \lfloor a$
by (*simp-all add*: *semi-factor-proj-comp lattice-meet-absorb1 lattice-meet-absorb2*)

end
theory *Semi-Factor-DI*
imports *Subdomain HOLCF−extra.DI HOLCF−extra.Algebraic-Lattice HOL−Library.Countable-Set*
begin

11 Semi-factors over dI-domains

Over a dI-domain the meet of any non-empty family of semi-factor deflations is again a semi-factor deflation. Its image is the intersection of the images and it is stable (*stable-defl-glb*), so only the slide rule needs proof. At a compact argument the family takes only finitely many values, and a finite subfamily, whose meet is a semi-factor by *semi-factor-defl-oo*, gives the slide rule there; algebraicity extends it to all arguments.

Joinability with a fixed element is admissible: a chain bounded with y stays below the chain of joins with y.

lemma *adm-joinable*: *adm* $(\lambda x.\ x \Uparrow (y :: 'a{::}bounded\text{-}complete))$
proof (*rule admI*)
fix $Y :: nat \Rightarrow 'a$ **assume** Y: *chain* Y **and** J: $\forall i.\ Y\ i \Uparrow y$
have *ch*: *chain* $(\lambda i.\ Y\ i \sqcup y)$
by (*rule chainI*, *rule join-mono1*) (*simp-all add*: J *chainE*[*OF* Y])
show $(\bigsqcup i.\ Y\ i) \Uparrow y$
proof (*rule joinableI*)
show $(\bigsqcup i.\ Y\ i) \sqsubseteq (\bigsqcup i.\ Y\ i \sqcup y)$ **by** (*rule lub-mono*[*OF* Y *ch*]) (*simp add*: J *joinableD*(*1*))
show $y \sqsubseteq (\bigsqcup i.\ Y\ i \sqcup y)$
by (*rule below-trans*[*OF joinableD*(*2*)[*OF* J[*rule-format*]] *is-ub-thelub*[*OF ch*]])
qed
qed

lemma *semi-factor-defl-finite-meet*:
fixes $F :: ('a{::}\{bifinite, distributive\text{-}cpo\} \to 'a)\ set$
assumes *finite* F **and** $\bigwedge d.\ d \in F \Longrightarrow$ *semi-factor-defl* d
shows $\exists e.$ *semi-factor-defl* $e \wedge (\forall z.\ e{\cdot}z = z \longleftrightarrow (\forall d{\in}F.\ d{\cdot}z = z))$
using *assms*

proof (*induct F rule*: *finite-induct*)
case *empty*
show *?case* **by** (*intro exI*[*of - ID*] *conjI*) (*simp-all add*: *semi-factor-defl-ID*)
next
case (*insert d F*)
then obtain *e* **where** *e*: *semi-factor-defl e* **and** *efx*: $\forall z.\ e{\cdot}z = z \longleftrightarrow (\forall d{\in}F.\ d{\cdot}z = z)$
by *blast*
have *d*: *semi-factor-defl d* **using** *insert.prems* **by** *simp*
note *fx* = *stable-defl-oo-fixed*[*OF semi-factor-defl.axioms*(*1*)[*OF d*] *semi-factor-defl.axioms*(*1*)[*OF e*]]
show *?case*
proof (*intro exI*[*of - d oo e*] *conjI allI*)
show *semi-factor-defl* (*d oo e*) **by** (*rule semi-factor-defl-oo*[*OF d e*])
show $(d\ oo\ e){\cdot}z = z \longleftrightarrow (\forall d'{\in}\mathit{insert}\ d\ F.\ d'{\cdot}z = z)$ **for** *z* **using** *fx efx* **by** *simp*
qed
qed

lemma *semi-factor-defl-glb-compact*:
fixes $\mathcal{D}$ :: ('*a*::*dI* → '*a*) *set*
assumes *ne*: $\mathcal{D} \neq \{\}$ **and** *sf*: $\bigwedge d.\ d \in \mathcal{D} \Longrightarrow$ *semi-factor-defl d*
and *k*: *compact x* **and** *y*: *glb* $\mathcal{D}{\cdot}y = y$ **and** *xy*: *glb* $\mathcal{D}{\cdot}x \sqsubseteq y$
shows $x \Uparrow y$
proof −
have *st*: $\bigwedge d.\ d \in \mathcal{D} \Longrightarrow$ *stable-defl d* **by** (*rule semi-factor-defl.axioms*(*1*)[*OF sf*])
note *fx* = *stable-defl-glb-fixes*[*OF ne st*]
have $(\lambda d.\ d{\cdot}x)\ `\ \mathcal{D} \subseteq \{z.\ z \sqsubseteq x\}$
using *st* **by** (*auto intro*: *deflation.below stable-defl.deflation*)
then have *finite* $((\lambda d.\ d{\cdot}x)\ `\mathcal{D})$ **by** (*rule finite-subset*) (*rule finite-below-compact*[*OF k*])
then have $\exists F{\subseteq}\mathcal{D}.$ *finite* $F \wedge (\lambda d.\ d{\cdot}x)\ `\ \mathcal{D} = (\lambda d.\ d{\cdot}x)\ `\ F$
by (*rule finite-subset-image*[*OF - subset-refl*])
then obtain *F* **where** *F*: $F \subseteq \mathcal{D}$ *finite F* **and** *FD*: $(\lambda d.\ d{\cdot}x)\ `\ \mathcal{D} = (\lambda d.\ d{\cdot}x)\ `\ F$
by (*elim exE conjE*) (*rule that*)
obtain *e* **where** *e*: *semi-factor-defl e* **and** *efx*: $\bigwedge z.\ e{\cdot}z = z \longleftrightarrow (\forall d{\in}F.\ d{\cdot}z = z)$
using *semi-factor-defl-finite-meet*[*OF F*(*2*)] *sf F*(*1*) **by** *blast*
interpret *e*: *semi-factor-defl e* **by** (*rule e*)
have *exF*: $d{\cdot}(e{\cdot}x) = e{\cdot}x$ **if** $d \in F$ **for** *d* **using** *efx*[*of e·x*] *e.idem that* **by** *blast*
have $d{\cdot}(e{\cdot}x) = e{\cdot}x$ **if** *d*: $d \in \mathcal{D}$ **for** *d*
proof −
have $d{\cdot}x \in (\lambda d.\ d{\cdot}x)\ `\ F$ **using** *FD d* **by** *blast*
then obtain d' **where** d': $d' \in F\ d{\cdot}x = d'{\cdot}x$ **by** *blast*
have $e{\cdot}x = d'{\cdot}(e{\cdot}x)$ **using** *exF*[*OF d'*(*1*)] **by** *simp*
also have $\ldots \sqsubseteq d'{\cdot}x$ **by** (*rule monofun-cfun-arg*[*OF e.below*])
finally have $e{\cdot}x \sqsubseteq d{\cdot}x$ **using** $d'(2)$ **by** *simp*

```
    then show ?thesis by (rule stable-defl.lower-fixed[OF st[OF d]])
  qed
  then have glb 𝒟·(e·x) = e·x using fx by blast
  then have e·x ⊑ glb 𝒟·x by (metis monofun-cfun-arg e.below)
  then have e·x ⊑ y using xy by (rule below-trans)
  moreover have e·y = y using efx y fx F(1) by blast
  ultimately show x ⇑ y using e.slide by blast
qed

lemma semi-factor-defl-glb:
  fixes 𝒟 :: ('a::dI → 'a) set
  assumes ne: 𝒟 ≠ {} and sf: ⋀d. d ∈ 𝒟 ⟹ semi-factor-defl d
  shows semi-factor-defl (glb 𝒟)
proof (rule semi-factor-deflI)
  have st: ⋀d. d ∈ 𝒟 ⟹ stable-defl d by (rule semi-factor-defl.axioms(1)[OF
sf])
  show stable-defl (glb 𝒟) by (rule stable-defl-glb[OF ne st])
  fix x y assume y: glb 𝒟·y = y and xy: glb 𝒟·x ⊑ y
  obtain a :: nat ⇒ 'a → 'a where a: approx-chain a using bifinite by blast
  interpret approx-chain a by (fact a)
  have ch: chain (λi. a i·x) by (rule ch2ch-Rep-cfunL[OF chain-approx])
  have a i·x ⇑ y for i
    by (rule semi-factor-defl-glb-compact[OF ne sf compact-approx y])
      (blast, rule below-trans[OF monofun-cfun-arg[OF approx-below] xy])
  then have (⨆i. a i·x) ⇑ y by (rule admD[OF adm-joinable ch])
  then show x ⇑ y by (simp add: contlub-cfun-fun[OF chain-approx, symmetric])
qed
```

11.1 Meets of semi-factors

Over a dI key the meet of a non-empty set of semi-factors is computed pointwise: its projection is the glb of the projections, so its image is the intersection of the images.

```
lemma semi-factor-p-glb:
  fixes 𝒜 :: 'a::dI semi-factor set
  assumes ne: 𝒜 ≠ {}
  shows semi-factor-p (glb 𝒜) = (⨅A∈𝒜. semi-factor-p A)
proof −
  let ?P = semi-factor-p ‘ 𝒜
  have neP: ?P ≠ {} using ne by blast
  have sfd: semi-factor-defl (glb ?P)
   by (rule semi-factor-defl-glb[OF neP]) (auto intro: semi-factor-defl-semi-factor-p)
  have glb 𝒜 = semi-factor-of-defl (glb ?P)
  proof (rule glb-eqI, rule is-glbI)
    show semi-factor-of-defl (glb ?P) ⊑ A if A ∈ 𝒜 for A
      unfolding semi-factor-below-iff-p semi-factor-p-of-defl[OF sfd]
      by (rule glb-below) (use that in blast)
    show B ⊑ semi-factor-of-defl (glb ?P) if ∀A∈𝒜. B ⊑ A for B
      unfolding semi-factor-below-iff-p semi-factor-p-of-defl[OF sfd]
```

by (*rule glb-greatest*[*OF neP*]) (*use that* **in** ‹*auto simp*: *semi-factor-below-iff-p*›)
qed
then show *?thesis* **by** (*simp add*: *semi-factor-p-of-defl*[*OF sfd*])
qed

11.2 Algebraicity

lemma *countable-compacts*: *countable* {*x* :: *'a*::*bifinite*. *compact x*}
proof −
obtain *f* :: *'a compact-basis* ⇒ *nat* **where** *inj f* **using** *compact-basis.countable*
..
then have *countable* (*range* (*Rep-compact-basis* :: *'a compact-basis* ⇒ *'a*))
by (*rule countable-image*[*OF countableI*])
moreover have *range* (*Rep-compact-basis* :: *'a compact-basis* ⇒ *'a*) = {*x*. *compact x*}
using *type-definition-compact-basis* **by** (*rule type-definition.Rep-range*)
ultimately show *?thesis* **by** *simp*
qed

lemma *stable-defl-semi-factor-p*: *stable-defl* (*semi-factor-p F*)
by (*rule semi-factor-defl.axioms*(*1*)[*OF semi-factor-defl-semi-factor-p*])

lemma *deflation-semi-factor-p*: *deflation* (*semi-factor-p F*)
by (*rule stable-defl.deflation*[*OF stable-defl-semi-factor-p*])

lemma *semi-factor-p-mono*: *F* ⊑ *G* ⟹ *semi-factor-p F*·*x* ⊑ *semi-factor-p G*·*x*
by (*rule monofun-cfun-fun*) (*simp add*: *semi-factor-below-iff-p*[*symmetric*])

Projections onto a lub of semi-factors are pointwise lubs (*proj-Sup*).

lemma *semi-factor-p-lub-is-lub*: (λ*F*. *semi-factor-p F*·*x*) ‘ *𝒳* <<| *semi-factor-p* (*lub 𝒳*)·*x*
proof −
have *lub 𝒳* = *Sup 𝒳* **by** (*rule lub-eqI*[*OF semi-factor-is-lub*])
then have *semi-factor-p* (*lub 𝒳*) = *stable-subdom-p* (*Sup* (*subdom-of-semi-factor* ‘ *𝒳*))
by (*simp add*: *semi-factor-p.rep-eq Sup-semi-factor.rep-eq*)
moreover have (λ*F*. *semi-factor-p F*·*x*) ‘ *𝒳* = (λ*A*. *stable-subdom-p A*·*x*) ‘ (*subdom-of-semi-factor* ‘ *𝒳*)
by (*simp add*: *image-image semi-factor-p.rep-eq*)
ultimately show *?thesis* **using** *proj-Sup*[**where** *𝒜* = *subdom-of-semi-factor* ‘ *𝒳* **and** *x* = *x*] **by** *simp*
qed

lemma *semi-factor-p-chain-lub*: *semi-factor-p* (⨆ *i*. *Y i*)·*x* = (⨆ *i*. *semi-factor-p* (*Y i*)·*x*)
proof −
have *range* (λ*i*. *semi-factor-p* (*Y i*)·*x*) <<| *semi-factor-p* (⨆ *i*. *Y i*)·*x*
using *semi-factor-p-lub-is-lub*[**where** *𝒳* = *range Y* **and** *x* = *x*] **by** (*simp add*: *image-image*)

then show *?thesis* **by** (*rule lub-eqI*[*symmetric*])
qed

The principal semi-factor of x: the least one containing x. Over a dI key its image is the intersection of the images containing x (*semi-factor-p-glb*).

definition *semi-factor-principal* :: *'a*::*dI* ⇒ *'a semi-factor* **where**
semi-factor-principal x = *glb* {*F*. *semi-factor-p F*·*x* = *x*}

lemma *semi-factor-principal-fixes*:
semi-factor-p (*semi-factor-principal x*)·*y* = *y* ⟷
(∀ *F*. *semi-factor-p F*·*x* = *x* ⟶ *semi-factor-p F*·*y* = *y*)
proof −
let *?A* = {*F* :: *'a semi-factor*. *semi-factor-p F*·*x* = *x*}
have *semi-factor-of-defl ID* ∈ *?A* **by** (*simp add*: *semi-factor-p-of-defl semi-factor-defl-ID*)
then have *ne*: *?A* ≠ {} **by** *blast*
have *semi-factor-p* (*semi-factor-principal x*) = *glb* (*semi-factor-p* ‘ *?A*)
unfolding *semi-factor-principal-def* **by** (*rule semi-factor-p-glb*[*OF ne*])
moreover have *glb* (*semi-factor-p* ‘ *?A*)·*y* = *y* ⟷ (∀ *d*∈*semi-factor-p* ‘ *?A*. *d*·*y* = *y*)
by (*rule stable-defl-glb-fixes*) (*use ne* **in** ‹*auto intro*: *stable-defl-semi-factor-p*›)
ultimately show *?thesis* **by** *auto*
qed

lemma *semi-factor-principal-self*: *semi-factor-p* (*semi-factor-principal x*)·*x* = *x*
by (*simp add*: *semi-factor-principal-fixes*)

lemma *semi-factor-principal-below-iff*: *semi-factor-principal x* ⊑ *F* ⟷ *semi-factor-p F*·*x* = *x*
proof
assume *semi-factor-principal x* ⊑ *F*
then have *semi-factor-p* (*semi-factor-principal x*)·*x* ⊑ *semi-factor-p F*·*x*
by (*rule semi-factor-p-mono*)
then have *x* ⊑ *semi-factor-p F*·*x* **by** (*simp add*: *semi-factor-principal-self*)
then show *semi-factor-p F*·*x* = *x* **by** (*rule below-antisym*[*OF deflation.below*[*OF deflation-semi-factor-p*]])
next
assume *semi-factor-p F*·*x* = *x*
then show *semi-factor-principal x* ⊑ *F* **unfolding** *semi-factor-principal-def* **by** (*intro glb-below*) *simp*
qed

lemma *compact-semi-factor-principal*:
assumes *k*: *compact k*
shows *compact* (*semi-factor-principal k*)
proof (*rule compactI2*)
fix *Y* :: *nat* ⇒ *'a semi-factor* **assume** *Y*: *chain Y* **and** *le*: *semi-factor-principal k* ⊑ (⨆ *i*. *Y i*)
have *ch*: *chain* (λ*i*. *semi-factor-p* (*Y i*)·*k*) **by** (*rule chainI*, *rule semi-factor-p-mono*, *rule chainE*[*OF Y*])

have $k = \mathit{semi\text{-}factor\text{-}p}\ (\bigsqcup i.\ Y\ i)\cdot k$ **using** *le* **by** (*simp add*: *semi-factor-principal-below-iff*)
also have $\ldots = (\bigsqcup i.\ \mathit{semi\text{-}factor\text{-}p}\ (Y\ i)\cdot k)$ **by** (*rule semi-factor-p-chain-lub*)
finally have $k \sqsubseteq (\bigsqcup i.\ \mathit{semi\text{-}factor\text{-}p}\ (Y\ i)\cdot k)$ **by** *simp*
then obtain i **where** $k \sqsubseteq \mathit{semi\text{-}factor\text{-}p}\ (Y\ i)\cdot k$ **using** *compactD2*[*OF k ch*] **by** *blast*
then have $\mathit{semi\text{-}factor\text{-}p}\ (Y\ i)\cdot k = k$ **by** (*rule below-antisym*[*OF deflation.below*[*OF deflation-semi-factor-p*]])
then have $\mathit{semi\text{-}factor\text{-}principal}\ k \sqsubseteq Y\ i$ **by** (*simp add*: *semi-factor-principal-below-iff*)
then show $\exists i.\ \mathit{semi\text{-}factor\text{-}principal}\ k \sqsubseteq Y\ i$ **by** *blast*
qed

Every semi-factor is the lub of the principals of its compact elements: its image is downward closed and the projection onto the lub fixes every compact of it, hence every element.

lemma *semi-factor-lub-principals*:
$F = \mathit{lub}\ \{\mathit{semi\text{-}factor\text{-}principal}\ k \mid k.\ \mathit{compact}\ k \wedge \mathit{semi\text{-}factor\text{-}p}\ F\cdot k = k\}$ (**is** $F = \mathit{lub}\ ?P$)
proof (*rule below-antisym*)
show $\mathit{lub}\ ?P \sqsubseteq F$ **by** (*rule lattice-lub-least*) (*auto simp*: *is-ub-def semi-factor-principal-below-iff*)
show $F \sqsubseteq \mathit{lub}\ ?P$
proof (*simp only*: *semi-factor-below-iff-p*, *rule cfun-belowI*)
fix x
let $?Z = \mathit{lub}\ ?P$ **and** $?y = \mathit{semi\text{-}factor\text{-}p}\ F\cdot x$
have *fix-Z*: $\mathit{semi\text{-}factor\text{-}p}\ ?Z\cdot k = k$ **if** $\mathit{compact}\ k\ \mathit{semi\text{-}factor\text{-}p}\ F\cdot k = k$ **for** k
proof −
have $\mathit{semi\text{-}factor\text{-}principal}\ k \sqsubseteq ?Z$ **by** (*rule lattice-lub-upper*) (*use that* **in** *blast*)
then show *?thesis* **by** (*simp add*: *semi-factor-principal-below-iff*)
qed
obtain C **where** C: $\mathit{chain}\ C\ \forall i.\ \mathit{compact}\ (C\ i)\ ?y = (\bigsqcup i.\ C\ i)$ **by** (*rule obtain-compact-chain*)
have *fixF*: $\mathit{semi\text{-}factor\text{-}p}\ F\cdot(C\ i) = C\ i$ **for** i
proof −
have $C\ i \sqsubseteq \mathit{semi\text{-}factor\text{-}p}\ F\cdot x$ **using** *is-ub-thelub*[*OF C(1)*] *C(3)* **by** *simp*
then show *?thesis* **by** (*rule stable-defl.lower-fixed*[*OF stable-defl-semi-factor-p*])
qed
have $\mathit{semi\text{-}factor\text{-}p}\ ?Z\cdot ?y = (\bigsqcup i.\ \mathit{semi\text{-}factor\text{-}p}\ ?Z\cdot(C\ i))$
unfolding *C(3)* **by** (*rule contlub-cfun-arg*[*OF C(1)*])
also have $\ldots = (\bigsqcup i.\ C\ i)$ **using** *C(2) fixF* **by** (*simp add*: *fix-Z*)
also have $\ldots = ?y$ **by** (*simp add*: *C(3)*)
finally have *yZ*: $\mathit{semi\text{-}factor\text{-}p}\ ?Z\cdot ?y = ?y$ **.**
have $?y \sqsubseteq x$ **by** (*rule deflation.below*[*OF deflation-semi-factor-p*])
then have $\mathit{semi\text{-}factor\text{-}p}\ ?Z\cdot ?y \sqsubseteq \mathit{semi\text{-}factor\text{-}p}\ ?Z\cdot x$ **by** (*rule monofun-cfun-arg*)
with *yZ* **show** $\mathit{semi\text{-}factor\text{-}p}\ F\cdot x \sqsubseteq \mathit{semi\text{-}factor\text{-}p}\ ?Z\cdot x$ **by** *simp*
qed
qed

lemma *semi-factor-algebraic*: $F = \mathit{lub}\ \{K.\ \mathit{compact}\ K \wedge K \sqsubseteq F\}$ **for** $F :: {'a}{::}\mathit{dI}$ *semi-factor*

proof (*rule below-antisym*)
have *lub* {*semi-factor-principal k* | *k*. *compact k* ∧ *semi-factor-p F*·*k* = *k*} ⊑ *lub* {*K*. *compact K* ∧ *K* ⊑ *F*}
by (*rule lattice-lub-mono*)
(*auto intro*: *compact-semi-factor-principal simp*: *semi-factor-principal-below-iff*)
then show *F* ⊑ *lub* {*K*. *compact K* ∧ *K* ⊑ *F*}
by (*rule below-trans*[*OF eq-imp-below*[*OF semi-factor-lub-principals*]])
show *lub* {*K*. *compact K* ∧ *K* ⊑ *F*} ⊑ *F* **by** (*rule lattice-lub-least*) (*auto simp*: *is-ub-def*)
qed

A compact semi-factor is the join of finitely many compact principals: enumerate the compacts of its image, and the joins of initial segments form a chain with lub the semi-factor.

lemma *compact-semi-factor-finite-join*:
assumes *K*: *compact K*
shows ∃ *A*. *finite A* ∧ *A* ⊆ {*k*. *compact k*} ∧ *K* = *lub* (*semi-factor-principal* ‘ *A*)
proof −
let *?P* = {*k*. *compact k* ∧ *semi-factor-p K*·*k* = *k*}
have *semi-factor-p K*·⊥ = ⊥ **by** (*rule bottomI*, *rule deflation.below*[*OF deflation-semi-factor-p*])
then have *neP*: *?P* ≠ {} **by** (*intro exI*[*of* - ⊥] *notI*) *simp*
have *cP*: *countable ?P* **by** (*rule countable-subset*[*OF* - *countable-compacts*]) *auto*
define *e* **where** *e* = *from-nat-into ?P*
have *e*: *range e* = *?P* **unfolding** *e-def* **by** (*rule range-from-nat-into*[*OF neP cP*])
have *eP*: *compact* (*e i*) ∧ *semi-factor-p K*·(*e i*) = *e i* **for** *i* **using** *rangeI*[*of e i*] *e* **by** *blast*
define *G* **where** *G n* = *lub* (*semi-factor-principal* ‘ *e* ‘ {..*n*}) **for** *n*
have *chG*: *chain G* **unfolding** *G-def* **by** (*rule chainI*, *rule lattice-lub-mono*) *auto*
have *GK*: *G n* ⊑ *K* **for** *n*
unfolding *G-def* **by** (*rule lattice-lub-least*) (*auto simp*: *is-ub-def semi-factor-principal-below-iff eP*)
have *K* ⊑ (⨆ *n*. *G n*)
proof −
have *K* = *lub* {*semi-factor-principal k* | *k*. *compact k* ∧ *semi-factor-p K*·*k* = *k*}
by (*rule semi-factor-lub-principals*)
also have ... ⊑ (⨆ *n*. *G n*)
proof (*rule lattice-lub-least*, *rule is-ubI*)
fix *X* **assume** *X* ∈ {*semi-factor-principal k* | *k*. *compact k* ∧ *semi-factor-p K*·*k* = *k*}
then obtain *k* **where** *X*: *X* = *semi-factor-principal k* **and** *k*: *k* ∈ *?P* **by** *blast*
then have *k* ∈ *range e* **using** *e* **by** *simp*
then obtain *n* **where** *n*: *k* = *e n* **by** *blast*
have *X* ⊑ *G n* **unfolding** *X n G-def* **by** (*rule lattice-lub-upper*) *simp*
also have *G n* ⊑ (⨆ *n*. *G n*) **by** (*rule is-ub-thelub*[*OF chG*])
finally show *X* ⊑ (⨆ *n*. *G n*) **.**

qed
finally show *?thesis* **.**
qed
then obtain *n* **where** $K \sqsubseteq G\ n$ **using** *compactD2*[*OF K chG*] **by** *blast*
then have $K = G\ n$ **using** *GK*[*of n*] **by** (*rule below-antisym*)
then show *?thesis* **unfolding** *G-def* **by** (*intro exI*[*of - e ‘ {..n}*] *conjI*) (*auto simp*: *eP*)
qed

lemma *countable-compact-semi-factors*: *countable* $\{K :: 'a::dI\ \textit{semi-factor}.\ \textit{compact}\ K\}$
proof (*rule countable-subset*)
show $\{K.\ \textit{compact}\ K\} \subseteq (\lambda A.\ \textit{lub}\ (\textit{semi-factor-principal} ‘ A)) ‘ \{A.\ \textit{finite}\ A \wedge A \subseteq \{k :: 'a.\ \textit{compact}\ k\}\}$
using *compact-semi-factor-finite-join* **by** *blast*
show *countable* $((\lambda A.\ \textit{lub}\ (\textit{semi-factor-principal} ‘ A)) ‘ \{A.\ \textit{finite}\ A \wedge A \subseteq \{k :: 'a.\ \textit{compact}\ k\}\})$
by (*rule countable-image*, *rule countable-Collect-finite-subset*, *rule countable-compacts*)
qed

lemma *semi-factor-compacts-enum*: $\exists\, c :: \textit{nat} \Rightarrow 'a::dI\ \textit{semi-factor}.\ \textit{range}\ c = \{K.\ \textit{compact}\ K\}$
using *range-from-nat-into*[*OF - countable-compact-semi-factors*] *compact-bottom*
by *blast*

instance *semi-factor* :: (*dI*) *bifinite*
by (*rule bifinite-algebraic-lattice*[*OF semi-factor-compacts-enum semi-factor-algebraic*])

end
theory *ValsetBase* **imports** *HOLCF HOLCF−extra.Distributive Subdomain*
begin
default-sort *type*
class *two* = **assumes** *exTwo*: $\exists\, x\ y :: 'a.\ x \neq y$
begin

lemma *obtain-neq*: **obtains** $y :: 'a$ **where** $x \neq y$
proof −
from *exTwo* **obtain** $a\ b :: 'a$ **where** $a \neq b$ **by** *blast*
consider $x = a \mid x \neq a$ **by** *fast*
then show *thesis* **apply** *cases* **using** ‹$a \neq b$› *that* **by** *simp-all*
qed

end

pcpodef ('*a*::*type*, '*b*) *tuple* = *UNIV* :: ('*a* ⇒ '*b*::*pcpo*) *set*
morphisms *tget Abs-tuple*
by *simp-all*

lemma *Abs-tuple-inverse'*: *tget* (*Abs-tuple t*) = *t* **using** *Abs-tuple-inverse* **by** *simp*

definition *ctget* ≡ Λ *t a*. *tget t* (*undiscr a*)

lemma *ctget-beta*: *ctget*·*t*·*a* = *tget t* (*undiscr a*)
unfolding *ctget-def*
by (*simp add*: *cont2cont-fun cont-tget*)

definition *ctabs* ≡ Λ *f*. *Abs-tuple* (λ*a*. *f*·(*Discr a*))

lemma *ctabs-beta*: *ctabs*·*f* = *Abs-tuple* (λ*a*. *f*·(*Discr a*))
unfolding *ctabs-def*
by (*simp add*: *cont-apply cont-Abs-tuple*)

lemma *ctabs-ctget-inverse*[*simp*]: *ctabs*·(*ctget*·*t*) = *t*
by (*simp add*: *ctabs-beta ctget-beta tget-inverse*)

lemma *ctget-ctabs-inverse*[*simp*]: *ctget*·(*ctabs*·*t*) = *t*
by (*intro cfun-eqI*) (*simp add*: *ctget-beta ctabs-beta Abs-tuple-inverse'*)

lemma *approx-chain-tuple*:
approx-chain a ⟹ *approx-chain* (λ*i*. *ctabs oo* (*a i*) *oo ctget*)
by (*simp add*: *approx-chain-iso*)

instance *tuple* :: (*countable*, *bifinite*) *bifinite*
proof
show ∃ *a*::*nat*⇒('*a*, '*b*) *tuple*→('*a*, '*b*) *tuple*. *approx-chain a*
using *approx-chain-tuple bifinite* **by** *blast*
qed

instantiation *tuple* :: (*countable*, *domain*) *domain*
begin
definition
emb = *emb oo ctget*
definition
prj = *ctabs oo prj*
definition
defl (*t* :: ('*a*, '*b*) *tuple itself*) = *DEFL*('*a discr* → '*b*)
definition
(*liftemb* :: ('*a*, '*b*) *tuple u* → *udom u*) = *u-map*·*emb*
definition
(*liftprj* :: *udom u* → ('*a*, '*b*) *tuple u*) = *u-map*·*prj*
definition
liftdefl (*t* :: ('*a*, '*b*) *tuple itself*) = *liftdefl-of*·(*DEFL*(('*a*, '*b*) *tuple*))

instance proof
have *ep-pair ctget ctabs* **by** *standard simp-all*
then show *ep-pair emb* (*prj* :: *udom* → ('*a*, '*b*) *tuple*)
unfolding *emb-tuple-def prj-tuple-def*
by (*intro ep-pair-comp ep-pair-emb-prj*)

show *cast·DEFL*((*'a*, *'b*) *tuple*) = *emb oo* (*prj* :: *udom* → (*'a*, *'b*) *tuple*)
unfolding *emb-tuple-def prj-tuple-def defl-tuple-def*
by (*simp add*: *cast-DEFL cfcomp1*)
qed (*fact liftemb-tuple-def liftprj-tuple-def liftdefl-tuple-def*)+

end

notation
tget (‹(‹*notation*=‹*infix tget*››-⟨-⟩)› [*999*, *0*] *999*)

definition *tupd* :: *'a* ⇒ (*'a*, *'b*::*pcpo*) *tuple* → *'b* → (*'a*, *'b*) *tuple* **where**
tupd a ≡ Λ *t b*. *Abs-tuple* ((*tget t*)(*a* := *b*))

lemma *tupd-beta*: *tupd a·t·b* = *Abs-tuple* ((*tget t*)(*a* := *b*))
unfolding *tupd-def*
by (*simp add*: *cont2cont-fun cont-apply cont-tget cont-Abs-tuple*)

lemma *tupd-fupd*: *tget* (*tupd a·t·b*) = (*tget t*)(*a* := *b*)
by (*simp add*: *tupd-beta Abs-tuple-inverse'*)

definition *tcut* (*A* :: *'a*::*type set*) ≡ Λ *t*. *Abs-tuple* (λ*a*. *if a* ∈ *A then tget t a else* ⊥)

lemma *tcut-beta*: *tcut A·t* = *Abs-tuple* (λ*a*. *if a* ∈ *A then tget t a else* ⊥)
unfolding *tcut-def*
by (*simp add*: *cont2cont-fun cont-tget cont-Abs-tuple*)

lemma *tcut-tget*: *tcut A·t*⟨*a*⟩ = (*if a* ∈ *A then tget t a else* ⊥)
by (*simp add*: *tcut-beta Abs-tuple-inverse'*)

lemma *tcut-mem*[*simp*]: *a* ∈ *A* ⟹ *tcut A·t*⟨*a*⟩ = *t*⟨*a*⟩
by (*simp add*: *tcut-tget*)

lemma *tcut-nomem*[*simp*]: *a* ∉ *A* ⟹ *tcut A·t*⟨*a*⟩ = ⊥
by (*simp add*: *tcut-tget*)

lemma *tuple-eqI*: (⋀*x*. *a*⟨*x*⟩ = *b*⟨*x*⟩) ⟹ *a* = *b* **unfolding** *tget-inject*[*symmetric*]
by *auto*

lemma *tuple-bounded-complete*:
fixes *A* :: (*'a*, *'b*::*bounded-complete*) *tuple set* **and** *b*
assumes *A* <| *b*
shows *A* <<| *Abs-tuple* (*lub* (*tget* ‘ *A*))
proof −
from *assms* **have** *ub*: *tget* ‘ *A* <| *tget b* **by** (*auto simp*: *is-ub-def below-tuple-def*)
from *ex-bound-lub*[*OF ub*] **have** *lubA*: *tget* ‘ *A* <<| *lub* (*tget* ‘ *A*) **.**
show *A* <<| *Abs-tuple* (*lub* (*tget* ‘ *A*))
proof (*rule is-lubI*)
show *A* <| *Abs-tuple* (*lub* (*tget* ‘ *A*))

```
    proof (rule is-ubI)
      fix t assume t ∈ A
      hence tget t ⊑ lub (tget ‘ A) using lubA by (meson imageI is-lubD1 is-ubD)
    thus t ⊑ Abs-tuple (lub (tget ‘ A)) by (simp add: below-tuple-def Abs-tuple-inverse')
    qed
  next
    fix u assume A <| u
    hence tget ‘ A <| tget u by (auto simp: is-ub-def below-tuple-def)
    with lubA have lub (tget ‘ A) ⊑ tget u by (rule is-lubD2)
   thus Abs-tuple (lub (tget ‘ A)) ⊑ u by (simp add: below-tuple-def Abs-tuple-inverse')
  qed
qed

instance tuple :: (type, bounded-complete) bounded-complete
  by intro-classes (blast intro: tuple-bounded-complete)

lemma tuple-lub:
  fixes A :: ('a, 'b::bounded-complete) tuple set
  assumes A <| b
  shows lub A = Abs-tuple (lub (tget ‘ A))
  using lub-eqI[OF tuple-bounded-complete, OF assms] .

lemma tget-join:
  fixes a b :: ('a, 'b::bounded-complete) tuple
  assumes a ⇑ b
  shows tget (a ⊔ b) = (λp. tget a p ⊔ tget b p)
proof −
  from assms obtain z where az: a ⊑ z and bz: b ⊑ z by (auto simp: joinable-def)
  hence abf: tget a ⇑ tget b by (auto simp: below-tuple-def intro: joinableI)
  from az bz have zub: {a, b} <| z by (auto simp: is-ub-def)
  have tget (a ⊔ b) = lub (tget ‘ {a, b})
    unfolding join-def using zub by (simp add: tuple-lub Abs-tuple-inverse')
  also have ... = tget a ⊔ tget b by (simp add: join-def)
  also have ... = (λp. tget a p ⊔ tget b p) using abf by (rule fun-join)
  finally show ?thesis .
qed

lemma tget-meet:
  fixes a b :: ('a, 'b::bounded-complete) tuple
  shows tget (a ⊓ b) = (λp. tget a p ⊓ tget b p)
proof −
  let ?X = Abs-tuple (λp. tget a p ⊓ tget b p)
  have le: tget (a ⊓ b) p ⊑ tget a p tget (a ⊓ b) p ⊑ tget b p for p
    using meet-below1[of a b, unfolded below-tuple-def, THEN fun-belowD]
          meet-below2[of a b, unfolded below-tuple-def, THEN fun-belowD] by auto
  have a ⊓ b = ?X
  proof (rule below-antisym)
    show a ⊓ b ⊑ ?X unfolding below-tuple-def Abs-tuple-inverse'
      by (auto simp: fun-below-iff le intro: meet-greatest)
```

```
    show ?X ⊑ a ⊓ b
    proof (rule meet-greatest)
      show ?X ⊑ a unfolding below-tuple-def Abs-tuple-inverse'
        by (auto simp: fun-below-iff meet-below1)
      show ?X ⊑ b unfolding below-tuple-def Abs-tuple-inverse'
        by (auto simp: fun-below-iff meet-below2)
    qed
  qed
  thus ?thesis by (simp add: Abs-tuple-inverse')
qed

instance tuple :: (type, distributive-cpo) distributive-cpo
proof
  fix a b c :: ('a, 'b) tuple
  assume ab: a ⇑ b
  hence ptw: tget a p ⇑ tget b p for p
    by (auto simp: joinable-def below-tuple-def intro: joinableI dest: fun-belowD)
  have cab: (c ⊓ a) ⇑ (c ⊓ b) by (rule joinableI[OF meet-below1 meet-below1])
  have tget (c ⊓ (a ⊔ b)) = (λp. tget c p ⊓ (tget a p ⊔ tget b p))
    using ab by (simp add: tget-meet tget-join)
  also have ... = (λp. (tget c p ⊓ tget a p) ⊔ (tget c p ⊓ tget b p))
    using ptw by (simp add: meet-join-distrib)
  also have ... = tget ((c ⊓ a) ⊔ (c ⊓ b))
    using cab by (simp add: tget-join tget-meet)
  finally show c ⊓ (a ⊔ b) = (c ⊓ a) ⊔ (c ⊓ b) by (simp add: tget-inject)
qed

instance tuple :: (countable, scott-domain) scott-domain ..

interpretation tcut-deflation: deflation tcut A for A
proof
  fix t :: ('a, 'b) tuple
  show tcut A·(tcut A·t) = tcut A·t
    by (rule tuple-eqI) (simp add: tcut-tget)
  show tcut A·t ⊑ t
    unfolding below-tuple-def below-fun-def
    by (simp add: tcut-tget)
qed

lemmas tcut-deflation = tcut-deflation.deflation-axioms

lemma tcut-mono: A ⊆ B ⟹ tcut A ⊑ tcut B
  unfolding below-cfun-def below-fun-def below-tuple-def
  by (auto simp add: tcut-tget)

lemma tcut-bot[simp]: tcut {} = ⊥
  by (intro cfun-eqI tuple-eqI) (simp add: tcut-tget tget-strict)

lemma deflation-le-cut:
```

assumes *deflation g*
deflation h
$g \sqsubseteq h$
and *fg*: $f = f$ *oo* g
shows
$f = f$ *oo* h
proof −
from *assms* **have** $g{\cdot}(h{\cdot}x) = g{\cdot}x$ **for** x **by** (*auto intro*: *deflation-below-comp1*)
then have *1*: g *oo* $h = g$ **by** (*simp add*: *cfun-eq-iff*)
from *fg* **have** f *oo* $h = (f$ *oo* $g)$ *oo* h **by** *simp*
also have ... = f *oo* $(g$ *oo* $h)$ **by** (*simp add*: *cfun-eq-iff*)
also note *1*
also note *fg*[*symmetric*]
finally show *?thesis* **..**
qed

default-sort *type*

definition *functions-with-support* **where**
functions-with-support = $\{(A, f).\ f = f$ *oo* *tcut* $A\}$

typedef $('a, 'b{::}pcpo, 'c{::}cpo)$ *tattr* = *functions-with-support* :: $('a$ *set* $\times$ $(('a, 'b)$ *tuple* $\to 'c))$ *set*
proof
show $(\{\}{::}'a$ *set*, $(\Lambda\ {-}.c)) \in$ *functions-with-support* **for** c
by (*simp add*: *functions-with-support-def cfun-eq-iff*)
qed

lemma *tattr-liftI*[*intro*]: $(\bigwedge x.\ f{\cdot}x = f{\cdot}(tcut\ A{\cdot}x)) \Longrightarrow (A, f) \in$ *functions-with-support*
unfolding *functions-with-support-def* **by** (*simp add*: *cfun-eq-iff*)

lemma *tattr-liftE*[*elim*]: $p \in$ *functions-with-support* $\Longrightarrow$
$(\bigwedge A\ f.\ p = (A, f) \Longrightarrow f = f$ *oo* *tcut* $A \Longrightarrow$ *thesis*$) \Longrightarrow$ *thesis*
unfolding *functions-with-support-def*
by (*simp split*: *prod.splits*)

setup-lifting *type-definition-tattr*

lift-definition *tattr-const* :: $'c \Rightarrow ('a, 'b{::}pcpo, 'c{::}cpo)$ *tattr* **is** $\lambda x.\ (\{\}, (\Lambda\ {-}.x))$
by *auto*

lemma *tattr-Un*: $f = f$ *oo* *tcut* $A \Longrightarrow f = f$ *oo* *tcut* $(A{\cup}B)$
by (*rule deflation-le-cut*[*of tcut A*]) (*auto simp add*: *tcut-deflation intro*: *tcut-mono*)

lemma *tattr-Un'*: $f = f$ *oo* *tcut* $A \Longrightarrow f{\cdot}x = f{\cdot}(tcut\ (A{\cup}B){\cdot}x)$
by (*drule tattr-Un*[*of f A B*]) (*simp add*: *cfun-eq-iff*)

lift-definition *tattr-slice* :: $'a$ *set* $\Rightarrow ('a, 'b{::}pcpo, 'c{::}cpo)$ *tattr* $\Rightarrow ('a, 'b, 'c)$ *tattr*
is $\lambda B\ (A, f).\ (A{\cup}B, f)$

by (*auto intro*: *tattr-Un'*)

lemma *cont-tget-fix*[*simp*]: $(\Lambda\ t.\ t\langle a\rangle)\cdot t = t\langle a\rangle$ **by** (*simp add*: *cont2cont-fun cont-tget*)

lift-definition *tattr-dim* :: $'a \Rightarrow ('a,\ 'b{::}pcpo,\ 'b)$ *tattr*
is $\lambda a.\ (\{a\},\ (\Lambda\ t.\ t\langle a\rangle))$
by (*auto intro*!: *cfun-eqI*)

lemma *tattr-apply*:
assumes *f1* = *f1 oo tcut A*
f2 = *f2 oo tcut B*
shows $(\Lambda\ t.\ f1{\cdot}t{\cdot}(f2{\cdot}t)){\cdot}t = (\Lambda\ t.\ f1{\cdot}t{\cdot}(f2{\cdot}t)){\cdot}(tcut\ (A{\cup}B){\cdot}t)$
using *tattr-Un'*[*OF assms*(*1*), *of t B*] *tattr-Un'*[*OF assms*(*2*), *of t A*]
by (*simp add*: *sup-commute*)

lift-definition *tattr-apply* :: $('a,\ 'b{::}pcpo,\ 'c{::}cpo \to 'd{::}cpo)$ *tattr* $\Rightarrow$ $('a,\ 'b,\ 'c)$ *tattr*
$\Rightarrow$ $('a,\ 'b,\ 'd)$ *tattr*
is $\lambda(A,\ f1)\ (B,\ f2).\ (A{\cup}B,\ (\Lambda\ t.\ f1{\cdot}t{\cdot}(f2{\cdot}t)))$
by (*auto intro*: *tattr-apply*)

lift-definition *tattr-dims* :: $('a,\ 'b{::}pcpo,\ 'c{::}cpo)$ *tattr* $\Rightarrow$ $'a$ *set*
is *fst*.

lift-definition *tattr-eval* :: $('a,\ 'b{::}pcpo,\ 'c{::}cpo)$ *tattr* $\Rightarrow$ $('a,\ 'b)$ *tuple* $\to$ $'c$
is *snd* .

abbreviation *tattr-eval'* *t f* $\equiv$ *tattr-eval f*$\cdot$*t*
abbreviation *tattr-slice'* *f B* $\equiv$ *tattr-slice B f*

notation
tattr-const (‹(‹*notation*=‹*prefix tattr-const*››⋆-)› [*1000*]*1000*)
and *tattr-dim* (‹(‹*notation*=‹*postfix tattr-const*››⋆⟨-⟩)› [*0*]*1000*)
and *tattr-slice'* (**infixl** ‹⋆'/› *300*)
and *tattr-apply* (**infixl** ‹⋆$› *200*)
and *tattr-eval'* (**infix** ‹*eval*› *100*)
and *tattr-dims* (‹$\mathcal{D}$›)

lemma *tattr-eval-const*[*simp*]: *t eval* ⋆*c* = *c*
by *transfer' simp*

lemma *tattr-eval-dim*[*simp*]: *t eval* ⋆⟨*a*⟩ = *t*⟨*a*⟩
by *transfer' simp*

lemma *tattr-eval-slice*[*simp*]: *t eval f* ⋆/ *A* = *t eval f*
by *transfer'* (*simp split*: *prod.splits*)

lemma *tattr-eval-apply*[*simp*]: *t eval* (*a* ⋆$ *b*) = (*t eval a*)$\cdot$(*t eval b*)
by *transfer'* (*simp split*: *prod.splits*)

lemma *tattr-dims-const*[*simp*]: $\mathcal{D}(\star c) = \{\}$
by *transfer′ simp*

lemma *tattr-dims-dim*[*simp*]: $\mathcal{D}(\star\langle d\rangle) = \{d\}$
by *transfer′ simp*

lemma *tattr-dims-slice*[*simp*]: $\mathcal{D}(f \star/ A) = \mathcal{D}(f) \cup A$
by *transfer′* (*simp split*: *prod.splits*)

lemma *tattr-dims-apply*[*simp*]: $\mathcal{D}(f \star\$ g) = \mathcal{D}(f) \cup \mathcal{D}(g)$
by *transfer′* (*simp split*: *prod.splits*)

lemmas *tattr-eval-simps* =
tattr-eval-const
tattr-eval-dim
tattr-eval-slice
tattr-eval-apply

lemmas *tattr-dims-simps* =
tattr-dims-const
tattr-dims-dim
tattr-dims-slice
tattr-dims-apply

11.3 Componentwise reasoning on tuples

lemma *tuple-below-iff*: $x \sqsubseteq y \longleftrightarrow (\forall\, d.\ x\langle d\rangle \sqsubseteq y\langle d\rangle)$
by (*simp add*: *below-tuple-def fun-below-iff*)

lemma *tuple-eq-iff*: $x = y \longleftrightarrow (\forall\, d.\ x\langle d\rangle = y\langle d\rangle)$
by (*auto simp*: *po-eq-conv tuple-below-iff*)

lemma *tuple-joinableI*:
fixes $x\ y$ :: ('*d*, '*v*::*bounded-complete*) *tuple*
assumes $\bigwedge d.\ x\langle d\rangle \Uparrow y\langle d\rangle$
shows $x \Uparrow y$
proof (*rule joinableI*)
let *?z* = *Abs-tuple* $(\lambda d.\ x\langle d\rangle \sqcup y\langle d\rangle)$ :: ('*d*, '*v*) *tuple*
show $x \sqsubseteq$ *?z* $y \sqsubseteq$ *?z*
using *joinableD*[*OF assms*] **by** (*simp-all add*: *tuple-below-iff Abs-tuple-inverse′*)
qed

The single-key record *tupd* $d{\cdot}\bot{\cdot}x$.

lemma *tupd-bot-tget*: $(tupd\ d{\cdot}\bot{\cdot}x)\langle e\rangle = (if\ e = d\ then\ x\ else\ \bot)$
by (*simp add*: *tupd-fupd tget-strict*)

The componentwise map of a family of functions.

definition *tmap* :: ('*d* ⇒ '*v*::*pcpo* → '*v*) ⇒ ('*d*, '*v*) *tuple* → ('*d*, '*v*) *tuple* **where**
tmap f = (Λ *t*. *Abs-tuple* $(\lambda d.\ f\ d{\cdot}(t\langle d\rangle))$)

lemma *tmap-tget* [*simp*]: $(tmap\ f{\cdot}t)\langle d\rangle = f\ d{\cdot}(t\langle d\rangle)$
unfolding *tmap-def*
by (*subst beta-cfun*)
(*simp-all add*: *cont-Abs-tuple cont2cont-lambda cont2cont-fun cont-tget Abs-tuple-inverse'*)

The component at d of a function on tuples, read off the single-key records.

definition *tcomp* :: $'d \Rightarrow (('d, 'v{::}pcpo)\ tuple \to ('d, 'v)\ tuple) \Rightarrow 'v \to 'v$ **where**
$tcomp\ d\ p = (\Lambda\ t.\ t\langle d\rangle)\ oo\ p\ oo\ tupd\ d{\cdot}\bot$

lemma *tcomp-beta*: $tcomp\ d\ p{\cdot}x = (p{\cdot}(tupd\ d{\cdot}\bot{\cdot}x))\langle d\rangle$
by (*simp add*: *tcomp-def*)

lemma *tcomp-tmap* [*simp*]: $tcomp\ d\ (tmap\ f) = f\ d$
by (*rule cfun-eqI*) (*simp add*: *tcomp-beta tupd-bot-tget*)

11.4 Semi-factors of the tuple domain

Records are ordered pointwise and their keys are independent, so a semi-factor of the tuples acts on each key separately: it is the componentwise map of its components, and these are semi-factors. Hence the semi-factors of the tuples are the families of semi-factors.

lemma *semi-factor-defl-tmap*:
fixes $f :: 'd{::}countable \Rightarrow 'v{::}\{bifinite, distributive\text{-}cpo\} \to 'v$
assumes f: $\bigwedge d.\ semi\text{-}factor\text{-}defl\ (f\ d)$
shows $semi\text{-}factor\text{-}defl\ (tmap\ f)$
proof (*rule semi-factor-deflI*)
have *sf*: $stable\text{-}defl\ (f\ d)$ **for** d **by** (*rule semi-factor-defl.axioms*(*1*)[*OF f*])
note $df = stable\text{-}defl.deflation[OF\ sf]$
show $stable\text{-}defl\ (tmap\ f)$
proof *standard*
fix t **show** $tmap\ f{\cdot}(tmap\ f{\cdot}t) = tmap\ f{\cdot}t$ **by** (*simp add*: *tuple-eq-iff deflation.idem*[*OF df*])
next
fix t **show** $tmap\ f{\cdot}t \sqsubseteq t$ **by** (*simp add*: *tuple-below-iff deflation.below*[*OF df*])
next
fix $x\ y :: ('d, 'v)\ tuple$ **assume** $tmap\ f{\cdot}y = y\ x \sqsubseteq y$
then show $tmap\ f{\cdot}x = x$
by (*auto simp*: *tuple-eq-iff tuple-below-iff intro*: *stable-defl.downclosed*[*OF sf*])
qed
fix $x\ y :: ('d, 'v)\ tuple$ **assume** *y*: $tmap\ f{\cdot}y = y$ **and** *le*: $tmap\ f{\cdot}x \sqsubseteq y$
show $x \Uparrow y$
proof (*rule tuple-joinableI*)
fix d
from y **have** $(tmap\ f{\cdot}y)\langle d\rangle = y\langle d\rangle$ **by** *simp*
then have $f\ d{\cdot}(y\langle d\rangle) = y\langle d\rangle$ **by** *simp*
moreover from *le* **have** $f\ d{\cdot}(x\langle d\rangle) \sqsubseteq y\langle d\rangle$ **by** (*simp add*: *tuple-below-iff*)
ultimately show $x\langle d\rangle \Uparrow y\langle d\rangle$ **by** (*rule semi-factor-defl.slide*[*OF f*])

qed
qed

lemma *deflation-tupd*:
fixes p :: ('d, 'v::*pcpo*) *tuple* → ('d, 'v) *tuple*
assumes p: *deflation p*
shows $p{\cdot}(tupd\ d{\cdot}\bot{\cdot}x) = tupd\ d{\cdot}\bot{\cdot}(tcomp\ d\ p{\cdot}x)$
proof (*rule tuple-eqI*)
fix e
have $(p{\cdot}(tupd\ d{\cdot}\bot{\cdot}x))\langle e\rangle \sqsubseteq (tupd\ d{\cdot}\bot{\cdot}x)\langle e\rangle$
using *deflation.below*[*OF p*] **by** (*simp add*: *tuple-below-iff*)
then show $(p{\cdot}(tupd\ d{\cdot}\bot{\cdot}x))\langle e\rangle = (tupd\ d{\cdot}\bot{\cdot}(tcomp\ d\ p{\cdot}x))\langle e\rangle$
by (*cases* $e = d$) (*simp-all add*: *tupd-bot-tget tcomp-beta*)
qed

lemma *stable-defl-tget*:
fixes p :: ('d::*countable*, 'v::{*bifinite*, *distributive-cpo*}) *tuple* → ('d, 'v) *tuple*
assumes p: *stable-defl p*
shows $(p{\cdot}t)\langle d\rangle = tcomp\ d\ p{\cdot}(t\langle d\rangle)$
proof (*rule below-antisym*)
interpret P: *stable-defl p* **by** (*rule p*)
let *?s* = $tupd\ d{\cdot}\bot{\cdot}(t\langle d\rangle)$ **and** *?u* = $tupd\ d{\cdot}\bot{\cdot}((p{\cdot}t)\langle d\rangle)$
have *?u* ⊑ $p{\cdot}t$ **by** (*simp add*: *tuple-below-iff tupd-bot-tget*)
then have $p{\cdot}$*?u* = *?u* **by** (*rule P.lower-fixed*)
moreover have *?u* ⊑ *?s* **using** *P.below*[*of t*] **by** (*simp add*: *tuple-below-iff tupd-bot-tget*)
ultimately have *?u* ⊑ $p{\cdot}$*?s* **by** (*rule P.below-image*)
then have $?u\langle d\rangle \sqsubseteq (p{\cdot}?s)\langle d\rangle$ **by** (*simp add*: *tuple-below-iff*)
then show $(p{\cdot}t)\langle d\rangle \sqsubseteq tcomp\ d\ p{\cdot}(t\langle d\rangle)$ **by** (*simp add*: *tupd-bot-tget tcomp-beta*)
have *?s* ⊑ *t* **by** (*simp add*: *tuple-below-iff tupd-bot-tget*)
then have $p{\cdot}$*?s* ⊑ $p{\cdot}t$ **by** (*rule monofun-cfun-arg*)
then have $(p{\cdot}?s)\langle d\rangle \sqsubseteq (p{\cdot}t)\langle d\rangle$ **by** (*simp add*: *tuple-below-iff*)
then show $tcomp\ d\ p{\cdot}(t\langle d\rangle) \sqsubseteq (p{\cdot}t)\langle d\rangle$ **by** (*simp add*: *tcomp-beta*)
qed

lemma *tmap-tcomp*:
fixes p :: ('d::*countable*, 'v::{*bifinite*, *distributive-cpo*}) *tuple* → ('d, 'v) *tuple*
assumes *stable-defl p* **shows** $tmap\ (\lambda d.\ tcomp\ d\ p) = p$
by (*rule cfun-eqI*) (*simp add*: *tuple-eq-iff stable-defl-tget*[*OF assms*])

lemma *semi-factor-defl-tcomp*:
fixes p :: ('d::*countable*, 'v::{*bifinite*, *distributive-cpo*}) *tuple* → ('d, 'v) *tuple*
assumes p: *semi-factor-defl p*
shows *semi-factor-defl* (*tcomp d p*)
proof (*rule semi-factor-deflI*)
interpret P: *semi-factor-defl p* **by** (*rule p*)
note *single* = *deflation-tupd*[*OF P.deflation*]
show *stable-defl* (*tcomp d p*)
proof *standard*

fix x
have $tcomp\ d\ p{\cdot}(tcomp\ d\ p{\cdot}x) = (p{\cdot}(tupd\ d{\cdot}\bot{\cdot}(tcomp\ d\ p{\cdot}x)))\langle d\rangle$ **by** (*rule tcomp-beta*)
also have $\ldots = (p{\cdot}(p{\cdot}(tupd\ d{\cdot}\bot{\cdot}x)))\langle d\rangle$ **by** (*simp only*: *single*)
also have $\ldots = tcomp\ d\ p{\cdot}x$ **by** (*simp add*: *P.idem tcomp-beta*)
finally show $tcomp\ d\ p{\cdot}(tcomp\ d\ p{\cdot}x) = tcomp\ d\ p{\cdot}x$ **.**
next
fix x
have $(p{\cdot}(tupd\ d{\cdot}\bot{\cdot}x))\langle d\rangle \sqsubseteq (tupd\ d{\cdot}\bot{\cdot}x)\langle d\rangle$ **using** *P.below* **by** (*simp add*: *tuple-below-iff*)
then show $tcomp\ d\ p{\cdot}x \sqsubseteq x$ **by** (*simp add*: *tcomp-beta tupd-bot-tget*)
next
fix $x\ y$ **assume** y: $tcomp\ d\ p{\cdot}y = y$ **and** xy: $x \sqsubseteq y$
have $p{\cdot}(tupd\ d{\cdot}\bot{\cdot}y) = tupd\ d{\cdot}\bot{\cdot}y$ **by** (*simp add*: *single y*)
moreover have $tupd\ d{\cdot}\bot{\cdot}x \sqsubseteq tupd\ d{\cdot}\bot{\cdot}y$ **using** *xy* **by** (*rule monofun-cfun-arg*)
ultimately have $p{\cdot}(tupd\ d{\cdot}\bot{\cdot}x) = tupd\ d{\cdot}\bot{\cdot}x$ **by** (*rule P.downclosed*)
then show $tcomp\ d\ p{\cdot}x = x$ **by** (*simp add*: *tcomp-beta tupd-bot-tget*)
qed
fix $x\ y$ **assume** y: $tcomp\ d\ p{\cdot}y = y$ **and** *le*: $tcomp\ d\ p{\cdot}x \sqsubseteq y$
have *fy*: $p{\cdot}(tupd\ d{\cdot}\bot{\cdot}y) = tupd\ d{\cdot}\bot{\cdot}y$ **by** (*simp add*: *single y*)
have $p{\cdot}(tupd\ d{\cdot}\bot{\cdot}x) \sqsubseteq tupd\ d{\cdot}\bot{\cdot}y$ **unfolding** *single* **using** *le* **by** (*rule monofun-cfun-arg*)
with *fy* **have** *j*: $tupd\ d{\cdot}\bot{\cdot}x \Uparrow tupd\ d{\cdot}\bot{\cdot}y$ **by** (*rule P.slide*)
have $(tupd\ d{\cdot}\bot{\cdot}x)\langle d\rangle \sqsubseteq (tupd\ d{\cdot}\bot{\cdot}x \sqcup tupd\ d{\cdot}\bot{\cdot}y)\langle d\rangle$
$(tupd\ d{\cdot}\bot{\cdot}y)\langle d\rangle \sqsubseteq (tupd\ d{\cdot}\bot{\cdot}x \sqcup tupd\ d{\cdot}\bot{\cdot}y)\langle d\rangle$
using *joinableD*[*OF j*] **by** (*simp-all add*: *tuple-below-iff*)
then show $x \Uparrow y$ **by** (*simp add*: *tupd-bot-tget*) (*rule joinableI*)
qed

The isomorphism between families of semi-factors and semi-factors of the tuples.

definition *semi-factor-of-family* ::
$('d{::}countable \Rightarrow 'v{::}\{bifinite, distributive\text{-}cpo\}\ semi\text{-}factor) \Rightarrow ('d, 'v)\ tuple\ semi\text{-}factor$
where *semi-factor-of-family F* = *semi-factor-of-defl* (*tmap* ($\lambda d.$ *semi-factor-p* (*F d*)))

definition *family-of-semi-factor* ::
$('d{::}countable, 'v{::}\{bifinite, distributive\text{-}cpo\})\ tuple\ semi\text{-}factor \Rightarrow 'd \Rightarrow 'v\ semi\text{-}factor$
where *family-of-semi-factor G d* = *semi-factor-of-defl* (*tcomp d* (*semi-factor-p G*))

lemma *semi-factor-p-of-family*:
semi-factor-p (*semi-factor-of-family F*) = *tmap* ($\lambda d.$ *semi-factor-p* (*F d*))
unfolding *semi-factor-of-family-def*
by (*rule semi-factor-p-of-defl*, *rule semi-factor-defl-tmap*, *rule semi-factor-defl-semi-factor-p*)

lemma *semi-factor-p-family-of*:
semi-factor-p (*family-of-semi-factor G d*) = *tcomp d* (*semi-factor-p G*)
unfolding *family-of-semi-factor-def*

by (*rule semi-factor-p-of-defl*, *rule semi-factor-defl-tcomp*, *rule semi-factor-defl-semi-factor-p*)

lemma *family-of-semi-factor-of-family* [*simp*]: *family-of-semi-factor* (*semi-factor-of-family* F) = F
by (*rule ext*) (*simp add*: *semi-factor-p-inject*[*symmetric*] *semi-factor-p-family-of semi-factor-p-of-family*)

lemma *semi-factor-of-family-of-semi-factor* [*simp*]:
semi-factor-of-family (*family-of-semi-factor* G) = G
using *semi-factor-defl.axioms*(*1*)[*OF semi-factor-defl-semi-factor-p*[*of* G]]
by (*simp add*: *semi-factor-p-inject*[*symmetric*] *semi-factor-p-of-family semi-factor-p-family-of tmap-tcomp*)

lemma *tmap-below-iff*: *tmap* $f \sqsubseteq$ *tmap* $g \longleftrightarrow (\forall\, d.\ f\ d \sqsubseteq g\ d)$
proof
assume *fg*: *tmap* $f \sqsubseteq$ *tmap* g
show $\forall\, d.\ f\ d \sqsubseteq g\ d$
proof (*intro allI cfun-belowI*)
fix $d\ x$
have (*tmap* $f{\cdot}(tupd\ d{\cdot}\bot{\cdot}x))\langle d\rangle \sqsubseteq$ (*tmap* $g{\cdot}(tupd\ d{\cdot}\bot{\cdot}x))\langle d\rangle$
using *monofun-cfun-fun*[*OF fg*] **by** (*simp add*: *tuple-below-iff*)
then show $f\ d{\cdot}x \sqsubseteq g\ d{\cdot}x$ **by** (*simp add*: *tupd-bot-tget*)
qed
qed (*auto intro*!: *cfun-belowI simp*: *tuple-below-iff monofun-cfun-fun*)

lemma *semi-factor-of-family-below-iff*:
semi-factor-of-family $F \sqsubseteq$ *semi-factor-of-family* $F' \longleftrightarrow F \sqsubseteq F'$
by (*simp add*: *semi-factor-below-iff-p semi-factor-p-of-family tmap-below-iff fun-below-iff*)

lemma *family-of-semi-factor-below-iff*:
family-of-semi-factor $G \sqsubseteq F \longleftrightarrow G \sqsubseteq$ *semi-factor-of-family* F
by (*metis semi-factor-of-family-below-iff semi-factor-of-family-of-semi-factor*)

lemma *below-family-of-semi-factor-iff*:
$F \sqsubseteq$ *family-of-semi-factor* $G \longleftrightarrow$ *semi-factor-of-family* $F \sqsubseteq G$
by (*metis semi-factor-of-family-below-iff semi-factor-of-family-of-semi-factor*)

Being an order isomorphism, it preserves all glbs and lubs.

lemma *semi-factor-of-family-glb*:
semi-factor-of-family (*glb* A) = *glb* (*semi-factor-of-family* ‘ A)
proof (*rule below-antisym*)
show *semi-factor-of-family* (*glb* A) $\sqsubseteq$ *glb* (*semi-factor-of-family* ‘ A)
by (*rule lattice-glb-greatest*) (*auto simp*: *semi-factor-of-family-below-iff intro*: *glb-below*)
show *glb* (*semi-factor-of-family* ‘ A) $\sqsubseteq$ *semi-factor-of-family* (*glb* A)
unfolding *family-of-semi-factor-below-iff*[*symmetric*]
by (*rule lattice-glb-greatest*) (*simp add*: *family-of-semi-factor-below-iff glb-below*)
qed

lemma *semi-factor-of-family-lub*:
semi-factor-of-family (*lub A*) = *lub* (*semi-factor-of-family* ‘ *A*)
proof (*rule below-antisym*)
show *semi-factor-of-family* (*lub A*) ⊑ *lub* (*semi-factor-of-family* ‘ *A*)
unfolding *below-family-of-semi-factor-iff*[*symmetric*]
by (*rule lattice-lub-least*) (*simp add*: *is-ub-def below-family-of-semi-factor-iff lattice-lub-upper*)
show *lub* (*semi-factor-of-family* ‘ *A*) ⊑ *semi-factor-of-family* (*lub A*)
by (*rule lattice-lub-least*)
(*auto simp*: *is-ub-def semi-factor-of-family-below-iff intro*: *lattice-lub-upper*)
qed

lemma *semi-factor-of-family-inf*:
semi-factor-of-family (*F* ⊓ *G*) = *semi-factor-of-family F* ⊓ *semi-factor-of-family G*
unfolding *meet-def* **using** *semi-factor-of-family-glb*[*of* {*F*, *G*}] **by** *simp*

lemma *semi-factor-of-family-sup*:
semi-factor-of-family (*F* ⊔ *G*) = *semi-factor-of-family F* ⊔ *semi-factor-of-family G*
unfolding *join-def* **using** *semi-factor-of-family-lub*[*of* {*F*, *G*}] **by** *simp*

lemma *semi-factor-of-family-top*: *semi-factor-of-family lattice-top* = *lattice-top*
by (*rule below-antisym*) (*simp-all add*: *family-of-semi-factor-below-iff*[*symmetric*])

lemma *semi-factor-of-family-bot*: *semi-factor-of-family* ⊥ = ⊥
by (*rule bottomI*) (*subst below-family-of-semi-factor-iff*[*symmetric*], *rule minimal*)

lemma *semi-factor-of-family-proj* [*simp*]: (*t* ⌊ *semi-factor-of-family F*)⟨*d*⟩ = *t*⟨*d*⟩ ⌊ *F d*
by (*simp add*: *semi-factor-p-of-family*)

11.5 Dimension sets as semi-factors of the tuple domain

The coordinate cut is the family that is the top on the dimensions and ⊥ elsewhere.

definition *dcut* :: *'a*::*countable set* ⇒ (*'a*, *'b*::*scott-domain*) *tuple semi-factor* **where**
dcut A = *semi-factor-of-family* (λ*d*. *if d* ∈ *A then lattice-top else* ⊥)

lemma *semi-factor-p-dcut*:
semi-factor-p (*dcut A*) = (*tcut A* :: (*'a*::*countable*, *'b*::*scott-domain*) *tuple* → (*'a*,*'b*) *tuple*)
by (*rule cfun-eqI*, *rule tuple-eqI*)
(*simp add*: *dcut-def semi-factor-p-of-family tcut-tget semi-factor-p-top semi-factor-p-bot*)

Projecting a tuple onto the dimensions of an LOD is exactly the coordinate cut.

lemma *dcut-proj*: *x* ⌊ *dcut A* = *tcut A*·*x*

by (*simp add*: *semi-factor-p-dcut*)

A semi-factor is determined by its projection.

lemma *semi-factor-eqI*:
assumes *semi-factor-p F* = *semi-factor-p G* **shows** *F* = *G*
using *assms* **by** (*simp add*: *semi-factor-p-inject*)

lemma *tcut-compose*: *tcut B*·(*tcut A*·*t*) = *tcut* (*A* ∩ *B*)·*t*
by (*rule tuple-eqI*) (*simp add*: *tcut-tget*)

dcut is a lattice homomorphism, as the indicator families are.

lemma *dcut-inf*: *dcut* (*A* ∩ *B*) = *dcut A* ⊓ *dcut B*
unfolding *dcut-def semi-factor-of-family-inf*[*symmetric*]
by (*rule arg-cong*[**where** *f* = *semi-factor-of-family*]) (*simp add*: *fun-eq-iff fun-meet lattice-meet-idem lattice-meet-bot-left lattice-meet-bot-right*)

lemma *tcut-joinable*: *tcut A*·*t* ⇑ *tcut B*·*t*
by (*rule joinableI*[*OF tcut-deflation.below tcut-deflation.below*])

lemma *tcut-join*: *tcut A*·*t* ⊔ *tcut B*·*t* = *tcut* (*A* ∪ *B*)·*t*
for *t* :: ('*a*, '*b*::*bounded-complete*) *tuple*
proof (*rule tuple-eqI*)
fix *a*
have *tget* (*tcut A*·*t* ⊔ *tcut B*·*t*) *a* = *tget* (*tcut A*·*t*) *a* ⊔ *tget* (*tcut B*·*t*) *a*
by (*subst tget-join*[*OF tcut-joinable*]) *simp*
thus (*tcut A*·*t* ⊔ *tcut B*·*t*)⟨*a*⟩ = (*tcut* (*A* ∪ *B*)·*t*)⟨*a*⟩
by (*cases a* ∈ *A*; *cases a* ∈ *B*) (*simp-all add*: *tcut-tget*)
qed

lemma *dcut-sup*: *dcut* (*A* ∪ *B*) = *dcut A* ⊔ *dcut B*
unfolding *dcut-def semi-factor-of-family-sup*[*symmetric*]
by (*rule arg-cong*[**where** *f* = *semi-factor-of-family*]) (*simp add*: *fun-eq-iff fun-join lattice-join-bot-left lattice-join-bot-right*)

lemma *dcut-bot*: *dcut* {} = ⊥
by (*simp add*: *dcut-def semi-factor-of-family-bot lambda-strict*)

lemma *dcut-top*: *dcut UNIV* = *lattice-top*
by (*simp add*: *dcut-def semi-factor-of-family-top*[*unfolded lattice-top-fun*])

lemma *tcut-below*: *tcut A*·*t* ⊑ *t*
by (*simp add*: *below-tuple-def fun-below-iff tcut-tget*)

lemma *tcut-mono-set*: *A* ⊆ *B* ⟹ *tcut A*·*t* ⊑ *tcut B*·*t*
by (*auto simp*: *below-tuple-def fun-below-iff tcut-tget*)

The support of a tuple: the coordinates it defines. A key carries its own dimensions.

definition *tsupp* :: ('*d*, '*v*::*pcpo*) *tuple* ⇒ '*d set* **where**

tsupp $K = \{d.\ K\langle d\rangle \neq \bot\}$

lemma *tsupp-tcut*: *tsupp* (*tcut* $A{\cdot}K$) = *tsupp* $K \cap A$
by (*auto simp*: *tsupp-def tcut-tget*)

lemma *tcut-tsupp*: *tcut* (*tsupp* K)$\cdot K = K$
by (*rule tuple-eqI*) (*simp add*: *tcut-tget tsupp-def*)

lemma *tcut-below-tcut*: *tsupp* $K \cap A \subseteq B \Longrightarrow$ *tcut* $A{\cdot}K \sqsubseteq$ *tcut* $B{\cdot}K$
by (*auto simp*: *below-tuple-def fun-below-iff tcut-tget tsupp-def*)

lemma *tsupp-mono*: $x \sqsubseteq y \Longrightarrow$ *tsupp* $x \subseteq$ *tsupp* y
by (*auto simp*: *tsupp-def below-tuple-def fun-below-iff*) (*metis bottomI*)

lemma *adm-finite-bex*: *finite* $A \Longrightarrow (\bigwedge a.\ a \in A \Longrightarrow$ *adm* $(P\ a)) \Longrightarrow$ *adm* $(\lambda x.\ \exists\, a{\in}A.\ P\ a\ x)$
by (*induct rule*: *finite-induct*) *simp-all*

Being defined on finitely many dimensions is an open condition.

lemma *adm-not-tsupp*: *finite* $D \Longrightarrow$ *adm* $(\lambda t.\ \neg\ D \subseteq$ *tsupp* $t)$
proof −
assume *finite D*
then have *adm* $(\lambda t.\ \exists\, d{\in}D.\ t\langle d\rangle \sqsubseteq \bot)$
by (*rule adm-finite-bex*) (*intro adm-below cont-const*, *simp add*: *cont2cont-fun cont-tget*)
then show *?thesis* **by** (*simp add*: *tsupp-def subset-iff Bex-def*)
qed

With chain-finite (e.g. flat) values, a cut to finitely many dimensions is compact, and so is a key with finite support.

lemma *compact-tcut*:
fixes t :: ('*d*, '*v*::{*pcpo*, *chfin*}) *tuple*
assumes *finite A*
shows *compact* (*tcut* $A{\cdot}t$)
proof (*rule compactI*)
have $(\neg$ *tcut* $A{\cdot}t \sqsubseteq u) \longleftrightarrow (\exists\, a{\in}A.\ \neg\ t\langle a\rangle \sqsubseteq u\langle a\rangle)$ **for** u
by (*auto simp*: *below-tuple-def fun-below-iff tcut-tget*)
moreover have *adm* $(\lambda u.\ \exists\, a{\in}A.\ \neg\ t\langle a\rangle \sqsubseteq u\langle a\rangle)$
using *assms* **by** (*rule adm-finite-bex*) (*simp add*: *cont2cont-fun cont-tget*)
ultimately show *adm* $(\lambda u.\ \neg$ *tcut* $A{\cdot}t \sqsubseteq u)$ **by** *simp*
qed

lemma *compact-tsupp*:
fixes K :: ('*d*, '*v*::{*pcpo*, *chfin*}) *tuple*
assumes *finite* (*tsupp* K)
shows *compact K*
using *compact-tcut*[*OF assms*, *of K*] **by** (*simp add*: *tcut-tsupp*)

lemma *tget-lub*: *chain* $Y \Longrightarrow (\bigsqcup i.\ Y\ i)\langle d\rangle = (\bigsqcup i.\ (Y\ i)\langle d\rangle)$

by (*simp add*: *lub-tuple Abs-tuple-inverse′ lub-fun ch2ch-cont cont-tget*)

Conversely, a compact key has finite support (whatever the values): otherwise cutting away ever smaller tails of an infinite sequence in its support approximates it by a chain of keys none of which is above it. So compactness, the finitely observable keys, is what keys should be; for chain-finite values it is exactly finite support.

lemma *compact-finite-tsupp*:
fixes K :: ('*d*, '*v*::*pcpo*) *tuple*
assumes *compact K*
shows *finite* (*tsupp K*)
proof (*rule ccontr*)
assume *infinite* (*tsupp K*)
then obtain f :: *nat* ⇒ '*d* **where** *f*: *inj f range f* ⊆ *tsupp K*
using *infinite-countable-subset* **by** *blast*
define Y **where** $Y\ i$ = *tcut* (− f ‘ {*i*..})·K **for** i
have Y: *chain Y*
by (*rule chainI*) (*auto simp*: *Y-def intro*: *tcut-mono-set*)
have $K \sqsubseteq (\bigsqcup i.\ Y\ i)$
proof −
have $K\langle d\rangle \sqsubseteq (\bigsqcup i.\ (Y\ i)\langle d\rangle)$ **for** d
proof −
have *ch*: *chain* (λ*i*. (*Y i*)⟨*d*⟩)
using Y **by** (*simp add*: *chain-def below-tuple-def fun-below-iff*)
obtain j **where** $(Y\ j)\langle d\rangle = K\langle d\rangle$
proof (*cases d* ∈ *range f*)
case *True*
then obtain j **where** *d*: $d = f\ j$ **by** *blast*
have $(Y\ (Suc\ j))\langle d\rangle = K\langle d\rangle$
unfolding *Y-def* **by** (*simp add*: *tcut-tget d inj-image-mem-iff*[*OF f*(*1*)])
then show *?thesis* **by** (*rule that*)
next
case *False*
then have $(Y\ 0)\langle d\rangle = K\langle d\rangle$ **by** (*auto simp*: *Y-def tcut-tget*)
then show *?thesis* **by** (*rule that*)
qed
then show *?thesis* **using** *is-ub-thelub*[*OF ch*, *of j*] **by** *simp*
qed
then show *?thesis* **by** (*simp add*: *below-tuple-def fun-below-iff tget-lub*[*OF Y*])
qed
then obtain i **where** $K \sqsubseteq Y\ i$ **using** *assms Y compact-below-lub-iff* **by** *blast*
then have $K\langle f\ i\rangle \sqsubseteq (Y\ i)\langle f\ i\rangle$ **by** (*simp add*: *below-tuple-def fun-below-iff*)
moreover have $(Y\ i)\langle f\ i\rangle = \bot$ **by** (*simp add*: *Y-def tcut-tget*)
moreover have $K\langle f\ i\rangle \neq \bot$ **using** *f*(*2*) **by** (*auto simp*: *tsupp-def*)
ultimately show *False* **by** *simp*
qed

Cutting a compact key leaves it compact.

lemma *compact-tcut-compact*:

fixes $K :: ('d, 'v{::}pcpo)\ tuple$
assumes *compact* K
shows *compact* $(tcut\ A{\cdot}K)$
proof (*rule compactI2*)
fix Y **assume** Y: *chain* Y **and** *below*: $tcut\ A{\cdot}K \sqsubseteq (\bigsqcup i.\ Y\ i)$
define Z **where** $Z\ i = Abs\text{-}tuple\ (\lambda d.\ if\ d \in A\ then\ (Y\ i)\langle d\rangle\ else\ K\langle d\rangle)$ **for** i
have Zd: $(Z\ i)\langle d\rangle = (if\ d \in A\ then\ (Y\ i)\langle d\rangle\ else\ K\langle d\rangle)$ **for** $i\ d$
by (*simp add*: *Z-def Abs-tuple-inverse'*)
have Z: *chain* Z
using Y **by** (*auto intro*!: *chainI simp*: *below-tuple-def fun-below-iff Zd chain-def*)
have $K \sqsubseteq (\bigsqcup i.\ Z\ i)$
proof −
have $K\langle d\rangle \sqsubseteq (\bigsqcup i.\ (Z\ i)\langle d\rangle)$ **for** d
proof (*cases* $d \in A$)
case *True*
from *below* **have** $(tcut\ A{\cdot}K)\langle d\rangle \sqsubseteq (\bigsqcup i.\ Y\ i)\langle d\rangle$ **by** (*simp add*: *below-tuple-def fun-below-iff*)
with *True* **show** *?thesis* **by** (*simp add*: *Zd tget-lub*[*OF Y*] *tcut-tget*)
next
case *False* **then show** *?thesis* **by** (*simp add*: *Zd*)
qed
then show *?thesis* **by** (*simp add*: *below-tuple-def fun-below-iff tget-lub*[*OF Z*])
qed
then obtain i **where** $K \sqsubseteq Z\ i$ **using** *assms Z compact-below-lub-iff* **by** *blast*
then have $tcut\ A{\cdot}K \sqsubseteq Y\ i$
by (*auto simp*: *below-tuple-def fun-below-iff Zd tcut-tget split*: *if-splits*)
then show $\exists i.\ tcut\ A{\cdot}K \sqsubseteq Y\ i$ **..**
qed

11.6 Compact tuples

A tuple is compact exactly when it has finite support and compact components. A component of a compact tuple is compact: a chain approximating the component, placed into the tuple, approximates the whole tuple.

lemma *compact-tget*:
fixes $K :: ('d, 'v{::}pcpo)\ tuple$
assumes *compact* K **shows** *compact* $(K\langle d\rangle)$
proof (*rule compactI2*)
fix $Y :: nat \Rightarrow 'v$ **assume** Y: *chain* Y **and** *le*: $K\langle d\rangle \sqsubseteq (\bigsqcup i.\ Y\ i)$
define Z **where** $Z\ i = Abs\text{-}tuple\ ((tget\ K)(d := Y\ i))$ **for** i
have Zd: $(Z\ i)\langle e\rangle = (if\ e = d\ then\ Y\ i\ else\ K\langle e\rangle)$ **for** $i\ e$
by (*simp add*: *Z-def Abs-tuple-inverse'*)
have Z: *chain* Z
using Y **by** (*simp add*: *chain-def tuple-below-iff Zd*)
have $K \sqsubseteq (\bigsqcup i.\ Z\ i)$
using *le* **by** (*simp add*: *tuple-below-iff tget-lub*[*OF Z*] *Zd Y*)
then obtain i **where** $K \sqsubseteq Z\ i$ **using** *compactD2*[*OF assms Z*] **by** *blast*
then have $K\langle d\rangle \sqsubseteq (Z\ i)\langle d\rangle$ **by** (*simp add*: *tuple-below-iff*)
then show $\exists i.\ K\langle d\rangle \sqsubseteq Y\ i$ **by** (*auto simp*: *Zd*)

qed

lemma *compact-tupleI*:
fixes K :: ('d, 'v::*pcpo*) *tuple*
assumes *finite* (*tsupp* K) **and** $\bigwedge d$. *compact* ($K\langle d\rangle$)
shows *compact* K
proof (*rule compactI*)
have ($\neg$ $K \sqsubseteq u$) $\longleftrightarrow$ ($\exists\, d{\in}$*tsupp* K. $\neg$ $K\langle d\rangle \sqsubseteq u\langle d\rangle$) **for** u
by (*auto simp*: *tuple-below-iff tsupp-def*) (*metis minimal*)
moreover have *adm* (λu. $\exists\, d{\in}$*tsupp* K. $\neg$ $K\langle d\rangle \sqsubseteq u\langle d\rangle$)
using *assms*(*1*) **proof** (*rule adm-finite-bex*)
show *adm* (λu. $\neg$ $K\langle d\rangle \sqsubseteq u\langle d\rangle$) **for** d
by (*rule adm-compact-not-below*[*OF assms*(*2*)]) (*simp add*: *cont2cont-fun cont-tget*)
qed
ultimately show *adm* (λu. $\neg$ $K \sqsubseteq u$) **by** *simp*
qed

end
theory *Congruence*
imports *HOLCF*
begin

default-sort *pcpo*

12 Continuous congruences (functional-dependence relations)

A *continuous congruence* on a cpo 'a is a relation R that is transitive, refines the cpo order ($\sqsubseteq$), and is admissible (Scott-closed) in each argument: a coarser "below". The motivating instance is the kernel order $\lambda x\ y.\ f{\cdot}x \sqsubseteq f{\cdot}y$ of a continuous map; a bigger relation identifies more points, i.e. is *coarser*. These are used only as *predicates*, to state functional dependence between levels of detail; levels of detail themselves are syntactic dimension sets (see the frame development). We deliberately do not build a type of congruences: the only reason to do so would be meets, and finitely flat-generated congruences are not closed under them.

12.1 Continuous congruences (admissible refining preorders)

locale *is-cong* =
fixes R :: '$a \Rightarrow$ '$a \Rightarrow$ *bool*
assumes *trans*: $R\ x\ y \Longrightarrow R\ y\ z \Longrightarrow R\ x\ z$
and *belowD*: $x \sqsubseteq y \Longrightarrow R\ x\ y$
and *adm1*: *adm* (λx. $R\ x\ y$)
and *adm2*: *adm* (λy. $R\ x\ y$)

begin
lemmas *axioms* = *trans belowD adm1 adm2*
end

lemma (**in** *is-cong*) *refl*: $R\ x\ x$
  by (*auto simp*: *trans belowD*)

lemma *is-cong-top*: *is-cong top*
  by (*simp add*: *is-cong-def*)

lemma *is-cong-below*: *is-cong* $(\lambda x\ y.\ x \sqsubseteq y)$
  by (*auto simp*: *is-cong-def intro*: *below-trans adm-below cont-id cont-const*)

lemma *is-cong-Inter*:
  assumes $\bigwedge i.\ i \in I \Longrightarrow$ *is-cong* $(R\ i)$
  shows *is-cong* $(\lambda x\ y.\ \forall\, i{\in}I.\ R\ i\ x\ y)$
proof
  show $\forall\, i \in I.\ R\ i\ x\ y \Longrightarrow \forall\, i \in I.\ R\ i\ y\ z \Longrightarrow \forall\, i \in I.\ R\ i\ x\ z$ **for** $x\ y\ z$
    using *assms*[*THEN is-cong.trans*] **by** *blast*
  show $x \sqsubseteq y \Longrightarrow \forall\, i \in I.\ R\ i\ x\ y$ **for** $x\ y$
    using *assms*[*THEN is-cong.belowD*] **by** *blast*
  show *adm* $(\lambda x.\ \forall\, i \in I.\ R\ i\ x\ y)$ **for** y
    using *assms*[*THEN is-cong.adm1*] *adm-ball*[*of I*] **by** *force*
  show *adm* $(\lambda y.\ \forall\, i \in I.\ R\ i\ x\ y)$ **for** x
    using *assms*[*THEN is-cong.adm2*] *adm-ball*[*of I*] **by** *force*
qed

lemma *Inf-iff*: *Inf* $\mathcal{R} = (\lambda x\ y.\ \forall\, R \in \mathcal{R}.\ R\ x\ y)$
  by *fast*

corollary *is-cong-Inf*: $(\bigwedge R.\ R \in \mathcal{R} \Longrightarrow$ *is-cong* $R) \Longrightarrow$ *is-cong* $(Inf\ \mathcal{R})$
  unfolding *Inf-iff*
  using *is-cong-Inter*[*of* $\mathcal{R}$ $(\lambda x.\ x)$] **.**

lemma *inf2-def*: *inf* $R\ S = (\lambda x\ y.\ R\ x\ y \wedge S\ x\ y)$
  by *auto*

lemma *is-cong-inf2*:
  assumes *is-cong* R *is-cong* S **shows** *is-cong* $(inf\ R\ S)$
  unfolding *inf2-def*
proof
  from *assms* **show** $R\ x\ y \wedge S\ x\ y \Longrightarrow R\ y\ z \wedge S\ y\ z \Longrightarrow R\ x\ z \wedge S\ x\ z$ **for** $x\ y\ z$
    by (*blast dest*: *is-cong.trans*)
  from *assms* **show** $x \sqsubseteq y \Longrightarrow R\ x\ y \wedge S\ x\ y$ **for** $x\ y$
    by (*blast dest*: *is-cong.belowD*)
  from *assms* **show** *adm* $(\lambda x.\ R\ x\ y \wedge S\ x\ y)$ **for** y
    by (*intro adm-conj*) (*blast intro*: *assms dest*: *is-cong.adm1*)+
  from *assms* **show** *adm* $(\lambda y.\ R\ x\ y \wedge S\ x\ y)$ **for** x
    by (*intro adm-conj*) (*blast intro*: *assms dest*: *is-cong.adm2*)+

qed

12.2 The generated congruence (least cong above a relation)

definition *gen* :: (*'a* ⇒ *'a* ⇒ *bool*) ⇒ (*'a* ⇒ *'a* ⇒ *bool*) **where**
gen B = (λ*x y*. ∀ *R*. *is-cong R* ⟶ (∀ *u v*. *B u v* ⟶ *R u v*) ⟶ *R x y*)

lemma *is-cong-gen*: *is-cong* (*gen B*)
proof −
have ∗: *gen B* = (λ*x y*. ∀ *R*∈{*R*. *is-cong R* ∧ (∀ *u v*. *B u v* ⟶ *R u v*)}. *R x y*)
by (*auto simp*: *gen-def fun-eq-iff*)
show *?thesis* **unfolding** ∗ **by** (*rule is-cong-Inter*) *auto*
qed

lemma *gen-upper*: *B x y* ⟹ *gen B x y*
by (*auto simp*: *gen-def*)

lemma *gen-least*: *is-cong R* ⟹ (⋀*u v*. *B u v* ⟹ *R u v*) ⟹ *gen B x y* ⟹ *R x y*
by (*auto simp*: *gen-def*)

lemma *map-is-cong*: **assumes** *is-cong R cont f* **shows** *is-cong* (*inv-imagep R f*)
unfolding *inv-imagep-def*
proof (*standard*)
from *assms*(*1*) **show** *R* (*f x*) (*f y*) ⟹ *R* (*f y*) (*f z*) ⟹ *R* (*f x*) (*f z*) **for** *x y z*
by (*rule is-cong.trans*)
from *assms*(*1*)[*THEN is-cong.belowD*] *assms*(*2*)[*THEN cont2monofunE*]
show *x* ⊑ *y* ⟹ *R* (*f x*) (*f y*) **for** *x y*
by *presburger*
from *assms*(*2*) *assms*(*1*)[*THEN is-cong.adm1*]
show *adm* (λ*x*. *R* (*f x*) (*f y*)) **for** *y* **by** (*rule adm-subst*)
from *assms*(*2*) *assms*(*1*)[*THEN is-cong.adm2*]
show *adm* (λ*y*. *R* (*f x*) (*f y*)) **for** *x* **by** (*rule adm-subst*)
qed

12.3 Flat LODs (the generators)

A flat congruence is one whose quotient has height at most 1: a bottom class of "undefined" points (each R-below everything) and an antichain of value classes above. These arise exactly from continuous maps into flat domains, and generate the LOD frame.

locale *is-flat-cong* = *is-cong* +
assumes *height1*: *R x y* ⟹ *R y x* ∨ (∀ *z*. *R x z*)

lemma *is-flat-cong-eq-on*:
fixes *f* :: *'a* → *'b*::*flat*
shows *is-flat-cong* (λ*x y*. *f*·*x* ⊑ *f*·*y*)

proof *unfold-locales*
show $f{\cdot}x \sqsubseteq f{\cdot}y \Longrightarrow f{\cdot}y \sqsubseteq f{\cdot}z \Longrightarrow f{\cdot}x \sqsubseteq f{\cdot}z$ **for** $x\ y\ z$ **by** (*rule below-trans*)
show $x \sqsubseteq y \Longrightarrow f{\cdot}x \sqsubseteq f{\cdot}y$ **for** $x\ y$ **by** (*rule monofun-cfun-arg*)
show *adm* $(\lambda x.\ f{\cdot}x \sqsubseteq f{\cdot}y)$ **for** y **by** *simp*
show *adm* $(\lambda y.\ f{\cdot}x \sqsubseteq f{\cdot}y)$ **for** x **by** *simp*
show $f{\cdot}x \sqsubseteq f{\cdot}y \Longrightarrow f{\cdot}y \sqsubseteq f{\cdot}x \vee (\forall z.\ f{\cdot}x \sqsubseteq f{\cdot}z)$ **for** $x\ y$
by (*metis ax-flat minimal*)
qed

The family generated by the flats under arbitrary cong-joins, i.e. boolean-level intersections (*Inf* at the predicate level = *Sup* at the cong level).

definition *is-flat-gen* :: $('a \Rightarrow 'a \Rightarrow bool) \Rightarrow bool$ **where**
is-flat-gen $R \longleftrightarrow (\exists \mathcal{F}.\ (\forall f{\in}\mathcal{F}.\ \textit{is-flat-cong}\ f) \wedge R = \textit{Inf}\ \mathcal{F})$

lemma *is-flat-gen-flat*: *is-flat-cong* $f \Longrightarrow$ *is-flat-gen* f
unfolding *is-flat-gen-def* **by** (*rule exI*[*of* - $\{f\}$]) *simp*

Meet of two flats is flat: the generated congruence stays height at most 1. No induction, no distributivity of $'a$ the symmetric companion U is squeezed in between.

lemma *is-flat-cong-meet*:
fixes $f\ g :: 'a \Rightarrow 'a \Rightarrow bool$
assumes f: *is-flat-cong* f **and** g: *is-flat-cong* g
shows *is-flat-cong* (*gen* (*sup* $f\ g$))
proof −
let *?R* = *gen* (*sup* $f\ g$)
have R: *is-cong ?R* **by** (*rule is-cong-gen*)
have cf: *is-cong* f **using** f **by** (*simp add*: *is-flat-cong-def*)
note fh = *is-flat-cong.height1*[*OF* f]
note gh = *is-flat-cong.height1*[*OF* g]
have *sup1*: *?R* $a\ b$ **if** $f\ a\ b$ **for** $a\ b$
using *that* **by** (*intro gen-upper*) *simp*
have *sup2*: *?R* $a\ b$ **if** $g\ a\ b$ **for** $a\ b$
using *that* **by** (*intro gen-upper*) *simp*
define $U :: 'a \Rightarrow 'a \Rightarrow bool$
where $U = (\lambda x\ y.\ (\forall z.\ ?R\ x\ z) \vee (?R\ x\ y \wedge ?R\ y\ x))$
have *Ucong*: *is-cong U*
proof *unfold-locales*
fix $x\ y\ z$ **assume** xy: $U\ x\ y$ **and** yz: $U\ y\ z$
show $U\ x\ z$ **unfolding** *U-def*
using xy[*unfolded U-def*] yz[*unfolded U-def*] *is-cong.trans*[*OF R*] **by** *metis*
next
fix $x\ y :: 'a$ **assume** $x \sqsubseteq y$
hence fxy: $f\ x\ y$ **by** (*rule is-cong.belowD*[*OF cf*])
hence *Rxy*: *?R* $x\ y$ **by** (*rule sup1*)
from fh[*OF fxy*] **show** $U\ x\ y$ **unfolding** *U-def*
proof (*elim disjE*)
assume $f\ y\ x$ **hence** *?R* $y\ x$ **by** (*rule sup1*)
with *Rxy* **show** $(\forall z.\ ?R\ x\ z) \vee (?R\ x\ y \wedge ?R\ y\ x)$ **by** *blast*

```
    next
      assume ∀ z. f x z hence ∀ z. ?R x z using sup1 by blast
      thus (∀ z. ?R x z) ∨ (?R x y ∧ ?R y x) by blast
    qed
  next
    fix y show adm (λx. U x y) unfolding U-def
      by (intro adm-disj adm-all adm-conj is-cong.adm1[OF R] is-cong.adm2[OF
R])
  next
    fix x show adm (λy. U x y) unfolding U-def
     by (intro adm-disj adm-const adm-conj is-cong.adm2[OF R] is-cong.adm1[OF
R])
  qed
  have supU: U u v if huv: sup f g u v for u v
    unfolding U-def using sup1 sup2 fh gh
    by (metis sup1 fh sup2 gh that sup-apply sup-bool-def)
  have RU: ?R x y ⟹ U x y for x y
    by (rule gen-least[OF Ucong supU])
  show is-flat-cong ?R
  proof unfold-locales
    show ?R x y ⟹ ?R y z ⟹ ?R x z for x y z by (rule is-cong.trans[OF R])
    show x ⊑ y ⟹ ?R x y for x y by (rule is-cong.belowD[OF R])
    show adm (λx. ?R x y) for y by (rule is-cong.adm1[OF R])
    show adm (λy. ?R x y) for x by (rule is-cong.adm2[OF R])
    show ?R x y ⟹ ?R y x ∨ (∀ z. ?R x z) for x y
      using RU unfolding U-def by blast
  qed
qed

lemma gen-inf: gen R = Inf {z. z ≥ R ∧ is-cong z}
  unfolding gen-def
  by blast

end
theory LOD
 imports ValsetBase HOLCF−extra.Filter-Sum HOLCF−extra.Domain-Order-Derive
Semi-Factor-DI
   HOL−Library.FSet
begin
```

13 Finitary semi-factors and locally finitely detailed domains

A semi-factor is *finitary* when all its elements are compact (equivalently, as a subdomain it has finite height): its projection takes every element, however large, to a compact one. Finitary semi-factors are closed under meets (intersection), joins (pointwise joins of compacts) and contain the

bottom semi-factor.

default-sort {*bifinite*, *distributive-cpo*}

A domain is *locally finitely detailed* when every compact element lives at some finite level of detail, i.e. lies in a finitary semi-factor: every finite piece of information has a finite, self-contained level. This is the semi-factor analogue of property I of dI-domains (for stable subdomains the least one containing a compact k is $\downarrow k$, and a distributive lattice of finite height is finite), and strictly stronger. (Compare also Libkin's requirement that the compact elements – the storable objects – form an ideal.)

default-sort *type*

definition *finitary* :: ('*a*::*pcpo* → '*a*) ⇒ *bool* **where**
  finitary d ⟷ (∀ *x*. *compact* (*d*·*x*))

definition *faithful* :: ('*a*::*pcpo* → '*a*) ⇒ *bool* **where**
  faithful d ⟷ (∀ *x*. *d*·*x* = ⊥ ⟶ *x* = ⊥)

lemma *finitary-bottom*: *finitary* ⊥
  by (*simp add*: *finitary-def*)

Over a locally finitely detailed dI-domain the compact semi-factors, which are the levels of detail, are finitary (*compact-semi-factor-proj* below). The constructions act on the deflations; *semi-factor-of-defl* turns the result back into a semi-factor.

definition *finitary-semi-factor* :: '*a*::{*bifinite*, *distributive-cpo*} *semi-factor* ⇒ *bool* **where**
  finitary-semi-factor F ⟷ *finitary* (*semi-factor-p F*)

lemma *ex-finitary-semi-factor-iff*:
  fixes *k* :: '*a*::{*bifinite*, *distributive-cpo*}
  shows (∃ *F*. *finitary-semi-factor F* ∧ *k* ⌊ *F* = *k*) ⟷ (∃ *e*. *semi-factor-defl e* ∧ *finitary e* ∧ *e*·*k* = *k*)
proof
  assume ∃ *F*. *finitary-semi-factor F* ∧ *k* ⌊ *F* = *k*
  then obtain *F* **where** *finitary-semi-factor F k* ⌊ *F* = *k* **by** *blast*
  then show ∃ *e*. *semi-factor-defl e* ∧ *finitary e* ∧ *e*·*k* = *k*
    by (*intro exI*[*of* - *semi-factor-p F*])
      (*simp add*: *semi-factor-defl-semi-factor-p finitary-semi-factor-def*)
next
  assume ∃ *e*. *semi-factor-defl e* ∧ *finitary e* ∧ *e*·*k* = *k*
  then obtain *e* **where** *semi-factor-defl e finitary e e*·*k* = *k* **by** *blast*
  then show ∃ *F*. *finitary-semi-factor F* ∧ *k* ⌊ *F* = *k*
    by (*intro exI*[*of* - *semi-factor-of-defl e*])
      (*simp add*: *finitary-semi-factor-def semi-factor-p-of-defl*)
qed

class *locally-finitely-detailed* = *bifinite* + *distributive-cpo* +
  assumes *compact-level*: *compact* $k \Longrightarrow \exists\, F.$ *finitary-semi-factor* $F \wedge k \lfloor F = k$

The domains of rows and keys: locally finitely detailed with algebraic semi-factors. For now the latter is property I of dI-domains; it is meant to weaken to algebraicity of the semi-factors alone, which separated sums also give.

class *lod-domain* = *dI* + *locally-finitely-detailed*

13.1 Levels through the constructions

lemma *semi-factor-defl-u-map*:
  assumes *f*: *semi-factor-defl* (*f* :: *'a*::{*bifinite*, *distributive-cpo*} → *'a*)
  shows *semi-factor-defl* (*u-map*·*f*)
proof (*rule semi-factor-deflI*)
 show *stable-defl* (*u-map*·*f*) **by** (*rule stable-defl-u-map*[*OF semi-factor-defl.axioms*(*1*)[*OF f*]])
  fix *x y* :: *'a u* **assume** *y*: *u-map*·*f*·*y* = *y* **and** *le*: *u-map*·*f*·*x* ⊑ *y*
  show *x* ⇑ *y*
  proof (*cases x*)
    case *bottom* **then show** *?thesis* **by** (*simp add*: *bottom-joinable*)
  next
    case (*up a*)
    with *le* **obtain** *b* **where** *b*: *y* = *up*·*b f*·*a* ⊑ *b* **by** (*cases y*) *auto*
    with *y* **have** *f*·*b* = *b* **by** *simp*
    from *semi-factor-defl.slide*[*OF f this b*(*2*)] *up b*(*1*) **show** *?thesis*
      by (*simp add*: *cfun-arg-joinable*)
  qed
qed

lemma *finitary-u-map*:
  assumes *finitary f* **shows** *finitary* (*u-map*·*f*)
  unfolding *finitary-def*
proof
  fix *x* **show** *compact* (*u-map*·*f*·*x*)
    using *assms* **unfolding** *finitary-def* **by** (*cases x*) (*simp-all add*: *compact-up*)
qed

lemma *semi-factor-defl-ssum-map*:
  assumes *f*: *semi-factor-defl f faithful f* **and** *g*: *semi-factor-defl g faithful g*
  shows *semi-factor-defl* (*ssum-map*·*f*·*g*)
proof (*rule semi-factor-deflI*)
  show *stable-defl* (*ssum-map*·*f*·*g*)
    by (*rule stable-defl-ssum-map*[*OF semi-factor-defl.axioms*(*1*)[*OF f*(*1*)]
        semi-factor-defl.axioms(*1*)[*OF g*(*1*)]])
  fix *x y* **assume** *y*: *ssum-map*·*f*·*g*·*y* = *y* **and** *le*: *ssum-map*·*f*·*g*·*x* ⊑ *y*
  show *x* ⇑ *y*
  proof (*cases x*)
    case *bottom* **then show** *?thesis* **by** (*simp add*: *bottom-joinable*)
  next

```
    case (sinl a)
    with f(2) have fa: f·a ≠ ⊥ unfolding faithful-def by blast
    from le sinl fa obtain b where b: y = sinl·b b ≠ ⊥ f·a ⊑ b by (cases y) auto
    with y have f·b = b by simp
    from semi-factor-defl.slide[OF f(1) this b(3)] sinl b(1) show ?thesis
      by (simp add: cfun-arg-joinable)
  next
    case (sinr a)
    with g(2) have ga: g·a ≠ ⊥ unfolding faithful-def by blast
    from le sinr ga obtain b where b: y = sinr·b b ≠ ⊥ g·a ⊑ b by (cases y) auto
    with y have g·b = b by simp
    from semi-factor-defl.slide[OF g(1) this b(3)] sinr b(1) show ?thesis
      by (simp add: cfun-arg-joinable)
  qed
qed

lemma finitary-ssum-map:
  assumes finitary f finitary g shows finitary (ssum-map·f·g)
  unfolding finitary-def
proof
  fix x show compact (ssum-map·f·g·x)
    using assms unfolding finitary-def by (cases x) (simp-all add: compact-sinl
compact-sinr)
qed

lemma semi-factor-defl-sprod-map:
  fixes f :: 'a::{bifinite, distributive-cpo, filter-sum} → 'a
    and g :: 'b::{bifinite, distributive-cpo, filter-sum} → 'b
  assumes f: semi-factor-defl f faithful f and g: semi-factor-defl g faithful g
  shows semi-factor-defl (sprod-map·f·g)
proof (rule semi-factor-deflI)
  show stable-defl (sprod-map·f·g)
    by (rule stable-defl-sprod-map[OF semi-factor-defl.axioms(1)[OF f(1)]
          semi-factor-defl.axioms(1)[OF g(1)]])
  fix x y assume y: sprod-map·f·g·y = y and le: sprod-map·f·g·x ⊑ y
  show x ⇑ y
  proof (cases x)
    case bottom then show ?thesis by (simp add: bottom-joinable)
  next
    case (spair a b)
    have fa: f·a ≠ ⊥ g·b ≠ ⊥ using f(2) g(2) spair unfolding faithful-def by
blast+
    from le spair fa obtain c e where c: y = (:c, e:) c ≠ ⊥ e ≠ ⊥ f·a ⊑ c g·b ⊑
e
      by (cases y) (auto simp: spair-below-iff)
    with y have f·c = c g·e = e by (auto simp: spair-eq-iff)
    with c have a ⇑ c b ⇑ e
     by (auto intro: semi-factor-defl.slide[OF f(1)] semi-factor-defl.slide[OF g(1)])
    with spair c show ?thesis by (simp add: spair-joinable-iff)
```

qed
qed

lemma *finitary-sprod-map*:
assumes *finitary f finitary g* **shows** *finitary* (*sprod-map*·*f*·*g*)
unfolding *finitary-def*
proof
fix *x* **show** *compact* (*sprod-map*·*f*·*g*·*x*)
using *assms* **unfolding** *finitary-def* **by** (*cases x*) (*simp-all add*: *compact-spair*)
qed

13.2 Levels through isomorphisms

lemma *semi-factor-defl-iso*:
fixes *abs* :: *'r*::{*bifinite*, *distributive-cpo*} → *'t*::{*bifinite*, *distributive-cpo*} **and** *rep*
assumes *iso-abs*: ⋀*x*. *rep*·(*abs*·*x*) = *x* **and** *iso-rep*: ⋀*y*. *abs*·(*rep*·*y*) = *y*
and *f*: *semi-factor-defl f*
shows *semi-factor-defl* (*abs oo f oo rep*)
proof (*rule semi-factor-deflI*)
have *iso*: *iso abs rep* **by** (*rule iso.intro*[*OF iso-abs iso-rep*])
interpret *iso abs rep* **by** (*fact iso*)
show *stable-defl* (*abs oo f oo rep*)
by (*rule stable-defl-iso*[*OF iso-abs iso-rep semi-factor-defl.axioms*(*1*)[*OF f*]])
fix *x y* **assume** *y*: (*abs oo f oo rep*)·*y* = *y* **and** *le*: (*abs oo f oo rep*)·*x* ⊑ *y*
have *rep*·(*abs*·(*f*·(*rep*·*y*))) = *rep*·*y* **using** *y* **by** *simp*
then have *fy*: *f*·(*rep*·*y*) = *rep*·*y* **by** *simp*
have *rep*·((*abs oo f oo rep*)·*x*) ⊑ *rep*·*y* **using** *le* **by** (*rule monofun-cfun-arg*)
then have *f*·(*rep*·*x*) ⊑ *rep*·*y* **by** *simp*
then have *rep*·*x* ⇑ *rep*·*y* **by** (*rule semi-factor-defl.slide*[*OF f fy*])
then show *x* ⇑ *y* **by** (*simp add*: *iso-rep-joinable-iff*[*OF iso*])
qed

lemma *finitary-iso*:
fixes *abs* :: *'r*::*pcpo* → *'t*::*pcpo* **and** *rep*
assumes *iso-abs*: ⋀*x*. *rep*·(*abs*·*x*) = *x* **and** *iso-rep*: ⋀*y*. *abs*·(*rep*·*y*) = *y*
and *f*: *finitary f*
shows *finitary* (*abs oo f oo rep*)
proof −
interpret *iso abs rep* **by** (*rule iso.intro*[*OF iso-abs iso-rep*])
show *?thesis* **using** *f* **unfolding** *finitary-def* **by** (*simp add*: *compact-abs*)
qed

Finitary deflations are not admissible (the *take* functions converge to *ID*), so they get no fixed-point rule; the map and iso rules still derive them through non-recursive types (*map-fix-const*).

lemmas [*domain-category-map*] = *finitary-u-map finitary-ssum-map finitary-sprod-map*
lemmas [*domain-category-iso*] = *finitary-iso*

13.3 Discriminator for separated sums

In a separated sum every nonbottom element lies above a unique atom. The discriminator sends it there: it is the lub of the step functions *stepfn a a* at the atoms, which are compact. It is a faithful finitary semi-factor deflation, whose image is $\bot$ and the atoms.

definition *discriminator* :: *'a*::{*bifinite*, *bounded-complete*, *separated-sum*} → *'a*
  where *discriminator* = *lub* {*stepfn a a* |*a*. *atom a*}

lemma *discriminator-bound*:
  {*stepfn a a* |*a*. *atom a*} <| (*ID* :: *'a*::{*bifinite*, *bounded-complete*, *separated-sum*} → *'a*)
  by (*auto intro*!: *is-ubI simp*: *stepfn-below-iff atom-compact*)

lemma *discriminator-app*:
  discriminator·*x* = *lub* ((λ*a*. *if a* ⊑ *x then a else* ⊥) ‘ {*a*. *atom a*})
proof −
  have *S*: {*stepfn a a* |*a*. *atom a*} = (λ*a*. *stepfn a a*) ‘ {*a*. *atom a*} **by** *blast*
  have (λ*f*. *f*·*x*) ‘ {*stepfn a a* |*a*. *atom a*} = (λ*a*. *stepfn a a*·*x*) ‘ {*a*. *atom a*}
    unfolding *S image-image* **..**
  also have ... = (λ*a*. *if a* ⊑ *x then a else* ⊥) ‘ {*a*. *atom a*}
    by (*rule image-cong*[*OF refl*]) (*simp add*: *stepfn-beta*[*OF atom-compact*])
  finally have (λ*f*. *f*·*x*) ‘ {*stepfn a a* |*a*. *atom a*} = (λ*a*. *if a* ⊑ *x then a else* ⊥) ‘ {*a*. *atom a*} **.**
  then show *?thesis*
    unfolding *discriminator-def cfun-lub*[*OF discriminator-bound*] **by** *simp*
qed

lemma *discriminator-eq*:
  assumes *a*: *atom a* **and** *ax*: *a* ⊑ *x*
  shows *discriminator*·*x* = *a*
  unfolding *discriminator-app*
proof (*rule lub-eqI*, *rule is-lubI*)
  show (λ*a*. *if a* ⊑ *x then a else* ⊥) ‘ {*a*. *atom a*} <| *a*
    by (*auto intro*!: *is-ubI dest*: *atom-unique*[*OF - a - ax*])
next
  fix *u* **assume** (λ*a*. *if a* ⊑ *x then a else* ⊥) ‘ {*a*. *atom a*} <| *u*
  moreover have *a* ∈ (λ*a*. *if a* ⊑ *x then a else* ⊥) ‘ {*a*. *atom a*}
    using *a ax* **by** (*auto intro*!: *image-eqI*[*of - - a*])
  ultimately show *a* ⊑ *u* **by** (*rule is-ubD*)
qed

lemma *discriminator-strict* [*simp*]: *discriminator*·⊥ = ⊥
  unfolding *discriminator-app*
  by (*rule lub-eqI*) (*auto intro*!: *is-lubI is-ubI dest*: *atom-nonbottom*)

lemma *discriminator-atom*:
  assumes *x* ≠ ⊥ **shows** *atom* (*discriminator*·*x*) **and** *discriminator*·*x* ⊑ *x*
proof −

obtain *a* **where** *atom a a* $\sqsubseteq$ *x* **using** *assms* **by** (*rule atom-below*)
then show *atom* (*discriminator*·*x*) *discriminator*·*x* $\sqsubseteq$ *x* **by** (*simp-all add*: *discriminator-eq*)
qed

lemma *discriminator-below*: *discriminator*·*x* $\sqsubseteq$ *x*
by (*cases x* = $\bot$) (*simp-all add*: *discriminator-atom*)

lemma *discriminator-fixes*: *discriminator*·*x* = *x* $\longleftrightarrow$ *x* = $\bot$ $\vee$ *atom x*
by (*metis discriminator-atom*(*1*) *discriminator-eq below-refl discriminator-strict*)

lemma *discriminator-faithful*: *faithful discriminator*
unfolding *faithful-def* **using** *discriminator-atom atom-nonbottom* **by** *blast*

lemma *discriminator-finitary*: *finitary discriminator*
unfolding *finitary-def*
by (*metis atom-compact compact-bottom discriminator-atom discriminator-strict*)

lemma *stable-discriminator*: *stable-defl discriminator*
proof *standard*
fix *x* **show** *discriminator*·(*discriminator*·*x*) = *discriminator*·*x*
by (*cases x* = $\bot$) (*simp-all add*: *discriminator-atom discriminator-eq*)
next
fix *x* **show** *discriminator*·*x* $\sqsubseteq$ *x* **by** (*rule discriminator-below*)
next
fix *x y* **assume** *discriminator*·*y* = *y* **and** *x* $\sqsubseteq$ *y*
then show *discriminator*·*x* = *x* **by** (*auto simp*: *discriminator-fixes dest*: *atomD*)
qed

lemma *discriminator-semi-factor*:
semi-factor-defl (*discriminator* :: *'a*::{*bifinite*, *distributive-cpo*, *separated-sum*} $\rightarrow$ *'a*)
proof (*rule semi-factor-deflI*[*OF stable-discriminator*])
fix *x y* :: *'a* **assume** *y*: *discriminator*·*y* = *y* **and** *le*: *discriminator*·*x* $\sqsubseteq$ *y*
show *x* $\Uparrow$ *y*
proof (*cases x* = $\bot$)
case *True* **then show** *?thesis* **by** (*simp add*: *bottom-joinable*)
next
case *False*
note *a* = *discriminator-atom*[*OF False*]
have *y* $\neq$ $\bot$ **using** *le atom-nonbottom*[*OF a*(*1*)] **by** (*metis below-bottom-iff*)
with *y* **have** *atom y* **by** (*simp add*: *discriminator-fixes*)
with *a*(*1*) *le* **have** *discriminator*·*x* = *y* **using** *atomD atom-nonbottom* **by** *blast*
with *a*(*2*) **have** *y* $\sqsubseteq$ *x* **by** *simp*
then show *?thesis* **by** (*rule joinableI*[*OF below-refl*])
qed
qed

lemma *discriminator-deflation*: *deflation discriminator*
by (*rule stable-defl.deflation*[*OF stable-discriminator*])

In a separated sum every nonbottom semi-factor is faithful: an element sent to $\bot$ is joinable with every image element (slide rule), hence shares an atom with a nonbottom one; the atom is in the image (stability), and below the element, so it is sent to $\bot$ as well.

lemma *semi-factor-defl-faithful*:
fixes $d :: 'a::\{bifinite, distributive\text{-}cpo, separated\text{-}sum\} \to 'a$
assumes *d*: *semi-factor-defl d* **and** *a*: $d{\cdot}a = a$ $a \neq \bot$
shows *faithful d*
unfolding *faithful-def*
proof (*intro allI impI*, *rule ccontr*)
interpret *D*: *semi-factor-defl d* **by** (*rule d*)
fix *x* **assume** *dx*: $d{\cdot}x = \bot$ **and** *x*: $x \neq \bot$
have *xa*: $x \Uparrow a$ **using** *a*(*1*) **by** (*rule D.slide*) (*simp add*: *dx*)
obtain *l* **where** *l*: *atom l* $l \sqsubseteq x$ $l \sqsubseteq a$
using *separated-sum*[*OF x a*(*2*) *joinableD*(*1*)[*OF xa*] *joinableD*(*2*)[*OF xa*]] **by** *blast*
have $d{\cdot}l = l$ **using** *a*(*1*) *l*(*3*) **by** (*rule D.downclosed*)
moreover have $d{\cdot}l \sqsubseteq d{\cdot}x$ **using** *l*(*2*) **by** (*rule monofun-cfun-arg*)
ultimately have $l = \bot$ **using** *dx* **by** *simp*
with *atom-nonbottom*[*OF l*(*1*)] **show** *False* **by** *contradiction*
qed

13.4 Finitary and faithful deflations under joins

lemma *finitary-join*:
fixes *p q* :: *'a*::*bounded-complete* $\to$ *'a*
assumes *p*: *deflation p* **and** *q*: *deflation q* **and** *finitary p finitary q*
shows *finitary* ($p \sqcup q$)
using *assms*(*3*,*4*)
unfolding *finitary-def cfun-join*[*OF deflation-joinable*[*OF p q*]]
by (*blast intro*: *compact-join joinableI deflation.below*[*OF p*] *deflation.below*[*OF q*])

Faithfulness is upward closed, so it passes to joins.

lemma *faithful-mono*: *faithful p* $\Longrightarrow$ $p \sqsubseteq q$ $\Longrightarrow$ *faithful q*
unfolding *faithful-def* **by** (*metis monofun-cfun-fun below-bottom-iff*)

lemma *faithful-joinI1*:
fixes *p q* :: *'a*::*bounded-complete* $\to$ *'a*
shows *deflation p* $\Longrightarrow$ *deflation q* $\Longrightarrow$ *faithful p* $\Longrightarrow$ *faithful* ($p \sqcup q$)
by (*erule faithful-mono*[*OF - joinableD*(*1*)[*OF deflation-joinable*]])

lemma *faithful-joinI2*:
fixes *p q* :: *'a*::*bounded-complete* $\to$ *'a*
shows *deflation p* $\Longrightarrow$ *deflation q* $\Longrightarrow$ *faithful q* $\Longrightarrow$ *faithful* ($p \sqcup q$)
by (*erule faithful-mono*[*OF - joinableD*(*2*)[*OF deflation-joinable*]])

In a separated sum, the faithful deflations are exactly those above the discriminator.

lemma *faithful-iff-discriminator-below*:
  fixes $d :: 'a::\{bifinite, bounded\text{-}complete, separated\text{-}sum\} \to 'a$
  assumes *d*: *deflation d*
  shows *faithful d* $\longleftrightarrow$ *discriminator* $\sqsubseteq$ *d*
proof
  assume *f*: *faithful d*
  show *discriminator* $\sqsubseteq$ *d*
  proof (*rule cfun-belowI*)
    fix *x* **show** *discriminator*·*x* $\sqsubseteq$ *d*·*x*
    proof (*cases x* $= \bot$)
      case *False*
      with *f* **have** *d*·*x* $\neq \bot$ **unfolding** *faithful-def* **by** *blast*
      then obtain *a* **where** *a*: *atom a a* $\sqsubseteq$ *d*·*x* **by** (*rule atom-below*)
      have *a* $\sqsubseteq$ *x* **using** *a*(*2*) *deflation.below*[*OF d*] **by** (*rule below-trans*)
      with *a* **show** *?thesis* **by** (*simp add*: *discriminator-eq*)
    qed *simp*
  qed
next
  assume *discriminator* $\sqsubseteq$ *d*
  with *discriminator-faithful* **show** *faithful d* **by** (*rule faithful-mono*)
qed

Hence the discriminator lies below every nonbottom semi-factor: it is the only atom of the lattice of semi-factors.

lemma *discriminator-below-semi-factor-defl*:
  fixes $d :: 'a::\{bifinite, distributive\text{-}cpo, separated\text{-}sum\} \to 'a$
  assumes *d*: *semi-factor-defl d* **and** *nz*: *d* $\neq \bot$
  shows *discriminator* $\sqsubseteq$ *d*
proof −
  interpret *D*: *semi-factor-defl d* **by** (*rule d*)
  from *nz* **obtain** *x* **where** *x*: *d*·*x* $\neq \bot$ **using** *cfun-eqI*[*of d* $\bot$] **by** *auto*
  have *faithful d* **by** (*rule semi-factor-defl-faithful*[*OF d D.idem x*])
 then show *?thesis* **by** (*simp add*: *faithful-iff-discriminator-below*[*OF D.deflation*])
qed

13.5 Local finite detail as a deflation predicate

Every compact element of the image is fixed by a finitary level of the whole type. The level is not relative to the image, which makes the predicate admissible for free: a compact element fixed by the limit of a chain is fixed at some finite stage. At *ID* it is *locally-finitely-detailed*. The strict combinators need faithful levels of their components; these come from the components being separated sums, as a sort constraint. Lifting needs nothing.

locale *locally-finitely-detailed-defl* = *deflation d* **for** $d :: 'a::\{bifinite, distributive\text{-}cpo\} \to 'a$ +

assumes *level-image*: *compact* $k \Longrightarrow d{\cdot}k = k \Longrightarrow \exists\, F.$ *finitary-semi-factor* $F \wedge k \lfloor F = k$

lemma (**in** *locally-finitely-detailed-defl*) *deflation*: *deflation d* **by** (*fact deflation-axioms*)

lemmas (**in** *locally-finitely-detailed-defl*) *level-image-defl* = *level-image*[*unfolded ex-finitary-semi-factor-iff*]

The introduction rule asks for the levels as deflations, which is the form the constructions produce.

lemma *locally-finitely-detailed-deflI*:
assumes *deflation d*
and $\bigwedge k.$ *compact* $k \Longrightarrow d{\cdot}k = k \Longrightarrow \exists\, e.$ *semi-factor-defl* $e \wedge$ *finitary* $e \wedge e{\cdot}k = k$
shows *locally-finitely-detailed-defl d*
using *assms*
by (*simp add*: *locally-finitely-detailed-defl-def locally-finitely-detailed-defl-axioms-def ex-finitary-semi-factor-iff*)

lemma *locally-finitely-detailed-defl-bottom*:
locally-finitely-detailed-defl ($\bot$ :: $'a$::{*bifinite*, *distributive-cpo*} $\to$ $'a$)
by (*rule locally-finitely-detailed-deflI*[*OF deflation-bottom*])
(*use semi-factor-defl-bottom finitary-bottom* **in** *auto*)

lemma *locally-finitely-detailed-defl-ID*:
locally-finitely-detailed-defl (*ID* :: $'a$::*locally-finitely-detailed* $\to$ $'a$)
by (*rule locally-finitely-detailed-deflI*[*OF deflation-ID*])
(*rule compact-level*[*unfolded ex-finitary-semi-factor-iff*]; *simp*)

lemma *locally-finitely-detailed-class-deflI*:
assumes *d*: *locally-finitely-detailed-defl* (*ID* :: $'a$::{*bifinite*, *distributive-cpo*} $\to$ $'a$)
shows *OFCLASS*($'a$, *locally-finitely-detailed-class*)
by *intro-classes*
(*rule locally-finitely-detailed-defl.level-image*[*OF d*]; *simp*)

lemma *adm-locally-finitely-detailed-defl*:
adm (*locally-finitely-detailed-defl* :: ($'a$::{*bifinite*, *distributive-cpo*} $\to$ $'a$) $\Rightarrow$ *bool*)
proof (*rule admI*)
fix *Y* :: *nat* $\Rightarrow$ $'a \to 'a$ **assume** *Y*: *chain Y* **and** *P*: $\forall\, i.$ *locally-finitely-detailed-defl* (*Y i*)
show *locally-finitely-detailed-defl* ($\bigsqcup i.$ *Y i*)
proof (*rule locally-finitely-detailed-deflI*)
show *deflation* ($\bigsqcup i.$ *Y i*)
by (*rule admD*[*OF adm-deflation Y*]) (*use P* **in** ‹*simp add*: *locally-finitely-detailed-defl-def*›)
fix *k* **assume** *k*: *compact k* **and** *fk*: ($\bigsqcup i.$ *Y i*)·*k* = *k*
have *chk*: *chain* ($\lambda i.$ *Y i*·*k*) **by** (*rule ch2ch-Rep-cfunL*[*OF Y*])
have *k* $\sqsubseteq$ ($\bigsqcup i.$ *Y i*·*k*) **using** *fk* **by** (*simp add*: *contlub-cfun-fun*[*OF Y*])
then obtain *n* **where** *k* $\sqsubseteq$ *Y n*·*k* **using** *compactD2*[*OF k chk*] **by** *blast*

```
    then have Y n·k = k
    using deflation.below[OF locally-finitely-detailed-defl.deflation[OF P[rule-format]]]
      by (rule below-antisym[rotated])
    then show ∃ e. semi-factor-defl e ∧ finitary e ∧ e·k = k
      by (rule locally-finitely-detailed-defl.level-image-defl[OF P[rule-format] k])
  qed
qed

lemma locally-finitely-detailed-defl-u-map:
  assumes f: locally-finitely-detailed-defl (f :: 'a::{bifinite, distributive-cpo} → 'a)
  shows locally-finitely-detailed-defl (u-map·f)
proof (rule locally-finitely-detailed-deflI)
  show deflation (u-map·f)
    by (rule deflation-u-map[OF locally-finitely-detailed-defl.deflation[OF f]])
  fix k :: 'a u assume k: compact k and fk: u-map·f·k = k
  show ∃ e. semi-factor-defl e ∧ finitary e ∧ e·k = k
  proof (cases k)
    case bottom
     then show ?thesis using semi-factor-defl-bottom finitary-bottom by (intro
exI[of - ⊥]) simp
  next
    case (up k')
    with k fk have compact k' f·k' = k' by simp-all
    from locally-finitely-detailed-defl.level-image-defl[OF f this] obtain e
      where e: semi-factor-defl e finitary e e·k' = k' by blast
    with up show ?thesis
     by (intro exI[of - u-map·e]) (simp add: semi-factor-defl-u-map finitary-u-map)
  qed
qed
```

The strict combinators need faithful levels of their components. A level fixing a nonbottom element of a separated sum is faithful (*semi-factor-defl-faithful*); the branch of a sum not containing the element gets the discriminator.

```
lemma locally-finitely-detailed-defl-ssum-map:
  assumes f: locally-finitely-detailed-defl (f :: 'a::{bifinite, distributive-cpo, sepa-
rated-sum} → 'a)
      and g: locally-finitely-detailed-defl (g :: 'b::{bifinite, distributive-cpo, sepa-
rated-sum} → 'b)
  shows locally-finitely-detailed-defl (ssum-map·f·g)
proof (rule locally-finitely-detailed-deflI)
  note df = locally-finitely-detailed-defl.deflation[OF f]
    and dg = locally-finitely-detailed-defl.deflation[OF g]
  show deflation (ssum-map·f·g) by (rule deflation-ssum-map[OF df dg])
  note d = discriminator-semi-factor discriminator-finitary discriminator-faithful
  fix k :: 'a ⊕ 'b assume k: compact k and fk: ssum-map·f·g·k = k
  show ∃ e. semi-factor-defl e ∧ finitary e ∧ e·k = k
  proof (cases k)
    case bottom
     then show ?thesis using semi-factor-defl-bottom finitary-bottom by (intro
```

exI[*of* - ⊥]) *simp*
next
case (*sinl a*)
with *k fk* **have** *compact a f*·*a* = *a* **by** (*simp-all add*: *deflation-strict*[*OF df*])
from *locally-finitely-detailed-defl.level-image-defl*[*OF f this*] **obtain** *e*
where *e*: *semi-factor-defl e finitary e e*·*a* = *a* **by** *blast*
have *faithful e* **using** *e*(*1*,*3*) *sinl*(*2*) **by** (*rule semi-factor-defl-faithful*)
with *sinl e* **show** *?thesis*
by (*intro exI*[*of* - *ssum-map*·*e*·*discriminator*] *conjI*
semi-factor-defl-ssum-map finitary-ssum-map d) *simp-all*
next
case (*sinr b*)
with *k fk* **have** *compact b g*·*b* = *b* **by** (*simp-all add*: *deflation-strict*[*OF dg*])
from *locally-finitely-detailed-defl.level-image-defl*[*OF g this*] **obtain** *e*
where *e*: *semi-factor-defl e finitary e e*·*b* = *b* **by** *blast*
have *faithful e* **using** *e*(*1*,*3*) *sinr*(*2*) **by** (*rule semi-factor-defl-faithful*)
with *sinr e* **show** *?thesis*
by (*intro exI*[*of* - *ssum-map*·*discriminator*·*e*] *conjI*
semi-factor-defl-ssum-map finitary-ssum-map d) *simp-all*
qed
qed

lemma *locally-finitely-detailed-defl-sprod-map*:
assumes *f*: *locally-finitely-detailed-defl* (*f* :: *'a*::{*bifinite*, *distributive-cpo*, *separated-sum*} → *'a*)
and *g*: *locally-finitely-detailed-defl* (*g* :: *'b*::{*bifinite*, *distributive-cpo*, *separated-sum*} → *'b*)
shows *locally-finitely-detailed-defl* (*sprod-map*·*f*·*g*)
proof (*rule locally-finitely-detailed-deflI*)
note *df* = *locally-finitely-detailed-defl.deflation*[*OF f*]
and *dg* = *locally-finitely-detailed-defl.deflation*[*OF g*]
show *deflation* (*sprod-map*·*f*·*g*) **by** (*rule deflation-sprod-map*[*OF df dg*])
fix *k* :: *'a* ⊗ *'b* **assume** *k*: *compact k* **and** *fk*: *sprod-map*·*f*·*g*·*k* = *k*
obtain *a b* **where** *ab*: *k* = (:*a*, *b*:) *compact a compact b f*·*a* = *a g*·*b* = *b*
proof (*cases k*)
case *bottom*
with *deflation-strict*[*OF df*] *deflation-strict*[*OF dg*] **show** *thesis*
by (*intro that*[*of* ⊥ ⊥]) *simp-all*
next
case (*spair a b*)
with *k fk* **show** *thesis*
by (*intro that*[*of a b*]) (*simp-all add*: *compact-spair-iff spair-eq-iff*
deflation-strict[*OF df*] *deflation-strict*[*OF dg*])
qed
from *locally-finitely-detailed-defl.level-image-defl*[*OF f ab*(*2*,*4*)] **obtain** *e*
where *e*: *semi-factor-defl e finitary e e*·*a* = *a* **by** *blast*
from *locally-finitely-detailed-defl.level-image-defl*[*OF g ab*(*3*,*5*)] **obtain** *e′*
where *e′*: *semi-factor-defl e′ finitary e′ e′*·*b* = *b* **by** *blast*
show ∃ *e*. *semi-factor-defl e* ∧ *finitary e* ∧ *e*·*k* = *k*

proof (*cases* $a = \bot \vee b = \bot$)
case *True*
then have $k = \bot$ **using** *ab(1)* **by** *auto*
then show *?thesis* **using** *semi-factor-defl-bottom finitary-bottom* **by** (*intro exI*[*of* - $\bot$]) *simp*
next
case *False*
then have *faithful e faithful e′*
using *semi-factor-defl-faithful*[*OF e*(*1*,*3*)] *semi-factor-defl-faithful*[*OF e′*(*1*,*3*)]
by *blast+*
with *False ab(1) e e′* **show** *?thesis*
by (*intro exI*[*of* - *sprod-map*·*e*·*e′*]) (*simp add*: *semi-factor-defl-sprod-map finitary-sprod-map*)
qed
qed

lemma *locally-finitely-detailed-defl-iso*:
fixes *abs* :: *′r*::{*bifinite*, *distributive-cpo*} → *′t*::{*bifinite*, *distributive-cpo*} **and** *rep*
assumes *iso-abs*: $\bigwedge x.\ rep{\cdot}(abs{\cdot}x) = x$ **and** *iso-rep*: $\bigwedge y.\ abs{\cdot}(rep{\cdot}y) = y$
and *f*: *locally-finitely-detailed-defl f*
shows *locally-finitely-detailed-defl* (*abs oo f oo rep*)
proof (*rule locally-finitely-detailed-deflI*)
have *iso*: *iso abs rep* **by** (*rule iso.intro*[*OF iso-abs iso-rep*])
interpret *iso abs rep* **by** (*fact iso*)
show *deflation* (*abs oo f oo rep*)
by (*rule deflation-abs-rep*[*OF abs-iso rep-iso locally-finitely-detailed-defl.deflation*[*OF f*]])
fix *k* **assume** *k*: *compact k* **and** *fk*: (*abs oo f oo rep*)·*k* = *k*
have *compact* (*rep*·*k*) **using** *k* **by** (*rule compact-rep*)
moreover have *f*·(*rep*·*k*) = *rep*·*k*
proof −
have *rep*·(*abs*·(*f*·(*rep*·*k*))) = *rep*·*k* **using** *fk* **by** *simp*
then show *?thesis* **by** *simp*
qed
ultimately obtain *e* **where** *e*: *semi-factor-defl e finitary e e*·(*rep*·*k*) = *rep*·*k*
using *locally-finitely-detailed-defl.level-image-defl*[*OF f*] **by** *blast*
show ∃ *e*. *semi-factor-defl e* ∧ *finitary e* ∧ *e*·*k* = *k*
using *e* **by** (*intro exI*[*of* - *abs oo e oo rep*])
(*simp add*: *semi-factor-defl-iso*[*OF iso-abs iso-rep*] *finitary-iso*[*OF iso-abs iso-rep*])
qed

lemmas [*domain-category-adm*] = *adm-locally-finitely-detailed-defl*
lemmas [*domain-category-bottom*] = *locally-finitely-detailed-defl-bottom*
lemmas [*domain-category-iso*] = *locally-finitely-detailed-defl-iso*
lemmas [*domain-category-map*] = *locally-finitely-detailed-defl-u-map*
locally-finitely-detailed-defl-ssum-map locally-finitely-detailed-defl-sprod-map
lemmas [*domain-category-ID*] = *locally-finitely-detailed-defl-ID*

lemmas [*domain-category-deflation*] = *locally-finitely-detailed-defl.deflation*
lemmas [*domain-category-class*] = *locally-finitely-detailed-class-deflI*

The class instances of the strict combinators, from the map rules at *ID*.

instance *ssum* :: ({*locally-finitely-detailed*, *separated-sum*}, {*locally-finitely-detailed*, *separated-sum*})
locally-finitely-detailed
by (*rule locally-finitely-detailed-class-deflI*, *subst ssum-map-ID*[*symmetric*],
rule locally-finitely-detailed-defl-ssum-map; *rule locally-finitely-detailed-defl-ID*)

instance *sprod* :: ({*locally-finitely-detailed*, *separated-sum*}, {*locally-finitely-detailed*, *separated-sum*})
locally-finitely-detailed
by (*rule locally-finitely-detailed-class-deflI*, *subst sprod-map-ID*[*symmetric*],
rule locally-finitely-detailed-defl-sprod-map; *rule locally-finitely-detailed-defl-ID*)

instance *u* :: (*locally-finitely-detailed*) *locally-finitely-detailed*
by (*rule locally-finitely-detailed-class-deflI*, *subst u-map-ID*[*symmetric*],
rule locally-finitely-detailed-defl-u-map, *rule locally-finitely-detailed-defl-ID*)

With finite height the whole domain is a finitary level of detail. This cannot be a subclass statement (*chfin* and *flat* do not entail *bifinite*), but it makes every bifinite flat type locally finitely detailed in one line.

lemma *chfin-locally-finitely-detailedI*:
OFCLASS('*a*::{*chfin*, *bifinite*, *distributive-cpo*}, *locally-finitely-detailed-class*)
by *intro-classes* (*unfold ex-finitary-semi-factor-iff*,
intro exI[*of - ID*], *simp add*: *semi-factor-defl-ID finitary-def*)

instance *lift* :: (*countable*) *locally-finitely-detailed*
by (*rule chfin-locally-finitely-detailedI*)

13.6 Tuples

Under the isomorphism with families of semi-factors, the finitary semi-factors of the tuples are the finitely supported families of finitary semi-factors.

lemma *finitary-semi-factor-of-family*:
fixes *F* :: '*d*::*countable* ⇒ '*v*::{*bifinite*, *distributive-cpo*} *semi-factor*
shows *finitary-semi-factor* (*semi-factor-of-family F*) ⟷
finite {*d*. *F d* ≠ ⊥} ∧ (∀ *d*. *finitary-semi-factor* (*F d*))
proof *safe*
assume *fin*: *finitary-semi-factor* (*semi-factor-of-family F*)
then have *cpt*: *compact* (*t* ↓ *semi-factor-of-family F*) **for** *t*
by (*simp add*: *finitary-semi-factor-def finitary-def*)
show *finitary-semi-factor* (*F d*) **for** *d*
unfolding *finitary-semi-factor-def finitary-def*
proof
fix *x*

```
  have compact ((tupd d·⊥·x ⌊ semi-factor-of-family F)⟨d⟩) by (rule compact-tget[OF
cpt])
    then show compact (x ⌊ F d) by (simp add: tupd-bot-tget)
  qed
  show finite {d. F d ≠ ⊥}
  proof (rule ccontr)
    assume inf: infinite {d. F d ≠ ⊥}
    have ∃ x. x ⌊ F d ≠ ⊥ if F d ≠ ⊥ for d
    proof (rule ccontr)
      assume ∄ x. x ⌊ F d ≠ ⊥
      then have semi-factor-p (F d) = semi-factor-p ⊥ by (simp add: cfun-eq-iff
semi-factor-p-bot)
      with that show False by (simp add: semi-factor-p-inject)
    qed
    then obtain x where x: ⋀d. F d ≠ ⊥ ⟹ x d ⌊ F d ≠ ⊥ by metis
    define t :: ('d, 'v) tuple where t = Abs-tuple x
    have {d. F d ≠ ⊥} ⊆ tsupp (t ⌊ semi-factor-of-family F)
      using x by (auto simp: tsupp-def t-def Abs-tuple-inverse')
    with inf compact-finite-tsupp[OF cpt] show False by (metis finite-subset)
  qed
next
  assume fin: finite {d. F d ≠ ⊥} and F: ∀ d. finitary-semi-factor (F d)
  show finitary-semi-factor (semi-factor-of-family F)
    unfolding finitary-semi-factor-def finitary-def
  proof
    fix t show compact (t ⌊ semi-factor-of-family F)
    proof (rule compact-tupleI)
      have tsupp (t ⌊ semi-factor-of-family F) ⊆ {d. F d ≠ ⊥}
        by (auto simp: tsupp-def semi-factor-p-bot)
      then show finite (tsupp (t ⌊ semi-factor-of-family F)) using fin by (rule
finite-subset)
      show compact ((t ⌊ semi-factor-of-family F)⟨d⟩) for d
        using F by (simp add: finitary-semi-factor-def finitary-def)
    qed
  qed
qed
```

A compact tuple has finite support and compact components. A finitary level containing it is the family of finitary levels containing the components on its support, and ⊥ elsewhere.

```
instance tuple :: (countable, locally-finitely-detailed) locally-finitely-detailed
proof
  fix K :: ('a, 'b) tuple assume K: compact K
  define F where F d = (if d ∈ tsupp K
     then (SOME G. finitary-semi-factor G ∧ K⟨d⟩ ⌊ G = K⟨d⟩) else ⊥) for d
  have F: finitary-semi-factor (F d) ∧ K⟨d⟩ ⌊ F d = K⟨d⟩ for d
  proof (cases d ∈ tsupp K)
    case True
    then show ?thesis using someI-ex[OF compact-level[OF compact-tget[OF K,
```

of d]]]
  by (*simp add*: *F-def*)
 next
  case *False*
  then show *?thesis*
  by (*simp add*: *F-def tsupp-def finitary-semi-factor-def finitary-def semi-factor-p-bot*)
 qed
 have {*d*. *F d* ≠ ⊥} ⊆ *tsupp K* **by** (*auto simp*: *F-def*)
  then have *finite* {*d*. *F d* ≠ ⊥} **using** *compact-finite-tsupp*[*OF K*] **by** (*rule finite-subset*)
 with *F* **have** *finitary-semi-factor* (*semi-factor-of-family F*)
  by (*simp add*: *finitary-semi-factor-of-family*)
 moreover have *K* ⌊ *semi-factor-of-family F* = *K* **using** *F* **by** (*simp add*: *tuple-eq-iff*)
 ultimately show ∃ *G*. *finitary-semi-factor G* ∧ *K* ⌊ *G* = *K* **by** *blast*
qed

14 Projections onto semi-factors

Levels of detail are compact semi-factors of the key domain, acting on keys by projection; masks are arbitrary semi-factors. The facts are stated for all semi-factors, with compactness of the semi-factor (and locally finitely detailed dI keys) where compactness of keys is needed. (They took over the names of the facts of the retired locale *lod-space*.)

abbreviation *proj* :: *'k*::{*bifinite*, *distributive-cpo*} *semi-factor* ⇒ *'k* → *'k* **where**
 proj ≡ *semi-factor-p*

14.1 Compact semi-factors project to compact keys

Over a dI-domain every compact semi-factor is a finite join of principals of compacts (*compact-semi-factor-finite-join*). Over a locally finitely detailed one the principal of a compact lies below a semi-factor all of whose elements are compact, and finite joins keep that property. So compact semi-factors project every key to a compact one. (Without local finite detail this fails: in the separated sum of the powerset of the naturals and a point, the principal of the point is the whole domain.) Projections of compact keys are compact over any dI-domain, everything below a compact element being compact.

lemma *compact-finite-lub*:
 fixes *u* :: *'a*::*cpo*
 assumes *fin*: *finite C* **and** *comp*: ⋀*c*. *c* ∈ *C* ⟹ *compact c* **and** *lub*: *C* <<| *u*
 shows *compact u*
proof (*rule compactI2*)
 fix *Y* :: *nat* ⇒ *'a* **assume** *Y*: *chain Y* **and** *uY*: *u* ⊑ (⨆ *i*. *Y i*)
 have ∃ *i*. *c* ⊑ *Y i* **if** *c* ∈ *C* **for** *c*
  by (*rule compactD2*[*OF comp*[*OF that*] *Y*])
   (*rule below-trans*[*OF is-ubD*[*OF is-lubD1*[*OF lub*] *that*] *uY*])

then obtain f **where** f: $\bigwedge c.\ c \in C \Longrightarrow c \sqsubseteq Y\ (f\ c)$ **by** *metis*
let *?n* = *Max* (*insert 0* (*f* ‘ *C*))
have $c \sqsubseteq Y$ *?n* **if** $c \in C$ **for** c
by (*rule below-trans*[*OF f*[*OF that*] *chain-mono*[*OF Y*]]) (*simp add*: *fin that*)
then have $u \sqsubseteq Y$ *?n* **by** (*intro is-lubD2*[*OF lub*] *is-ubI*)
then show $\exists i.\ u \sqsubseteq Y\ i$ **by** *blast*
qed

lemma *compact-proj-below*:
assumes $F \sqsubseteq G$ **and** $\bigwedge y.$ *compact* $(y \lfloor G)$
shows *compact* $(x \lfloor F)$
by (*metis assms semi-factor-p-mono stable-defl.lower-fixed*[*OF stable-defl-semi-factor-p*])

lemma *compact-proj-lub*:
assumes *finite* $\mathcal{A}$ **and** $\bigwedge F\ y.\ F \in \mathcal{A} \Longrightarrow$ *compact* $(y \lfloor F)$
shows *compact* $(x \lfloor$ *lub* $\mathcal{A})$
by (*rule compact-finite-lub*[*OF - - semi-factor-p-lub-is-lub*]) (*use assms* **in** *auto*)

lemma *compact-proj-principal*:
fixes k :: *'a*::*lod-domain*
assumes *compact* k
shows *compact* $(x \lfloor$ *semi-factor-principal* $k)$
proof −
obtain F **where** F: *finitary-semi-factor* $F\ k \lfloor F = k$ **using** *compact-level*[*OF assms*] **by** *blast*
have *semi-factor-principal* $k \sqsubseteq F$ **using** $F(2)$ **by** (*simp add*: *semi-factor-principal-below-iff*)
then show *?thesis*
by (*rule compact-proj-below*) (*use* $F(1)$ **in** ‹*simp add*: *finitary-semi-factor-def finitary-def*›)
qed

theorem *compact-semi-factor-proj*:
fixes K :: *'a*::*lod-domain semi-factor*
assumes *compact* K
shows *compact* $(x \lfloor K)$
proof −
obtain A **where** A: *finite* A $A \subseteq \{k.$ *compact* $k\}$ $K =$ *lub* (*semi-factor-principal* ‘ A)
using *compact-semi-factor-finite-join*[*OF assms*] **by** *blast*
show *?thesis* **unfolding** $A(3)$
by (*rule compact-proj-lub*) (*use* $A(1,2)$ **in** ‹*auto intro*: *compact-proj-principal*›)
qed

lemma *compact-proj-compact*: *compact* $k \Longrightarrow$ *compact* $(k \lfloor (F$ :: *'a*::*dI semi-factor*$))$
by (*rule compact-below-compact*[*OF - deflation.below*[*OF deflation-semi-factor-p*]])

lemma *proj-inf*: *proj* $(a \sqcap b)\cdot k =$ *proj* $a\cdot$(*proj* $b\cdot k$)
by (*simp add*: *semi-factor-proj-comp lattice-meet-commute*)

lemma *proj-sup*: *proj* $(a \sqcup b)\cdot k = proj\ a\cdot k \sqcup proj\ b\cdot k$
by (*rule semi-factor-p-sup*)

lemma *proj-bot*: *proj* $\bot = \bot$
by (*rule semi-factor-p-bot*)

lemma *compact-semi-factor-sup*:
compact $a \Longrightarrow$ *compact* $b \Longrightarrow$ *compact* $(a \sqcup b :: 'k{::}\{bifinite, distributive\text{-}cpo\}$ *semi-factor*)
by (*simp add*: *compact-join*)

lemma *compact-fold-sup*:
$(\bigwedge x.\ x \in set\ xs \Longrightarrow compact\ x) \Longrightarrow compact\ b$
$\Longrightarrow$ *compact* (*fold* $(\sqcup)$ *xs* $b :: 'k{::}\{bifinite, distributive\text{-}cpo\}$ *semi-factor*)
by (*induct xs arbitrary*: *b*) (*simp-all add*: *compact-semi-factor-sup*)

lemma *proj-defl*: *deflation* (*proj a*)
by (*rule deflation-semi-factor-p*)

lemma *proj-slide*: $proj\ a\cdot y = y \Longrightarrow proj\ a\cdot x \sqsubseteq y \Longrightarrow x \Uparrow y$
by (*rule semi-factor-defl.slide*[*OF semi-factor-defl-semi-factor-p*])

lemma *proj-compact*: *compact a* $\Longrightarrow$ *compact* (*proj a*·*k*)
for $a :: 'k{::}lod\text{-}domain\ semi\text{-}factor$
by (*rule compact-semi-factor-proj*)

lemma *compact-lives*: *compact k* $\Longrightarrow \exists c.\ compact\ c \wedge proj\ c\cdot k = k$ **for** $k :: 'k{::}dI$
by (*blast intro*: *compact-semi-factor-principal semi-factor-principal-self*)

lemma *proj-join*: $x \Uparrow y \Longrightarrow proj\ a\cdot(x \sqcup y) = proj\ a\cdot x \sqcup proj\ a\cdot y$
by (*rule stable-defl-image-join*[*OF stable-defl-semi-factor-p*])

lemma *proj-downclosed*: $proj\ a\cdot y = y \Longrightarrow x \sqsubseteq y \Longrightarrow proj\ a\cdot x = x$
by (*rule stable-defl.downclosed*[*OF stable-defl-semi-factor-p*])

lemma *proj-below*: $proj\ a\cdot k \sqsubseteq k$
by (*rule deflation.below*[*OF proj-defl*])

lemma *proj-idem* [*simp*]: $proj\ a\cdot(proj\ a\cdot k) = proj\ a\cdot k$
by (*rule deflation.idem*[*OF proj-defl*])

lemma *proj-commute*: $proj\ a\cdot(proj\ b\cdot k) = proj\ b\cdot(proj\ a\cdot k)$
by (*simp only*: *proj-inf*[*symmetric*] *lattice-meet-commute*)

lemma *proj-le*: $a \sqsubseteq b \Longrightarrow proj\ a\cdot(proj\ b\cdot k) = proj\ a\cdot k$
by (*simp only*: *proj-inf*[*symmetric*] *lattice-meet-absorb1*)

lemma *proj-mono*: $a \sqsubseteq b \Longrightarrow proj\ a\cdot k \sqsubseteq proj\ b\cdot k$

by (*metis proj-le proj-below*)

14.2 Where a key lives

definition *lives-at* :: *'k*::{*bifinite*, *distributive-cpo*} *semi-factor* ⇒ *'k* ⇒ *bool* **where**
lives-at c k ⟷ *proj c*·*k* = *k*

The LODs a key lives at form a filter, non-empty exactly for compact keys.

definition *lods-of* :: *'k*::{*bifinite*, *distributive-cpo*} ⇒ *'k semi-factor set* **where**
lods-of k = {*c*. *lives-at c k*}

lemma *lives-at-up*: *lives-at c k* ⟹ *c* ⊑ *c'* ⟹ *lives-at c' k*
unfolding *lives-at-def* **by** (*metis proj-le proj-commute*)

lemma *lives-at-inf*: *lives-at c k* ⟹ *lives-at c' k* ⟹ *lives-at* (*c* ⊓ *c'*) *k*
unfolding *lives-at-def* **by** (*simp add*: *proj-inf*)

lemma *lives-at-proj*: *lives-at a* (*proj a*·*k*)
unfolding *lives-at-def* **by** *simp*

lemma *compact-iff-lives*: *compact k* ⟷ (∃ *c*. *compact c* ∧ *lives-at c k*)
for *k* :: *'k*::*lod-domain*
unfolding *lives-at-def* **by** (*metis compact-lives proj-compact*)

lemma *lods-of-nonempty*: *compact k* ⟹ *lods-of k* ≠ {} **for** *k* :: *'k*::*dI*
unfolding *lods-of-def lives-at-def* **using** *compact-lives* **by** *blast*

Projection keeps maximal keys maximal among the keys living at the LOD.

lemma *proj-maximal*:
assumes *max*: ⋀*y*. *x* ⊑ *y* ⟹ *y* = *x* **and** *t*: *lives-at a t* **and** *le*: *proj a*·*x* ⊑ *t*
shows *t* ⊑ *proj a*·*x*
proof −
from *t* **have** *tt*: *proj a*·*t* = *t* **by** (*simp add*: *lives-at-def*)
from *proj-slide*[*OF tt le*] **have** *j*: *x* ⇑ *t* **.**
then have *x* ⊔ *t* = *x* **by** (*intro max*) (*rule joinableD*(*1*))
then have *t* ⊑ *x* **using** *joinableD*(*2*)[*OF j*] **by** *simp*
then have *proj a*·*t* ⊑ *proj a*·*x* **by** (*rule monofun-cfun-arg*)
then show *?thesis* **by** (*simp add*: *tt*)
qed

14.3 Keys maximal at an LOD

definition *maximal-at* :: *'k*::{*bifinite*, *distributive-cpo*} *semi-factor* ⇒ *'k* ⇒ *bool* **where**
maximal-at c k ⟷ *proj c*·*k* = *k* ∧ (∀ *t*. *proj c*·*t* = *t* ⟶ *k* ⊑ *t* ⟶ *t* = *k*)

Maximality passes down to coarser LODs: anything above the projection at the coarser LOD slides into a join with the key, which lives at the finer LOD and so cannot exceed it.

lemma *maximal-at-proj*:
assumes *ab*: $a \sqsubseteq b$ **and** *k*: *maximal-at b k*
shows *maximal-at a* (*proj a*·*k*)
unfolding *maximal-at-def*
proof (*intro conjI allI impI*)
show *proj a*·(*proj a*·*k*) = *proj a*·*k* **by** *simp*
fix *t* **assume** *t*: *proj a*·*t* = *t* **and** *le*: *proj a*·*k* ⊑ *t*
have *kb*: *proj b*·*k* = *k* **using** *k* **by** (*simp add*: *maximal-at-def*)
have *tb*: *proj b*·*t* = *t* **using** *t lives-at-up*[*OF - ab*] **by** (*simp add*: *lives-at-def*)
have *j*: *k* ⇑ *t* **by** (*rule proj-slide*[*OF t le*])
have *proj b*·(*k* ⊔ *t*) = *k* ⊔ *t* **by** (*simp add*: *proj-join*[*OF j*] *kb tb*)
then have *k* ⊔ *t* = *k* **using** *k joinableD*(*1*)[*OF j*] **by** (*simp add*: *maximal-at-def*)
then have *t* ⊑ *k* **using** *joinableD*(*2*)[*OF j*] **by** *simp*
then have *t* ⊑ *proj a*·*k* **using** *monofun-cfun-arg*[*of t k proj a*] **by** (*simp add*: *t*)
with *le* **show** *t* = *proj a*·*k* **by** (*rule below-antisym*[*rotated*])
qed

lemma *maximal-at-lives*: *maximal-at c k* ⟹ *proj c*·*k* = *k*
by (*simp add*: *maximal-at-def*)

lemma *maximal-at-compact*: *maximal-at c k* ⟹ *compact c* ⟹ *compact k*
for *c* :: *'k*::*lod-domain semi-factor*
by (*metis maximal-at-lives proj-compact*)

A key below one living at the same LOD is that key, when it is maximal there.

lemma *maximal-at-eq*: *maximal-at c k* ⟹ *proj c*·*t* = *t* ⟹ *k* ⊑ *t* ⟹ *t* = *k*
by (*simp add*: *maximal-at-def*)

Joins of compatible maximal keys are maximal at the join of their LODs.

lemma *maximal-at-join*:
assumes *a*: *maximal-at A a* **and** *b*: *maximal-at B b* **and** *j*: *a* ⇑ *b*
shows *maximal-at* (*A* ⊔ *B*) (*a* ⊔ *b*)
unfolding *maximal-at-def*
proof (*intro conjI allI impI*)
have *aA*: *proj A*·*a* = *a* **and** *bB*: *proj B*·*b* = *b* **using** *a b* **by** (*simp-all add*: *maximal-at-def*)
have *pa*: *proj A*·(*a* ⊔ *b*) = *a*
proof −
have *proj A*·*a* ⊑ *proj A*·(*a* ⊔ *b*) **using** *joinableD*(*1*)[*OF j*] **by** (*rule monofun-cfun-arg*)
then have *a* ⊑ *proj A*·(*a* ⊔ *b*) **by** (*simp only*: *aA*)
then show *?thesis* **using** *a* **by** (*simp add*: *maximal-at-def*)
qed
have *pb*: *proj B*·(*a* ⊔ *b*) = *b*
proof −
have *proj B*·*b* ⊑ *proj B*·(*a* ⊔ *b*) **using** *joinableD*(*2*)[*OF j*] **by** (*rule monofun-cfun-arg*)
then have *b* ⊑ *proj B*·(*a* ⊔ *b*) **by** (*simp only*: *bB*)

```
  then show ?thesis using b by (simp add: maximal-at-def)
 qed
 show proj (A ⊔ B)·(a ⊔ b) = a ⊔ b by (simp only: proj-sup pa pb)
 fix t assume t: proj (A ⊔ B)·t = t and le: a ⊔ b ⊑ t
  have at: a ⊑ t and bt: b ⊑ t using le joinableD[OF j] by (blast intro: be-
low-trans)+
 have proj A·t = a
 proof −
   have a ⊑ proj A·t using monofun-cfun-arg[OF at, of proj A] by (simp add:
aA)
   then show ?thesis using a by (simp add: maximal-at-def)
 qed
 moreover have proj B·t = b
 proof −
   have b ⊑ proj B·t using monofun-cfun-arg[OF bt, of proj B] by (simp add:
bB)
   then show ?thesis using b by (simp add: maximal-at-def)
 qed
 ultimately show t = a ⊔ b using t by (simp add: proj-sup)
qed
```

A key maximal at a that has a common upper bound with K lies above the projection of K: the projection of their join is above the key, hence equal to it. (Under a pinning slicer, conformity provides the common upper bound.)

```
lemma proj-below-maximal:
 assumes K1: maximal-at a K1 and KK2: K ⊑ K2 and K1K2: K1 ⊑ K2
 shows proj a·K ⊑ K1
proof −
 have j: K ⇑ K1 by (rule joinableI[OF KK2 K1K2])
  have proj a·K1 ⊑ proj a·(K ⊔ K1) using joinableD(2)[OF j] by (rule mono-
fun-cfun-arg)
 then have K1 ⊑ proj a·(K ⊔ K1) using K1 by (simp add: maximal-at-def)
 then have eq: proj a·(K ⊔ K1) = K1 using K1 by (simp add: maximal-at-def)
  have proj a·K ⊑ proj a·(K ⊔ K1) using joinableD(1)[OF j] by (rule mono-
fun-cfun-arg)
 then show ?thesis by (simp only: eq)
qed
```

Round trip between two LODs that agree up to the LOD of a pinned key: from a key $K1$ at a compatible with the pin, go to a' through the join with the pin, and back.

```
lemma proj-round-trip:
 assumes aL: a ⊔ L = a' ⊔ L and K: maximal-at L K
   and K1: proj a·K1 = K1 and le: proj a·K ⊑ K1
 shows proj a·(proj a'·(K1 ⊔ K) ⊔ K) = K1
proof −
 have j: K1 ⇑ K using proj-slide[OF K1 le] by (simp add: joinable-sym)
 let ?J = K1 ⊔ K
 have JK: K ⊑ ?J using j by (rule joinableD)
```

have *jJ*: *proj a′·?J ⇑ K* **by** (*rule joinableI*[*OF proj-below JK*])
have *KL*: *proj L·K = K* **using** *K* **by** (*simp add*: *maximal-at-def*)
have *LJ*: *proj L·?J = K*
proof −
have *proj L·K ⊑ proj L·?J* **using** *JK* **by** (*rule monofun-cfun-arg*)
then have *K ⊑ proj L·?J* **by** (*simp only*: *KL*)
then show *?thesis* **using** *K* **by** (*simp add*: *maximal-at-def*)
qed
have *a-split*: *a = (a ⊓ a′) ⊔ (a ⊓ L)*
by (*metis aL lattice-meet-join-absorb lattice-meet-join-distrib1*)
have *proj a·?J = proj (a ⊓ a′)·?J ⊔ proj a·K*
by (*subst a-split*) (*simp add*: *proj-sup proj-inf LJ*)
moreover have *proj a·?J = K1*
by (*simp add*: *proj-join*[*OF j*] *K1 join-absorb2*[*OF le*])
ultimately have *K1 = proj (a ⊓ a′)·?J ⊔ proj a·K* **by** *simp*
then show *?thesis* **by** (*simp add*: *proj-join*[*OF jJ*] *proj-inf*)
qed

14.4 Masks

A *mask* is a semi-factor, possibly non-compact, acting on LODs by meet: truncating an LOD to a mask keeps the part of it within the mask. (This replaces an abstract type of truncations, i.e. of maps obeying $\tau\ (x \sqcap y) = x \sqcap \tau\ y$, which on a complete lattice are exactly the meets with a fixed element.)

lemma *truncate-inf*: *τ ⊓ (x ⊓ y) = x ⊓ (τ ⊓ y)* **for** *τ* :: *'l*::*lattice-complete*
by (*rule lattice-meet-left-commute*)

context
fixes *τ* :: *'l*::{*lattice-complete*, *distributive-cpo*}
begin

lemma *truncate-below*: *(τ ⊓ x) ⊑ x*
by (*rule lattice-meet-le2*)

lemma *truncate-idem*: *(τ ⊓ (τ ⊓ x)) = (τ ⊓ x)*
by (*rule lattice-meet-left-idem*)

lemma *truncate-mono*: *x ⊑ y ⟹ (τ ⊓ x) ⊑ (τ ⊓ y)*
by (*rule lattice-meet-mono*[*OF below-refl*])

lemma *truncate-inf-hom*: *(τ ⊓ (x ⊓ y)) = (τ ⊓ x) ⊓ (τ ⊓ y)*
by (*simp add*: *lattice-meet-ac*)

lemma *truncate-sup*: *(τ ⊓ (x ⊔ y)) = (τ ⊓ x) ⊔ (τ ⊓ y)*
by (*rule lattice-meet-join-distrib1*)

lemma *truncate-bot*: *(τ ⊓ ⊥) = ⊥*

by (*rule lattice-meet-bot-right*)

lemma *truncate-fix-down*: $(\tau \sqcap c) = c \Longrightarrow d \sqsubseteq c \Longrightarrow (\tau \sqcap d) = d$
by (*metis lattice-meet-absorb2 lattice-meet-le1 below-trans*)

lemma *truncate-fix-sup*: $(\tau \sqcap c) = c \Longrightarrow (\tau \sqcap d) = d \Longrightarrow (\tau \sqcap (c \sqcup d)) = c \sqcup d$
by (*simp add*: *truncate-sup*)

lemma *truncate-greatest*: $(\tau \sqcap c) = c \Longrightarrow c \sqsubseteq x \Longrightarrow c \sqsubseteq (\tau \sqcap x)$
by (*metis truncate-mono*)

Within a, truncating more than a is truncating a.

lemma *truncate-inf-sup*: $(\tau \sqcap (a \sqcup b)) \sqcap a = (\tau \sqcap a)$
by (*metis lattice-meet-assoc lattice-meet-commute lattice-meet-join-absorb*)

end

lemma *truncate-inf-comp*: $(\varrho \sqcap \tau) \sqcap x = \varrho \sqcap (\tau \sqcap x)$ **for** $x :: 'l$::*lattice-complete*
by (*rule lattice-meet-assoc*)

lemma *truncate-top*: *lattice-top* $\sqcap x = x$ **for** $x :: 'l$::*lattice-complete*
by (*rule lattice-meet-top-left*)

lemma *truncate-bot-trunc*: $\bot \sqcap x = \bot$ **for** $x :: 'l$::*lattice-complete*
by (*rule lattice-meet-bot-left*)

14.5 Masks acting on keys

A mask acts on keys by projection, through any LOD the key lives at.

lemma *truncate-proj-irrelevant*:
assumes *c*: *lives-at c k* **and** *c'*: *lives-at c' k*
shows *proj* $((\tau \sqcap c))\cdot k =$ *proj* $((\tau \sqcap c'))\cdot k$
proof −
have *e*: $(\tau \sqcap c) \sqcap c' = (\tau \sqcap c') \sqcap c$ **by** (*metis truncate-inf lattice-meet-commute*)
have *proj* $((\tau \sqcap c))\cdot k =$ *proj* $((\tau \sqcap c))\cdot($*proj* $c'\cdot k)$ **using** *c'* **by** (*simp add*: *lives-at-def*)
also have $\ldots =$ *proj* $((\tau \sqcap c) \sqcap c')\cdot k$ **by** (*simp only*: *proj-inf*)
also have $\ldots =$ *proj* $((\tau \sqcap c') \sqcap c)\cdot k$ **by** (*simp only*: *e*)
also have $\ldots =$ *proj* $((\tau \sqcap c'))\cdot($*proj* $c\cdot k)$ **by** (*simp only*: *proj-inf*)
also have $\ldots =$ *proj* $((\tau \sqcap c'))\cdot k$ **using** *c* **by** (*simp add*: *lives-at-def*)
finally show *?thesis* **.**
qed

definition *truncate-key* :: $'k$::{*bifinite*, *distributive-cpo*} *semi-factor* $\Rightarrow$ $'k \Rightarrow 'k$
where
truncate-key τ *k* = *proj* $\tau\cdot k$

lemma *truncate-key-eq*:
assumes *c*: *lives-at c k*

shows *truncate-key* τ k = *proj* $((\tau \sqcap c))\cdot k$
using c **by** (*simp add*: *truncate-key-def proj-inf lives-at-def*)

14.6 Compact maps

A map into a pcpo is compact in the pointwise order exactly when it has finite support and compact values.

lemma *compact-fun-iff*:
fixes $f :: 'a \Rightarrow 'b$::*pcpo*
shows *compact* $f \longleftrightarrow$ *finite* $\{x.\ f\ x \neq \bot\} \wedge (\forall x.\ \mathit{compact}\ (f\ x))$
proof
assume c: *compact* f
have *compact* $(f\ x)$ **for** x
proof (*rule compactI2*)
fix $C :: \mathit{nat} \Rightarrow 'b$ **assume** C: *chain* C **and** *le*: $f\ x \sqsubseteq (\bigsqcup i.\ C\ i)$
let $?Y = \lambda i.\ f(x := C\ i)$
have Y: *chain* $?Y$ **by** (*rule chainI*, *rule fun-belowI*) (*simp add*: *chainE*[*OF C*])
have $f \sqsubseteq (\bigsqcup i.\ ?Y\ i)$
proof (*rule fun-belowI*)
fix y **show** $f\ y \sqsubseteq (\bigsqcup i.\ ?Y\ i)\ y$ **by** (*cases* $y = x$) (*simp-all add*: *lub-fun*[*OF Y*] *le*)
qed
then obtain i **where** $f \sqsubseteq ?Y\ i$ **using** *compactD2*[*OF c Y*] **by** *blast*
then show $\exists i.\ f\ x \sqsubseteq C\ i$ **by** (*metis fun-belowD fun-upd-same*)
qed
moreover have *finite* $\{x.\ f\ x \neq \bot\}$
proof (*rule ccontr*)
assume *infinite* $\{x.\ f\ x \neq \bot\}$
then obtain $g :: \mathit{nat} \Rightarrow 'a$ **where** g: *inj g range* $g \subseteq \{x.\ f\ x \neq \bot\}$
using *infinite-countable-subset* **by** *blast*
let $?Y = \lambda n\ y.$ *if* $y \in g$ ‘ $\{n..\}$ *then* $\bot$ *else* $f\ y$
have Y: *chain* $?Y$
proof (*rule chainI*, *rule fun-belowI*)
fix $n\ y$ **show** $?Y\ n\ y \sqsubseteq ?Y\ (\mathit{Suc}\ n)\ y$ **by** (*cases* $y \in g$ ‘ $\{n..\}$) *auto*
qed
have $f \sqsubseteq (\bigsqcup n.\ ?Y\ n)$
proof (*rule fun-belowI*)
fix y
obtain n **where** n: $y \notin g$ ‘ $\{n..\}$
proof (*cases* $y \in$ *range* g)
case *True*
then obtain k **where** $y = g\ k$ **by** *blast*
then have $y \notin g$ ‘ $\{\mathit{Suc}\ k..\}$ **using** $g(1)$ **by** (*auto simp*: *inj-eq*)
then show *?thesis* **by** (*rule that*)
next
case *False*
then show *?thesis* **by** (*intro that*[*of 0*]) *auto*
qed
then have $f\ y = ?Y\ n\ y$ **by** *simp*

also have ... $\sqsubseteq$ ($\bigsqcup$ *n. ?Y n y*) **by** (*rule is-ub-thelub*[*OF ch2ch-fun*[*OF Y*]])
finally show *f y* $\sqsubseteq$ ($\bigsqcup$ *n. ?Y n*) *y* **by** (*simp add*: *lub-fun*[*OF Y*])
qed
then obtain *n* **where** *f* $\sqsubseteq$ *?Y n* **using** *compactD2*[*OF c Y*] **by** *blast*
then have *f* (*g n*) $\sqsubseteq$ $\bot$ **using** *fun-belowD*[*of f ?Y n g n*] **by** *simp*
then have *f* (*g n*) = $\bot$ **by** (*rule bottomI*)
with *g*(*2*) **show** *False* **by** *blast*
qed
ultimately show *finite* {*x*. *f x* $\neq$ $\bot$} $\wedge$ ($\forall$ *x*. *compact* (*f x*)) **by** *blast*
next
assume *finite* {*x*. *f x* $\neq$ $\bot$} $\wedge$ ($\forall$ *x*. *compact* (*f x*))
then have *fin*: *finite* {*x*. *f x* $\neq$ $\bot$} **and** *comp*: $\bigwedge$*x*. *compact* (*f x*) **by** *blast*+
show *compact f*
proof (*rule compactI2*)
fix *Y* :: *nat* $\Rightarrow$ *'a* $\Rightarrow$ *'b* **assume** *Y*: *chain Y* **and** *le*: *f* $\sqsubseteq$ ($\bigsqcup$ *i*. *Y i*)
have $\exists$ *i*. *f x* $\sqsubseteq$ *Y i x* **for** *x*
proof −
have *f x* $\sqsubseteq$ ($\bigsqcup$ *i*. *Y i x*) **using** *fun-belowD*[*OF le*, *of x*] **by** (*simp add*: *lub-fun*[*OF Y*])
then show *?thesis* **by** (*rule compactD2*[*OF comp ch2ch-fun*[*OF Y*]])
qed
then obtain *h* **where** *h*: $\bigwedge$*x*. *f x* $\sqsubseteq$ *Y* (*h x*) *x* **by** *metis*
let *?n* = *Max* (*insert 0* (*h* ' {*x*. *f x* $\neq$ $\bot$}))
have *f x* $\sqsubseteq$ *Y ?n x* **for** *x*
proof (*cases f x* = $\bot$)
case *True* **then show** *?thesis* **by** *simp*
next
case *False*
then have *h x* $\leq$ *?n* **by** (*intro Max-ge*) (*use fin* **in** *auto*)
then show *?thesis* **by** (*rule below-trans*[*OF h fun-belowD*[*OF chain-mono*[*OF Y*]]])
qed
then show $\exists$ *i*. *f* $\sqsubseteq$ *Y i* **by** (*blast intro*: *fun-belowI*)
qed
qed

14.7 Levels of detail

LODs are the compact semi-factors of the keys; the synonym only shortens the type.

type-synonym *'k lod* = *'k semi-factor*

end
theory *Slicer*
imports *ValsetBase Congruence LOD HOL−Library.FSet*
begin

default-sort *type*

15 Levels of detail as syntactic dimension sets

Dimensions are drawn from a (countable, nominal) label set $'d$; each label is evaluated on rows by a continuous row-level expression into a value domain. Two labels may share an expression: they are then functionally dependent, but distinct. The key of a row is the tuple of all its dimensions' values; a level of detail then cuts it.

The image of a row under the cut key need *not* be a stable subdomain of the tuple domain (with an expression dimension $c \approx a + b$ the allowable sub-rows are not closed under existing joins), and we never require it to be: the domain stays the source rows, and functional dependence is stated relationally via kernel orders.

locale *dim-eval* =
  fixes *deval* :: $'d \Rightarrow ('r{::}pcpo \to 'v{::}pcpo)$ $(\langle[\![\text{-}]\!]\rangle)$
begin

definition *keyof* :: $'r \to ('d, 'v)$ *tuple* **where**
  keyof = $(\Lambda\ r.$ *Abs-tuple* $(\lambda d.\ [\![d]\!]\cdot r))$

lemma *keyof-beta*: *keyof*$\cdot r$ = *Abs-tuple* $(\lambda d.\ [\![d]\!]\cdot r)$
  unfolding *keyof-def* **by** (*simp add*: *cont-Abs-tuple*)

lemma *keyof-tget* [*simp*]: (*keyof*$\cdot r)\langle d\rangle = [\![d]\!]\cdot r$
  by (*simp add*: *keyof-beta Abs-tuple-inverse'*)

15.1 Kernel orders and functional dependence

The kernel order of a projection p on keys (in practice a deflation, such as an LOD's): rows compared by what p keeps of their keys.

definition *kern* :: $(('d, 'v)$ *tuple* $\to ('d, 'v)$ *tuple*$) \Rightarrow 'r \Rightarrow 'r \Rightarrow$ *bool* **where**
  kern $p\ x\ y \longleftrightarrow p\cdot($*keyof*$\cdot x) \sqsubseteq p\cdot($*keyof*$\cdot y)$

lemma *is-cong-kern*: *is-cong* (*kern* p)
  unfolding *kern-def* **by** *unfold-locales* (*auto intro*: *below-trans monofun-cfun-arg*)

lemma *kern-bot*: *kern* $\bot\ x\ y$
  by (*simp add*: *kern-def*)

lemma *kern-below*:
  assumes *deflation* p *deflation* q $p \sqsubseteq q$ **and** *kern* $q\ x\ y$
  shows *kern* $p\ x\ y$
proof −
  have $p\cdot(q\cdot($*keyof*$\cdot x)) \sqsubseteq p\cdot(q\cdot($*keyof*$\cdot y))$ **using** *assms*(*4*) **unfolding** *kern-def* **by** (*rule monofun-cfun-arg*)
  then show *?thesis* **unfolding** *kern-def* **by** (*simp add*: *deflation-below-comp1*[*OF assms*(*1*−*3*)])
qed

Functional dependence of p on q, relative to a support predicate S on source rows: rows that q cannot tell apart (in the kernel order), p cannot either. Taking S true everywhere gives schema-level dependence; taking S to be what a filter selects gives the data-level one.

definition *fdep* :: ('*r* ⇒ *bool*) ⇒ (('*d*, '*v*) *tuple* → ('*d*, '*v*) *tuple*) ⇒ (('*d*, '*v*) *tuple* → ('*d*, '*v*) *tuple*) ⇒ *bool* **where**
fdep S p q ⟷ (∀ *x y*. *S x* ⟶ *S y* ⟶ *kern q x y* ⟶ *kern p x y*)

lemma *fdep-refl*: *fdep S p p*
by (*simp add*: *fdep-def*)

lemma *fdep-trans*: *fdep S p q* ⟹ *fdep S q r* ⟹ *fdep S p r*
unfolding *fdep-def* **by** *blast*

lemma *fdep-below*: *deflation p* ⟹ *deflation q* ⟹ *p* ⊑ *q* ⟹ *fdep S p q*
unfolding *fdep-def* **by** (*blast intro*: *kern-below*)

lemma *fdep-support-mono*: (⋀*x*. *S' x* ⟹ *S x*) ⟹ *fdep S p q* ⟹ *fdep S' p q*
unfolding *fdep-def* **by** *blast*

end

16 Filters

A filter is a continuous predicate into *one* (Sierpinski space): a row *passes* it when it maps to *ONE*, a Scott-open predicate. Several filters are combined by continuous conjunction.

definition *passes* :: ('*r*::*pcpo* → *one*) ⇒ '*r* ⇒ *bool* **where**
passes φ r ⟷ *φ*·*r* = *ONE*

lemma *passes-up*: *passes φ x* ⟹ *x* ⊑ *y* ⟹ *passes φ y*
unfolding *passes-def* **by** (*cases φ*·*y*) (*auto dest*: *monofun-cfun-arg*[*of x y φ*])

lemma *passes-open*: *adm* (*λr*. ¬ *passes φ r*)
proof −
have ¬ *passes φ r* ⟷ *φ*·*r* ⊑ ⊥ **for** *r*
unfolding *passes-def* **by** (*cases φ*·*r*) *simp-all*
then show *?thesis* **by** *simp*
qed

definition *fconj* :: ('*r*::*pcpo* → *one*) ⇒ ('*r* → *one*) ⇒ ('*r* → *one*) **where**
fconj φ ψ = (Λ *r*. *seq*·(*φ*·*r*)·(*ψ*·*r*))

lemma *passes-fconj* [*simp*]: *passes* (*fconj φ ψ*) *r* ⟷ *passes φ r* ∧ *passes ψ r*
unfolding *passes-def fconj-def* **by** (*auto simp*: *seq-conv-if split*: *if-splits*)

lemma *passes-ftrue* [*simp*]: *passes* (Λ -. *ONE*) *r*

by (*simp add*: *passes-def*)

17 Datasets

A dataset is a Scott-closed set S of rows whose *maximal* elements are the actual data points; the rest are their partial approximations (sub-rows). These need not be maximal in the row *type*. Restricting to a predicate keeps the data points satisfying it and everything below them. The result is generally *not* a stable subdomain (it need not be closed under joins of compatible sub-rows), but it is again a dataset, with exactly those points as its maximal elements. We assume finitely many data points, which also makes closedness automatic.

definition *pts* :: *'r*::*pcpo set* ⇒ *'r set* **where**
  pts S = {*m* ∈ *S*. ∀ *y*∈*S*. *m* ⊑ *y* ⟶ *y* = *m*}

definition *filtered* :: *'r*::*pcpo set* ⇒ (*'r* ⇒ *bool*) ⇒ *'r set* **where**
  filtered S P = {*x*. ∃ *m* ∈ *pts S*. *P m* ∧ *x* ⊑ *m*}

lemma *adm-below-finite*: *finite P* ⟹ *adm* (λ*x*. ∃ *m*∈*P*. *x* ⊑ *m*)
  by (*induct rule*: *finite-induct*) *simp-all*

locale *dataset* =
  fixes *S* :: *'r*::*pcpo set*
  assumes *downward*: *y* ∈ *S* ⟹ *x* ⊑ *y* ⟹ *x* ∈ *S*
    and *max-above*: *x* ∈ *S* ⟹ ∃ *m*∈*pts S*. *x* ⊑ *m*
    and *finite-pts*: *finite* (*pts S*)
begin

lemma *S-eq*: *S* = {*x*. ∃ *m*∈*pts S*. *x* ⊑ *m*}
  using *max-above downward* **unfolding** *pts-def* **by** *blast*

lemma *closed*: *adm* (λ*x*. *x* ∈ *S*)
  by (*subst S-eq*) (*simp add*: *adm-below-finite finite-pts*)

lemma *filtered-subset*: *filtered S P* ⊆ *S*
  unfolding *filtered-def pts-def* **using** *downward* **by** *blast*

lemma *pts-filtered*: *pts* (*filtered S P*) = {*m* ∈ *pts S*. *P m*}
  using *filtered-subset* **unfolding** *filtered-def pts-def* **by** (*auto 6 3*)

lemma *dataset-filtered*: *dataset* (*filtered S P*)
proof
  show *y* ∈ *filtered S P* ⟹ *x* ⊑ *y* ⟹ *x* ∈ *filtered S P* **for** *x y*
    unfolding *filtered-def* **by** (*blast intro*: *below-trans*)
  show *x* ∈ *filtered S P* ⟹ ∃ *m*∈*pts* (*filtered S P*). *x* ⊑ *m* **for** *x*
    unfolding *pts-filtered* **by** (*auto simp*: *filtered-def*)
  show *finite* (*pts* (*filtered S P*))

unfolding *pts-filtered* **using** *finite-pts* **by** *simp*
qed

lemma *filtered-filtered*: *filtered* (*filtered S P*) *Q* = *filtered S* (λ*x*. *P x* ∧ *Q x*)
unfolding *filtered-def*[*of filtered S P*] *pts-filtered*
by (*auto simp*: *filtered-def*)

lemma *filtered-top*: *filtered S* (λ-. *True*) = *S*
by (*subst* (*2*) *S-eq*) (*simp add*: *filtered-def*)

end

18 Slicers

18.1 Selections

What slices select: Scott-open predicates, i.e. upward closed with an admissible complement. These correspond one-to-one to filters $'r \to one$, and form a distributive bounded lattice under the pointwise operations.

typedef *'r sel* = {*P* :: *'r*::*pcpo* ⇒ *bool*. (∀ *x y*. *P x* ⟶ *x* ⊑ *y* ⟶ *P y*) ∧ *adm* (λ*x*. ¬ *P x*)}
morphisms *selects Abs-sel*
by (*rule exI*[*of* - λ-. *True*]) *simp*

setup-lifting *type-definition-sel*

lemma *selects-up*: *selects s x* ⟹ *x* ⊑ *y* ⟹ *selects s y*
using *selects*[*of s*] **by** *blast*

lemma *selects-open*: *adm* (λ*x*. ¬ *selects s x*)
using *selects*[*of s*] **by** *blast*

lemma *sel-eqI*: (⋀*x*. *selects a x* ⟷ *selects b x*) ⟹ *a* = *b*
by (*simp add*: *selects-inject*[*symmetric*] *fun-eq-iff*)

lift-definition *sel-of* :: (*'r*::*pcpo* → *one*) ⇒ *'r sel* **is** *passes*
using *passes-up passes-open* **by** *blast*

instantiation *sel* :: (*pcpo*) {*distrib-lattice*, *bounded-lattice*}
begin

lift-definition *less-eq-sel* :: *'a sel* ⇒ *'a sel* ⇒ *bool* **is** (≤) .
lift-definition *less-sel* :: *'a sel* ⇒ *'a sel* ⇒ *bool* **is** (<) .
lift-definition *inf-sel* :: *'a sel* ⇒ *'a sel* ⇒ *'a sel* **is** *inf*
by (*auto intro*!: *adm-disj*)
lift-definition *sup-sel* :: *'a sel* ⇒ *'a sel* ⇒ *'a sel* **is** *sup*
by (*auto intro*!: *adm-conj*)
lift-definition *bot-sel* :: *'a sel* **is** *bot* **by** *simp*

lift-definition *top-sel* :: *'a sel* **is** *top* **by** *simp*

instance proof
fix *x y z* :: *'a sel*
show $x < y \longleftrightarrow x \leq y \wedge \neg\ y \leq x$ **by** *transfer* (*rule less-le-not-le*)
show $x \leq x$ **by** *transfer* (*rule order-refl*)
show $x \leq y \Longrightarrow y \leq z \Longrightarrow x \leq z$ **by** *transfer* (*erule* (*1*) *order-trans*)
show $x \leq y \Longrightarrow y \leq x \Longrightarrow x = y$ **by** *transfer* (*erule* (*1*) *antisym*)
show *inf x y* $\leq x$ **by** *transfer* (*rule inf-le1*)
show *inf x y* $\leq y$ **by** *transfer* (*rule inf-le2*)
show $x \leq y \Longrightarrow x \leq z \Longrightarrow x \leq$ *inf y z* **by** *transfer* (*erule* (*1*) *le-infI*)
show $x \leq$ *sup x y* **by** *transfer* (*rule sup-ge1*)
show $y \leq$ *sup x y* **by** *transfer* (*rule sup-ge2*)
show $y \leq x \Longrightarrow z \leq x \Longrightarrow$ *sup y z* $\leq x$ **by** *transfer* (*erule* (*1*) *le-supI*)
show *sup x* (*inf y z*) = *inf* (*sup x y*) (*sup x z*) **by** *transfer* (*rule sup-inf-distrib1*)
show *bot* $\leq x$ **by** *transfer* (*rule bot-least*)
show $x \leq$ *top* **by** *transfer* (*rule top-greatest*)
qed

end

lemma *selects-inf* [*simp*]: *selects* (*inf a b*) *x* $\longleftrightarrow$ *selects a x* $\wedge$ *selects b x*
by (*simp add*: *inf-sel.rep-eq*)

lemma *selects-top* [*simp*]: *selects top x*
by (*simp add*: *top-sel.rep-eq*)

18.2 Semilattice homomorphisms

Slices are an algebra given by laws (e.g. a quotient of syntax), not a type class: conjunction alone is assumed to form a semilattice with unit. Their selections, their dimensions and their coarsening are all homomorphisms of such, into different targets.

locale *semilattice-hom* = *src*: *semilattice-neutr f z* + *tgt*: *semilattice-neutr g w*
for *f* :: *'a* $\Rightarrow$ *'a* $\Rightarrow$ *'a* **and** *z* :: *'a* **and** *g* :: *'b* $\Rightarrow$ *'b* $\Rightarrow$ *'b* **and** *w* :: *'b* +
fixes *h* :: *'a* $\Rightarrow$ *'b*
assumes *hom-f*: *h* (*f a b*) = *g* (*h a*) (*h b*)
and *hom-z*: *h z* = *w*

18.3 Slice evaluation

Slices are given by their conjunction $*$ and its unit **1**, nothing more: the slicer constructs only conjoin. The order is the one conjunction induces. Slices act on keys, which carry levels of detail (semi-factors of the keys, compact ones for LODs). Each slice comes with the LOD it is written at, *slod*: what it *looks at*, not what it selects (an upper bound, not the least one). Its meaning *sopen* is a Scott-open predicate on keys, and a meet-

semilattice homomorphism. A slice reads a key only through its truncation to the slice's LOD, so it applies to keys finer than its LOD as well. *holds* is the plain predicate.

The slicer interface is a record of operations. Besides evaluation, a slicer *coarsens* slices to a given LOD and *pins* keys; their laws are explained below.

record ('*f*, '*k*) *slicer-ops* =
  sl-conj :: '*f* ⇒ '*f* ⇒ '*f*
  sl-unit :: '*f*
  sl-lod :: '*f* ⇒ '*k semi-factor*
  sl-open :: '*f* ⇒ '*k sel*
  sl-coarsen :: '*k semi-factor* ⇒ '*f* ⇒ '*f*
  sl-pin :: '*k* ⇒ '*f*

hide-const (**open**) *slicer-ops.truncate*

locale *slicer* = *conj*: *semilattice-neutr sl-conj slicing sl-unit slicing* +
  slod: *semilattice-hom sl-conj slicing sl-unit slicing* (⊔) ⊥ *sl-lod slicing* +
  sopen: *semilattice-hom sl-conj slicing sl-unit slicing inf top sl-open slicing*
  for *slicing* :: ('*f*, '*k*::*lod-domain*) *slicer-ops* +
 assumes *sopen-lod*: *selects* (*sl-open slicing f*) (*proj* (*sl-lod slicing f*)·*k*) ⟷ *selects* (*sl-open slicing f*) *k*
    and *slod-compact*: *compact* (*sl-lod slicing f*)
    and *coarsen-conj*: *sl-coarsen slicing A* (*sl-conj slicing a b*) =
      sl-conj slicing (*sl-coarsen slicing A a*) (*sl-coarsen slicing A b*)
    and *coarsen-inside*: *sl-lod slicing f* ⊑ *A* ⟹ *sl-coarsen slicing A f* = *f*
    and *slod-coarsen*: *sl-lod slicing* (*sl-coarsen slicing A f*) ⊑ *sl-lod slicing f* ⊓ *A*
    and *coarsen-cut*: *selects* (*sl-open slicing f*) *k* ⟹
      selects (*sl-open slicing* (*sl-coarsen slicing A f*)) (*proj A*·*k*)
    and *coarsen-slod*: *sl-coarsen slicing A f* = *sl-coarsen slicing* (*A* ⊓ *sl-lod slicing f*) *f*
    and *coarsen-coarsen*: *sl-coarsen slicing A* (*sl-coarsen slicing B f*) = *sl-coarsen slicing* (*A* ⊓ *B*) *f*
    and *sopen-spin*: *compact K* ⟹ *selects* (*sl-open slicing* (*sl-pin slicing K*)) *k* ⟷ *K* ⊑ *k*
begin

abbreviation *conj* :: '*f* ⇒ '*f* ⇒ '*f* (**infixl** ‹∗› *70*) **where** *conj* ≡ *sl-conj slicing*
abbreviation *sunit* :: '*f* (‹**1**›) **where** *sunit* ≡ *sl-unit slicing*
abbreviation *slod* :: '*f* ⇒ '*k semi-factor* **where** *slod* ≡ *sl-lod slicing*
abbreviation *sopen* :: '*f* ⇒ '*k sel* **where** *sopen* ≡ *sl-open slicing*
abbreviation *coarsen* :: '*k semi-factor* ⇒ '*f* ⇒ '*f* **where** *coarsen* ≡ *sl-coarsen slicing*
abbreviation *spin* :: '*k* ⇒ '*f* **where** *spin* ≡ *sl-pin slicing*

lemmas *slod-conj* [*simp*] = *slod.hom-f* **and** *slod-unit* [*simp*] = *slod.hom-z*

definition *sle* :: '*f* ⇒ '*f* ⇒ *bool* (**infix** ‹≤› *50*) **where**
  a ≤ *b* ⟷ *a* = *a* ∗ *b*

definition *slt* :: *'f* ⇒ *'f* ⇒ *bool* (**infix** ‹<› *50*) **where**
  $a < b \longleftrightarrow a \leq b \wedge a \neq b$

sublocale *sord*: *semilattice-neutr-order conj sunit sle slt*
  by *unfold-locales* (*simp-all add*: *sle-def slt-def*)

lemma *sopen-mono*: $a \leq b \Longrightarrow$ *sopen a* ≤ *sopen b*
  unfolding *sle-def* **by** (*metis sopen.hom-f inf.orderI*)

definition *holds* :: *'f* ⇒ *'k* ⇒ *bool* **where**
  holds f = *selects* (*sopen f*)

lemma *holds-conj* [*simp*]: *holds* (*a* ∗ *b*) *k* ⟷ *holds a k* ∧ *holds b k*
  by (*simp add*: *holds-def sopen.hom-f*)

lemma *holds-unit* [*simp*]: *holds* **1** *k*
  by (*simp add*: *holds-def sopen.hom-z*)

lemma *holds-up*: *holds f k* ⟹ *k* ⊑ *k'* ⟹ *holds f k'*
  unfolding *holds-def* **by** (*rule selects-up*)

lemma *holds-open*: *adm* (λ*k*. ¬ *holds f k*)
  unfolding *holds-def* **by** (*rule selects-open*)

lemma *holds-mono*: *a* ≤ *b* ⟹ *holds a k* ⟹ *holds b k*
  unfolding *sle-def* **by** (*metis holds-conj*)

lemma *holds-lod*: *holds f* (*proj* (*slod f*)·*k*) ⟷ *holds f k*
  by (*simp add*: *holds-def sopen-lod*)

Truncating to any LOD the slice is written within does not change whether it holds.

lemma *holds-proj*: *slod f* ⊑ *c* ⟹ *holds f* (*proj c*·*k*) ⟷ *holds f k*
  by (*metis holds-lod proj-le*)

lemma *holds-bot-lod*:
  assumes *slod f* = ⊥ **shows** *holds f k* ⟷ *holds f* ⊥
  using *holds-lod*[*of f k*] **by** (*simp add*: *assms proj-bot*)

18.4 Coarsening

A slicer adds *coarsening* of a slice to a given LOD. *GROUP BY ONLY* coarsens to the new frame LOD; *FOR* coarsens to the slice's own LOD with dimensions dropped by a transformer. Coarsening distributes over conjunction, leaves slices written inside the target alone, keeps only what the target looks at, holds at the truncation of every key the original holds at, and coarsening twice is coarsening once, to the meet of the targets. It

only depends on the part of the target the slice is written at: coarsening is about what is cut away. It is *not* extensional: coarsening $b = 2 \wedge b \neq 2$ away from b gives *True*, coarsening *False* gives *False*.

lemma *holds-coarsen-cut*: *holds f k* $\Longrightarrow$ *holds* (*coarsen A f*) (*proj A·k*)
unfolding *holds-def* **by** (*rule coarsen-cut*)

lemma *coarsen-unit* [*simp*]: *coarsen A* **1** = **1**
by (*rule coarsen-inside*) *simp*

lemma *coarsen-hom*: *semilattice-hom conj sunit conj sunit* (*coarsen A*)
by *unfold-locales* (*fact coarsen-conj coarsen-unit*)+

lemma *coarsen-mono*: *a* $\leq$ *b* $\Longrightarrow$ *coarsen A a* $\leq$ *coarsen A b*
unfolding *sle-def* **by** (*metis coarsen-conj*)

lemma *slod-coarsen-target*: *slod* (*coarsen A f*) $\sqsubseteq$ *A*
using *slod-coarsen*[*of A f*] **by** (*rule below-trans*[*OF - lattice-meet-le2*])

lemma *holds-coarsen*: *holds f k* $\Longrightarrow$ *holds* (*coarsen A f*) *k*
using *holds-coarsen-cut proj-below* **by** (*rule holds-up*)

lemma *holds-coarsen-reads*: *holds* (*coarsen A f*) (*proj A·k*) $\longleftrightarrow$ *holds* (*coarsen A f*) *k*
by (*rule holds-proj*[*OF slod-coarsen-target*])

A slice written entirely outside the target coarsens to a constant, which is true as soon as the slice holds anywhere. Syntactically this fails: *False* $\wedge$ *a* = *1* coarsens to *False* away from *a*.

lemma *coarsen-outside*:
assumes *slod f* $\sqcap$ *A* = $\bot$ **and** *holds f t*
shows *holds* (*coarsen A f*) *k*
proof −
have *lod*: *slod* (*coarsen A f*) = $\bot$
using *slod-coarsen*[*of A f*] *assms*(*1*) **by** *simp*
have *holds* (*coarsen A f*) (*proj A·t*) **using** *assms*(*2*) **by** (*rule holds-coarsen-cut*)
then have *holds* (*coarsen A f*) $\bot$ **using** *holds-bot-lod*[*OF lod*, *of proj A·t*] **by** *blast*
then show *?thesis* **by** (*rule holds-up*) *simp*
qed

Coarsening to less is weaker.

lemma *holds-coarsen-le*:
assumes *A* $\sqsubseteq$ *B* **and** *holds* (*coarsen B f*) *k* **shows** *holds* (*coarsen A f*) *k*
proof −
have *coarsen A f* = *coarsen A* (*coarsen B f*) **by** (*simp add*: *coarsen-coarsen lattice-meet-absorb1*[*OF assms*(*1*)])
then show *?thesis* **using** *holds-coarsen*[*OF assms*(*2*)] **by** *simp*
qed

lemma *coarsen-top* [*simp*]: *coarsen lattice-top f* = *f*
by (*rule coarsen-inside*) *simp*

18.5 Pins

spin K is the slice pinning the key K: on keys, the principal upset of K. It is only specified for compact keys, whose upsets are open. The slicer picks the LOD a pin is written at; nothing is assumed about it, but it is always one the key lives at. Nothing is assumed of coarsened pins either: by *coarsen-cut* they hold above the truncated key, and a syntactic slicer may lose more (coarsening *date* = *d* to the month drops it).

lemma *holds-spin*: *compact K* $\Longrightarrow$ *holds* (*spin K*) *k* $\longleftrightarrow$ *K* $\sqsubseteq$ *k*
unfolding *holds-def* **by** (*rule sopen-spin*)

lemma *slod-spin-lives*:
assumes *compact K* **shows** *proj* (*slod* (*spin K*))·*K* = *K*
proof −
have *holds* (*spin K*) *K* **by** (*simp add*: *holds-spin*[*OF assms*])
then have *holds* (*spin K*) (*proj* (*slod* (*spin K*))·*K*) **by** (*simp only*: *holds-lod*)
then have *K* $\sqsubseteq$ *proj* (*slod* (*spin K*))·*K* **by** (*simp only*: *holds-spin*[*OF assms*])
then show *?thesis* **using** *proj-below* **by** (*rule below-antisym*[*rotated*])
qed

lemma *holds-coarsen-spin*: *compact K* $\Longrightarrow$ *proj A*·*K* $\sqsubseteq$ *k* $\Longrightarrow$ *holds* (*coarsen A* (*spin K*)) *k*
by (*rule holds-up*[*OF holds-coarsen-cut*]) (*simp-all add*: *holds-spin*)

A slice below a pin only holds above the pinned key.

lemma *holds-pinned*:
assumes *compact K* **and** *f* $\leq$ *spin K* **and** *holds f k*
shows *K* $\sqsubseteq$ *k*
proof −
have *holds* (*spin K*) *k* **using** *assms*(*2*,*3*) **by** (*rule holds-mono*)
then show *?thesis* **by** (*simp only*: *holds-spin*[*OF assms*(*1*)])
qed

end

18.6 Rows versus keys

A row predicate reading only what a projection p keeps of the key is the same thing as a key predicate: its up-image is open (when the projected keys are compact), and its preimage gives back the row predicate. Preimages of open key predicates are open for any p.

context *dim-eval*
begin

definition *keyup* :: $(('d, 'v)\ tuple \to ('d, 'v)\ tuple) \Rightarrow ('r \Rightarrow bool) \Rightarrow ('d, 'v)\ tuple \Rightarrow bool$ **where**
  $keyup\ p\ V\ t \longleftrightarrow (\exists r.\ V\ r \wedge p{\cdot}(keyof{\cdot}r) \sqsubseteq t)$

definition *rowpre* :: $(('d, 'v)\ tuple \to ('d, 'v)\ tuple) \Rightarrow (('d, 'v)\ tuple \Rightarrow bool) \Rightarrow 'r \Rightarrow bool$ **where**
  $rowpre\ p\ U\ r \longleftrightarrow U\ (p{\cdot}(keyof{\cdot}r))$

lemma *keyup-up*: $keyup\ p\ V\ t \Longrightarrow t \sqsubseteq t' \Longrightarrow keyup\ p\ V\ t'$
  unfolding *keyup-def* **by** (*blast intro*: *below-trans*)

lemma *rowpre-keyup*:
  assumes $\bigwedge x\ y.\ kern\ p\ x\ y \Longrightarrow V\ x \Longrightarrow V\ y$
  shows $rowpre\ p\ (keyup\ p\ V) = V$
  using *assms* **unfolding** *fun-eq-iff rowpre-def keyup-def kern-def* **by** *blast*

lemma *rowpre-up*: $(\bigwedge t\ t'.\ U\ t \Longrightarrow t \sqsubseteq t' \Longrightarrow U\ t') \Longrightarrow rowpre\ p\ U\ r \Longrightarrow r \sqsubseteq r' \Longrightarrow rowpre\ p\ U\ r'$
  unfolding *rowpre-def* **by** (*blast intro*: *monofun-cfun-arg*)

lemma *rowpre-open*: $adm\ (\lambda t.\ \neg\ U\ t) \Longrightarrow adm\ (\lambda r.\ \neg\ rowpre\ p\ U\ r)$
  unfolding *rowpre-def* **using** *adm-subst*[*of* $\lambda r.\ p{\cdot}(keyof{\cdot}r)\ \lambda t.\ \neg\ U\ t$] **by** *simp*

lemma *keyup-open*:
  assumes $\bigwedge r.\ compact\ (p{\cdot}(keyof{\cdot}r))$
  shows $adm\ (\lambda t.\ \neg\ keyup\ p\ V\ t)$
proof (*rule admI*)
  fix $Y :: nat \Rightarrow ('d, 'v)\ tuple$
  assume *Y*: $chain\ Y\ \forall i.\ \neg\ keyup\ p\ V\ (Y\ i)$
  show $\neg\ keyup\ p\ V\ (\bigsqcup i.\ Y\ i)$
  proof
    assume $keyup\ p\ V\ (\bigsqcup i.\ Y\ i)$
    then obtain *r* **where** $V\ r\ p{\cdot}(keyof{\cdot}r) \sqsubseteq (\bigsqcup i.\ Y\ i)$ **by** (*auto simp*: *keyup-def*)
   then obtain *i* **where** $p{\cdot}(keyof{\cdot}r) \sqsubseteq Y\ i$ **using** $Y(1)$ *assms compact-below-lub-iff*
by *blast*
    with $\langle V\ r \rangle\ Y(2)$ **show** *False* **by** (*auto simp*: *keyup-def*)
  qed
qed

end

end
theory *Relspace*
  imports *Main*
begin

19 Relational vocabulary over a set of rows

A *relspace* is just a set of rows D. Key spaces are given by arbitrary functions (projections) on rows; nothing here depends on their nature. All relations are stated through *co-occurrence* of keys, so that D is mentioned only in its definition.

locale *relspace* =
  fixes $D :: 'd\ set$
begin

definition *cooc* :: $'a \Rightarrow ('d \Rightarrow 'a) \Rightarrow ('d \Rightarrow 'b) \Rightarrow 'b \Rightarrow bool$
    (- ⟨-⋈-⟩ - [*51*, *0*, *0*, *51*] *50*) **where**
  $k\ \langle p \bowtie q\rangle\ \kappa \longleftrightarrow (\exists\, d{\in}D.\ p\ d = k \wedge q\ d = \kappa)$

lemma *cooc-fibres*: $k\ \langle p \bowtie q\rangle\ \kappa \longleftrightarrow D \cap p\ -`\ \{k\} \cap q\ -`\ \{\kappa\} \neq \{\}$
  by (*auto simp*: *cooc-def*)

19.1 Functional dependence

definition *determines* :: $('d \Rightarrow 'a) \Rightarrow ('d \Rightarrow 'b) \Rightarrow bool$ (**infix** $\twoheadrightarrow$ *50*) **where**
  $p \twoheadrightarrow q \longleftrightarrow (\forall\, k\ \kappa\ \kappa'.\ k\ \langle p \bowtie q\rangle\ \kappa \longrightarrow k\ \langle p \bowtie q\rangle\ \kappa' \longrightarrow \kappa = \kappa')$

lemma *determines-rows*: $p \twoheadrightarrow q \longleftrightarrow (\forall\, d{\in}D.\ \forall\, d'{\in}D.\ p\ d = p\ d' \longrightarrow q\ d = q\ d')$
  unfolding *determines-def cooc-def* **by** *metis*

lemma *determines-comp*: $p \twoheadrightarrow f \circ p$
  by (*simp add*: *determines-rows*)

lemma *determines-refl*: $p \twoheadrightarrow p$
  by (*simp add*: *determines-rows*)

lemma *determines-trans*: $p \twoheadrightarrow q \Longrightarrow q \twoheadrightarrow r \Longrightarrow p \twoheadrightarrow r$
  unfolding *determines-rows* **by** *blast*

lemma *determines-pair*: $p \twoheadrightarrow q \Longrightarrow p \twoheadrightarrow r \Longrightarrow p \twoheadrightarrow (\lambda d.\ (q\ d,\ r\ d))$
  unfolding *determines-rows* **by** *blast*

definition *fequiv* :: $('d \Rightarrow 'a) \Rightarrow ('d \Rightarrow 'b) \Rightarrow bool$ (**infix** $\Leftrightarrow$ *50*) **where**
  $p \Leftrightarrow q \longleftrightarrow p \twoheadrightarrow q \wedge q \twoheadrightarrow p$

19.2 Splitting (unconditional independence)

definition *splits* :: $('d \Rightarrow 'a) \Rightarrow ('d \Rightarrow 'b) \Rightarrow bool$ **where**
  $splits\ p\ q \longleftrightarrow (\forall\, k\ \kappa\ k'\ \kappa'.\ k\ \langle p \bowtie q\rangle\ \kappa \longrightarrow k'\ \langle p \bowtie q\rangle\ \kappa' \longrightarrow k\ \langle p \bowtie q\rangle\ \kappa')$

lemma *splits-rows*:
  $splits\ p\ q \longleftrightarrow (\forall\, d_1{\in}D.\ \forall\, d_2{\in}D.\ \exists\, d{\in}D.\ p\ d = p\ d_1 \wedge q\ d = q\ d_2)$
  unfolding *splits-def cooc-def* **by** *blast*

lemma *splits-sym*: *splits p q* $\Longrightarrow$ *splits q p*
unfolding *splits-rows* **by** *metis*

lemma *splits-antimono*: *splits p q* $\Longrightarrow$ $p \Rrightarrow p'$ $\Longrightarrow$ $q \Rrightarrow q'$ $\Longrightarrow$ *splits p′ q′*
unfolding *splits-rows determines-rows* **by** *metis*

19.3 Divisibility at a grain

t divides a at grain n: up to functional equivalence, a is t joined with a cofactor that is independent of t and n together. Cofactors are labelled by rows.

definition *divides* :: $('d \Rightarrow 'c) \Rightarrow ('d \Rightarrow 'b) \Rightarrow ('d \Rightarrow 'a) \Rightarrow$ *bool* **where**
divides n t a $\longleftrightarrow$ $(\exists\, i :: 'd \Rightarrow 'd.$ *splits* $(\lambda d.\ (t\ d,\ n\ d))\ i \wedge a \Leftrightarrow (\lambda d.\ (t\ d,\ i\ d)))$

19.4 Equivalence up to a grain

a and t are equivalent at grain n when co-occurring keys have the same co-occurring n-keys.

definition *equiv-upto* :: $('d \Rightarrow 'c) \Rightarrow ('d \Rightarrow 'a) \Rightarrow ('d \Rightarrow 'b) \Rightarrow$ *bool* **where**
equiv-upto n a t $\longleftrightarrow$
$(\forall\, k\ \kappa.\ k\ \langle a \bowtie t\rangle\ \kappa \longrightarrow \{\nu.\ k\ \langle a \bowtie n\rangle\ \nu\} = \{\nu.\ \kappa\ \langle t \bowtie n\rangle\ \nu\})$

lemma *equiv-upto-refl*: *equiv-upto n a a*
unfolding *equiv-upto-def cooc-def* **by** *auto*

lemma *equiv-upto-sym*:
assumes *equiv-upto n a t* **shows** *equiv-upto n t a*
unfolding *equiv-upto-def*
proof (*intro allI impI*)
fix $k\ \kappa$ **assume** $k\ \langle t \bowtie a\rangle\ \kappa$
then have $\kappa\ \langle a \bowtie t\rangle\ k$ **by** (*auto simp*: *cooc-def*)
with *assms* **show** $\{\nu.\ k\ \langle t \bowtie n\rangle\ \nu\} = \{\nu.\ \kappa\ \langle a \bowtie n\rangle\ \nu\}$
unfolding *equiv-upto-def* **by** *simp*
qed

lemma *equiv-upto-trans*:
assumes *at*: *equiv-upto n a t* **and** *ts*: *equiv-upto n t s* **shows** *equiv-upto n a s*
unfolding *equiv-upto-def*
proof (*intro allI impI*)
fix $k\ \sigma$ **assume** $k\ \langle a \bowtie s\rangle\ \sigma$
then obtain d **where** $d \in D\ a\ d = k\ s\ d = \sigma$ **by** (*auto simp*: *cooc-def*)
then have $k\ \langle a \bowtie t\rangle\ t\ d\ t\ d\ \langle t \bowtie s\rangle\ \sigma$ **by** (*auto simp*: *cooc-def*)
with *at ts* **show** $\{\nu.\ k\ \langle a \bowtie n\rangle\ \nu\} = \{\nu.\ \sigma\ \langle s \bowtie n\rangle\ \nu\}$
unfolding *equiv-upto-def* **by** *simp*
qed

lemma *fequiv-equiv-upto*:
assumes $a \Leftrightarrow t$ **shows** *equiv-upto n a t*

```
  unfolding equiv-upto-def
proof (intro allI impI)
  fix k κ assume k ⟨a⋈t⟩ κ
  then obtain d₀ where d₀: d₀ ∈ D a d₀ = k t d₀ = κ by (auto simp: cooc-def)
  from assms have a ⇛ t t ⇛ a by (auto simp: fequiv-def)
  with d₀ show {ν. k ⟨a⋈n⟩ ν} = {ν. κ ⟨t⋈n⟩ ν}
    unfolding cooc-def determines-rows by metis
qed

lemma divides-equiv-upto:
  assumes divides n t a shows equiv-upto n a t
proof −
  from assms obtain i :: 'd ⇒ 'd
    where sp: splits (λd. (t d, n d)) i and eq: a ⇔ (λd. (t d, i d))
    by (auto simp: divides-def)
  from eq have a-ti: a ⇛ (λd. (t d, i d)) and ti-a: (λd. (t d, i d)) ⇛ a
    by (auto simp: fequiv-def)
  show ?thesis unfolding equiv-upto-def
  proof (intro allI impI)
    fix k κ assume k ⟨a⋈t⟩ κ
    then obtain d₀ where d₀: d₀ ∈ D a d₀ = k t d₀ = κ by (auto simp: cooc-def)
    have κ ⟨t⋈n⟩ ν if k ⟨a⋈n⟩ ν for ν
    proof −
      from that obtain d where d: d ∈ D a d = k n d = ν by (auto simp: cooc-def)
      from a-ti d d₀ have (t d, i d) = (t d₀, i d₀) unfolding determines-rows by
metis
      with d d₀ show ?thesis by (auto simp: cooc-def)
    qed
    moreover have k ⟨a⋈n⟩ ν if κ ⟨t⋈n⟩ ν for ν
    proof −
      from that obtain d where d: d ∈ D t d = κ n d = ν by (auto simp: cooc-def)
      from sp d(1) d₀(1) obtain e where e: e ∈ D (t e, n e) = (t d, n d) i e = i
d₀
        unfolding splits-rows by blast
      from e d d₀ have (t e, i e) = (t d₀, i d₀) by simp
      with ti-a e(1) d₀(1) have a e = a d₀ unfolding determines-rows by metis
      with e d d₀ show ?thesis by (auto simp: cooc-def)
    qed
    ultimately show {ν. k ⟨a⋈n⟩ ν} = {ν. κ ⟨t⋈n⟩ ν} by blast
  qed
qed
```

19.5 Equivalence modulo a grain

a and t are equivalent modulo n when rows with distinct n-keys agree on a exactly when they agree on t. This is a condition on pairs of rows, so it is inherited by every subset of rows; it is the largest such relation within *equiv-upto*. At row grain it is functional equivalence; it is vacuous when both sides determine n.

definition *equiv-mod* :: $('d \Rightarrow 'c) \Rightarrow ('d \Rightarrow 'a) \Rightarrow ('d \Rightarrow 'b) \Rightarrow bool$ **where**
equiv-mod n a t $\longleftrightarrow$ $(\forall\, d{\in}D.\ \forall\, d'{\in}D.\ n\ d \neq n\ d' \longrightarrow (a\ d = a\ d' \longleftrightarrow t\ d = t\ d'))$

lemma *equiv-mod-refl*: *equiv-mod n a a*
by (*simp add*: *equiv-mod-def*)

lemma *equiv-mod-sym*: *equiv-mod n a t* $\Longrightarrow$ *equiv-mod n t a*
by (*simp add*: *equiv-mod-def*)

lemma *equiv-mod-trans*: *equiv-mod n a t* $\Longrightarrow$ *equiv-mod n t s* $\Longrightarrow$ *equiv-mod n a s*
by (*simp add*: *equiv-mod-def*)

Adding the same component to both sides, and replacing either side by a functionally equivalent one, preserve the equivalence.

lemma *equiv-mod-pair*: *equiv-mod n a t* $\Longrightarrow$ *equiv-mod n* $(\lambda d.\ (a\ d,\ m\ d))$ $(\lambda d.\ (t\ d,\ m\ d))$
by (*auto simp*: *equiv-mod-def*)

lemma *equiv-mod-fequiv*: $a \Leftrightarrow a' \Longrightarrow t \Leftrightarrow t' \Longrightarrow$ *equiv-mod n a t* $\Longrightarrow$ *equiv-mod n a' t'*
unfolding *equiv-mod-def fequiv-def determines-rows* **by** *metis*

lemma *fequiv-equiv-mod*: $a \Leftrightarrow t \Longrightarrow$ *equiv-mod n a t*
unfolding *equiv-mod-def fequiv-def determines-rows* **by** *metis*

lemma *equiv-mod-equiv-upto*:
assumes *equiv-mod n a t* **shows** *equiv-upto n a t*
unfolding *equiv-upto-def*
proof (*intro allI impI*)
fix $k\ \kappa$ **assume** $k\ \langle a{\bowtie}t\rangle\ \kappa$
then obtain d_0 **where** d_0: $d_0 \in D\ a\ d_0 = k\ t\ d_0 = \kappa$ **by** (*auto simp*: *cooc-def*)
have $\kappa\ \langle t{\bowtie}n\rangle\ \nu$ **if** $k\ \langle a{\bowtie}n\rangle\ \nu$ **for** ν
proof −
from *that* **obtain** d **where** d: $d \in D\ a\ d = k\ n\ d = \nu$ **by** (*auto simp*: *cooc-def*)
show *?thesis*
proof (*cases n d* = *n* d_0)
case *True* **with** $d\ d_0$ **show** *?thesis* **by** (*auto simp*: *cooc-def*)
next
case *False*
with *assms d* d_0 **have** *t d* = *t* d_0 **unfolding** *equiv-mod-def* **by** *metis*
with $d\ d_0$ **show** *?thesis* **by** (*auto simp*: *cooc-def*)
qed
qed
moreover have $k\ \langle a{\bowtie}n\rangle\ \nu$ **if** $\kappa\ \langle t{\bowtie}n\rangle\ \nu$ **for** ν
proof −
from *that* **obtain** d **where** d: $d \in D\ t\ d = \kappa\ n\ d = \nu$ **by** (*auto simp*: *cooc-def*)
show *?thesis*
proof (*cases n d* = *n* d_0)

case *True* **with** *d* d_0 **show** *?thesis* **by** (*auto simp*: *cooc-def*)
next
case *False*
with *assms d* d_0 **have** *a d* = *a* d_0 **unfolding** *equiv-mod-def* **by** *metis*
with *d* d_0 **show** *?thesis* **by** (*auto simp*: *cooc-def*)
qed
qed
ultimately show $\{\nu.\ k\ \langle a{\bowtie}n\rangle\ \nu\} = \{\nu.\ \kappa\ \langle t{\bowtie}n\rangle\ \nu\}$ **by** *blast*
qed

end

lemma *equiv-mod-subset*:
$D' \subseteq D \Longrightarrow$ *relspace.equiv-mod D n a t* $\Longrightarrow$ *relspace.equiv-mod D' n a t*
unfolding *relspace.equiv-mod-def* **by** *blast*

end
theory *Schema*
imports *LOD HOLCF−extra.LowerPD-Representation HOLCF−extra.Lattice-Complete*
begin

default-sort {*bifinite*, *distributive-cpo*}

20 Schemas: presence rules and chase rules

A schema on a row domain consists of presence rules and chase rules.

A presence rule is triggered by a *decisive* antecedent: a predicate of one part of the row, false at bottom and constant above every nonbottom value of that part (so it is decided as soon as the part is present, as for presence itself or for *is-na* on a separated sum). Its consequent is any up-closed predicate (presence, maximality, a decided value test).

A chase rule acts on pairs of rows: when its guard holds of (x, y) it raises x by a contribution computed from x and y, in the rows with a top adjoined (an incompatible contribution is a conflict). Guards are up-closed and Scott-open. Functional dependencies are chase rules (guard: equal maximal determinants, contribution: the partner's dependent part), and so are join columns, which stay uncollapsed: under joint presence each key side is raised by the other side transported into its coordinates (a unary rule, ignoring the partner).

Each presence rule is either *upheld* by every chase rule (one step between conforming rows produces a row satisfying it) or *repaired* by a set of functional dependencies among the chase rules: their determinants lie inside the trigger part and are forced maximal by the antecedent, and the consequent is carried along their determinants and dependents. A triggered row then got its trigger part from a conforming source row, and the repairing depen-

dencies copy the source's evidence across.

datatype *'r presence-rule* =
Presence-Rule (*trigger*: *'r semi-factor*) (*antecedent*: *'r* ⇒ *bool*) (*consequent*: *'r* ⇒ *bool*)

datatype *'r chase-rule* =
Chase-Rule (*guard*: *'r* ⇒ *'r* ⇒ *bool*) (*contribution*: *'r* → *'r* → *'r top*)

20.1 Antecedents and consequents

definition *decisive* :: *'r semi-factor* ⇒ (*'r* ⇒ *bool*) ⇒ *bool* **where**
decisive p P ⟷ (∀ *x*. *P x* = *P* (*x* ⌊ *p*)) ∧ ¬ *P* ⊥ ∧
(∀ *x y*. *x* ⌊ *p* ≠ ⊥ ⟶ *x* ⊑ *y* ⟶ *P y* = *P x*)

definition *up-closed* :: (*'r*::*po* ⇒ *bool*) ⇒ *bool* **where**
up-closed C ⟷ (∀ *x y*. *C x* ⟶ *x* ⊑ *y* ⟶ *C y*)

lemma *decisive-part*: *decisive p P* ⟹ *P x* = *P* (*x* ⌊ *p*)
unfolding *decisive-def* **by** *blast*

lemma *decisive-bot*: *decisive p P* ⟹ ¬ *P* ⊥
unfolding *decisive-def* **by** *blast*

lemma *decisive-above*: *decisive p P* ⟹ *x* ⌊ *p* ≠ ⊥ ⟹ *x* ⊑ *y* ⟹ *P y* = *P x*
unfolding *decisive-def* **by** *blast*

lemma *decisive-present*: *decisive p P* ⟹ *P x* ⟹ *x* ⌊ *p* ≠ ⊥
unfolding *decisive-def* **by** *metis*

lemma *up-closedD*: *up-closed C* ⟹ *C x* ⟹ *x* ⊑ *y* ⟹ *C y*
unfolding *up-closed-def* **by** *blast*

Decisive antecedents are Scott-open, so their negations are admissible; up-closed consequents are admissible outright.

lemma *adm-not-decisive*:
assumes *P*: *decisive p P*
shows *adm* (λ*x*. ¬ *P x*)
proof (*rule admI*)
fix *Y* **assume** *Y*: *chain Y* **and** *nP*: ∀ *i*. ¬ *P* (*Y i*)
show ¬ *P* (⨆ *i*. *Y i*)
proof
assume *lub*: *P* (⨆ *i*. *Y i*)
have (⨆ *i*. *Y i*) ⌊ *p* ≠ ⊥ **using** *decisive-present*[*OF P lub*] **.**
then have (⨆ *i*. *Y i* ⌊ *p*) ≠ ⊥ **using** *Y* **by** (*simp add*: *contlub-cfun-arg*)
then obtain *i* **where** *i*: *Y i* ⌊ *p* ≠ ⊥ **by** *fastforce*
have *P* (⨆ *i*. *Y i*) = *P* (*Y i*) **by** (*rule decisive-above*[*OF P i is-ub-thelub*[*OF Y*]])
with *lub nP* **show** *False* **by** *simp*
qed

qed

lemma *adm-up-closed*: *up-closed C* $\Longrightarrow$ *adm C*
by (*rule admI*) (*blast intro*: *up-closedD is-ub-thelub*)

20.2 Guards

definition *open-guard* :: ($'r \Rightarrow 'r \Rightarrow$ *bool*) $\Rightarrow$ *bool* **where**
open-guard g $\longleftrightarrow$ ($\forall x\ y\ x'\ y'.\ g\ x\ y \longrightarrow x \sqsubseteq x' \longrightarrow y \sqsubseteq y' \longrightarrow g\ x'\ y'$) $\wedge$
($\forall X\ Y.$ *chain X* $\longrightarrow$ *chain Y* $\longrightarrow$ *g* ($\bigsqcup i.\ X\ i$) ($\bigsqcup i.\ Y\ i$) $\longrightarrow$ ($\exists i.\ g\ (X\ i)\ (Y\ i)$)))

lemma *open-guard-up*: *open-guard g* $\Longrightarrow$ *g x y* $\Longrightarrow$ $x \sqsubseteq x' \Longrightarrow y \sqsubseteq y' \Longrightarrow$ *g x' y'*
unfolding *open-guard-def* **by** *blast*

lemma *open-guard-lub*:
open-guard g $\Longrightarrow$ *chain X* $\Longrightarrow$ *chain Y* $\Longrightarrow$ *g* ($\bigsqcup i.\ X\ i$) ($\bigsqcup i.\ Y\ i$) $\Longrightarrow$ $\exists i.\ g\ (X\ i)\ (Y\ i)$
unfolding *open-guard-def* **by** *blast*

lemma *open-guard-lub1*: *open-guard g* $\Longrightarrow$ *chain X* $\Longrightarrow$ *g* ($\bigsqcup i.\ X\ i$) *y* $\Longrightarrow$ $\exists i.\ g\ (X\ i)\ y$
using *open-guard-lub*[*of g X* λ-. *y*] **by** *simp*

lemma *open-guard-lub2*: *open-guard g* $\Longrightarrow$ *chain Y* $\Longrightarrow$ *g x* ($\bigsqcup i.\ Y\ i$) $\Longrightarrow$ $\exists i.\ g\ x\ (Y\ i)$
using *open-guard-lub*[*of g* λ-. *x Y*] **by** *simp*

A maximal part does not move under refinement, and with finitary determinants it is compact, so it is reached after finitely many approximants.

lemma *chain-part-reaches*:
assumes *C*: *chain C* **and** *k*: ($\bigsqcup i.\ C\ i$) $\lfloor\ p = k$ **and** *ck*: *compact k*
obtains *n* **where** $\bigwedge j.\ n \leq j \Longrightarrow C\ j \lfloor\ p = k$
proof −
have *lub*: $k = (\bigsqcup i.\ C\ i \lfloor\ p)$ **using** *k C* **by** (*simp add*: *contlub-cfun-arg*)
have *Cp*: *chain* ($\lambda i.\ C\ i \lfloor\ p$) **using** *C* **by** *simp*
obtain *n* **where** *n*: $k \sqsubseteq C\ n \lfloor\ p$
using *compact-below-lub-iff*[*OF ck Cp*] *lub* **by** *auto*
have $C\ j \lfloor\ p = k$ **if** *j*: $n \leq j$ **for** *j*
proof (*rule below-antisym*)
show $C\ j \lfloor\ p \sqsubseteq k$ **unfolding** *lub* **by** (*rule is-ub-thelub*[*OF Cp*])
show $k \sqsubseteq C\ j \lfloor\ p$ **using** *n monofun-cfun-arg*[*OF chain-mono*[*OF C j*]] **by** (*rule below-trans*)
qed
then show *?thesis* **by** (*rule that*)
qed

20.3 Functional dependencies and key joins as chase rules

A set of rows satisfies a strong functional dependency $a \to b$ when rows whose a-parts are equal and maximal have equal b-parts. A maximal a-part does not move under refinement.

definition *fd* :: *'r semi-factor* ⇒ *'r semi-factor* ⇒ *'r set* ⇒ *bool* **where**
fd a b R ⟷
(∀ *x*∈*R*. ∀ *y*∈*R*. *maximal-at a* (*x* ⌊ *a*) ⟶ *x* ⌊ *a* = *y* ⌊ *a* ⟶ *x* ⌊ *b* = *y* ⌊ *b*)

lemma *fdD*:
fd a b R ⟹ *x* ∈ *R* ⟹ *y* ∈ *R* ⟹ *maximal-at a* (*x* ⌊ *a*) ⟹ *x* ⌊ *a* = *y* ⌊ *a* ⟹
x ⌊ *b* = *y* ⌊ *b*
unfolding *fd-def* **by** *blast*

lemma *maximal-at-proj-above*:
assumes *m*: *maximal-at a* (*x* ⌊ *a*) **and** *le*: *x* ⊑ *y*
shows *y* ⌊ *a* = *x* ⌊ *a*
proof (*rule maximal-at-eq*[*OF m*])
show *proj a*·(*y* ⌊ *a*) = *y* ⌊ *a* **by** *simp*
show *x* ⌊ *a* ⊑ *y* ⌊ *a* **using** *le* **by** (*rule monofun-cfun-arg*)
qed

definition *fd-guard* :: *'r semi-factor* ⇒ *'r* ⇒ *'r* ⇒ *bool* **where**
fd-guard a x y ⟷ *maximal-at a* (*x* ⌊ *a*) ∧ *x* ⌊ *a* = *y* ⌊ *a*

definition *fd-rule* :: *'r semi-factor* ⇒ *'r semi-factor* ⇒ *'r chase-rule* **where**
fd-rule a b = *Chase-Rule* (*fd-guard a*) (Λ *x y*. *down*·(*y* ⌊ *b*))

lemma *fd-rule-simps* [*simp*]:
guard (*fd-rule a b*) = *fd-guard a contribution* (*fd-rule a b*)·*x*·*y* = *down*·(*y* ⌊ *b*)
by (*simp-all add*: *fd-rule-def*)

lemma *open-fd-guard*:
assumes *a*: *finitary-semi-factor a*
shows *open-guard* (*fd-guard a*)
unfolding *open-guard-def*
proof (*intro conjI allI impI*)
fix *x y x′ y′* **assume** *g*: *fd-guard a x y* **and** *x*: *x* ⊑ *x′* **and** *y*: *y* ⊑ *y′*
have *m*: *maximal-at a* (*x* ⌊ *a*) **and** *eq*: *x* ⌊ *a* = *y* ⌊ *a* **using** *g* **unfolding** *fd-guard-def* **by** *auto*
have *my*: *maximal-at a* (*y* ⌊ *a*) **using** *m eq* **by** *simp*
show *fd-guard a x′ y′*
using *m maximal-at-proj-above*[*OF m x*] *maximal-at-proj-above*[*OF my y*] *eq*
by (*simp add*: *fd-guard-def*)
next
fix *X Y* :: *nat* ⇒ - **assume** *X*: *chain X* **and** *Y*: *chain Y* **and** *g*: *fd-guard a* (⨆ *i*. *X i*) (⨆ *i*. *Y i*)
define *k* **where** *k* = (⨆ *i*. *X i*) ⌊ *a*
have *m*: *maximal-at a k* **and** *yk*: (⨆ *i*. *Y i*) ⌊ *a* = *k* **using** *g* **by** (*auto simp add*:

fd-guard-def k-def)
have *ck*: *compact k* **using** *a* **by** (*simp add*: *k-def finitary-semi-factor-def finitary-def*)
obtain *n* **where** *n*: $\bigwedge j.\ n \leq j \Longrightarrow X\ j \lfloor a = k$ **using** *chain-part-reaches*[*OF X k-def*[*symmetric*] *ck*] **by** *blast*
obtain *n'* **where** *n'*: $\bigwedge j.\ n' \leq j \Longrightarrow Y\ j \lfloor a = k$ **using** *chain-part-reaches*[*OF Y yk ck*] **by** *blast*
have *fd-guard a* (*X* (*max n n'*)) (*Y* (*max n n'*)) **using** *n n' m* **by** (*simp add*: *fd-guard-def*)
then show $\exists i.$ *fd-guard a* (*X i*) (*Y i*) **by** *blast*
qed

A key join between parts S and T (the two key sides), with transports φ from S-coordinates to T-coordinates and ψ back: under joint presence each side is raised by the other.

definition *both-present* :: *'r semi-factor* ⇒ *'r semi-factor* ⇒ *'r* ⇒ *'r* ⇒ *bool* **where**
both-present S T x y ⟷ $x \lfloor S \neq \bot \wedge x \lfloor T \neq \bot$

definition *key-join-rules* :: *'r semi-factor* ⇒ *'r semi-factor* ⇒ (*'r* → *'r*) ⇒ (*'r* → *'r*) ⇒ *'r chase-rule set* **where**
key-join-rules S T φ ψ =
{*Chase-Rule* (*both-present S T*) (Λ *x y*. *down*·(*φ*·(*x* ⌊ *S*))),
Chase-Rule (*both-present S T*) (Λ *x y*. *down*·(*ψ*·(*x* ⌊ *T*)))}

lemma *present-lub*: *chain X* ⟹ $(\bigsqcup i.\ X\ i) \lfloor S \neq \bot \Longrightarrow \exists n.\ \forall j{\geq}n.\ X\ j \lfloor S \neq \bot$
proof −
assume *X*: *chain X* **and** *p*: $(\bigsqcup i.\ X\ i) \lfloor S \neq \bot$
then have $(\bigsqcup i.\ X\ i \lfloor S) \neq \bot$ **by** (*simp add*: *contlub-cfun-arg*)
then obtain *n* **where** *n*: $X\ n \lfloor S \neq \bot$ **by** *fastforce*
have $X\ j \lfloor S \neq \bot$ **if** $n \leq j$ **for** *j*
using *n monofun-cfun-arg*[*OF chain-mono*[*OF X that*], *of proj S*] **by** (*auto simp add*: *bottomI*)
then show *?thesis* **by** *blast*
qed

lemma *open-both-present*: *open-guard* (*both-present S T*)
unfolding *open-guard-def*
proof (*intro conjI allI impI*)
fix *x y x' y'* **assume** *g*: *both-present S T x y* **and** *x*: $x \sqsubseteq x'$
have $x \lfloor S \sqsubseteq x' \lfloor S\ x \lfloor T \sqsubseteq x' \lfloor T$ **using** *x* **by** (*simp-all add*: *monofun-cfun-arg*)
then show *both-present S T x' y'* **using** *g* **by** (*auto simp add*: *both-present-def bottomI*)
next
fix *X Y* :: *nat* ⇒ - **assume** *X*: *chain X* **and** *g*: *both-present S T* $(\bigsqcup i.\ X\ i)$ $(\bigsqcup i.\ Y\ i)$
obtain *n* **where** *n*: $\forall j{\geq}n.\ X\ j \lfloor S \neq \bot$ **using** *present-lub*[*OF X*] *g* **by** (*auto simp add*: *both-present-def*)
obtain *n'* **where** *n'*: $\forall j{\geq}n'.\ X\ j \lfloor T \neq \bot$ **using** *present-lub*[*OF X*] *g* **by** (*auto simp add*: *both-present-def*)

have *both-present S T* (*X* (*max n n'*)) (*Y* (*max n n'*)) **using** *n n'* **by** (*simp add*: *both-present-def*)
then show ∃ *i*. *both-present S T* (*X i*) (*Y i*) **by** *blast*
qed

lemma *open-key-join-rules*: *c* ∈ *key-join-rules S T φ ψ* ⟹ *open-guard* (*guard c*)
by (*auto simp add*: *key-join-rules-def open-both-present*)

20.4 The combined chase step

One step applies every rule to a pair of rows and takes the join of the row and all firing contributions, in the complete lattice of rows with a top.

definition *guarded* :: *'r chase-rule* ⇒ *'r* ⇒ *'r* ⇒ *'r top* **where**
guarded c x y = (*if guard c x y then contribution c·x·y else* ⊥)

definition *rules-step* :: *'r chase-rule set* ⇒ *'r* ⇒ *'r* ⇒ *'r top* **where**
rules-step C x y = *down·x* ⊔ *lub* ((λ*c*. *guarded c x y*) ‘ *C*)

lemma *rules-step-above*: *down·x* ⊑ *rules-step C x y*
unfolding *rules-step-def* **by** (*rule joinableD*(*1*)[*OF joinable-lattice*])

lemma *rules-step-guarded*: *c* ∈ *C* ⟹ *guarded c x y* ⊑ *rules-step C x y*
unfolding *rules-step-def*
by (*rule below-trans*[*OF lattice-lub-upper joinableD*(*2*)[*OF joinable-lattice*]]) *simp*

lemma *cont-guarded-if*:
assumes *up*: ⋀*x x'*. *g x* ⟹ *x* ⊑ *x'* ⟹ *g x'* **and** *op*: ⋀*Y*. *chain Y* ⟹ *g* (⨆ *i*. *Y i*) ⟹ ∃ *i*. *g* (*Y i*)
and *f*: *cont f*
shows *cont* (λ*x*. *if g x then f x else* ⊥)
proof (*rule contI2*)
show *monofun* (λ*x*. *if g x then f x else* ⊥)
by (*rule monofunI*) (*auto dest*: *up intro*: *cont2monofunE*[*OF f*])
fix *Y* **assume** *Y*: *chain Y* **and** *ch*: *chain* (λ*i*. *if g* (*Y i*) *then f* (*Y i*) *else* ⊥)
show (*if g* (⨆ *i*. *Y i*) *then f* (⨆ *i*. *Y i*) *else* ⊥) ⊑ (⨆ *i*. *if g* (*Y i*) *then f* (*Y i*) *else* ⊥)
proof (*cases g* (⨆ *i*. *Y i*))
case *True*
obtain *n* **where** *n*: *g* (*Y n*) **using** *op*[*OF Y True*] **by** *blast*
have *gn*: *g* (*Y* (*i* + *n*)) **for** *i* **by** (*rule up*[*OF n chain-mono*[*OF Y*]]) *simp*
have *f* (⨆ *i*. *Y i*) = (⨆ *i*. *f* (*Y* (*i* + *n*)))
by (*simp add*: *cont2contlubE*[*OF f Y*] *lub-range-shift*[*OF ch2ch-cont*[*OF f Y*]])
also have ... = (⨆ *i*. *if g* (*Y* (*i* + *n*)) *then f* (*Y* (*i* + *n*)) *else* ⊥) **by** (*simp add*: *gn*)
also have ... = (⨆ *i*. *if g* (*Y i*) *then f* (*Y i*) *else* ⊥) **by** (*rule lub-range-shift*[*OF ch*])
finally show *?thesis* **using** *True* **by** *simp*
qed *simp*
qed

lemma *cont-lattice-lub*:
fixes f :: *'i::type* ⇒ *'a::cpo* ⇒ *'b::lattice-complete*
assumes f: $\bigwedge i.\ i \in I \Longrightarrow$ *cont* $(f\ i)$
shows *cont* $(\lambda x.$ *lub* $((\lambda i.\ f\ i\ x)\ `\ I))$
proof (*rule contI2*)
show *monofun* $(\lambda x.$ *lub* $((\lambda i.\ f\ i\ x)\ `\ I))$
proof (*rule monofunI*, *rule lattice-lub-least*, *rule is-ubI*, *elim imageE*)
fix *x y z i* **assume** *xy*: $x \sqsubseteq y$ **and** *z*: $z = f\ i\ x$ **and** *i*: $i \in I$
have $f\ i\ x \sqsubseteq f\ i\ y$ **by** (*rule cont2monofunE*[*OF f*[*OF i*] *xy*])
also have $f\ i\ y \sqsubseteq$ *lub* $((\lambda i.\ f\ i\ y)\ `\ I)$ **using** *i* **by** (*simp add*: *lattice-lub-upper*)
finally show $z \sqsubseteq$ *lub* $((\lambda i.\ f\ i\ y)\ `\ I)$ **by** (*simp add*: *z*)
qed
fix *Y* **assume** *Y*: *chain Y* **and** *ch*: *chain* $(\lambda n.$ *lub* $((\lambda i.\ f\ i\ (Y\ n))\ `\ I))$
show *lub* $((\lambda i.\ f\ i\ (\bigsqcup n.\ Y\ n))\ `\ I) \sqsubseteq (\bigsqcup n.$ *lub* $((\lambda i.\ f\ i\ (Y\ n))\ `\ I))$
proof (*rule lattice-lub-least*, *rule is-ubI*, *elim imageE*)
fix *z i* **assume** *z*: $z = f\ i\ (\bigsqcup n.\ Y\ n)$ **and** *i*: $i \in I$
have $f\ i\ (\bigsqcup n.\ Y\ n) = (\bigsqcup n.\ f\ i\ (Y\ n))$ **by** (*rule cont2contlubE*[*OF f*[*OF i*] *Y*])
also have $\ldots \sqsubseteq (\bigsqcup n.$ *lub* $((\lambda i.\ f\ i\ (Y\ n))\ `\ I))$
by (*rule lub-mono*[*OF ch2ch-cont*[*OF f*[*OF i*] *Y*] *ch*]) (*simp add*: *lattice-lub-upper i*)
finally show $z \sqsubseteq (\bigsqcup n.$ *lub* $((\lambda i.\ f\ i\ (Y\ n))\ `\ I))$ **by** (*simp add*: *z*)
qed
qed

lemma *cont-lattice-join*:
fixes *f g* :: *'a::cpo* ⇒ *'b::lattice-complete*
assumes *f*: *cont f* **and** *g*: *cont g*
shows *cont* $(\lambda x.\ f\ x \sqcup g\ x)$
proof −
have *img*: $(\lambda b.$ (*if b then f else g*) *x*) ` *UNIV* = {*f x*, *g x*} **for** *x* **by** (*auto simp add*: *UNIV-bool*)
have $(\lambda x.\ f\ x \sqcup g\ x) = (\lambda x.$ *lub* $((\lambda b.$ (*if b then f else g*) *x*) ` *UNIV*))
by (*simp add*: *img lub-eqI*[*OF joinable-lub*[*OF joinable-lattice*]])
moreover have *cont* (*if b then f else g*) **for** *b* **by** (*cases b*) (*simp-all add*: *f g*)
ultimately show *?thesis* **using** *cont-lattice-lub*[*of UNIV* $\lambda b.$ *if b then f else g*]
by *simp*
qed

lemma *cont-guarded1*: *open-guard* (*guard c*) ⟹ *cont* $(\lambda x.$ *guarded c x y*)
unfolding *guarded-def*
by (*rule cont-guarded-if*) (*auto dest*: *open-guard-lub1 intro*: *open-guard-up*)

lemma *cont-guarded2*: *open-guard* (*guard c*) ⟹ *cont* $(\lambda y.$ *guarded c x y*)
unfolding *guarded-def*
by (*rule cont-guarded-if*) (*auto dest*: *open-guard-lub2 intro*: *open-guard-up*)

lemma *cont-rules-step1*: $\forall\, c{\in}C.$ *open-guard* (*guard c*) ⟹ *cont* $(\lambda x.$ *rules-step C x y*)

unfolding *rules-step-def*
by (*intro cont-lattice-join cont-lattice-lub cont-guarded1*) *simp-all*

lemma *cont-rules-step2*: $\forall\, c{\in}C.$ *open-guard* (*guard c*) $\Longrightarrow$ *cont* ($\lambda y.$ *rules-step C x y*)
unfolding *rules-step-def*
by (*intro cont-lattice-join cont-lattice-lub cont-guarded2*) *simp-all*

definition *step-cfun* :: *'r chase-rule set* $\Rightarrow$ *'r* $\to$ *'r* $\to$ *'r top* **where**
step-cfun C = (Λ *x y. rules-step C x y*)

lemma *step-cfun-beta*: $\forall\, c{\in}C.$ *open-guard* (*guard c*) $\Longrightarrow$ *step-cfun C*·*x*·*y* = *rules-step C x y*
unfolding *step-cfun-def*
by (*simp add*: *cont2cont-LAM cont-rules-step1 cont-rules-step2*)

Lifted to rows with a top, which is absorbing.

definition *step-top* :: *'r chase-rule set* $\Rightarrow$ *'r top* $\to$ *'r top* $\to$ *'r top* **where**
step-top C = (Λ *U V. Iftop Top* (Λ *u. Iftop Top* (*step-cfun C*·*u*) *V*) *U*)

lemma *cont-step-top-inner*: *cont* ($\lambda u.$ *Iftop Top* (*step-cfun C*·*u*) *V*)
by (*cases V*) (*simp-all add*: *Iftop-down Iftop-top*)

lemma *step-top-down*: *step-top C*·(*down*·*u*)·(*down*·*v*) = *step-cfun C*·*u*·*v*
proof −
have *inner*: (Λ *u. Iftop Top* (*step-cfun C*·*u*) *V*)·*w* = *Iftop Top* (*step-cfun C*·*w*) *V* **for** *V w*
by (*simp add*: *cont-step-top-inner*)
have *c1*: *cont* ($\lambda V.$ *Iftop Top* (Λ *u. Iftop Top* (*step-cfun C*·*u*) *V*) *U*) **for** *U*
by (*cases U*) (*simp-all add*: *Iftop-down Iftop-top inner cont-Iftop*)
have *c2*: *cont* ($\lambda U.$ *Iftop Top* (Λ *u. Iftop Top* (*step-cfun C*·*u*) *V*) *U*) **for** *V*
by (*rule cont-Iftop*) *simp*
show *?thesis*
unfolding *step-top-def* **by** (*simp add*: *cont2cont-LAM c1 c2 Iftop-down inner cont-Iftop*)
qed

20.5 The chase on the Hoare powerdomain

One round of steps applies the step to every pair of members, as a double bind with no explicit sup over the rules. The chase of X is the least Y above X closed under steps.

definition *chase-steps* :: *'r chase-rule set* $\Rightarrow$ *'r top lower-pd* $\to$ *'r top lower-pd* **where**
chase-steps C = (Λ *Y. lower-bind*·*Y*·(Λ *U. lower-bind*·*Y*·(Λ *V.* {|*step-top C*·*U*·*V*|}))))

lemma *chase-steps-beta*: *chase-steps C*·*Y* = *lower-bind*·*Y*·(Λ *U. lower-bind*·*Y*·(Λ *V.* {|*step-top C*·*U*·*V*|}))

by (*simp add*: *chase-steps-def*)

definition *pd-chase* :: *'r chase-rule set* ⇒ *'r top lower-pd* → *'r top lower-pd* **where**
pd-chase C = (Λ *X*. *fix*·(Λ *Y*. *X* ∪♭ *Y* ∪♭ *chase-steps C*·*Y*))

lemma *pd-chase-beta*: *pd-chase C*·*X* = *fix*·(Λ *Y*. *X* ∪♭ *Y* ∪♭ *chase-steps C*·*Y*)
by (*simp add*: *pd-chase-def*)

lemma *chase-steps-mem*:
assumes *U*: *U* ∈♭ *Y* **and** *V*: *V* ∈♭ *Y*
shows *step-top C*·*U*·*V* ∈♭ *chase-steps C*·*Y*
proof −
have {*step-top C*·*U*·*V*}♭ ⊑ *lower-bind*·*Y*·(Λ *V*. {*step-top C*·*U*·*V*}♭)
using *lower-bind-mem*[*OF V*, *of* Λ *V*. {*step-top C*·*U*·*V*}♭] **by** *simp*
also have ... ⊑ *chase-steps C*·*Y*
using *lower-bind-mem*[*OF U*, *of* Λ *U*. *lower-bind*·*Y*·(Λ *V*. {*step-top C*·*U*·*V*}♭)]
by (*simp add*: *chase-steps-beta*)
finally show *?thesis* **by** (*simp add*: *lower-mem-equiv*)
qed

lemma *chase-steps-below*:
assumes *st*: ⋀*U V*. *U* ∈♭ *Y* ⟹ *V* ∈♭ *Y* ⟹ *step-top C*·*U*·*V* ∈♭ *Z*
shows *chase-steps C*·*Y* ⊑ *Z*
unfolding *chase-steps-beta*
proof (*rule lower-bind-below*, *simp*)
fix *U* **assume** *U*: *U* ∈♭ *Y*
show *lower-bind*·*Y*·(Λ *V*. {*step-top C*·*U*·*V*}♭) ⊑ *Z*
by (*rule lower-bind-below*) (*simp add*: *lower-mem-equiv*[*symmetric*] *st*[*OF U*])
qed

lemma *pd-chase-unfold*: *pd-chase C*·*X* = *X* ∪♭ *pd-chase C*·*X* ∪♭ *chase-steps C*·(*pd-chase C*·*X*)
unfolding *pd-chase-beta* **by** (*subst fix-eq*) *simp*

lemma *pd-chase-above*: *X* ⊑ *pd-chase C*·*X*
proof −
have *X* ⊑ *X* ∪♭ *pd-chase C*·*X* ∪♭ *chase-steps C*·(*pd-chase C*·*X*)
by (*rule below-trans*[*OF lower-plus-below1 lower-plus-below1*])
also have ... = *pd-chase C*·*X* **by** (*rule pd-chase-unfold*[*symmetric*])
finally show *?thesis* **.**
qed

lemma *pd-chase-closed*: *chase-steps C*·(*pd-chase C*·*X*) ⊑ *pd-chase C*·*X*
proof −
have *chase-steps C*·(*pd-chase C*·*X*) ⊑ *X* ∪♭ *pd-chase C*·*X* ∪♭ *chase-steps C*·(*pd-chase C*·*X*)
by (*rule lower-plus-below2*)
also have ... = *pd-chase C*·*X* **by** (*rule pd-chase-unfold*[*symmetric*])
finally show *?thesis* **.**

qed

lemma *pd-chase-least*: $X \sqsubseteq Y \Longrightarrow$ *chase-steps* $C{\cdot}Y \sqsubseteq Y \Longrightarrow$ *pd-chase* $C{\cdot}X \sqsubseteq Y$
unfolding *pd-chase-beta* **by** (*rule fix-least-below*) (*simp add*: *lower-plus-least*)

20.6 Lawful datasets

definition *obeys* :: $'r$ *presence-rule* $\Rightarrow$ $'r \Rightarrow$ *bool* **where**
obeys $r\ x \longleftrightarrow$ (*antecedent* $r\ x \longrightarrow$ *consequent* $r\ x$)

definition *row-obeys* :: $'r$ *presence-rule set* $\Rightarrow$ $'r \Rightarrow$ *bool* **where**
row-obeys $R\ x \longleftrightarrow (\forall\, r{\in}R.\ \textit{obeys}\ r\ x)$

Both conditions are stated on compact members, which makes them admissible. Presence: every compact member lies below a conforming member. Chase: steps between compact members do not conflict and land in the set.

definition *obeys-presence-pd* :: $'r$ *presence-rule set* $\Rightarrow$ $'r$ *lower-pd* $\Rightarrow$ *bool* **where**
obeys-presence-pd $R\ X \longleftrightarrow$
$(\forall\, x.\ \textit{compact}\ x \longrightarrow x \in\flat\ X \longrightarrow (\exists\, l.\ \textit{row-obeys}\ R\ l \wedge x \sqsubseteq l \wedge l \in\flat\ X))$

definition *chase-closed-pd* :: $'r$ *chase-rule set* $\Rightarrow$ $'r$ *lower-pd* $\Rightarrow$ *bool* **where**
chase-closed-pd $C\ X \longleftrightarrow (\forall\, x\ y.\ \textit{compact}\ x \longrightarrow \textit{compact}\ y \longrightarrow x \in\flat\ X \longrightarrow y \in\flat\ X \longrightarrow$
$(\exists\, w.\ w \in\flat\ X \wedge \textit{rules-step}\ C\ x\ y = \textit{down}{\cdot}w))$

lemma *adm-obeys-presence-pd*: *adm* (*obeys-presence-pd* (R :: $'r$ *presence-rule set*))
proof (*rule admI*)
fix Y :: *nat* $\Rightarrow$ $'r$ *lower-pd*
assume Y: *chain* Y **and** *all*: $\forall\, i.$ *obeys-presence-pd* R ($Y\ i$)
show *obeys-presence-pd* R ($\bigsqcup i.\ Y\ i$)
unfolding *obeys-presence-pd-def*
proof (*intro allI impI*)
fix x **assume** x: *compact* x **and** *mem*: $x \in\flat$ ($\bigsqcup i.\ Y\ i$)
obtain i **where** $x \in\flat\ Y\ i$ **by** (*rule lower-mem-compact-lub*[*OF* $Y\ x$ *mem*])
then obtain l **where** *row-obeys* $R\ l\ x \sqsubseteq l\ l \in\flat\ Y\ i$
using *all* x **unfolding** *obeys-presence-pd-def* **by** *blast*
then show $\exists\, l.$ *row-obeys* $R\ l \wedge x \sqsubseteq l \wedge l \in\flat$ ($\bigsqcup i.\ Y\ i$) **using** *lower-mem-lub*[*OF* Y] **by** *blast*
qed
qed

lemma *adm-chase-closed-pd*: *adm* (*chase-closed-pd* (C :: $'r$ *chase-rule set*))
proof (*rule admI*)
fix Y :: *nat* $\Rightarrow$ $'r$ *lower-pd*
assume Y: *chain* Y **and** *all*: $\forall\, i.$ *chase-closed-pd* C ($Y\ i$)
show *chase-closed-pd* C ($\bigsqcup i.\ Y\ i$)
unfolding *chase-closed-pd-def*
proof (*intro allI impI*)
fix $x\ y$ **assume** x: *compact* x **and** y: *compact* y

```
    and mx: x ∈♭ (⨆ i. Y i) and my: y ∈♭ (⨆ i. Y i)
   obtain i where i: x ∈♭ Y i by (rule lower-mem-compact-lub[OF Y x mx])
   obtain j where j: y ∈♭ Y j by (rule lower-mem-compact-lub[OF Y y my])
   have x ∈♭ Y (max i j) y ∈♭ Y (max i j) using lower-mem-chain[OF Y] i j by
simp-all
   then obtain w where w ∈♭ Y (max i j) rules-step C x y = down·w
    using all x y unfolding chase-closed-pd-def by blast
  then show ∃ w. w ∈♭ (⨆ i. Y i) ∧ rules-step C x y = down·w using lower-mem-lub[OF
Y] by blast
 qed
qed
```

With open guards, closure under steps between compact members extends to all members, limits included: the steps between the approximants converge to the step between the members.

```
theorem chase-closed-pd-mem:
 fixes x y :: 'r
 assumes og: ∀ c∈C. open-guard (guard c) and cl: chase-closed-pd C X
   and x: x ∈♭ X and y: y ∈♭ X
 shows ∃ w. w ∈♭ X ∧ rules-step C x y = down·w
proof −
 obtain ap :: nat ⇒ 'r → 'r where ap: approx-chain ap using bifinite by blast
 define xs where xs = (λi. ap i·x)
 define ys where ys = (λi. ap i·y)
 have xs: chain xs and ys: chain ys
   unfolding xs-def ys-def using approx-chain.chain-approx[OF ap] by (simp-all
add: ch2ch-Rep-cfunL)
 have lub-x: (⨆ i. xs i) = x and lub-y: (⨆ i. ys i) = y
   using approx-chain.chain-approx[OF ap] approx-chain.lub-approx[OF ap]
   by (simp-all add: xs-def ys-def flip: contlub-cfun-fun)
 have ∃ w. w ∈♭ X ∧ rules-step C (xs i) (ys i) = down·w for i
 proof −
   have xs i ∈♭ X ys i ∈♭ X
     using x y approx-chain.approx-below[OF ap] by (simp-all add: xs-def ys-def
lower-mem-below)
   then show ?thesis using cl approx-chain.compact-approx[OF ap]
    unfolding chase-closed-pd-def xs-def ys-def by blast
 qed
 then obtain ws where ws: ⋀i. ws i ∈♭ X and eq: ⋀i. rules-step C (xs i) (ys
i) = down·(ws i)
   by metis
 define s where s = step-cfun C
 have s: s·u·v = rules-step C u v for u v by (simp add: s-def step-cfun-beta og)
 have ws-chain: chain ws
 proof (rule chainI)
   fix i
   have s·(xs i)·(ys i) ⊑ s·(xs (Suc i))·(ys (Suc i))
     by (intro monofun-cfun-arg monofun-cfun chainE xs ys)
   then show ws i ⊑ ws (Suc i) by (simp add: s eq)
```

qed
have *rules-step C x y* = *s*·(⨆ *i*. *xs i*)·(⨆ *i*. *ys i*) **by** (*simp add*: *s lub-x lub-y*)
also have ... = (⨆ *i*. *s*·(*xs i*)·(*ys i*))
by (*simp add*: *contlub-cfun-arg*[*OF xs*] *lub-APP*[*OF ch2ch-Rep-cfunR*[*OF xs*] *ys*])
also have ... = (⨆ *i*. *down*·(*ws i*)) **by** (*simp add*: *s eq*)
also have ... = *down*·(⨆ *i*. *ws i*) **by** (*simp add*: *contlub-cfun-arg*[*OF ws-chain*])
finally show *?thesis* **using** *admD*[*OF adm-lower-mem ws-chain*] *ws* **by** *blast*
qed

20.7 The chase as the least conforming dataset above

Through *down*, a dataset is closed exactly when one round of steps stays inside it.

lemma *chase-closed-pd-steps*:
assumes *og*: ∀ *c*∈*C*. *open-guard* (*guard c*)
shows *chase-closed-pd C X* ⟷ *chase-steps C*·(*lower-map*·*down*·*X*) ⊑ *lower-map*·*down*·*X*
proof
assume *cl*: *chase-closed-pd C X*
show *chase-steps C*·(*lower-map*·*down*·*X*) ⊑ *lower-map*·*down*·*X*
proof (*rule chase-steps-below*)
fix *U V* **assume** *U* ∈♭ *lower-map*·*down*·*X V* ∈♭ *lower-map*·*down*·*X*
then obtain *u v* **where** *u*: *u* ∈♭ *X U* = *down*·*u* **and** *v*: *v* ∈♭ *X V* = *down*·*v*
by (*auto simp add*: *lower-mem-map-down*)
obtain *w* **where** *w* ∈♭ *X rules-step C u v* = *down*·*w*
using *chase-closed-pd-mem*[*OF og cl u*(*1*) *v*(*1*)] **by** *blast*
then show *step-top C*·*U*·*V* ∈♭ *lower-map*·*down*·*X*
by (*auto simp add*: *u v step-top-down step-cfun-beta og lower-mem-map-down*)
qed
next
assume *st*: *chase-steps C*·(*lower-map*·*down*·*X*) ⊑ *lower-map*·*down*·*X*
show *chase-closed-pd C X*
unfolding *chase-closed-pd-def*
proof (*intro allI impI*)
fix *x y* **assume** *x*: *x* ∈♭ *X* **and** *y*: *y* ∈♭ *X*
have *step-top C*·(*down*·*x*)·(*down*·*y*) ∈♭ *chase-steps C*·(*lower-map*·*down*·*X*)
by (*intro chase-steps-mem lower-map-mem x y*)
then have *rules-step C x y* ∈♭ *lower-map*·*down*·*X*
using *st* **by** (*auto simp add*: *lower-mem-equiv step-top-down step-cfun-beta og intro*: *below-trans*)
then show ∃ *w*. *w* ∈♭ *X* ∧ *rules-step C x y* = *down*·*w* **by** (*simp add*: *lower-mem-map-down*)
qed
qed

theorem *pd-chase-fixed-iff*:
assumes *og*: ∀ *c*∈*C*. *open-guard* (*guard c*)
shows *pd-chase C*·(*lower-map*·*down*·*X*) = *lower-map*·*down*·*X* ⟷ *chase-closed-pd C X*
proof

assume *fx*: *pd-chase C·(lower-map·down·X) = lower-map·down·X*
show *chase-closed-pd C X*
using *pd-chase-closed*[*of C lower-map·down·X*] **by** (*simp add*: *fx chase-closed-pd-steps*[*OF og*])
next
assume *cl*: *chase-closed-pd C X*
show *pd-chase C·(lower-map·down·X) = lower-map·down·X*
proof (*rule below-antisym*)
show *pd-chase C·(lower-map·down·X)* $\sqsubseteq$ *lower-map·down·X*
by (*rule pd-chase-least*) (*simp-all add*: *chase-closed-pd-steps*[*OF og*, *symmetric*] *cl*)
qed (*rule pd-chase-above*)
qed

corollary *pd-chase-fixed*:
$\forall$ *c*$\in$*C*. *open-guard* (*guard c*) $\Longrightarrow$ *chase-closed-pd C X* $\Longrightarrow$ *pd-chase C·(lower-map·down·X) = lower-map·down·X*
by (*simp add*: *pd-chase-fixed-iff*)

A chase without conflict is again a dataset, and it is closed.

lemma *pd-chase-Top-free*:
assumes *og*: $\forall$ *c*$\in$*C*. *open-guard* (*guard c*) **and** *nT*: $\neg$ *Top* $\in\flat$ *pd-chase C·(lower-map·down·X)*
obtains *X'* **where** *pd-chase C·(lower-map·down·X) = lower-map·down·X' X* $\sqsubseteq$ *X' chase-closed-pd C X'*
proof −
obtain *X'* **where** *eq*: *pd-chase C·(lower-map·down·X) = lower-map·down·X'*
using *nT* **by** (*rule lower-map-down-onto*)
have *X* $\sqsubseteq$ *X'* **using** *pd-chase-above*[*of lower-map·down·X C*] **by** (*simp add*: *eq lower-map-down-below-iff*)
moreover have *chase-closed-pd C X'*
using *pd-chase-closed*[*of C lower-map·down·X*] **by** (*simp add*: *eq chase-closed-pd-steps*[*OF og*])
ultimately show *?thesis* **using** *eq that* **by** *blast*
qed

The chase is the least closed dataset above X, and it conflicts exactly when there is no closed dataset above X.

theorem *pd-chase-least-closed*:
assumes *og*: $\forall$ *c*$\in$*C*. *open-guard* (*guard c*) **and** *le*: *X* $\sqsubseteq$ *Z* **and** *cl*: *chase-closed-pd C Z*
obtains *X'* **where** *pd-chase C·(lower-map·down·X) = lower-map·down·X'* *X* $\sqsubseteq$ *X' chase-closed-pd C X' X'* $\sqsubseteq$ *Z*
proof −
have *below*: *pd-chase C·(lower-map·down·X)* $\sqsubseteq$ *lower-map·down·Z*
by (*rule pd-chase-least*)
(*simp-all add*: *lower-map-down-below-iff le chase-closed-pd-steps*[*OF og*, *symmetric*] *cl*)
have $\neg$ *Top* $\in\flat$ *pd-chase C·(lower-map·down·X)*

using *below Top-not-mem-map-down*[*of Z*] **by** (*auto simp add*: *lower-mem-equiv intro*: *below-trans*)
then obtain X' **where** *eq*: *pd-chase C*·(*lower-map*·*down*·*X*) = *lower-map*·*down*·X'
and $X \sqsubseteq X'$ *chase-closed-pd C X'*
by (*rule pd-chase-Top-free*[*OF og*])
moreover have $X' \sqsubseteq Z$ **using** *below* **by** (*simp add*: *eq lower-map-down-below-iff*)
ultimately show *?thesis* **by** (*rule that*)
qed

theorem *pd-chase-conflict-iff*:
assumes *og*: ∀ *c*∈*C*. *open-guard* (*guard c*)
shows *Top* ∈♭ *pd-chase C*·(*lower-map*·*down*·*X*) ⟷ ¬ (∃ *Z*. *X* ⊑ *Z* ∧ *chase-closed-pd C Z*)
proof
assume *T*: *Top* ∈♭ *pd-chase C*·(*lower-map*·*down*·*X*)
show ¬ (∃ *Z*. *X* ⊑ *Z* ∧ *chase-closed-pd C Z*)
proof
assume ∃ *Z*. *X* ⊑ *Z* ∧ *chase-closed-pd C Z*
then obtain *Z* **where** *X* ⊑ *Z chase-closed-pd C Z* **by** *blast*
then obtain X' **where** *pd-chase C*·(*lower-map*·*down*·*X*) = *lower-map*·*down*·X'
by (*rule pd-chase-least-closed*[*OF og*])
with *T* **show** *False* **by** (*simp add*: *Top-not-mem-map-down*)
qed
next
assume *nc*: ¬ (∃ *Z*. *X* ⊑ *Z* ∧ *chase-closed-pd C Z*)
show *Top* ∈♭ *pd-chase C*·(*lower-map*·*down*·*X*)
proof (*rule ccontr*)
assume ¬ *Top* ∈♭ *pd-chase C*·(*lower-map*·*down*·*X*)
then obtain X' **where** *X* ⊑ X' *chase-closed-pd C X'* **by** (*rule pd-chase-Top-free*[*OF og*])
with *nc* **show** *False* **by** *blast*
qed
qed

Monotonicity is free: the chase is continuous, so conflicts propagate upwards.

corollary *pd-chase-conflict-up*:
X ⊑ *Y* ⟹ *Top* ∈♭ *pd-chase C*·*X* ⟹ *Top* ∈♭ *pd-chase C*·*Y*
by (*rule lower-mem-below*[*OF - below-refl*], *unfold lower-mem-equiv*)
(*erule below-trans*, *erule monofun-cfun-arg*)

20.8 The laws on maximal members

A maximal member absorbs a step exactly when every firing rule contributes below it. Since every member lies below a maximal one and the step is monotone, closure under steps is the same as the rules holding at the maximal members.

lemma *rules-step-absorbed*:

rules-step C m y = down·m ⟷ (∀ c∈C. guard c m y ⟶ contribution c·m·y ⊑ down·m)
proof (*intro iffI ballI impI*)
fix *c* **assume** *eq*: *rules-step C m y = down·m* **and** *c*: *c ∈ C* **and** *g*: *guard c m y*
show *contribution c·m·y ⊑ down·m*
using *rules-step-guarded*[*OF c, of m y*] *g* **by** (*simp add*: *eq guarded-def*)
next
assume *all*: *∀ c∈C. guard c m y ⟶ contribution c·m·y ⊑ down·m*
have *lub* ((*λc. guarded c m y*) ‘ *C*) *⊑ down·m*
by (*rule lattice-lub-least*, *rule is-ubI*) (*auto simp add*: *guarded-def all*)
then have *rules-step C m y ⊑ down·m*
unfolding *rules-step-def* **by** (*rule join-belowI*[*OF below-refl*])
then show *rules-step C m y = down·m* **using** *rules-step-above* **by** (*rule below-antisym*)
qed

definition *rule-holds* :: *′r chase-rule ⇒ ′r lower-pd ⇒ bool* **where**
rule-holds c X ⟷
(*∀ m∈lower-maximal X. ∀ y. y ∈♭ X ⟶ guard c m y ⟶ contribution c·m·y ⊑ down·m*)

theorem *chase-closed-pd-iff-rule-holds*:
assumes *og*: *∀ c∈C. open-guard* (*guard c*)
shows *chase-closed-pd C X ⟷* (*∀ c∈C. rule-holds c X*)
proof
assume *cl*: *chase-closed-pd C X*
show *∀ c∈C. rule-holds c X*
unfolding *rule-holds-def*
proof (*intro ballI allI impI*)
fix *c m y* **assume** *c*: *c ∈ C* **and** *m*: *m ∈ lower-maximal X* **and** *y*: *y ∈♭ X* **and** *g*: *guard c m y*
have *mx*: *m ∈♭ X* **using** *m* **by** (*simp add*: *lower-maximal-def*)
obtain *w* **where** *w*: *w ∈♭ X* **and** *eq*: *rules-step C m y = down·w*
using *chase-closed-pd-mem*[*OF og cl mx y*] **by** *blast*
have *m ⊑ w* **using** *rules-step-above*[*of m C y*] **by** (*simp add*: *eq*)
then have *w = m* **using** *m w* **by** (*simp add*: *lower-maximal-def*)
then have *rules-step C m y = down·m* **by** (*simp add*: *eq*)
then show *contribution c·m·y ⊑ down·m* **using** *c g* **by** (*simp add*: *rules-step-absorbed*)
qed
next
assume *all*: *∀ c∈C. rule-holds c X*
show *chase-closed-pd C X*
unfolding *chase-closed-pd-def*
proof (*intro allI impI*)
fix *x y* **assume** *x*: *x ∈♭ X* **and** *y*: *y ∈♭ X*
obtain *m* **where** *m*: *m ∈ lower-maximal X x ⊑ m* **using** *x* **by** (*rule lower-mem-maximal-above*)
have *rules-step C m y = down·m*
using *all m*(*1*) *y* **unfolding** *rules-step-absorbed rule-holds-def* **by** *blast*
moreover have *rules-step C x y ⊑ rules-step C m y*

by (*rule cont2monofunE*[*OF cont-rules-step1*[*OF og*] *m*(*2*)])
ultimately have *le*: *rules-step C x y* $\sqsubseteq$ *down·m* **by** *simp*
then obtain *w* **where** *w*: *rules-step C x y* = *down·w* **by** (*cases rules-step C x y*) *auto*
have *w* $\in\flat$ *X* **using** *le m*(*1*) **by** (*auto simp add*: *w lower-maximal-def intro*: *lower-mem-below*)
then show $\exists$ *w*. *w* $\in\flat$ *X* $\wedge$ *rules-step C x y* = *down·w* **using** *w* **by** *blast*
qed
qed

For a functional dependency the law is the dependency on the maximal members.

theorem *rule-holds-fd*: *rule-holds* (*fd-rule a b*) *X* $\longleftrightarrow$ *fd a b* (*lower-maximal X*)
proof
assume *h*: *rule-holds* (*fd-rule a b*) *X*
show *fd a b* (*lower-maximal X*)
unfolding *fd-def*
proof (*intro ballI impI*)
fix *x y* **assume** *x*: *x* $\in$ *lower-maximal X* **and** *y*: *y* $\in$ *lower-maximal X*
and *m*: *maximal-at a* (*x* $\lfloor$ *a*) **and** *eq*: *x* $\lfloor$ *a* = *y* $\lfloor$ *a*
have *absorbed*: *v* $\lfloor$ *b* $\sqsubseteq$ *u* $\lfloor$ *b*
if *u*: *u* $\in$ *lower-maximal X* **and** *v*: *v* $\in$ *lower-maximal X* **and** *g*: *fd-guard a u v* **for** *u v*
proof −
have *v* $\in\flat$ *X* **using** *v* **by** (*simp add*: *lower-maximal-def*)
then have *v* $\lfloor$ *b* $\sqsubseteq$ *u* **using** *h u g* **by** (*simp add*: *rule-holds-def*)
then have *proj b*·(*v* $\lfloor$ *b*) $\sqsubseteq$ *u* $\lfloor$ *b* **by** (*rule monofun-cfun-arg*)
then show *?thesis* **by** *simp*
qed
have *fd-guard a x y fd-guard a y x* **using** *m eq* **by** (*auto simp add*: *fd-guard-def*)
then show *x* $\lfloor$ *b* = *y* $\lfloor$ *b*
using *absorbed*[*OF x y*] *absorbed*[*OF y x*] **by** (*blast intro*: *below-antisym*)
qed
next
assume *f*: *fd a b* (*lower-maximal X*)
show *rule-holds* (*fd-rule a b*) *X*
unfolding *rule-holds-def*
proof (*intro ballI allI impI*)
fix *m y* **assume** *m*: *m* $\in$ *lower-maximal X* **and** *y*: *y* $\in\flat$ *X* **and** *g*: *guard* (*fd-rule a b*) *m y*
obtain *m′* **where** *m′*: *m′* $\in$ *lower-maximal X y* $\sqsubseteq$ *m′* **using** *y* **by** (*rule lower-mem-maximal-above*)
have *ma*: *maximal-at a* (*m* $\lfloor$ *a*) **and** *eq*: *m* $\lfloor$ *a* = *y* $\lfloor$ *a*
using *g* **unfolding** *fd-rule-simps fd-guard-def* **by** *auto*
have *my*: *maximal-at a* (*y* $\lfloor$ *a*) **using** *ma* **unfolding** *eq* **.**
have *m′* $\lfloor$ *a* = *y* $\lfloor$ *a* **by** (*rule maximal-at-proj-above*[*OF my m′*(*2*)])
then have *m* $\lfloor$ *a* = *m′* $\lfloor$ *a* **using** *eq* **by** *simp*
then have *m* $\lfloor$ *b* = *m′* $\lfloor$ *b* **by** (*rule fdD*[*OF f m*(*1*) *m′*(*1*) *ma*])
moreover have *y* $\lfloor$ *b* $\sqsubseteq$ *m′* $\lfloor$ *b* **using** *m′*(*2*) **by** (*rule monofun-cfun-arg*)

ultimately have $y \lfloor b \sqsubseteq m \lfloor b$ **by** *simp*
also have $m \lfloor b \sqsubseteq m$ **by** (*rule proj-below*)
finally show *contribution* (*fd-rule a b*)·m·$y \sqsubseteq$ *down*·m **by** *simp*
qed
qed

corollary *fd-maximal-members*:
$\forall\, c{\in}C.$ *open-guard* (*guard c*) $\Longrightarrow$ *chase-closed-pd C X* $\Longrightarrow$ *fd-rule a b* $\in C \Longrightarrow$
fd a b (*lower-maximal X*)
using *chase-closed-pd-iff-rule-holds rule-holds-fd* **by** *blast*

21 The schema locale

definition *upholds* :: *'r presence-rule set* $\Rightarrow$ *'r chase-rule* $\Rightarrow$ *'r presence-rule* $\Rightarrow$ *bool* **where**
upholds R c r $\longleftrightarrow$ ($\forall\, x\ y\ z.$ *row-obeys R x* $\longrightarrow$ *row-obeys R y* $\longrightarrow$ *guard c x y* $\longrightarrow$
down·$x \sqcup$ *contribution c*·x·y = *down*·$z \longrightarrow$ *obeys r z*)

definition *repairs* :: (*'r semi-factor* $\times$ *'r semi-factor*) *set* $\Rightarrow$ *'r presence-rule* $\Rightarrow$ *bool* **where**
repairs D r $\longleftrightarrow$
($\forall\,(a,\ b){\in}D.$ $a \sqsubseteq$ *trigger r* $\wedge$ ($\forall\, x.$ *antecedent r x* $\longrightarrow$ *maximal-at a* ($x \lfloor a$))) $\wedge$
($\forall\, x\ y.$ *consequent r y* $\longrightarrow$ ($\forall\,(a,\ b){\in}D.$ $y \lfloor a \sqsubseteq x \lfloor a \wedge y \lfloor b \sqsubseteq x \lfloor b$) $\longrightarrow$
consequent r x)

record *'r schema-rules* =
presence :: *'r presence-rule set*
chase :: *'r chase-rule set*

hide-const (**open**) *schema-rules.truncate*

locale *schema* =
fixes *rules* :: *'r schema-rules*
assumes *antecedent-decisive*: $r \in$ *presence rules* $\Longrightarrow$ *decisive* (*trigger r*) (*antecedent r*)
and *consequent-up-closed*: $r \in$ *presence rules* $\Longrightarrow$ *up-closed* (*consequent r*)
and *guard-open*: $\forall\, c{\in}$*chase rules*. *open-guard* (*guard c*)
and *rule-maintained*: $r \in$ *presence rules* $\Longrightarrow$
($\forall\, c{\in}$*chase rules*. *upholds* (*presence rules*) *c r*) $\vee$
($\exists\, D.$ ($\forall\,(a,\ b){\in}D.$ *fd-rule a b* $\in$ *chase rules*) $\wedge$ *repairs D r*)
begin

abbreviation *presence-rules* :: *'r presence-rule set* **where** *presence-rules* $\equiv$ *presence rules*
abbreviation *chase-rules* :: *'r chase-rule set* **where** *chase-rules* $\equiv$ *chase rules*

abbreviation *conforming-row* :: *'r* $\Rightarrow$ *bool* **where**
conforming-row $\equiv$ *row-obeys presence-rules*

definition *conforms* :: *'r lower-pd* ⇒ *bool* **where**
conforms X ⟷ *obeys-presence-pd presence-rules X* ∧ *chase-closed-pd chase-rules X*

lemma *conforming-row-bot*: *conforming-row* ⊥
unfolding *row-obeys-def obeys-def* **by** (*blast dest*: *decisive-bot*[*OF antecedent-decisive*])

lemma *adm-conforming-row*: *adm conforming-row*
unfolding *row-obeys-def obeys-def imp-conv-disj*
by (*intro adm-ball adm-disj adm-not-decisive*[*OF antecedent-decisive*]
adm-up-closed[*OF consequent-up-closed*])

Lawful rows are closed under compatible joins: a triggered antecedent of the join is decided by the joinand whose trigger part is present, and that joinand's consequent goes up.

lemma *conforming-row-join*:
assumes *x*: *conforming-row x* **and** *y*: *conforming-row y* **and** *j*: *x* ⇑ *y*
shows *conforming-row* (*x* ⊔ *y*)
unfolding *row-obeys-def obeys-def*
proof (*intro ballI impI*)
fix *r* **assume** *r*: *r* ∈ *presence-rules* **and** *P*: *antecedent r* (*x* ⊔ *y*)
note *D* = *antecedent-decisive*[*OF r*] **and** *C* = *consequent-up-closed*[*OF r*]
have (*x* ⊔ *y*) ⌊ *trigger r* ≠ ⊥ **by** (*rule decisive-present*[*OF D P*])
then have *x* ⌊ *trigger r* ≠ ⊥ ∨ *y* ⌊ *trigger r* ≠ ⊥ **by** (*auto simp add*: *proj-join*[*OF j*])
then show *consequent r* (*x* ⊔ *y*)
proof
assume *x* ⌊ *trigger r* ≠ ⊥
then have *antecedent r x* **using** *P decisive-above*[*OF D - joinableD*(*1*)[*OF j*]]
by *blast*
then have *consequent r x* **using** *x r* **by** (*simp add*: *row-obeys-def obeys-def*)
then show *?thesis* **by** (*rule up-closedD*[*OF C - joinableD*(*1*)[*OF j*]])
next
assume *y* ⌊ *trigger r* ≠ ⊥
then have *antecedent r y* **using** *P decisive-above*[*OF D - joinableD*(*2*)[*OF j*]]
by *blast*
then have *consequent r y* **using** *y r* **by** (*simp add*: *row-obeys-def obeys-def*)
then show *?thesis* **by** (*rule up-closedD*[*OF C - joinableD*(*2*)[*OF j*]])
qed
qed

lemma *adm-conforms*: *adm conforms*
unfolding *conforms-def* **by** (*intro adm-conj adm-obeys-presence-pd adm-chase-closed-pd*)

theorem *conforms-chase-fixed*: *conforms X* ⟹ *pd-chase chase-rules*·(*lower-map*·*down*·*X*) = *lower-map*·*down*·*X*
unfolding *conforms-def* **using** *pd-chase-fixed*[*OF guard-open*] **by** *blast*

corollary *conforms-fd*:
conforms X $\Longrightarrow$ *fd-rule a b* $\in$ *chase-rules* $\Longrightarrow$ *fd a b* (*lower-maximal X*)
unfolding *conforms-def* **using** *fd-maximal-members*[*OF guard-open*] **by** *blast*

lemma *conforms-iff-rule-holds*:
conforms X $\longleftrightarrow$ *obeys-presence-pd presence-rules X* $\wedge$ ($\forall$ *c*$\in$*chase-rules*. *rule-holds c X*)
unfolding *conforms-def* **by** (*simp add*: *chase-closed-pd-iff-rule-holds*[*OF guard-open*])

end

end
theory *Views*
imports *Schema*
begin

default-sort {*bifinite*, *distributive-cpo*}

22 Views and functional dependencies

A *view* reads a section of the row and a part of its key: a pair of a column-side and a key-side semi-factor. Functional dependencies between views cover all four combinations of sides (columns to columns, columns to keys, keys to columns, keys to keys) at once. The key of a row is computed from a declared column support, so columns-to-keys dependencies hold on every set of rows; the others are properties of datasets.

type-synonym ('*r*, '*k*) *view* = '*r semi-factor* $\times$ '*k semi-factor*

Projecting a pair of a row and a key onto a view's parts.

definition *pproj* :: ('*r*, '*k*) *view* $\Rightarrow$ '*r* $\times$ '*k* $\rightarrow$ '*r* $\times$ '*k* **where**
pproj v = (Λ *p*. (*fst p* $\lfloor$ *fst v*, *proj* (*snd v*)$\cdot$(*snd p*)))

lemma *pproj-apply* [*simp*]: *pproj v*$\cdot$(*r*, *k*) = (*r* $\lfloor$ *fst v*, *proj* (*snd v*)$\cdot$*k*)
by (*simp add*: *pproj-def*)

A keying computes keys from rows and declares the column support of each key part.

record ('*r*, '*k*) *keying* =
key-fun :: '*r* $\rightarrow$ '*k*
key-supp :: '*k semi-factor* $\Rightarrow$ '*r semi-factor*

hide-const (**open**) *keying.truncate*

locale *keyed* =
fixes *keying* :: ('*r*::{*domain*, *lod-domain*}, '*k*::*lod-domain*) *keying*
assumes *csupp-reads*: *proj a*$\cdot$(*key-fun keying*$\cdot$(*x* $\lfloor$ *key-supp keying a*)) = *proj a*$\cdot$(*key-fun keying*$\cdot$*x*)

and *csupp-bot*: *key-supp keying* $\bot = \bot$
and *csupp-sup*: *key-supp keying* $(a \sqcup b) =$ *key-supp keying* $a \sqcup$ *key-supp keying* b
and *csupp-compact*: *compact* $a \Longrightarrow$ *compact* (*key-supp keying* a)
begin

abbreviation *rkey* :: $'r \to 'k$ **where** *rkey* $\equiv$ *key-fun keying*
abbreviation *csupp* :: $'k$ *semi-factor* $\Rightarrow$ $'r$ *semi-factor* **where** *csupp* $\equiv$ *key-supp keying*

abbreviation *rproj* :: $'k$ *semi-factor* $\Rightarrow$ $'r \Rightarrow 'k$ **where**
rproj a x $\equiv$ *proj a*·(*rkey*·*x*)

definition *vpart* :: $('r, 'k)$ *view* $\Rightarrow$ $'r \Rightarrow 'r \times 'k$ **where**
vpart v x = (*x* ⌊ *fst v*, *rproj* (*snd v*) *x*)

lemma *vpart-pair* [*simp*]: *vpart* (*c*, *a*) *x* = (*x* ⌊ *c*, *rproj a x*)
by (*simp add*: *vpart-def*)

A view part projects onto the parts of smaller views.

lemma *vpart-pproj*: $v \sqsubseteq w \Longrightarrow$ *pproj* v·(*vpart w x*) = *vpart v x*
by (*simp add*: *vpart-def below-prod-def proj-le*)

A key part is read off any section covering its support.

lemma *rproj-section*: *csupp a* $\sqsubseteq c \Longrightarrow$ *rproj a* (*y* ⌊ *c*) = *rproj a y*
by (*metis csupp-reads proj-le*)

lemma *csupp-mono*: $a \sqsubseteq b \Longrightarrow$ *csupp a* $\sqsubseteq$ *csupp b*
by (*metis csupp-sup join-absorb lattice-join-ge1*)

22.1 Dependencies on a set of rows

A view determines a function on a set of rows when the function is monotone in the view there: a row reading less on the view gives less. On all rows this makes the function factor continuously through the view; in particular rows agreeing on the view agree on the function. A dependency between views is the case where the function is the other view.

definition *determines* :: $'r$ *set* $\Rightarrow$ $('r, 'k)$ *view* $\Rightarrow$ $('r \to 'b{::}cpo) \Rightarrow$ *bool* **where**
determines P v g $\longleftrightarrow$ $(\forall x{\in}P.\ \forall y{\in}P.$ *vpart v x* $\sqsubseteq$ *vpart v y* $\longrightarrow$ $g{\cdot}x \sqsubseteq g{\cdot}y)$

The view part as a continuous function, for dependencies between views.

definition *vproj* :: $('r, 'k)$ *view* $\Rightarrow$ $'r \to 'r \times 'k$ **where**
vproj v = (Λ *x*. *vpart v x*)

lemma *vproj-apply* [*simp*]: *vproj* $v{\cdot}x$ = *vpart v x*
by (*simp add*: *vproj-def vpart-def*)

abbreviation *vfd* :: *'r set* ⇒ (*'r*, *'k*) *view* ⇒ (*'r*, *'k*) *view* ⇒ *bool* **where**
vfd P v w ≡ *determines P v* (*vproj w*)

lemma *determines-belowD*:
determines P v g ⟹ *x* ∈ *P* ⟹ *y* ∈ *P* ⟹ *vpart v x* ⊑ *vpart v y* ⟹ *g*·*x* ⊑ *g*·*y*
unfolding *determines-def vproj-apply* **by** *blast*

lemma *determinesD*: *determines P v g* ⟹ *x* ∈ *P* ⟹ *y* ∈ *P* ⟹ *vpart v x* = *vpart v y* ⟹ *g*·*x* = *g*·*y*
by (*rule below-antisym*) (*simp-all add*: *determines-belowD*)

lemma *vfdD*: *vfd P v w* ⟹ *x* ∈ *P* ⟹ *y* ∈ *P* ⟹ *vpart v x* = *vpart v y* ⟹ *vpart w x* = *vpart w y*
using *determinesD*[*of P v vproj w x y*] **by** *simp*

lemma *determines-subset*: *determines P v g* ⟹ *Q* ⊆ *P* ⟹ *determines Q v g*
unfolding *determines-def vproj-apply* **by** *blast*

lemma *determines-trans*: *vfd P u v* ⟹ *determines P v g* ⟹ *determines P u g*
unfolding *determines-def vproj-apply* **by** *blast*

lemma *vpart-below-iff*: *vpart* (*c*, *a*) *x* ⊑ *vpart* (*c*, *a*) *y* ⟷ *x* ⌊ *c* ⊑ *y* ⌊ *c* ∧ *rproj a x* ⊑ *rproj a y*
by (*simp add*: *below-prod-def*)

lemma *determines-key*: *determines P* (*c*, *a*) (*h oo proj a oo rkey*)
unfolding *determines-def vproj-apply* **by** (*simp add*: *vpart-below-iff monofun-cfun-arg*)

lemma *determines-comp*:
assumes *g*: *determines P v g*
shows *determines P v* (*h oo g*)
unfolding *determines-def vproj-apply*
using *determines-belowD*[*OF g*] **by** (*simp add*: *monofun-cfun-arg*)

lemma *determines-pair*:
assumes *f*: *determines P v f* **and** *g*: *determines P v g*
shows *determines P v* (Λ *x*. *h*·(*f*·*x*)·(*g*·*x*))
unfolding *determines-def vproj-apply*
using *determines-belowD*[*OF f*] *determines-belowD*[*OF g*] **by** (*simp add*: *monofun-cfun*)

lemma *determines-prod*:
assumes *f*: *determines P v f* **and** *g*: *determines P v g*
shows *determines P v* (Λ *x*. (*f*·*x*, *g*·*x*))
unfolding *determines-def vproj-apply*
using *determines-belowD*[*OF f*] *determines-belowD*[*OF g*] **by** (*simp add*: *below-prod-def*)

Views are ordered structurally, as a product: a larger view reads more.

lemma *vpart-le*:
assumes *le*: $v \sqsubseteq w$ **and** *below*: *vpart w x* $\sqsubseteq$ *vpart w y*
shows *vpart v x* $\sqsubseteq$ *vpart v y*
proof −
have *c*: $x \upharpoonright fst\ w \sqsubseteq y \upharpoonright fst\ w$ **and** *k*: *rproj* (*snd w*) *x* $\sqsubseteq$ *rproj* (*snd w*) *y*
using *below* **by** (*simp-all add*: *vpart-def below-prod-def*)
have $x \upharpoonright fst\ v \sqsubseteq y \upharpoonright fst\ v$
using *monofun-cfun-arg*[*OF c*, *of proj* (*fst v*)] *le* **by** (*simp add*: *below-prod-def proj-le*)
moreover have *rproj* (*snd v*) *x* $\sqsubseteq$ *rproj* (*snd v*) *y*
using *monofun-cfun-arg*[*OF k*, *of proj* (*snd v*)] *le* **by** (*simp add*: *below-prod-def proj-le*)
ultimately show *?thesis* **by** (*simp add*: *vpart-def below-prod-def*)
qed

lemma *vfd-refl*: $w \sqsubseteq v \Longrightarrow$ *vfd P v w*
unfolding *determines-def vproj-apply* **by** (*blast intro*: *vpart-le*)

lemma *determines-mono*: *determines P v g* $\Longrightarrow$ $v \sqsubseteq w \Longrightarrow$ *determines P w g*
by (*rule determines-trans*[*OF vfd-refl*])

Projections to two LODs below a third join monotonically.

lemma *proj-sup-mono*:
fixes *x y* :: *'a*::{*bifinite*, *distributive-cpo*}
assumes *proj a*·*x* $\sqsubseteq$ *proj a*·*y* **and** *proj b*·*x* $\sqsubseteq$ *proj b*·*y*
shows *proj* (*a* $\sqcup$ *b*)·*x* $\sqsubseteq$ *proj* (*a* $\sqcup$ *b*)·*y*
proof −
have *ub*: *proj a*·*y* $\sqsubseteq$ *proj a*·*y* $\sqcup$ *proj b*·*y proj b*·*y* $\sqsubseteq$ *proj a*·*y* $\sqcup$ *proj b*·*y*
using *join-is-lub*[*OF proj-below proj-below*, *of a y b*, *THEN is-lubD1*, *THEN is-ubD*] **by** *simp-all*
show *?thesis* **unfolding** *proj-sup*
by (*rule join-belowI*) (*rule below-trans*[*OF assms*(*1*) *ub*(*1*)], *rule below-trans*[*OF assms*(*2*) *ub*(*2*)])
qed

lemma *vpart-join*:
assumes *v*: *vpart v x* $\sqsubseteq$ *vpart v y* **and** *w*: *vpart w x* $\sqsubseteq$ *vpart w y*
shows *vpart* (*v* $\sqcup$ *w*) *x* $\sqsubseteq$ *vpart* (*v* $\sqcup$ *w*) *y*
proof −
have $x \upharpoonright fst\ v \sqsubseteq y \upharpoonright fst\ v\ x \upharpoonright fst\ w \sqsubseteq y \upharpoonright fst\ w$
rproj (*snd v*) *x* $\sqsubseteq$ *rproj* (*snd v*) *y rproj* (*snd w*) *x* $\sqsubseteq$ *rproj* (*snd w*) *y*
using *v w* **by** (*simp-all add*: *vpart-def below-prod-def*)
then show *?thesis* **unfolding** *vpart-def lattice-join-prod fst-conv snd-conv below-prod-def*
by (*simp add*: *proj-sup-mono*)
qed

lemma *vfd-sup*: *vfd P u v* $\Longrightarrow$ *vfd P u w* $\Longrightarrow$ *vfd P u* (*v* $\sqcup$ *w*)
unfolding *determines-def vproj-apply* **by** (*blast intro*: *vpart-join*)

Columns-to-keys dependencies are a matter of computation.

lemma *vfd-support*: *csupp a* $\sqsubseteq$ *c* $\Longrightarrow$ *vfd P* (*c*, $\bot$) ($\bot$, *a*)
  unfolding *determines-def vproj-apply*
proof (*intro ballI impI*)
  fix *x y* **assume** *le*: *csupp a* $\sqsubseteq$ *c* **and** *below*: *vpart* (*c*, $\bot$) *x* $\sqsubseteq$ *vpart* (*c*, $\bot$) *y*
  have *x* $\lfloor$ *c* $\sqsubseteq$ *y* $\lfloor$ *c* **using** *below* **by** (*simp add*: *vpart-below-iff*)
   then have *rproj a* (*x* $\lfloor$ *c*) $\sqsubseteq$ *rproj a* (*y* $\lfloor$ *c*) **by** (*rule monofun-cfun-arg*[*OF monofun-cfun-arg*])
  then have *rproj a x* $\sqsubseteq$ *rproj a y* **by** (*simp only*: *rproj-section*[*OF le*])
  then show *vpart* ($\bot$, *a*) *x* $\sqsubseteq$ *vpart* ($\bot$, *a*) *y* **by** (*simp add*: *vpart-below-iff proj-bot*)
qed

22.2 The key part, the section, and the determination order

The key part of a view drops its column part: it is the meet with the fixed view ($\bot$, $\top$), so it is below the view and preserves joins. The section of a view is the pure-column view reading the columns and the declared support of the key part. Views are also preordered semantically, by *determination* on all rows: the structural order implies it, the key part is below the view and the section above it. On pure-column views the two orders agree.

definition *vkey* :: ('*r*, '*k*) *view* $\Rightarrow$ ('*r*, '*k*) *view* **where**
  vkey v = ($\bot$, *snd v*)

definition *vsec* :: ('*r*, '*k*) *view* $\Rightarrow$ ('*r*, '*k*) *view* **where**
  vsec v = (*fst v* $\sqcup$ *csupp* (*snd v*), $\bot$)

definition *vdet* :: ('*r*, '*k*) *view* $\Rightarrow$ ('*r*, '*k*) *view* $\Rightarrow$ *bool* (**infix** ‹$\preceq$› *50*) **where**
  v $\preceq$ *w* $\longleftrightarrow$ *vfd UNIV w v*

lemma *vkey-meet*: *vkey v* = *v* $\sqcap$ ($\bot$, *lattice-top*)
 by (*simp add*: *vkey-def prod-meet lattice-meet-commute*[*of - lattice-top*] *lattice-meet-top-left*)

lemma *vkey-below*: *vkey v* $\sqsubseteq$ *v*
  by (*simp add*: *vkey-def below-prod-def*)

lemma *vkey-idem* [*simp*]: *vkey* (*vkey v*) = *vkey v*
  by (*simp add*: *vkey-def*)

lemma *vkey-join*: *vkey* (*v* $\sqcup$ *w*) = *vkey v* $\sqcup$ *vkey w*
  by (*simp add*: *vkey-def lattice-join-prod*)

lemma *vsec-idem* [*simp*]: *vsec* (*vsec v*) = *vsec v*
  by (*simp add*: *vsec-def csupp-bot*)

lemma *vdet-refl* [*simp*]: *v* $\preceq$ *v*
  unfolding *vdet-def* **by** (*rule vfd-refl*) *simp*

lemma *vdet-trans*: *u* $\preceq$ *v* $\Longrightarrow$ *v* $\preceq$ *w* $\Longrightarrow$ *u* $\preceq$ *w*

unfolding *vdet-def* **by** (*rule determines-trans*)

lemma *below-vdet*: $v \sqsubseteq w \Longrightarrow v \preceq w$
unfolding *vdet-def* **by** (*rule vfd-refl*)

lemma *vdet-vfd*: $v \preceq w \Longrightarrow vfd\ P\ w\ v$
unfolding *vdet-def* **by** (*erule determines-subset*) *simp*

lemma *determines-vdet*: $determines\ P\ v\ g \Longrightarrow v \preceq w \Longrightarrow determines\ P\ w\ g$
by (*rule determines-trans*[*OF vdet-vfd*])

lemma *vkey-vdet*: $vkey\ v \preceq v$
by (*rule below-vdet*[*OF vkey-below*])

lemma *vdet-vsec*: $v \preceq vsec\ v$
unfolding *vdet-def determines-def vproj-apply*
proof (*intro ballI impI*)
fix *x y* **assume** *below*: $vpart\ (vsec\ v)\ x \sqsubseteq vpart\ (vsec\ v)\ y$
let *?s* = $fst\ v \sqcup csupp\ (snd\ v)$
have *s*: $x \lfloor ?s \sqsubseteq y \lfloor ?s$ **using** *below* **by** (*simp add*: *vsec-def vpart-def below-prod-def*)
have $x \lfloor fst\ v \sqsubseteq y \lfloor fst\ v$
using *monofun-cfun-arg*[*OF s, of proj* (*fst v*)] **by** (*simp add*: *proj-le lattice-join-ge1*)
moreover have $rproj\ (snd\ v)\ x \sqsubseteq rproj\ (snd\ v)\ y$
proof −
have *le*: $csupp\ (snd\ v) \sqsubseteq ?s$ **by** (*rule lattice-join-ge2*)
have $rproj\ (snd\ v)\ (x \lfloor ?s) \sqsubseteq rproj\ (snd\ v)\ (y \lfloor ?s)$ **by** (*rule monofun-cfun-arg*[*OF monofun-cfun-arg*[*OF s*]])
then show *?thesis* **by** (*simp only*: *rproj-section*[*OF le*])
qed
ultimately show $vpart\ v\ x \sqsubseteq vpart\ v\ y$ **by** (*simp add*: *vpart-def below-prod-def*)
qed

Factoring: a continuous function determined by a view on all rows factors continuously through the view's section, which rows realise (a section of a row is a row).

theorem *determines-factor*:
assumes $determines\ UNIV\ v\ f$
shows $f = f\ oo\ proj\ (fst\ v \sqcup csupp\ (snd\ v))$
proof (*rule cfun-eqI*)
fix *x*
let *?s* = $fst\ v \sqcup csupp\ (snd\ v)$
have *p*: $vsec\ v = (?s, \bot)$ **by** (*rule vsec-def*)
have *s*: $determines\ UNIV\ (?s, \bot)\ f$
using *determines-vdet*[*OF assms vdet-vsec*] **unfolding** *p* **.**
have $vpart\ (?s, \bot)\ (x \lfloor ?s) = vpart\ (?s, \bot)\ x$ **by** (*simp add*: *proj-bot*)
then have $f{\cdot}(x \lfloor ?s) = f{\cdot}x$ **by** (*rule determinesD*[*OF s UNIV-I UNIV-I*])
then show $f{\cdot}x = (f\ oo\ proj\ ?s){\cdot}x$ **by** (*simp only*: *cfcomp2*)

qed

In general a function factors continuously through the view's parts, read as a pair, into the values with a top adjoined: the least upper bound of what rows reading at most a given pair give (a left Kan extension along the view part). For a compact view it is continuous and reads only the view's parts of its argument; on view parts it is the function, if the view determines it.

definition *vfactor-set* :: ('*r*, '*k*) *view* ⇒ ('*r* → '*b*::*bounded-complete*) ⇒ '*r* × '*k* ⇒ '*b top set* **where**
vfactor-set v f p = {*down*·(*f*·*x*) |*x*. *vpart v x* ⊑ *p*}

definition *vfactor* :: ('*r*, '*k*) *view* ⇒ ('*r* → '*b*::*bounded-complete*) ⇒ '*r* × '*k* → '*b top* **where**
vfactor v f = (Λ *p*. *lub* (*vfactor-set v f p*))

lemma *compact-vpart*: *compact v* ⟹ *compact x* ⟹ *compact* (*vpart v x*)
by (*simp add*: *vpart-def compact-proj-compact proj-compact*[*OF compact-snd*])

lemma *lattice-lub-max*: *x* ∈ *S* ⟹ (⋀*y*. *y* ∈ *S* ⟹ *y* ⊑ *x*) ⟹ *lub S* = (*x* :: '*a*::*lattice-complete*)
by (*rule below-antisym*) (*blast intro*: *lattice-lub-least is-ubI*, *rule lattice-lub-upper*)

lemma *vfactor-beta*:
fixes *f* :: '*r* → '*b*::*bounded-complete*
assumes *cv*: *compact v*
shows *vfactor v f*·*p* = *lub* (*vfactor-set v f p*)
unfolding *vfactor-def*
proof (*rule beta-cfun*)
let *?G* = λ*p*. *lub* (*vfactor-set v f p*)
have *mono*: *?G p* ⊑ *?G q* **if** *p* ⊑ *q* **for** *p q*
by (*rule lattice-lub-least*, *rule is-ubI*, *rule lattice-lub-upper*)
(*auto simp*: *vfactor-set-def intro*: *below-trans*[*OF - that*])
obtain *A* :: *nat* ⇒ '*r* → '*r* **where** *A*: *approx-chain A* **using** *bifinite* **by** *blast*
have *app*: *x* = (⨆*j*. *A j*·*x*) **for** *x*
using *approx-chain.lub-approx*[*OF A*] *approx-chain.chain-approx*[*OF A*]
by (*metis ID1 contlub-cfun-fun*)
have *chain-bound*: *?G* (⨆ *i*. *Y i*) ⊑ (⨆ *i*. *?G* (*Y i*)) **if** *Y*: *chain Y* **for** *Y*
proof (*rule lattice-lub-least*, *rule is-ubI*)
have *GY*: *chain* (λ*i*. *?G* (*Y i*)) **using** *Y* **by** (*auto intro*: *chainI mono chainE*)
fix *u* **assume** *u* ∈ *vfactor-set v f* (⨆ *i*. *Y i*)
then obtain *x* **where** *u*: *u* = *down*·(*f*·*x*) **and** *x*: *vpart v x* ⊑ (⨆ *i*. *Y i*)
by (*auto simp*: *vfactor-set-def*)
have *chA*: *chain* (λ*j*. *A j*·*x*) **using** *approx-chain.chain-approx*[*OF A*] **by** (*simp add*: *ch2ch-Rep-cfunL*)
have *each*: *down*·(*f*·(*A j*·*x*)) ⊑ (⨆ *i*. *?G* (*Y i*)) **for** *j*
proof −
have *vpart v* (*A j*·*x*) ⊑ *vpart v x*
using *monofun-cfun-arg*[*OF approx-chain.approx-below*[*OF A*], *of vproj v*]
by *simp*

```
    then have vpart v (A j·x) ⊑ (⨆ i. Y i) using x by (rule below-trans)
    then obtain i where i: vpart v (A j·x) ⊑ Y i
     using compactD2[OF compact-vpart[OF cv approx-chain.compact-approx[OF
A]] Y] by blast
    have down·(f·(A j·x)) ⊑ ?G (Y i)
      by (rule lattice-lub-upper) (auto simp: vfactor-set-def intro: i)
    also have ... ⊑ (⨆ i. ?G (Y i)) by (rule is-ub-thelub[OF GY])
    finally show ?thesis .
  qed
  have u = (⨆ j. down·(f·(A j·x)))
    unfolding u by (subst app[of x]) (simp add: contlub-cfun-arg chA)
  then show u ⊑ (⨆ i. ?G (Y i)) using each by (simp add: lub-below chA)
 qed
 show cont ?G by (rule contI2) (auto intro: monofunI mono chain-bound)
qed

lemma vfactor-reads:
 assumes cv: compact v
 shows vfactor v f·(s, κ) = vfactor v f·(s ⌊ fst v, proj (snd v)·κ)
proof −
 have vpart v x ⊑ (s, κ) ⟷ vpart v x ⊑ (s ⌊ fst v, proj (snd v)·κ) for x
 proof
   assume vpart v x ⊑ (s, κ)
    then have x ⌊ fst v ⊑ s rproj (snd v) x ⊑ κ by (simp-all add: vpart-def
below-prod-def)
   then show vpart v x ⊑ (s ⌊ fst v, proj (snd v)·κ)
     using monofun-cfun-arg[of - - proj (fst v)] monofun-cfun-arg[of - - proj (snd
v)]
     by (fastforce simp: vpart-def below-prod-def)
 next
   assume vpart v x ⊑ (s ⌊ fst v, proj (snd v)·κ)
   then show vpart v x ⊑ (s, κ) by (rule below-trans) (simp add: below-prod-def
proj-below)
 qed
 then have vfactor-set v f (s, κ) = vfactor-set v f (s ⌊ fst v, proj (snd v)·κ)
   by (simp add: vfactor-set-def)
 then show ?thesis by (simp add: vfactor-beta[OF cv])
qed

theorem vfactor-vpart:
 assumes cv: compact v and d: determines UNIV v f
 shows vfactor v f·(vpart v x) = down·(f·x)
 unfolding vfactor-beta[OF cv]
proof (rule lattice-lub-max)
 show down·(f·x) ∈ vfactor-set v f (vpart v x) by (auto simp: vfactor-set-def)
 show u ⊑ down·(f·x) if u ∈ vfactor-set v f (vpart v x) for u
   using that determines-belowD[OF d] by (auto simp: vfactor-set-def)
qed
```

corollary *vfactor-vproj*:
compact v $\Longrightarrow$ *determines UNIV v f* $\Longrightarrow$ *vfactor v f oo vproj v* = *down oo f*
by (*rule cfun-eqI*) (*simp add*: *vfactor-vpart*)

corollary *determines-factor-ex*:
determines UNIV v f $\Longrightarrow$ $\exists h.$ *f* = *h oo proj* (*fst v* $\sqcup$ *csupp* (*snd v*))
by (*blast dest*: *determines-factor*)

Joins: a view determined by two views' join is bounded by both; a join is the least upper bound up to determination.

lemma *vdet-join*: $u \preceq w \Longrightarrow v \preceq w \Longrightarrow u \sqcup v \preceq w$
unfolding *vdet-def* **by** (*rule vfd-sup*)

lemma *vdet-join-mono*: $u \preceq u' \Longrightarrow v \preceq v' \Longrightarrow u \sqcup v \preceq u' \sqcup v'$
by (*rule vdet-join*) (*blast intro*: *vdet-trans below-vdet lattice-join-ge1 lattice-join-ge2*)+

lemma *view-split*: *v* = (*fst v*, $\bot$) $\sqcup$ ($\bot$, *snd v*) **for** *v* :: ('*r*, '*k*) *view*
by (*simp add*: *lattice-join-prod lattice-join-bot-left lattice-join-bot-right*)

A key part is *exact* when it determines its declared column support on all rows: the key is one-to-one with those columns. For exact key parts the section is equivalent to the view.

definition *exact-key* :: '*k semi-factor* $\Rightarrow$ *bool* **where**
exact-key a $\longleftrightarrow$ (*csupp a*, $\bot$) $\preceq$ ($\bot$, *a*)

theorem *vsec-vdet-exact*:
assumes *exact-key* (*snd v*)
shows *vsec v* $\preceq$ *v*
proof −
have *vsec v* = (*fst v*, $\bot$) $\sqcup$ (*csupp* (*snd v*), $\bot$)
by (*simp add*: *vsec-def lattice-join-prod lattice-join-bot-left*)
moreover have (*fst v*, $\bot$) $\preceq$ *v* **by** (*rule below-vdet*) (*simp add*: *below-prod-def*)
moreover have (*csupp* (*snd v*), $\bot$) $\preceq$ *v*
using *assms* **unfolding** *exact-key-def*
by (*rule vdet-trans*) (*rule below-vdet*, *simp add*: *below-prod-def*)
ultimately show *?thesis* **by** (*simp add*: *vdet-join*)
qed

lemma *vdet-sections*: (*c*, $\bot$) $\preceq$ (*c'*, $\bot$) $\longleftrightarrow$ *c* $\sqsubseteq$ *c'*
proof
assume *d*: (*c*, $\bot$) $\preceq$ (*c'*, $\bot$)
have *proj* (*c* $\sqcap$ *c'*)·*x* = *proj c*·*x* **for** *x*
proof −
have *vpart* (*c'*, $\bot$) (*x* $\lfloor$ *c'*) = *vpart* (*c'*, $\bot$) *x* **by** (*simp add*: *proj-bot*)
then have *vpart* (*c*, $\bot$) (*x* $\lfloor$ *c'*) = *vpart* (*c*, $\bot$) *x*
using *d* **unfolding** *vdet-def* **by** (*blast dest*: *vfdD*)
then show *?thesis* **by** (*simp add*: *proj-inf proj-bot*)
qed
then have *c* $\sqcap$ *c'* = *c* **by** (*simp add*: *cfun-eqI semi-factor-p-inject*[*symmetric*])

then show $c \sqsubseteq c'$ **by** (*metis lattice-meet-le2*)
qed (*simp add*: *below-vdet below-prod-def*)

22.3 Weak dependencies

A weak dependency only holds between rows where both sides are present, as for the two sides of a join. Weak dependencies do not compose with each other (the middle view may be absent in one row), but they compose with dependencies on either side, given that presence passes from the outer view to the middle one.

definition *vpresent* :: ('*r*, '*k*) *view* ⇒ '*r* ⇒ *bool* **where**
vpresent v x ⟷ *vpart v x* ≠ ⊥

definition *weak-fd* :: '*r set* ⇒ ('*r*, '*k*) *view* ⇒ ('*r*, '*k*) *view* ⇒ *bool* **where**
weak-fd P v w ⟷ (∀ *x*∈*P*. ∀ *y*∈*P*. *vpart v x* = *vpart v y* ⟶ *vpresent v x* ⟶ *vpresent w x* ⟶ *vpresent w y* ⟶ *vpart w x* = *vpart w y*)

lemma *vfd-weak-fd*: *vfd P v w* ⟹ *weak-fd P v w*
unfolding *weak-fd-def* **by** (*blast dest*: *vfdD*)

lemma *vfd-weak-fd-trans*:
vfd P u v ⟹ *weak-fd P v w* ⟹ (∀ *x*∈*P*. *vpresent u x* ⟶ *vpresent v x*) ⟹ *weak-fd P u w*
unfolding *weak-fd-def* **by** (*blast dest*: *vfdD*)

lemma *weak-fd-vfd-trans*:
weak-fd P u v ⟹ *vfd P v w* ⟹ (∀ *x*∈*P*. *vpresent w x* ⟶ *vpresent v x*) ⟹ *weak-fd P u w*
unfolding *weak-fd-def* **by** (*blast dest*: *vfdD*)

The chase only guarantees the guarded form: equal *maximal* determinants. It becomes a dependency between views when the determinant is maximal where present and the dependent part is absent where the determinant is.

lemma *fd-vfd*:
assumes *f*: *fd a b P* **and** *m*: ∀ *x*∈*P*. *x* ⌊ *a* ≠ ⊥ ⟶ *maximal-at a* (*x* ⌊ *a*)
and *abs*: ∀ *x*∈*P*. *x* ⌊ *a* = ⊥ ⟶ *x* ⌊ *b* = ⊥
shows *vfd P* (*a*, ⊥) (*b*, ⊥)
unfolding *determines-def vproj-apply*
proof (*intro ballI impI*)
fix *x y* **assume** *x*: *x* ∈ *P* **and** *y*: *y* ∈ *P* **and** *below*: *vpart* (*a*, ⊥) *x* ⊑ *vpart* (*a*, ⊥) *y*
have *le*: *x* ⌊ *a* ⊑ *y* ⌊ *a* **using** *below* **by** (*simp add*: *vpart-below-iff*)
have *x* ⌊ *b* ⊑ *y* ⌊ *b*
proof (*cases x* ⌊ *a* = ⊥)
case *True*
then show *?thesis* **using** *abs x* **by** *simp*
next
case *False*

then have *mx*: *maximal-at a* (*x* ⌊ *a*) **using** *m x* **by** *blast*
have *eq*: *x* ⌊ *a* = *y* ⌊ *a* **by** (*rule maximal-at-eq*[*OF mx - le*, *symmetric*]) *simp*
show *?thesis* **using** *fdD*[*OF f x y mx eq*] **by** *simp*
qed
then show *vpart* (*b*, ⊥) *x* ⊑ *vpart* (*b*, ⊥) *y* **by** (*simp add*: *vpart-below-iff proj-bot*)
qed

end

23 Dependencies over all conforming datasets

locale *keyed-schema* = *schema rules* + *keyed keying*
for *rules* :: *'r*::{*domain*, *lod-domain*} *schema-rules* **and** *keying* :: (*'r*, *'k*::*lod-domain*) *keying*
begin

definition *universally* :: (*'r set* ⇒ *bool*) ⇒ *bool* **where**
universally Q ⟷ (∀ *X*. *conforms X* ⟶ *Q* (*lower-maximal X*))

lemma *universallyD*: *universally Q* ⟹ *conforms X* ⟹ *Q* (*lower-maximal X*)
unfolding *universally-def* **by** *blast*

lemma *universally-mono*: *universally Q* ⟹ (⋀*P*. *Q P* ⟹ *T P*) ⟹ *universally T*
unfolding *universally-def* **by** *blast*

lemma *universally-mono2*:
universally Q ⟹ *universally Q'* ⟹ (⋀*P*. *Q P* ⟹ *Q' P* ⟹ *T P*) ⟹ *universally T*
unfolding *universally-def* **by** *blast*

A functional dependency of the schema holds universally between views, given maximal determinants and absent dependents where the determinant is absent.

lemma *universally-fd-rule*:
assumes *r*: *fd-rule a b* ∈ *chase-rules*
and *m*: *universally* (λ*P*. ∀ *x*∈*P*. *x* ⌊ *a* ≠ ⊥ ⟶ *maximal-at a* (*x* ⌊ *a*))
and *abs*: *universally* (λ*P*. ∀ *x*∈*P*. *x* ⌊ *a* = ⊥ ⟶ *x* ⌊ *b* = ⊥)
shows *universally* (λ*P*. *vfd P* (*a*, ⊥) (*b*, ⊥))
unfolding *universally-def*
using *fd-vfd conforms-fd*[*OF - r*] *universallyD*[*OF m*] *universallyD*[*OF abs*] **by** *blast*

23.1 Strong keys

A key is *strong* when it is functionally equivalent to its column support in every conforming dataset; one direction holds by computation. Strong keys

transfer column dependencies to keys: recover the columns from the key, then compute the other key from them.

definition *strong-key* :: *'k semi-factor* ⇒ *bool* **where**
strong-key a ⟷ *universally* (λ*P*. *vfd P* (⊥, *a*) (*csupp a*, ⊥))

lemma *strong-key-equiv*:
strong-key a ⟹ *conforms X* ⟹
vfd (*lower-maximal X*) (⊥, *a*) (*csupp a*, ⊥) ∧ *vfd* (*lower-maximal X*) (*csupp a*, ⊥) (⊥, *a*)
unfolding *strong-key-def* **by** (*auto dest*: *universallyD intro*: *vfd-support*)

theorem *strong-key-transfer*:
assumes *a*: *strong-key a* **and** *c*: *universally* (λ*P*. *vfd P* (*csupp a*, ⊥) (*c*, ⊥))
and *b*: *csupp b* ⊑ *c*
shows *universally* (λ*P*. *vfd P* (⊥, *a*) (⊥, *b*))
using *a c* **unfolding** *strong-key-def*
by (*rule universally-mono2*) (*blast intro*: *determines-trans vfd-support*[*OF b*])

23.2 Supports of content functions

A content function reads a section of the row together with a part of its key: on all rows, these two (its column and key supports) determine it, which is what it depends on across datasets. Within every conforming dataset the key part alone determines it, as when it reads a value column whose primary key is in the key.

definition *fun-support* :: (*'r*, *'k*) *view* ⇒ (*'r* → *'b*::*cpo*) ⇒ *bool* **where**
fun-support v f ⟷ *determines UNIV v f* ∧ *universally* (λ*P*. *determines P* (*vkey v*) *f*)

A function whose view is its own key part is *key-pure*: determined by its key part on all rows. A function reading only a section is supported by any key part determining that section in data.

lemma *fun-support-key*: *fun-support* (*vkey v*) *f* ⟷ *determines UNIV* (*vkey v*) *f*
unfolding *fun-support-def universally-def vkey-idem* **by** (*blast intro*: *determines-subset*)

A view is *keyed* when within every conforming dataset its key part determines it; then every function of its parts is supported by it.

definition *keyed-view* :: (*'r*, *'k*) *view* ⇒ *bool* **where**
keyed-view v ⟷ *universally* (λ*P*. *vfd P* (*vkey v*) *v*)

lemma *keyed-view-key*: *keyed-view* (⊥, *a*)
unfolding *keyed-view-def universally-def vkey-def* **by** (*simp add*: *vfd-refl*)

lemma *keyed-view-sup*: *keyed-view v* ⟹ *keyed-view w* ⟹ *keyed-view* (*v* ⊔ *w*)
unfolding *keyed-view-def*
proof (*erule universally-mono2*, *assumption*)
fix *P* **assume** *v*: *vfd P* (*vkey v*) *v* **and** *w*: *vfd P* (*vkey w*) *w*

have *vfd P* (*vkey* (*v* ⊔ *w*)) *v vfd P* (*vkey* (*v* ⊔ *w*)) *w*
unfolding *vkey-join* **by** (*rule determines-mono*[*OF v lattice-join-ge1*], *rule determines-mono*[*OF w lattice-join-ge2*])
then show *vfd P* (*vkey* (*v* ⊔ *w*)) (*v* ⊔ *w*) **by** (*rule vfd-sup*)
qed

lemma *keyed-view-columns*:
assumes *fd*: *universally* (λ*P*. *vfd P* (⊥, *a*) (*c*, ⊥))
shows *keyed-view* (*c*, *a*)
proof −
have *j*: (*c*, *a*) = (*c*, ⊥) ⊔ (⊥, *a*) **using** *view-split*[*of* (*c*, *a*)] **by** *simp*
have *universally* (λ*P*. *vfd P* (⊥, *a*) ((*c*, ⊥) ⊔ (⊥, *a*)))
using *fd* **by** (*rule universally-mono*) (*rule vfd-sup*[*OF - vfd-refl*[*OF below-refl*]])
then show *?thesis* **unfolding** *keyed-view-def vkey-def* **by** (*simp only*: *j*[*symmetric*] *snd-conv*)
qed

lemma *keyed-view-determines*:
keyed-view v ⟹ *conforms X* ⟹ *determines* (*lower-maximal X*) (*vkey v*) (*h oo vproj v*)
unfolding *keyed-view-def* **by** (*blast dest*: *universallyD intro*: *determines-comp*)

lemma *fun-support-columns*:
assumes *reads*: ⋀*x*. *f*·(*x* ⌊ *c*) = *f*·*x* **and** *ka*: *universally* (λ*P*. *vfd P* (⊥, *a*) (*c*, ⊥))
shows *fun-support* (*c*, *a*) *f*
proof −
have *c*: *determines P* (*c*, ⊥) *f* **for** *P*
unfolding *determines-def vproj-apply vpart-below-iff* **by** (*metis reads mono-fun-cfun-arg*)
have *determines UNIV* (*c*, *a*) *f*
by (*rule determines-mono*[*OF c*]) (*simp add*: *below-prod-def*)
moreover have *universally* (λ*P*. *determines P* (⊥, *a*) *f*)
using *ka* **by** (*rule universally-mono*) (*rule determines-trans*[*OF - c*])
ultimately show *?thesis* **unfolding** *fun-support-def vkey-def* **by** *simp*
qed

end

end
theory *Frame*
imports *Slicer Views*
begin

default-sort *type*

24 Frames

A frame has a level of detail and two levels of filters. The *ambient* filter is an open predicate on rows: expressions may add to it (an MDX-style subselect), but never take it apart or discard it. The *slice* is a syntactic filter on keys that expressions coarsen; dropping detail from the frame always coarsens the slice as well. The frame's LOD partitions the selected rows into groups by their projected keys.

record (*'l*::*type*, *'f*::*type*, *'r*::*pcpo*) *frame* =
  flod :: *'l*
  amb :: *'r sel*
  slice :: *'f*

The record package's own *truncate* operation would shadow the truncation action.

hide-const (**open**) *frame.truncate*

25 Frame transformers

The level-of-detail constructs act on the frame an inner expression is evaluated under. *finclude D* adds detail *D* (requested detail: *INCLUDE* is not a combinator of its own, it compiles into the grouping inside an aggregate); *frestrict φ* adds *φ* to the ambient filter; *ffor τ G* needs the slicer: it drops detail by a truncation *τ*, from the frame and from the slice, and conjoins *G*. Pegging conjoins a key's pin to the slice.

definition *finclude* :: *'l*::*po* ⇒ (*'l*, *'f*::*type*, *'r*::*pcpo*) *frame* ⇒ (*'l*, *'f*, *'r*) *frame*
where
  finclude D F = *F*(|*flod* := *flod F* ⊔ *D*|)

lemma *flod-finclude*: *flod* (*finclude D F*) = *flod F* ⊔ *D*
  by (*simp add*: *finclude-def*)

lemma *amb-finclude* [*simp*]: *amb* (*finclude D F*) = *amb F*
  by (*simp add*: *finclude-def*)

lemma *slice-finclude* [*simp*]: *slice* (*finclude D F*) = *slice F*
  by (*simp add*: *finclude-def*)

definition *frestrict* :: *'r*::*pcpo sel* ⇒ (*'l*::*type*, *'f*::*type*, *'r*) *frame* ⇒ (*'l*, *'f*, *'r*) *frame*
where
  frestrict φ F = *F*(|*amb* := *inf* (*amb F*) *φ*|)

lemma *flod-frestrict* [*simp*]: *flod* (*frestrict φ F*) = *flod F*
  by (*simp add*: *frestrict-def*)

lemma *slice-frestrict* [*simp*]: *slice* (*frestrict φ F*) = *slice F*

by (*simp add*: *frestrict-def*)

lemma *amb-frestrict* [*simp*]: *amb* (*frestrict* φ *F*) = *inf* (*amb F*) φ
by (*simp add*: *frestrict-def*)

26 Frame semantics

A dataspace fixes a slicer and a schema with keys; datasets are elements of the lower powerdomain of rows, their data points the maximal members. Rows have keys, via *rkey*; a frame selects the rows in its ambient filter whose key its slice holds at.

locale *dataspace* = *slicer slicing* + *keyed-schema rules keying*
for *slicing* :: ('*f*, '*k*::{*domain*, *lod-domain*}) *slicer-ops*
and *rules* :: '*r*::{*domain*, *lod-domain*} *schema-rules* **and** *keying* :: ('*r*, '*k*) *keying*
begin

definition *frows* :: ('*k semi-factor*, '*f*, '*r*) *frame* ⇒ '*r* ⇒ *bool* **where**
frows F r ⟷ *selects* (*amb F*) *r* ∧ *holds* (*slice F*) (*rkey*·*r*)

lemma *frows-cong*: *amb F* = *amb F'* ⟹ *slice F* = *slice F'* ⟹ *frows F* = *frows F'*
by (*simp add*: *frows-def fun-eq-iff*)

lemma *frows-finclude* [*simp*]: *frows* (*finclude D F*) = *frows F*
by (*simp add*: *frows-def fun-eq-iff*)

lemma *frows-frestrict*: *frows* (*frestrict* φ *F*) *r* ⟷ *frows F r* ∧ *selects* φ *r*
by (*auto simp*: *frows-def*)

definition *ffor* :: '*k semi-factor* ⇒ '*f* ⇒ ('*k semi-factor*, '*f*, '*r*) *frame* ⇒ ('*k semi-factor*, '*f*, '*r*) *frame* **where**
ffor τ *G F* = *F*(|*flod* := (τ ⊓ *flod F*), *slice* := *coarsen* τ (*slice F*) ∗ *G*|)

lemma *flod-ffor* [*simp*]: *flod* (*ffor* τ *G F*) = (τ ⊓ *flod F*)
and *amb-ffor* [*simp*]: *amb* (*ffor* τ *G F*) = *amb F*
and *slice-ffor* [*simp*]: *slice* (*ffor* τ *G F*) = *coarsen* τ (*slice F*) ∗ *G*
by (*simp-all add*: *ffor-def*)

definition *peg* :: '*k* ⇒ ('*k semi-factor*, '*f*, '*r*) *frame* ⇒ ('*k semi-factor*, '*f*, '*r*) *frame* **where**
peg K F = *F*(|*slice* := *slice F* ∗ *spin K*|)

lemma *flod-peg* [*simp*]: *flod* (*peg K F*) = *flod F*
by (*simp add*: *peg-def*)

lemma *amb-peg* [*simp*]: *amb* (*peg K F*) = *amb F*
by (*simp add*: *peg-def*)

lemma *slice-peg* [*simp*]: *slice* (*peg K F*) = *slice F* ∗ *spin K*
by (*simp add*: *peg-def*)

lemma *frows-peg-sub*: *frows* (*peg K F*) *r* ⟹ *frows F r*
by (*simp add*: *frows-def*)

lemma *frows-peg*: *compact K* ⟹ *frows* (*peg K F*) *r* ⟷ *frows F r* ∧ *K* ⊑ *rkey*·*r*
by (*auto simp*: *frows-def holds-spin*)

Pegging commutes with the transformers that keep the slice.

lemma *peg-finclude*: *peg K* (*finclude D F*) = *finclude D* (*peg K F*)
by (*simp add*: *peg-def finclude-def*)

lemma *peg-frestrict*: *peg K* (*frestrict* φ *F*) = *frestrict* φ (*peg K F*)
by (*simp add*: *peg-def frestrict-def*)

end

27 Key images

The keys a frame has at an LOD are the projections of the data points it selects.

context *dataspace*
begin

definition *keyimg* :: *'r lower-pd* ⇒ (*'k semi-factor*, *'f*, *'r*) *frame* ⇒ *'k semi-factor* ⇒ *'k* ⇒ *bool* **where**
keyimg data F c k ⟷ (∃ *x*∈*lower-maximal data*. *frows F x* ∧ *k* = *proj c*·(*rkey*·*x*))

lemma *keyimg-cong*: *frows F* = *frows F'* ⟹ *keyimg data F c* = *keyimg data F' c*
by (*simp add*: *keyimg-def fun-eq-iff*)

lemma *keyimg-mono*: (⋀*r*. *frows F r* ⟹ *frows F' r*) ⟹ *keyimg data F c k* ⟹ *keyimg data F' c k*
unfolding *keyimg-def* **by** *blast*

Keys conform to the slice.

lemma *keyimg-slicer*: *keyimg data F c k* ⟹ ∃ *k'*. *k* ⊑ *k'* ∧ *holds* (*slice F*) *k'*
unfolding *keyimg-def frows-def* **using** *proj-below* **by** *blast*

lemma *keyimg-peg-iff*: *compact K* ⟹ *keyimg data* (*peg K F*) *c K* ⟷ *keyimg data F c K*
unfolding *keyimg-def* **by** (*auto simp*: *frows-peg proj-below*)

end

context *dataspace*

begin

An LOD is read on rows through their keys; reading at a join of LODs is reading at both.

lemma *rproj-sup-eq*: *rproj* (*a* ⊔ *b*) *y* = *rproj* (*a* ⊔ *b*) *z* ⟷ *rproj a y* = *rproj a z* ∧ *rproj b y* = *rproj b z*
proof
  assume *e*: *rproj* (*a* ⊔ *b*) *y* = *rproj* (*a* ⊔ *b*) *z*
  show *rproj a y* = *rproj a z* ∧ *rproj b y* = *rproj b z*
    using *arg-cong*[*OF e*, *of Rep-cfun* (*proj a*)] *arg-cong*[*OF e*, *of Rep-cfun* (*proj b*)]
    by (*simp add*: *proj-le lattice-join-ge1 lattice-join-ge2*)
qed (*simp add*: *proj-sup*)

end

end
theory *Domain-Transfer*
  imports *LOD HOLCF−extra.Domain-Order-Derive*
begin

28 Transferring the classes along domain isomorphisms

The domain package presents each domain as isomorphic, through *abs* and *rep* satisfying the laws of *iso*, to a combination of its constructor arguments. The order-theoretic transfer (bounds, lubs, meets; *bounded-complete-iso*, *distributive-cpo-iso*) is in *HOLCF−extra.Domain-Order*; levels of detail transfer by conjugation with the isomorphism (*semi-factor-defl-iso*, *finitary-iso* in *LOD-Factor*).

lemma *locally-finitely-detailed-iso*:
  fixes *abs* :: *'r*::*locally-finitely-detailed* → *'t*::{*bifinite*, *distributive-cpo*} **and** *rep*
  assumes *iso abs rep* **shows** *OFCLASS*(*'t*, *locally-finitely-detailed-class*)
proof *intro-classes*
  interpret *iso abs rep* **by** *fact*
  fix *k* :: *'t* **assume** *compact k*
  then have *compact* (*rep*·*k*) **by** (*rule compact-rep*)
  from *compact-level*[*OF this*, *unfolded ex-finitary-semi-factor-iff*] **obtain** *e*
    where *e*: *semi-factor-defl e finitary e e*·(*rep*·*k*) = *rep*·*k* **by** *blast*
  show ∃ *F*. *finitary-semi-factor F* ∧ *k* ⌊ *F* = *k*
    unfolding *ex-finitary-semi-factor-iff*
    using *e* **by** (*intro exI*[*of - abs oo e oo rep*])
      (*simp add*: *semi-factor-defl-iso*[*OF abs-iso rep-iso*] *finitary-iso*[*OF abs-iso rep-iso*])
qed

29 Example: a non-recursive domain

A lazy binary sum is represented by $'a\ u \oplus 'b\ u$; every class transfers from the combinator instances.

domain ($'a$::*domain*, $'b$::*domain*) *either* = *L* (**lazy** $'a$) | *R* (**lazy** $'b$)

lemma *either-iso*: *iso either-abs either-rep*
by *unfold-locales* (*rule either.abs-iso*, *rule either.rep-iso*)

Finitary levels are not admissible, but *either* is not recursive, so the map rule goes through without fixed-point induction.

lemma *finitary-either-map* [*domain-category-map*]:
finitary v $\Longrightarrow$ *finitary w* $\Longrightarrow$ *finitary* (*either-map*·*v*·*w*)
by (*domain-category-map-rule map-def*: *either-map-def iso-rews*: *either.iso-rews*)

instance *either* :: ({*domain*, *bounded-complete*}, {*domain*, *bounded-complete*}) *bounded-complete*
by (*rule bounded-complete-iso*[*OF either-iso*])

instance *either* :: ({*domain*, *distributive-cpo*}, {*domain*, *distributive-cpo*}) *distributive-cpo*
by (*rule distributive-cpo-iso*[*OF either-iso*])

instance *either* :: ({*domain*, *locally-finitely-detailed*}, {*domain*, *locally-finitely-detailed*}) *locally-finitely-detailed*
by (*rule locally-finitely-detailed-iso*[*OF either-iso*])

30 Example: a recursive domain

Strict lists recurse through $\otimes$. The classes are derived in dependency order; local finite detail needs the components of the smash product, here the element type and the recursive occurrence, to be separated sums.

domain ($'a$::*domain*) *sseq* = *SNil* | *SCons* $'a$ $'a$ *sseq*

lemma *bounded-complete-defl-sseq-map* [*domain-category-map*]:
bounded-complete-defl v $\Longrightarrow$ *bounded-complete-defl* (*sseq-map*·*v*)
by (*domain-category-map-rule map-def*: *sseq-map-def iso-rews*: *sseq.iso-rews*)

lemma *filter-sum-defl-sseq-map* [*domain-category-map*]:
filter-sum-defl v $\Longrightarrow$ *filter-sum-defl* (*sseq-map*·*v*)
by (*domain-category-map-rule map-def*: *sseq-map-def iso-rews*: *sseq.iso-rews*)

lemma *separated-sum-defl-sseq-map* [*domain-category-map*]:
separated-sum-defl v $\Longrightarrow$ *separated-sum-defl* (*sseq-map*·*v*)
by (*domain-category-map-rule map-def*: *sseq-map-def iso-rews*: *sseq.iso-rews*)

instance *sseq* :: ({*domain*, *bounded-complete*}) *bounded-complete*
by (*domain-category-instance map-ID*: *sseq-map-ID*)

instance *sseq* :: ({*domain*, *filter-sum*}) *filter-sum*
  by (*domain-category-instance map-ID*: *sseq-map-ID*)

instance *sseq* :: ({*domain*, *separated-sum*}) *separated-sum*
  by (*domain-category-instance map-ID*: *sseq-map-ID*)

lemma *distributive-defl-sseq-map* [*domain-category-map*]:
  distributive-defl (*v* :: *'a*::{*domain*, *bounded-complete*, *filter-sum*} → *'a*) ⟹
    distributive-defl (*sseq-map*·*v*)
  by (*domain-category-map-rule map-def*: *sseq-map-def iso-rews*: *sseq.iso-rews*)

instance *sseq* :: ({*domain*, *distributive-cpo*, *filter-sum*}) *distributive-cpo*
  by (*domain-category-instance map-ID*: *sseq-map-ID*)

lemma *locally-finitely-detailed-defl-sseq-map* [*domain-category-map*]:
  locally-finitely-detailed-defl (*v* :: *'a*::{*domain*, *distributive-cpo*, *separated-sum*} →
'a) ⟹
    locally-finitely-detailed-defl (*sseq-map*·*v*)
  by (*domain-category-map-rule map-def*: *sseq-map-def iso-rews*: *sseq.iso-rews*)

instance *sseq* :: ({*domain*, *separated-sum*, *locally-finitely-detailed*}) *locally-finitely-detailed*
  by (*domain-category-instance map-ID*: *sseq-map-ID*)

end
theory *Value*
  imports *Domain-Transfer*
begin

31 Values

Bottom is a data-quality lapse or divergence: a dangling foreign key, a failed cast, a short row. Missing values that are not lapses live at three levels.

- *Data* (records and record-level expressions), *'v datum*: *NA* (not applicable; joins through it give *NA*), *Val*·*Null* (present but unknown; joins through it give ⊥), or a known value. Source nulls map to one of the two per column.
- *Keys* (dimensions), *'v with-na*: nulls become *NA*, which can be pinned (*to-key*).
- *Measures*, *'v with-null*: *NA* becomes ⊥, i.e. no entry (*from-na*); aggregations treat ⊥ and *NA* alike. A record-level function is *NA-blind* when it commutes with *squash*, the projection forgetting *NA*; lifted operators are, coalescing is not.

31.1 Scalars

Scalars have strict fields, so there are no per-type roots: *scalar* is flat.

domain *scalar* = *SBool tr* | *SInt int lift*

lemma *bounded-complete-defl-scalar-map* [*domain-category-map*]: *bounded-complete-defl scalar-map*
by (*domain-category-map-rule map-def*: *scalar-map-def iso-rews*: *scalar.iso-rews*)

lemma *separated-sum-defl-scalar-map* [*domain-category-map*]: *separated-sum-defl scalar-map*
by (*domain-category-map-rule map-def*: *scalar-map-def iso-rews*: *scalar.iso-rews*)

instance *scalar* :: *bounded-complete*
by (*domain-category-instance map-ID*: *scalar-map-ID*)

instance *scalar* :: *separated-sum*
by (*domain-category-instance map-ID*: *scalar-map-ID*)

lemma *distributive-defl-scalar-map* [*domain-category-map*]: *distributive-defl scalar-map*
by (*domain-category-map-rule map-def*: *scalar-map-def iso-rews*: *scalar.iso-rews*)

instance *scalar* :: *distributive-cpo*
by (*domain-category-instance map-ID*: *scalar-map-ID*)

lemma *locally-finitely-detailed-defl-scalar-map* [*domain-category-map*]:
locally-finitely-detailed-defl scalar-map
by (*domain-category-map-rule map-def*: *scalar-map-def iso-rews*: *scalar.iso-rews*)

instance *scalar* :: *locally-finitely-detailed*
by (*domain-category-instance map-ID*: *scalar-map-ID*)

31.2 Measures: values with nulls

domain ('*v*::*domain*) *with-null* = *Null* | *Known* '*v*

lemma *bounded-complete-defl-with-null-map* [*domain-category-map*]:
bounded-complete-defl v ⟹ *bounded-complete-defl* (*with-null-map*·*v*)
by (*domain-category-map-rule map-def*: *with-null-map-def iso-rews*: *with-null.iso-rews*)

lemma *separated-sum-defl-with-null-map* [*domain-category-map*]:
separated-sum-defl v ⟹ *separated-sum-defl* (*with-null-map*·*v*)
by (*domain-category-map-rule map-def*: *with-null-map-def iso-rews*: *with-null.iso-rews*)

instance *with-null* :: ({*domain*, *bounded-complete*}) *bounded-complete*
by (*domain-category-instance map-ID*: *with-null-map-ID*)

instance *with-null* :: ({*domain*, *separated-sum*}) *separated-sum*
by (*domain-category-instance map-ID*: *with-null-map-ID*)

lemma *distributive-defl-with-null-map* [*domain-category-map*]:
distributive-defl (*v* :: *'a*::{*domain*, *bounded-complete*} → *'a*) ⟹
distributive-defl (*with-null-map*·*v*)
by (*domain-category-map-rule map-def*: *with-null-map-def iso-rews*: *with-null.iso-rews*)

instance *with-null* :: ({*domain*, *distributive-cpo*}) *distributive-cpo*
by (*domain-category-instance map-ID*: *with-null-map-ID*)

lemma *locally-finitely-detailed-defl-with-null-map* [*domain-category-map*]:
locally-finitely-detailed-defl (*v* :: *'a*::{*domain*, *distributive-cpo*, *separated-sum*} →
'a) ⟹
locally-finitely-detailed-defl (*with-null-map*·*v*)
by (*domain-category-map-rule map-def*: *with-null-map-def iso-rews*: *with-null.iso-rews*)

instance *with-null* :: ({*domain*, *separated-sum*, *locally-finitely-detailed*}) *locally-finitely-detailed*
by (*domain-category-instance map-ID*: *with-null-map-ID*)

31.3 Keys: values with NA

The lazy root *Val*·⊥ is "not *NA*, value not observed": it gives the not-null level of detail. Keys from data are maximal and never take that value.

domain (*'a*::*domain*) *with-na* = *NA* | *Val* (**lazy** *'a*)

lemma *bounded-complete-defl-with-na-map* [*domain-category-map*]:
bounded-complete-defl v ⟹ *bounded-complete-defl* (*with-na-map*·*v*)
by (*domain-category-map-rule map-def*: *with-na-map-def iso-rews*: *with-na.iso-rews*)

lemma *filter-sum-defl-with-na-map* [*domain-category-map*]:
filter-sum-defl v ⟹ *filter-sum-defl* (*with-na-map*·*v*)
by (*domain-category-map-rule map-def*: *with-na-map-def iso-rews*: *with-na.iso-rews*)

lemma *separated-sum-defl-with-na-map* [*domain-category-map*]:
deflation v ⟹ *separated-sum-defl* (*with-na-map*·*v*)
by (*domain-category-map-rule map-def*: *with-na-map-def iso-rews*: *with-na.iso-rews*)

instance *with-na* :: ({*domain*, *bounded-complete*}) *bounded-complete*
by (*domain-category-instance map-ID*: *with-na-map-ID*)

instance *with-na* :: (*domain*) *separated-sum*
by (*domain-category-instance map-ID*: *with-na-map-ID*)

lemma *distributive-defl-with-na-map* [*domain-category-map*]:
distributive-defl (*v* :: *'a*::{*domain*, *bounded-complete*} → *'a*) ⟹
distributive-defl (*with-na-map*·*v*)
by (*domain-category-map-rule map-def*: *with-na-map-def iso-rews*: *with-na.iso-rews*)

instance *with-na* :: ({*domain*, *distributive-cpo*}) *distributive-cpo*
by (*domain-category-instance map-ID*: *with-na-map-ID*)

lemma *locally-finitely-detailed-defl-with-na-map* [*domain-category-map*]:
  locally-finitely-detailed-defl v $\Longrightarrow$ *locally-finitely-detailed-defl* (*with-na-map*·*v*)
 by (*domain-category-map-rule map-def*: *with-na-map-def iso-rews*: *with-na.iso-rews*)

instance *with-na* :: ({*domain*, *locally-finitely-detailed*}) *locally-finitely-detailed*
  by (*domain-category-instance map-ID*: *with-na-map-ID*)

31.4 Data

A datum is *NA*, *Val*·*Null*, or a known value.

type-synonym $'v$ *datum* = $'v$ *with-null with-na*

32 Coercions between the levels

Binding through *NA*: *NA* propagates, a value is passed on. A join through a relation that may be inapplicable is a bind.

fixrec *na-bind* :: ($'a$::*domain* $\rightarrow$ $'b$::*domain with-na*) $\rightarrow$ $'a$ *with-na* $\rightarrow$ $'b$ *with-na*
where
  na-bind·*f*·*NA* = *NA*
| *na-bind*·*f*·(*Val*·*x*) = *f*·*x*

lemma *na-bind-strict* [*simp*]: *na-bind*·*f*·$\bot$ = $\bot$
  by *fixrec-simp*

Measures: *NA* becomes $\bot$ (no entry).

fixrec *from-na* :: $'a$::*domain with-na* $\rightarrow$ $'a$ **where**
  from-na·*NA* = $\bot$
| *from-na*·(*Val*·*x*) = *x*

lemma *from-na-strict* [*simp*]: *from-na*·$\bot$ = $\bot$
  by *fixrec-simp*

Measures embed back strictly: *na-emb* and *from-na* form an embedding-projection pair. Their composition *squash* is the projection that forgets *NA* (and the not-null root, which data never takes).

definition *na-emb* :: $'a$::*domain* $\rightarrow$ $'a$ *with-na* **where**
  na-emb = *strictify*·*Val*

lemma *na-emb-strict* [*simp*]: *na-emb*·$\bot$ = $\bot$
  by (*simp add*: *na-emb-def*)

lemma *na-emb-defined* [*simp*]: *x* $\neq$ $\bot$ $\Longrightarrow$ *na-emb*·*x* = *Val*·*x*
  by (*simp add*: *na-emb-def strictify-conv-if*)

lemma *ep-pair-na*: *ep-pair na-emb from-na*

proof
  fix $x :: 'a$ **show** *from-na*·(*na-emb*·x) = x **by** (*cases* $x = \bot$) *simp-all*
next
  fix $y :: 'a$ *with-na* **show** *na-emb*·(*from-na*·y) ⊑ y
    by (*cases* y) (*simp-all add*: *na-emb-def strictify-conv-if*)
qed

lemma *from-na-na-emb* [*simp*]: *from-na*·(*na-emb*·x) = x
  by (*rule ep-pair.e-inverse*[*OF ep-pair-na*])

definition *squash* :: $'a$::*domain with-na* → $'a$ *with-na* **where**
  squash = *na-emb oo from-na*

lemma *squash-simps* [*simp*]: *squash*·⊥ = ⊥ *squash*·*NA* = ⊥ *squash*·(*Val*·x) = *na-emb*·x
  by (*simp-all add*: *squash-def*)

lemma *squash-na-emb* [*simp*]: *squash*·(*na-emb*·x) = *na-emb*·x
  unfolding *squash-def* **by** (*simp add*: *ep-pair.e-inverse*[*OF ep-pair-na*])

lemma *deflation-squash*: *deflation squash*
  unfolding *squash-def* **by** (*rule ep-pair.deflation-e-p*[*OF ep-pair-na*])

lemma *from-na-squash* [*simp*]: *from-na*·(*squash*·x) = *from-na*·x
  unfolding *squash-def* **by** (*simp add*: *ep-pair.e-inverse*[*OF ep-pair-na*])

Keys: nulls become *NA*.

fixrec *null-na* :: $'v$::*domain with-null* → $'v$ *with-na* **where**
  null-na·*Null* = *NA*
| $v \neq \bot \Longrightarrow$ *null-na*·(*Known*·v) = *Val*·v

lemma *null-na-strict* [*simp*]: *null-na*·⊥ = ⊥
  by *fixrec-simp*

definition *to-key* :: $'v$::*domain datum* → $'v$ *with-na* **where**
  to-key = *na-bind*·*null-na*

Measures project to plain values, *Null* becoming ⊥. *Known* is strict, so it is the embedding itself.

fixrec *from-null* :: $'v$::*domain with-null* → $'v$ **where**
  from-null·*Null* = ⊥
| $v \neq \bot \Longrightarrow$ *from-null*·(*Known*·v) = v

lemma *from-null-strict* [*simp*]: *from-null*·⊥ = ⊥
  by *fixrec-simp*

lemma *ep-pair-null*: *ep-pair* (*Known* :: $'v$::*domain* → $'v$ *with-null*) *from-null*
proof
  fix $x :: 'v$ **show** *from-null*·(*Known*·x) = x **by** (*cases* $x = \bot$) *simp-all*

next
  fix y :: *'v with-null* **show** *Known·(from-null·y)* $\sqsubseteq$ y **by** (*cases y*) *simp-all*
qed

lemma *from-null-Known* [*simp*]: *from-null·(Known·x)* = x
  by (*rule ep-pair.e-inverse*[*OF ep-pair-null*])

The plain value of a datum: every missing value becomes $\bot$.

definition *plain* :: *'v*::*domain datum* → *'v* **where**
  plain = *from-null oo from-na*

lemma *plain-simps* [*simp*]:
  plain·⊥ = ⊥ *plain·NA* = ⊥ *plain·(Val·x)* = *from-null·x*
  by (*simp-all add*: *plain-def*)

lemma *plain-na-emb* [*simp*]: *plain·(na-emb·x)* = *from-null·x*
  by (*simp add*: *plain-def*)

33 Record-level operators

Operators are lifted layer by layer, each layer's missing value infectious. The outer layer is checked first, so *NA* wins over *Null*; NA-blindness forces that choice.

fixrec *na-lift2* :: (*'a*::*domain* → *'b*::*domain* → *'c*::*domain*) → *'a with-na* → *'b with-na* → *'c with-na*
  where
  na-lift2·f·NA·NA = *NA*
| *na-lift2·f·NA·(Val·y)* = *NA*
| *na-lift2·f·(Val·x)·NA* = *NA*
| *na-lift2·f·(Val·x)·(Val·y)* = *na-emb·(f·x·y)*

fixrec *null-lift2* ::
  (*'a*::*domain* → *'b*::*domain* → *'c*::*domain*) → *'a with-null* → *'b with-null* → *'c with-null*
  where
  null-lift2·f·Null·Null = *Null*
| $y \neq \bot \Longrightarrow$ *null-lift2·f·Null·(Known·y)* = *Null*
| $x \neq \bot \Longrightarrow$ *null-lift2·f·(Known·x)·Null* = *Null*
| $x \neq \bot \Longrightarrow y \neq \bot \Longrightarrow$ *null-lift2·f·(Known·x)·(Known·y)* = *Known·(f·x·y)*

fixrec *na-lift1* :: (*'a*::*domain* → *'b*::*domain*) → *'a with-na* → *'b with-na* **where**
  na-lift1·f·NA = *NA*
| *na-lift1·f·(Val·x)* = *na-emb·(f·x)*

fixrec *null-lift1* :: (*'a*::*domain* → *'b*::*domain*) → *'a with-null* → *'b with-null* **where**
  null-lift1·f·Null = *Null*
| $x \neq \bot \Longrightarrow$ *null-lift1·f·(Known·x)* = *Known·(f·x)*

abbreviation *dlift1* :: (*'a*::*domain* → *'b*::*domain*) → *'a datum* → *'b datum* **where**
  dlift1 ≡ *na-lift1 oo null-lift1*

lemma *na-lift1-strict* [*simp*]: *na-lift1*·*f*·⊥ = ⊥
  by *fixrec-simp*

lemma *null-lift1-strict* [*simp*]: *null-lift1*·*f*·⊥ = ⊥
  by *fixrec-simp*

abbreviation *dlift2* :: (*'a*::*domain* → *'b*::*domain* → *'c*::*domain*) → *'a datum* → *'b datum* → *'c datum* **where**
  dlift2 ≡ *na-lift2 oo null-lift2*

lemma *na-lift2-strict* [*simp*]: *na-lift2*·*f*·⊥·*y* = ⊥ *na-lift2*·*f*·*x*·⊥ = ⊥
  by *fixrec-simp* (*cases x*; *fixrec-simp*)

lemma *null-lift2-strict* [*simp*]: *null-lift2*·*f*·⊥·*y* = ⊥ *null-lift2*·*f*·*x*·⊥ = ⊥
  by *fixrec-simp* (*cases x*; *fixrec-simp*)

A record-level function is NA-blind when swapping *NA* for ⊥ in its inputs does the same to its output.

definition *na-blind1* :: (*'a*::*domain with-na* → *'b*::*domain with-na*) ⇒ *bool* **where**
  na-blind1 g ⟷ (∀ *x*. *g*·(*squash*·*x*) = *squash*·(*g*·*x*))

definition *na-blind2* ::
  (*'a*::*domain with-na* → *'b*::*domain with-na* → *'c*::*domain with-na*) ⇒ *bool* **where**
  na-blind2 g ⟷ (∀ *x y*. *g*·(*squash*·*x*)·(*squash*·*y*) = *squash*·(*g*·*x*·*y*))

lemma *na-blind2-na-lift2*:
  assumes *l*: ⋀*y*. *f*·⊥·*y* = ⊥ **and** *r*: ⋀*x*. *f*·*x*·⊥ = ⊥
  shows *na-blind2* (*na-lift2*·*f*)
  unfolding *na-blind2-def*
proof (*intro allI*)
  fix *x y* **show** *na-lift2*·*f*·(*squash*·*x*)·(*squash*·*y*) = *squash*·(*na-lift2*·*f*·*x*·*y*)
    by (*cases x*; *cases y*) (*auto simp*: *l r strictify-conv-if na-emb-def*)
qed

The inner layer is strict, so every lifted record-level operator is NA-blind.

lemma *na-blind2-dlift2*: *na-blind2* (*dlift2*·*f*)
  by (*simp add*: *na-blind2-na-lift2*)

On plain values, a lifted strict operator is the operator itself.

lemma *plain-dlift1*:
  assumes *f*·⊥ = ⊥ **shows** *plain*·(*na-lift1*·(*null-lift1*·*f*)·*x*) = *f*·(*plain*·*x*)
proof (*cases x*)
  case (*Val a*) **then show** *?thesis* **using** *assms* **by** (*cases a*) *simp-all*
qed (*simp-all add*: *assms*)

lemma *plain-dlift2*:

assumes $\bigwedge y.\ f{\cdot}\bot{\cdot}y = \bot\ \bigwedge x.\ f{\cdot}x{\cdot}\bot = \bot$
shows *plain*$\cdot$(*na-lift2*$\cdot$(*null-lift2*$\cdot f$)$\cdot x\cdot y$) = $f\cdot$(*plain*$\cdot x$)$\cdot$(*plain*$\cdot y$)
proof (*cases x*)
case *x*: (*Val a*) **show** *?thesis*
proof (*cases y*)
case *y*: (*Val b*) **show** *?thesis* **unfolding** *x y* **using** *assms* **by** (*cases a*; *cases b*) *simp-all*
qed (*simp-all add*: *x assms*)
qed (*cases y*; *simp add*: *assms*)+

33.1 Equality

Equality of flat values, strict in both arguments. *tr* is *tr*, so this covers both fields of *scalar*.

definition *lift-eq* :: *'a lift* → *'a lift* → *tr* **where**
lift-eq = (*FLIFT a*. *FLIFT b*. *Def* (*a* = *b*))

lemma *lift-eq-Def* [*simp*]: *lift-eq*$\cdot$(*Def a*)$\cdot$(*Def b*) = *Def* (*a* = *b*)
by (*simp add*: *lift-eq-def*)

lemma *lift-eq-strict* [*simp*]: *lift-eq*$\cdot\bot\cdot y = \bot$ *lift-eq*$\cdot x\cdot\bot = \bot$
by (*simp-all add*: *lift-eq-def*) (*cases x*; *simp*)

lemma *lift-eq-defined*: $x \neq \bot \Longrightarrow y \neq \bot \Longrightarrow$ *lift-eq*$\cdot x\cdot y$ = *Def* (*x* = *y*)
by (*cases x*; *cases y*) *simp-all*

Scalars of different types are unequal; under static typing that case does not arise.

fixrec *scalar-eq* :: *scalar* → *scalar* → *tr* **where**
$a \neq \bot \Longrightarrow b \neq \bot \Longrightarrow$ *scalar-eq*$\cdot$(*SBool*$\cdot a$)$\cdot$(*SBool*$\cdot b$) = *lift-eq*$\cdot a\cdot b$
| $m \neq \bot \Longrightarrow n \neq \bot \Longrightarrow$ *scalar-eq*$\cdot$(*SInt*$\cdot m$)$\cdot$(*SInt*$\cdot n$) = *lift-eq*$\cdot m\cdot n$
| $a \neq \bot \Longrightarrow n \neq \bot \Longrightarrow$ *scalar-eq*$\cdot$(*SBool*$\cdot a$)$\cdot$(*SInt*$\cdot n$) = *FF*
| $m \neq \bot \Longrightarrow b \neq \bot \Longrightarrow$ *scalar-eq*$\cdot$(*SInt*$\cdot m$)$\cdot$(*SBool*$\cdot b$) = *FF*

lemma *scalar-eq-strict* [*simp*]: *scalar-eq*$\cdot\bot\cdot y = \bot$ *scalar-eq*$\cdot x\cdot\bot = \bot$
by *fixrec-simp* (*cases x*; *fixrec-simp*)

lemma *scalar-eq-defined*: $x \neq \bot \Longrightarrow y \neq \bot \Longrightarrow$ *scalar-eq*$\cdot x\cdot y$ = *Def* (*x* = *y*)
by (*cases x*; *cases y*) (*simp-all add*: *lift-eq-defined FF-def*)

Record-level equality is the lifted scalar equality, hence NA-blind.

abbreviation *datum-eq* :: *scalar datum* → *scalar datum* → *tr datum* **where**
datum-eq ≡ *dlift2*$\cdot$*scalar-eq*

33.2 Truth

The connectives are strict, so lifted to data they make *NA* and *Null* infectious (*NOT NA* = *NA*). *truth-d* projects a datum to *tr*, sending every missing

value and every non-boolean to $\bot$; it is a homomorphism for the connectives. A slicer condition holds where *truth-d* is *TT*.

definition *tr-and* :: $tr \to tr \to tr$ **where**
  $tr\text{-}and = (FLIFT\ a.\ FLIFT\ b.\ Def\ (a \wedge b))$

definition *tr-or* :: $tr \to tr \to tr$ **where**
  $tr\text{-}or = (FLIFT\ a.\ FLIFT\ b.\ Def\ (a \vee b))$

lemma *tr-and-simps* [*simp*]:
  $tr\text{-}and{\cdot}\bot{\cdot}y = \bot\ tr\text{-}and{\cdot}x{\cdot}\bot = \bot\ tr\text{-}and{\cdot}(Def\ a){\cdot}(Def\ b) = Def\ (a \wedge b)$
  by (*simp-all add*: *tr-and-def*) (*cases x rule*: *lift.exhaust*; *simp*)

lemma *tr-or-simps* [*simp*]:
  $tr\text{-}or{\cdot}\bot{\cdot}y = \bot\ tr\text{-}or{\cdot}x{\cdot}\bot = \bot\ tr\text{-}or{\cdot}(Def\ a){\cdot}(Def\ b) = Def\ (a \vee b)$
  by (*simp-all add*: *tr-or-def*) (*cases x rule*: *lift.exhaust*; *simp*)

fixrec *sbool* :: *scalar* $\to$ *tr* **where**
  $b \neq \bot \Longrightarrow sbool{\cdot}(SBool{\cdot}b) = b$

lemma *sbool-strict* [*simp*]: $sbool{\cdot}\bot = \bot$
  by *fixrec-simp*

fixrec *snot* :: *scalar* $\to$ *scalar* **where**
  $a \neq \bot \Longrightarrow snot{\cdot}(SBool{\cdot}a) = SBool{\cdot}(neg{\cdot}a)$

fixrec *sand* :: *scalar* $\to$ *scalar* $\to$ *scalar* **where**
  $a \neq \bot \Longrightarrow b \neq \bot \Longrightarrow sand{\cdot}(SBool{\cdot}a){\cdot}(SBool{\cdot}b) = SBool{\cdot}(tr\text{-}and{\cdot}a{\cdot}b)$

fixrec *sor* :: *scalar* $\to$ *scalar* $\to$ *scalar* **where**
  $a \neq \bot \Longrightarrow b \neq \bot \Longrightarrow sor{\cdot}(SBool{\cdot}a){\cdot}(SBool{\cdot}b) = SBool{\cdot}(tr\text{-}or{\cdot}a{\cdot}b)$

Unconditional constructor equations; ill-typed applications give $\bot$.

lemma *sbool-simps* [*simp*]: $sbool{\cdot}(SBool{\cdot}b) = b\ sbool{\cdot}(SInt{\cdot}n) = \bot$
  by (*cases* $b = \bot$; *simp*) (*cases* $n = \bot$; *fixrec-simp*)

lemma *snot-simps* [*simp*]: $snot{\cdot}(SBool{\cdot}a) = SBool{\cdot}(neg{\cdot}a)\ snot{\cdot}(SInt{\cdot}n) = \bot$
  by (*cases* $a = \bot$; *simp*; *fixrec-simp*) (*cases* $n = \bot$; *fixrec-simp*)

lemma *sand-simps* [*simp*]:
  $sand{\cdot}(SBool{\cdot}a){\cdot}(SBool{\cdot}b) = SBool{\cdot}(tr\text{-}and{\cdot}a{\cdot}b)$
  $sand{\cdot}(SInt{\cdot}m){\cdot}y = \bot\ sand{\cdot}x{\cdot}(SInt{\cdot}n) = \bot$
    apply (*cases* $a = \bot$; *cases* $b = \bot$; *simp*; *fixrec-simp*)
   apply (*cases* $m = \bot$; *fixrec-simp*)
  apply (*cases x*; *cases* $n = \bot$; *fixrec-simp*)
  done

lemma *sor-simps* [*simp*]:
  $sor{\cdot}(SBool{\cdot}a){\cdot}(SBool{\cdot}b) = SBool{\cdot}(tr\text{-}or{\cdot}a{\cdot}b)$
  $sor{\cdot}(SInt{\cdot}m){\cdot}y = \bot\ sor{\cdot}x{\cdot}(SInt{\cdot}n) = \bot$

```
   apply (cases a = ⊥; cases b = ⊥; simp; fixrec-simp)
  apply (cases m = ⊥; fixrec-simp)
 apply (cases x; cases n = ⊥; fixrec-simp)
 done

definition truth :: scalar with-na → tr where
 truth = sbool oo from-na

definition truth-d :: scalar datum → tr where
 truth-d = sbool oo plain

lemma scalar-connectives-strict [simp]:
 snot·⊥ = ⊥ sand·⊥·y = ⊥ sand·x·⊥ = ⊥ sor·⊥·y = ⊥ sor·x·⊥ = ⊥
    apply fixrec-simp
   apply fixrec-simp
  apply (cases x; fixrec-simp)
 apply fixrec-simp
 apply (cases x; fixrec-simp)
 done

lemma sbool-connectives:
 sbool·(snot·s) = neg·(sbool·s)
 sbool·(sand·s·t) = tr-and·(sbool·s)·(sbool·t)
 sbool·(sor·s·t) = tr-or·(sbool·s)·(sbool·t)
 by (cases s; simp; cases t; simp)+

lemma truth-d-not: truth-d·(dlift1·snot·x) = neg·(truth-d·x)
 by (simp add: truth-d-def plain-dlift1 sbool-connectives)

lemma truth-d-and: truth-d·(dlift2·sand·x·y) = tr-and·(truth-d·x)·(truth-d·y)
 by (simp add: truth-d-def plain-dlift2 sbool-connectives)

lemma truth-d-or: truth-d·(dlift2·sor·x·y) = tr-or·(truth-d·x)·(truth-d·y)
 by (simp add: truth-d-def plain-dlift2 sbool-connectives)
```

34 The not-null level of keys

The atoms of *'a with-na* are *NA* and *Val·⊥*, so its discriminator is the projection *not-null*, which observes only whether a key is *NA*. It is a finitary semi-factor, i.e. a level of detail, and *is-na* factors through it.

```
fixrec is-na :: 'a::domain with-na → tr where
 is-na·NA = TT
| is-na·(Val·x) = FF

lemma is-na-strict [simp]: is-na·⊥ = ⊥
 by fixrec-simp

fixrec not-null :: 'a::domain with-na → 'a with-na where
```

```
  not-null·NA = NA
| not-null·(Val·x) = Val·⊥

lemma not-null-strict [simp]: not-null·⊥ = ⊥
  by fixrec-simp

lemma atom-NA: atom NA
  by (rule atomI) (simp, case-tac z, simp-all)

lemma atom-Val-bottom: atom (Val·⊥)
  by (rule atomI) (simp, case-tac z, simp-all)

lemma discriminator-with-na:
  discriminator·x = not-null·(x :: 'a::{domain, bounded-complete} with-na)
proof (cases x)
  case bottom then show ?thesis by simp
next
  case NA then show ?thesis by (simp add: discriminator-eq[OF atom-NA be-
low-refl])
next
  case (Val v)
  have Val·⊥ ⊑ x unfolding Val by (rule monofun-cfun-arg) simp
 then show ?thesis unfolding Val by (simp add: discriminator-eq[OF atom-Val-bottom])
qed

lemma not-null-level:
  semi-factor-defl (not-null :: 'a::{domain, distributive-cpo} with-na → 'a with-na)
  finitary (not-null :: 'a::{domain, distributive-cpo} with-na → 'a with-na)
  using discriminator-semi-factor discriminator-finitary
  by (simp-all add: cfun-eqI[OF discriminator-with-na, symmetric])

lemma is-na-not-null: is-na·(not-null·x) = is-na·x
  by (cases x) simp-all

end
theory Content-Law
  imports Frame HOLCF−Library.List-Cpo
begin

default-sort type
```

35 The content law

An expression reads its data through a finite list of *generalized tables*. An entry's *selector* (a, ϱ) picks the fiber: the rows of the frame whose key agrees with the key at hand at $a \sqcup \varrho \sqcap L$, for the frame LOD L, among the rows passing the ambient filter and the slice coarsened to ϱ. Its guard cuts the fiber down, and its *view* (c, κ) reads the rest: the content maps each row's

key at κ to its section at c, which the key determines. The *content law* says that the semantics at a key is a function of these contents.

35.1 Observations

Observations are continuous maps from rows into observed values. A selection reads as a continuous truth value; selections pull back along continuous maps.

definition *sel-chi* :: *'r::pcpo sel* ⇒ *'r* → *tr* **where**
sel-chi P = (Λ *x*. *if selects P x then TT else* ⊥)

lemma *cont-sel-chi*: *cont* (λ*x*. *if selects P x then TT else* (⊥::*tr*))
proof (*rule contI2*)
show *m*: *monofun* (λ*x*. *if selects P x then TT else* (⊥::*tr*))
by (*rule monofunI*) (*auto dest*: *selects-up*)
fix *Y* :: *nat* ⇒ *'a* **assume** *Y*: *chain Y*
have *ch*: *chain* (λ*i*. *if selects P* (*Y i*) *then TT else* (⊥::*tr*))
by (*rule ch2ch-monofun*[*OF m Y*])
show (*if selects P* (⨆ *i*. *Y i*) *then TT else* ⊥) ⊑ (⨆ *i*. *if selects P* (*Y i*) *then TT else* (⊥::*tr*))
proof (*cases* ∃ *i*. *selects P* (*Y i*))
case *True*
then obtain *i* **where** *i*: *selects P* (*Y i*) **by** *blast*
have (*if selects P* (*Y i*) *then TT else* (⊥::*tr*)) ⊑ (⨆ *i*. *if selects P* (*Y i*) *then TT else* (⊥::*tr*))
by (*rule is-ub-thelub*[*OF ch*])
then show *?thesis* **using** *i* **by** *simp*
next
case *False*
then have ¬ *selects P* (⨆ *i*. *Y i*) **by** (*intro admD*[*OF selects-open Y*]) *blast*
then show *?thesis* **by** *simp*
qed
qed

lemma *sel-chi-apply*: *sel-chi P*·*x* = (*if selects P x then TT else* ⊥)
unfolding *sel-chi-def* **by** (*rule beta-cfun*[*OF cont-sel-chi*])

lift-definition *sel-vimage* :: (*'a::pcpo* → *'b::pcpo*) ⇒ *'b sel* ⇒ *'a sel* **is** λ*g P x*. *P* (*g*·*x*)
proof (*intro conjI allI impI*)
fix *g* :: *'a* → *'b* **and** *P* :: *'b* ⇒ *bool* **and** *x y* :: *'a*
assume *P*: (∀ *x y*. *P x* ⟶ *x* ⊑ *y* ⟶ *P y*) ∧ *adm* (λ*x*. ¬ *P x*)
show *P* (*g*·*y*) **if** *P* (*g*·*x*) *x* ⊑ *y* **using** *P that monofun-cfun-arg* **by** *blast*
next
fix *g* :: *'a* → *'b* **and** *P* :: *'b* ⇒ *bool*
assume *P*: (∀ *x y*. *P x* ⟶ *x* ⊑ *y* ⟶ *P y*) ∧ *adm* (λ*x*. ¬ *P x*)
show *adm* (λ*x*. ¬ *P* (*g*·*x*)) **by** (*rule adm-subst*[*OF cont-Rep-cfun2*]) (*use P* **in** *blast*)

qed

lemma *selects-sel-vimage* [*simp*]: *selects* (*sel-vimage g P*) *x* ⟷ *selects P* (*g*·*x*)
by (*simp add*: *sel-vimage.rep-eq*)

The truth value TT is maximal and compact, so being TT is a selection.

lift-definition *sel-TT* :: *tr sel* **is** λ*t*. *t* = *TT*
proof (*intro conjI allI impI*)
show *y* = *TT* **if** *x* = *TT x* ⊑ *y* **for** *x y* :: *tr* **using** *that* **by** *simp*
show *adm* (λ*t*. *t* ≠ *TT*) **by** (*simp add*: *compact-TT eq-commute*[*of* - *TT*])
qed

lemma *selects-sel-TT* [*simp*]: *selects sel-TT t* ⟷ *t* = *TT*
by (*simp add*: *sel-TT.rep-eq*)

Guards are continuous truth-valued functions of the parts of a declared view: the section of the row and the part of its key that they read. On rows they are read through their view, so they read nothing else.

record (′*r*::{*domain*, *distributive-cpo*}, ′*k*::{*domain*, *distributive-cpo*}, ′*b*::*cpo*) *vfn* =
vfun :: ′*r* × ′*k* → ′*b*
oview :: (′*r*, ′*k*) *view*

hide-const (**open**) *vfn.truncate*

type-synonym (′*r*, ′*k*) *guard* = (′*r*, ′*k*, *tr*) *vfn*

abbreviation *ocols* :: (′*r*::{*domain*, *distributive-cpo*}, ′*k*::{*domain*, *distributive-cpo*}, ′*b*::*cpo*) *vfn* ⇒ ′*r semi-factor* **where**
ocols f ≡ *fst* (*oview f*)

abbreviation *okeys* :: (′*r*::{*domain*, *distributive-cpo*}, ′*k*::{*domain*, *distributive-cpo*}, ′*b*::*cpo*) *vfn* ⇒ ′*k semi-factor* **where**
okeys f ≡ *snd* (*oview f*)

Reading the section at a view.

definition *vsection* :: (′*r*::{*domain*, *distributive-cpo*}, ′*k*::{*domain*, *distributive-cpo*}) *view* ⇒ (′*r*, ′*k*, ′*r*) *vfn* **where**
vsection v = (|*vfun* = *cfst*, *oview* = *v*|)

lemma *oview-vsection* [*simp*]: *oview* (*vsection v*) = *v*
by (*simp add*: *vsection-def*)

The guard every row passes, and the conjunction of guards, on the joined view.

definition *gtrue* :: (′*r*::{*domain*, *distributive-cpo*}, ′*k*::{*domain*, *distributive-cpo*}) *guard* **where**
gtrue = (|*vfun* = Λ -. *TT*, *oview* = ⊥|)

lemma *oview-gtrue* [*simp*]: *oview gtrue* = ⊥
by (*simp add*: *gtrue-def*)

definition *gand* :: ('*r*::{*domain*, *distributive-cpo*}, '*k*::{*domain*, *distributive-cpo*})
guard ⇒ ('*r*, '*k*) *guard* ⇒ ('*r*, '*k*) *guard* **where**
gand γ δ = (|*vfun* = Λ *p*. (*vfun* γ·(*pproj* (*oview* γ)·*p*)) *andalso* (*vfun* δ·(*pproj* (*oview* δ)·*p*)),
oview = *oview* γ ⊔ *oview* δ|)

lemma *gand-view* [*simp*]: *oview* (*gand* γ δ) = *oview* γ ⊔ *oview* δ
by (*simp add*: *gand-def*)

An entry: a selector, picking the fiber; a guard, cutting it down; and a view, whose key part keys the content and whose section is recorded at each key.

record '*k selector* =
lower-part :: '*k semi-factor*
upper-part :: '*k semi-factor*

hide-const (**open**) *selector.truncate*

abbreviation *sel* :: '*k semi-factor* ⇒ '*k semi-factor* ⇒ '*k selector* **where**
sel a ϱ ≡ (|*lower-part* = *a*, *upper-part* = ϱ|)

record ('*k*::{*domain*, *distributive-cpo*}, '*r*::{*domain*, *distributive-cpo*}) *entry* =
esel :: '*k selector*
eguard :: ('*r*, '*k*) *guard*
eview :: ('*r*, '*k*) *view*

hide-const (**open**) *entry.truncate*

abbreviation *elower* :: ('*k*::{*domain*, *distributive-cpo*}, '*r*::{*domain*, *distributive-cpo*})
entry ⇒ '*k semi-factor* **where**
elower e ≡ *lower-part* (*esel e*)

abbreviation *eupper* :: ('*k*::{*domain*, *distributive-cpo*}, '*r*::{*domain*, *distributive-cpo*})
entry ⇒ '*k semi-factor* **where**
eupper e ≡ *upper-part* (*esel e*)

abbreviation *ekeys* :: ('*k*::{*domain*, *distributive-cpo*}, '*r*::{*domain*, *distributive-cpo*})
entry ⇒ '*k semi-factor* **where**
ekeys e ≡ *snd* (*eview e*)

abbreviation *ecols* :: ('*k*::{*domain*, *distributive-cpo*}, '*r*::{*domain*, *distributive-cpo*})
entry ⇒ '*r semi-factor* **where**
ecols e ≡ *fst* (*eview e*)

abbreviation *mk-entry* ::
'*k selector* ⇒ ('*r*, '*k*) *guard* ⇒ ('*r*, '*k*) *view* ⇒ ('*k*::{*domain*, *distributive-cpo*},

$'r$::{*domain*, *distributive-cpo*}) *entry* **where**
mk-entry S γ v ≡ (|*esel* = *S*, *eguard* = *γ*, *eview* = *v*|)

record (*'k*::*domain*, *'f*::*type*, *'r*::{*bifinite*, *distributive-cpo*}, *'v*::*type*) *expr* =
emind :: *'k semi-factor*
edims :: (*'k*, *'r*) *entry list*
esem :: *'r lower-pd* ⇒ (*'k semi-factor*, *'f*, *'r*) *frame* ⇒ *'k* ⇀ *'v*
csem :: (*'k* ⇒ *'r option*) *list* ⇒ *'v option*

hide-const (**open**) *expr.truncate*

Contents are maps from keys to sections. A graph, a single-valued set of pairs, gives one; the value it records at a key is *cval*.

definition *cval* :: (*'k* × *'b*) *set* ⇒ *'k* ⇒ *'b* **where**
cval X q = (*THE v*. (*q*, *v*) ∈ *X*)

lemma *cval-singleton* [*simp*]: *cval* {(*q*, *v*)} *q* = *v*
by (*simp add*: *cval-def*)

lemma *cval-eq*: *single-valued X* ⟹ (*q*, *v*) ∈ *X* ⟹ *cval X q* = *v*
unfolding *cval-def* **by** (*rule the-equality*) (*auto dest*: *single-valuedD*)

definition *gmap* :: (*'k* × *'b*) *set* ⇒ *'k* ⇒ *'b option* **where**
gmap X q = (*if q* ∈ *fst* ‘ *X then Some* (*cval X q*) *else None*)

definition *graph* :: (*'k* ⇒ *'b option*) ⇒ (*'k* × *'b*) *set* **where**
graph m = {(*q*, *s*). *m q* = *Some s*}

lemma *dom-gmap* [*simp*]: *dom* (*gmap X*) = *fst* ‘ *X*
by (*auto simp*: *gmap-def dom-def*)

lemma *gmap-empty-iff* [*simp*]: *gmap X* = *Map.empty* ⟷ *X* = {}
unfolding *dom-eq-empty-conv*[*symmetric*] *dom-gmap* **by** *simp*

lemma *gmap-apply*: *single-valued X* ⟹ (*q*, *s*) ∈ *X* ⟹ *gmap X q* = *Some s*
unfolding *gmap-def* **by** (*simp add*: *cval-eq rev-image-eqI*)

lemma *gmap-singleton* [*simp*]: *gmap* {(*q*, *s*)} = [*q* ↦ *s*]
by (*auto simp*: *gmap-def fun-eq-iff*)

lemma *single-valued-graph*: *single-valued* (*graph m*)
by (*auto simp*: *single-valued-def graph-def*)

lemma *graph-gmap*:
assumes *X*: *single-valued X*
shows *graph* (*gmap X*) = *X*
proof (*rule set-eqI*, *rule iffI*)
fix *x* **assume** *x* ∈ *graph* (*gmap X*)
then obtain *q s* **where** *x*: *x* = (*q*, *s*) **and** *q*: *gmap X q* = *Some s* **by** (*auto simp*:

graph-def)
then obtain *s'* **where** *s'*: (*q*, *s'*) ∈ *X* **by** (*force simp*: *gmap-def split*: *if-splits*)
then show *x* ∈ *X* **using** *q gmap-apply*[*OF X s'*] **by** (*simp add*: *x*)
next
fix *x* **assume** *x* ∈ *X*
then show *x* ∈ *graph* (*gmap X*) **using** *gmap-apply*[*OF X*] **by** (*cases x*) (*simp add*: *graph-def*)
qed

lemma *gmap-graph* [*simp*]: *gmap* (*graph m*) = *m*
proof
fix *q* **show** *gmap* (*graph m*) *q* = *m q*
proof (*cases m q*)
case *None*
then have *q* ∉ *fst* ' *graph m* **by** (*force simp*: *graph-def*)
then show *?thesis* **using** *None* **by** (*simp add*: *gmap-def*)
next
case (*Some s*)
then have (*q*, *s*) ∈ *graph m* **by** (*simp add*: *graph-def*)
from *gmap-apply*[*OF single-valued-graph this*] *Some* **show** *?thesis* **by** *simp*
qed
qed

35.2 Groups and contents

context *dataspace*
begin

Reading a guard or a section on rows.

definition *ofun* :: ('*r*, '*k*, '*b*::*cpo*) *vfn* ⇒ '*r* → '*b* **where**
ofun f = *vfun f oo vproj* (*oview f*)

lemma *ofun-apply*: *ofun f*·*y* = *vfun f*·(*vpart* (*oview f*) *y*)
by (*simp add*: *ofun-def*)

lemma *ofun-vsection* [*simp*]: *ofun* (*vsection v*)·*y* = *y* ⌊ *fst v*
by (*simp add*: *ofun-apply vsection-def vpart-def*)

lemma *ofun-gtrue* [*simp*]: *ofun gtrue*·*y* = *TT*
by (*simp add*: *ofun-apply gtrue-def*)

lemma *gand-apply*: *ofun* (*gand γ δ*)·*y* = (*ofun γ*·*y andalso ofun δ*·*y*)
by (*simp add*: *ofun-apply gand-def vpart-pproj lattice-join-ge1 lattice-join-ge2*)

lemma *gand-TT* [*simp*]: *ofun* (*gand γ δ*)·*y* = *TT* ⟷ *ofun γ*·*y* = *TT* ∧ *ofun δ*·*y* = *TT*
unfolding *gand-apply* **by** (*cases ofun γ*·*y rule*: *trE*) *simp-all*

The rows a test passes are a selection.

definition *gsel* :: (*'r*, *'k*) *guard* ⇒ *'r sel* **where**
gsel γ = *sel-vimage* (*ofun γ*) *sel-TT*

lemma *selects-gsel* [*simp*]: *selects* (*gsel γ*) *y* ⟷ *ofun γ*·*y* = *TT*
by (*simp add*: *gsel-def*)

Whether a row passes a slice, as a guard.

definition *guard-holds* :: *'f* ⇒ (*'r*, *'k*) *guard* **where**
guard-holds X = (|*vfun* = *sel-chi* (*sopen X*) *oo csnd*, *oview* = (⊥, *slod X*)|)

lemma *guard-holds-apply*: *ofun* (*guard-holds X*)·*y* = (*if holds X* (*rkey*·*y*) *then TT else* ⊥)
by (*simp add*: *ofun-apply guard-holds-def sel-chi-apply holds-def sopen-lod*)

lemma *guard-holds-view* [*simp*]: *oview* (*guard-holds X*) = (⊥, *slod X*)
by (*simp add*: *guard-holds-def*)

lemma *guard-holds-TT* [*simp*]: *ofun* (*guard-holds X*)·*y* = *TT* ⟷ *holds X* (*rkey*·*y*)
by (*simp add*: *guard-holds-apply*)

35.3 Supports of guards

A guard or section is *supported* when its view is keyed: within every conforming dataset its key part determines it on rows.

definition *supported* :: (*'r*, *'k*, *'b*::*cpo*) *vfn* ⇒ *bool* **where**
supported f ⟷ *keyed-view* (*oview f*)

lemma *supported-fun-support*: *supported f* ⟹ *fun-support* (*oview f*) (*ofun f*)
proof −
assume *f*: *supported f*
have *determines UNIV* (*oview f*) (*vfun f oo vproj* (*oview f*))
by (*rule determines-comp*) (*rule vfd-refl*[*OF below-refl*])
moreover have *universally* (λ*P*. *determines P* (*vkey* (*oview f*)) (*vfun f oo vproj* (*oview f*)))
unfolding *universally-def* **using** *keyed-view-determines f*[*unfolded supported-def*]
by *blast*
ultimately show *?thesis* **unfolding** *fun-support-def ofun-def* **by** *blast*
qed

lemma *supported-determines*:
supported f ⟹ *conforms X* ⟹ *determines* (*lower-maximal X*) (*vkey* (*oview f*)) (*ofun f*)
unfolding *supported-def ofun-def* **by** (*rule keyed-view-determines*)

Rows agreeing on a key part covering a supported function's key part agree on it.

lemma *supported-agree*:

assumes *h*: *supported h* **and** *d*: *conforms data* **and** *x*: $x \in$ *lower-maximal data* **and** *y*: $y \in$ *lower-maximal data*
and *le*: *okeys h* $\sqsubseteq$ *G* **and** *e*: *rproj G y* = *rproj G x*
shows *ofun h*·*y* = *ofun h*·*x*
proof (*rule determinesD*[*OF supported-determines*[*OF h d*] *y x*])
have *rproj* (*okeys h*) *y* = *rproj* (*okeys h*) *x*
using *arg-cong*[*OF e*, *of Rep-cfun* (*proj* (*okeys h*))] **by** (*simp add*: *proj-le*[*OF le*])
then show *vpart* (*vkey* (*oview h*)) *y* = *vpart* (*vkey* (*oview h*)) *x* **by** (*simp add*: *vkey-def proj-bot*)
qed

Rows agreeing on a key part covering a keyed view's key part agree on its section.

lemma *view-agree*:
assumes *v*: *keyed-view v* **and** *d*: *conforms data* **and** *x*: $x \in$ *lower-maximal data* **and** *y*: $y \in$ *lower-maximal data*
and *le*: *snd v* $\sqsubseteq$ *G* **and** *e*: *rproj G y* = *rproj G x*
shows *y* ⌊ *fst v* = *x* ⌊ *fst v*
proof −
have *supported* (*vsection v*) **using** *v* **by** (*simp add*: *supported-def*)
from *supported-agree*[*OF this d x y - e*] *le* **show** *?thesis* **by** *simp*
qed

lemma *keyed-view-bot-view*: *keyed-view* (⊥ :: (*'r*, *'k*) *view*)
unfolding *inst-prod-pcpo* **by** (*rule keyed-view-key*)

A guard is *key-pure* when its view is its key part: then it is determined by its key part on all rows, independently of any dataset. Slice tests are key-pure.

definition *key-pure* :: (*'r*, *'k*, *'b*::*cpo*) *vfn* ⇒ *bool* **where**
key-pure f ⟷ *vkey* (*oview f*) = *oview f*

lemma *key-pure-determines*: *key-pure f* ⟹ *determines UNIV* (*oview f*) (*ofun f*)
unfolding *ofun-def* **by** (*rule determines-comp*) (*rule vfd-refl*[*OF below-refl*])

lemma *key-pure-guard-holds*: *key-pure* (*guard-holds X*)
by (*simp add*: *key-pure-def vkey-def*)

lemma *supported-guard-holds*: *supported* (*guard-holds X*)
by (*simp add*: *supported-def keyed-view-key*)

lemma *supported-gtrue*: *supported gtrue*
unfolding *supported-def oview-gtrue inst-prod-pcpo* **by** (*rule keyed-view-key*)

lemma *supported-gand*: *supported* γ ⟹ *supported* δ ⟹ *supported* (*gand* γ δ)
by (*simp add*: *supported-def keyed-view-sup*)

An entry is *well-formed* when its view is keyed and its guard supported. Then the view's key part determines its section on the rows of a conforming

dataset: contents are graphs. This is a property of the expression, independent of the frame.

definition *entry-wf* :: (*'k*, *'r*) *entry* ⇒ *bool* **where**
entry-wf e ⟷ *keyed-view* (*eview e*) ∧ *supported* (*eguard e*)

lemma *okeys-gtrue* [*simp*]: *okeys gtrue* = ⊥
unfolding *oview-gtrue inst-prod-pcpo* **by** (*rule snd-conv*)

lemma *entry-wf-view*: *keyed-view v* ⟹ *supported γ* ⟹ *entry-wf* (*mk-entry S γ v*)
by (*simp add*: *entry-wf-def*)

lemma *entry-wf-guard*: *entry-wf e* ⟹ *supported γ* ⟹ *entry-wf* (*e*(|*eguard* := *gand γ* (*eguard e*)|))
by (*simp add*: *entry-wf-def supported-gand*)

definition *group* :: *'r lower-pd* ⇒ *'k semi-factor* ⇒ *'k semi-factor* ⇒ (*'k semi-factor*, *'f*, *'r*) *frame* ⇒ *'k* ⇒ *'r set* **where**
group data a ϱ F k = {*y* ∈ *lower-maximal data*. *selects* (*amb F*) *y* ∧ *holds* (*coarsen ϱ* (*slice F*)) (*rkey*·*y*) ∧
rproj (*a* ⊔ (*ϱ* ⊓ *flod F*)) *y* = *proj* (*a* ⊔ (*ϱ* ⊓ *flod F*))·*k*}

The fiber of an entry: the rows of its group passing its guard. Its content graph pairs each row's key at the view's key part with its section at the view's columns; the content is that graph as a map.

definition *efiber* :: *'r lower-pd* ⇒ (*'k*, *'r*) *entry* ⇒ (*'k semi-factor*, *'f*, *'r*) *frame* ⇒ *'k* ⇒ *'r set* **where**
efiber data e F k = {*y* ∈ *group data* (*elower e*) (*eupper e*) *F k*. *ofun* (*eguard e*)·*y* = *TT*}

definition *cpair* :: (*'k*, *'r*) *entry* ⇒ *'r* ⇒ *'k* × *'r* **where**
cpair e y = (*rproj* (*ekeys e*) *y*, *y* ⌊ *ecols e*)

lemma *cpair-vpart*: *cpair e y* = (*snd* (*vpart* (*eview e*) *y*), *fst* (*vpart* (*eview e*) *y*))
by (*simp add*: *cpair-def vpart-def*)

definition *cgraph* :: *'r lower-pd* ⇒ (*'k*, *'r*) *entry* ⇒ (*'k semi-factor*, *'f*, *'r*) *frame* ⇒ *'k* ⇒ (*'k* × *'r*) *set* **where**
cgraph data e F k = *cpair e* ‘ *efiber data e F k*

definition *content* :: *'r lower-pd* ⇒ (*'k*, *'r*) *entry* ⇒ (*'k semi-factor*, *'f*, *'r*) *frame* ⇒ *'k* ⇒ *'k* ⇒ *'r option* **where**
content data e F k = *gmap* (*cgraph data e F k*)

abbreviation *contents* :: *'r lower-pd* ⇒ (*'k*, *'r*) *entry list* ⇒ (*'k semi-factor*, *'f*, *'r*) *frame* ⇒ *'k* ⇒ (*'k* ⇒ *'r option*) *list* **where**
contents data es F k ≡ *map* (λ*e*. *content data e F k*) *es*

A *row entry* keys its view and guard within its lower part: rows of a fiber,

which agree there, agree on the whole content pair, so its content has at most one key.

definition *row-entry* :: ('*k*, '*r*) *entry* ⇒ *bool* **where**
row-entry e ⟷ *ekeys e* ⊑ *elower e* ∧ *okeys* (*eguard e*) ⊑ *elower e*

lemma *rproj-agree*: *s* ⊑ *G* ⟹ *rproj G y* = *rproj G x* ⟹ *rproj s y* = *rproj s x*
by (*metis proj-le*)

lemma *row-entry-image*:
assumes *e*: *entry-wf e* **and** *r*: *row-entry e* **and** *d*: *conforms data* **and** *Y*: *Y* ⊆ *lower-maximal data*
and *x*: *x* ∈ *Y* **and** *aG*: *elower e* ⊑ *G* **and** *agree*: ⋀*y*. *y* ∈ *Y* ⟹ *rproj G y* = *rproj G x*
shows *cpair e* ‘ {*y* ∈ *Y*. *ofun* (*eguard e*)·*y* = *TT*} = (*if ofun* (*eguard e*)·*x* = *TT then* {*cpair e x*} *else* {})
proof −
have *sG*: *ekeys e* ⊑ *G* **and** *gG*: *okeys* (*eguard e*) ⊑ *G*
using *r aG* **unfolding** *row-entry-def* **by** (*blast intro*: *below-trans*)+
have *xm*: *x* ∈ *lower-maximal data* **using** *x Y* **by** *blast*
have *kv*: *keyed-view* (*eview e*) **and** *sup*: *supported* (*eguard e*) **using** *e* **by** (*simp-all add*: *entry-wf-def*)
let *?g* = λ*y*. *ofun* (*eguard e*)·*y*
have *g*: *?g y* = *?g x* **if** *y*: *y* ∈ *Y* **for** *y*
using *supported-agree*[*OF sup d xm - gG agree*[*OF y*]] *y Y* **by** *blast*
have *h*: *cpair e y* = *cpair e x* **if** *y*: *y* ∈ *Y* **for** *y*
using *rproj-agree*[*OF sG agree*[*OF y*]] *view-agree*[*OF kv d xm - sG agree*[*OF y*]] *y Y*
unfolding *cpair-def* **by** *blast*
show *?thesis*
proof (*cases ?g x* = *TT*)
case *True*
have {*y* ∈ *Y*. *?g y* = *TT*} = *Y*
proof (*rule set-eqI*, *rule iffI*)
fix *y* **assume** *y* ∈ *Y*
then show *y* ∈ {*y* ∈ *Y*. *?g y* = *TT*} **using** *True* **by** (*simp add*: *g*)
qed *simp*
moreover have *cpair e* ‘ *Y* = (λ-. *cpair e x*) ‘ *Y* **by** (*rule image-cong*[*OF refl h*])
moreover have *Y* ≠ {} **using** *x* **by** *blast*
ultimately show *?thesis* **using** *True* **by** (*simp only*: *image-constant-conv if-False*) *simp*
next
case *False*
have {*y* ∈ *Y*. *?g y* = *TT*} = {}
proof (*rule equals0I*)
fix *y* **assume** *y* ∈ {*y* ∈ *Y*. *?g y* = *TT*}
then have *y*: *y* ∈ *Y ?g y* = *TT* **by** *simp-all*
then have *?g x* = *TT* **by** (*simp only*: *g*[*OF y*(*1*)])
then show *False* **using** *False* **by** *simp*

qed
then show *?thesis* **using** *False* **by** *simp*
qed
qed

Content graphs of well-formed entries are single-valued.

lemma *cgraph-single-valued*:
assumes *e*: *entry-wf e* **and** *d*: *conforms data*
shows *single-valued* (*cgraph data e F k*)
proof (*rule single-valuedI*)
fix *q u u′* **assume** *u*: (*q*, *u*) ∈ *cgraph data e F k* **and** *u′*: (*q*, *u′*) ∈ *cgraph data e F k*
obtain *y* **where** *y*: *y* ∈ *efiber data e F k q* = *rproj* (*ekeys e*) *y u* = *y* ⌊ *ecols e*
using *u* **unfolding** *cgraph-def cpair-def* **by** *blast*
obtain *y′* **where** *y′*: *y′* ∈ *efiber data e F k q* = *rproj* (*ekeys e*) *y′ u′* = *y′* ⌊ *ecols e*
using *u′* **unfolding** *cgraph-def cpair-def* **by** *blast*
have *m*: *y* ∈ *lower-maximal data y′* ∈ *lower-maximal data*
using *y*(*1*) *y′*(*1*) **by** (*auto simp*: *efiber-def group-def*)
have *kv*: *keyed-view* (*eview e*) **using** *e* **by** (*simp add*: *entry-wf-def*)
have *y′* ⌊ *ecols e* = *y* ⌊ *ecols e*
by (*rule view-agree*[*OF kv d m*(*1*) *m*(*2*) *below-refl*]) (*use y*(*2*) *y′*(*2*) **in** *simp*)
then show *u* = *u′* **using** *y*(*3*) *y′*(*3*) **by** *simp*
qed

lemma *graph-content*: *entry-wf e* ⟹ *conforms data* ⟹ *graph* (*content data e F k*) = *cgraph data e F k*
unfolding *content-def* **by** (*rule graph-gmap*[*OF cgraph-single-valued*])

35.4 Column supports of frames and sections of datasets

A selection *reads* a section of the row when it is decided there. A frame reads a section covering its ambient filter and the column support of its slice's LOD. The *point section* of a dataset at a section is the set of its data points cut down to it; it is a plain set, not down-closed: contents are sets of values over data points.

definition *sel-reads* :: *′r semi-factor* ⇒ *′r sel* ⇒ *bool* **where**
sel-reads c P ⟷ (∀ *y*. *selects P* (*y* ⌊ *c*) ⟷ *selects P y*)

definition *frame-reads* :: *′r semi-factor* ⇒ (*′k semi-factor*, *′f*, *′r*) *frame* ⇒ *bool* **where**
frame-reads c F ⟷ *sel-reads c* (*amb F*) ∧ *csupp* (*slod* (*slice F*)) ⊑ *c*

definition *pts-section* :: *′r semi-factor* ⇒ *′r lower-pd* ⇒ *′r set* **where**
pts-section c data = (λ*y*. *y* ⌊ *c*) ‘ *lower-maximal data*

lemma *image-filter-factor*:
assumes ⋀*y*. *Q* (*g y*) = *Q y* **and** ⋀*y*. *h* (*g y*) = *h y*

shows h ‘ $\{y \in M.\ Q\ y\} = h$ ‘ $\{z \in g$ ‘ $M.\ Q\ z\}$
proof −
have $\{z \in g$ ‘ $M.\ Q\ z\} = g$ ‘ $\{y \in M.\ Q\ y\}$ **using** *assms(1)* **by** *auto*
then show *?thesis* **by** (*simp add*: *image-image assms(2)*)
qed

lemma *holds-section*: *csupp* (*slod X*) $\sqsubseteq c \Longrightarrow$ *holds X* (*rkey*·(*y* ⌊ *c*)) $\longleftrightarrow$ *holds X* (*rkey*·*y*)
by (*metis holds-lod rproj-section*)

A view's parts are read off any section covering its columns and its key part's support.

lemma *vpart-section*: *fst* (*vsec v*) $\sqsubseteq c \Longrightarrow$ *vpart v* (*y* ⌊ *c*) = *vpart v y*
unfolding *vsec-def fst-conv*
by (*simp add*: *vpart-def proj-le*[*OF below-trans*[*OF lattice-join-ge1*]]
rproj-section[*OF below-trans*[*OF lattice-join-ge2*]])

lemma *ofun-section*: *fst* (*vsec* (*oview f*)) $\sqsubseteq c \Longrightarrow$ *ofun f*·(*y* ⌊ *c*) = *ofun f*·*y*
by (*simp add*: *ofun-apply vpart-section*)

The content of an entry is computed from the point section at any section covering the group LOD's support, the frame, and the views of the guard and the entry.

lemma *content-section*:
assumes *F*: *frame-reads c F* **and** *G*: *csupp* (*elower e* ⊔ (*eupper e* ⊓ *flod F*)) $\sqsubseteq c$
and *fc*: *fst* (*vsec* (*eview e*)) $\sqsubseteq c$ **and** *gc*: *fst* (*vsec* (*oview* (*eguard e*))) $\sqsubseteq c$
shows *content data e F k* = *gmap* (*cpair e* ‘ $\{z \in$ *pts-section c data.*
selects (*amb F*) *z* ∧ *holds* (*coarsen* (*eupper e*) (*slice F*)) (*rkey*·*z*) ∧
rproj (*elower e* ⊔ (*eupper e* ⊓ *flod F*)) *z* = *proj* (*elower e* ⊔ (*eupper e* ⊓ *flod F*))·*k* ∧
ofun (*eguard e*)·*z* = *TT*$\}$)
proof −
let *?G* = *elower e* ⊔ (*eupper e* ⊓ *flod F*)
let *?P* = λ*z*. *selects* (*amb F*) *z* ∧ *holds* (*coarsen* (*eupper e*) (*slice F*)) (*rkey*·*z*) ∧ *rproj ?G z* = *proj ?G*·*k* ∧
ofun (*eguard e*)·*z* = *TT*
let *?h* = *cpair e*
have *sl*: *csupp* (*slod* (*coarsen* (*eupper e*) (*slice F*))) $\sqsubseteq c$
using *F csupp-mono*[*OF below-trans*[*OF slod-coarsen lattice-meet-le1*]] **unfolding** *frame-reads-def*
by (*blast intro*: *below-trans*)
have *P*: *?P* (*y* ⌊ *c*) $\longleftrightarrow$ *?P y* **for** *y*
using *F* **unfolding** *frame-reads-def sel-reads-def*
by (*simp add*: *holds-section*[*OF sl*] *rproj-section*[*OF G*] *ofun-section*[*OF gc*])
have *h*: *?h* (*y* ⌊ *c*) = *?h y* **for** *y* **by** (*simp only*: *cpair-vpart vpart-section*[*OF fc*])
have *efiber data e F k* = $\{y \in$ *lower-maximal data. ?P y*$\}$ **by** (*auto simp*: *efiber-def group-def*)
moreover have *?h* ‘ $\{y \in$ *lower-maximal data. ?P y*$\}$ = *?h* ‘ $\{z \in$ *pts-section c data. ?P z*$\}$

unfolding *pts-section-def* **by** (*rule image-filter-factor*) (*fact P h*)+
ultimately show *?thesis* **unfolding** *content-def cgraph-def* **by** *simp*
qed

Groups with the same parameters and rows coincide.

lemma *group-cong*:
assumes *amb F = amb F' slice F = slice F'*
and $a \sqcup (\varrho \sqcap \mathit{flod}\ F) = a' \sqcup (\varrho \sqcap \mathit{flod}\ F')$
and $\mathit{proj}\ (a \sqcup (\varrho \sqcap \mathit{flod}\ F))\cdot k = \mathit{proj}\ (a' \sqcup (\varrho \sqcap \mathit{flod}\ F'))\cdot k'$
shows *group data a ϱ F k = group data a' ϱ F' k'*
using *assms* **unfolding** *group-def* **by** *simp*

A lower part covering the masked part of the minimal LOD absorbs it.

lemma *sat-absorb*:
fixes *a ϱ m L* :: *'k semi-factor*
assumes $(\varrho \sqcap m) \sqsubseteq a$
shows $a \sqcup (\varrho \sqcap (m \sqcup L)) = a \sqcup (\varrho \sqcap L)$
proof −
have $a \sqcup (\varrho \sqcap (m \sqcup L)) = (a \sqcup (\varrho \sqcap m)) \sqcup (\varrho \sqcap L)$
by (*simp only*: *truncate-sup lattice-join-assoc*)
also have $a \sqcup (\varrho \sqcap m) = a$ **by** (*rule lattice-join-absorb1* [*OF assms*])
finally show *?thesis* **.**
qed

end

Projecting to a finer LOD after a coarser one changes nothing.

lemma *proj-absorb*: $a \sqsubseteq b \Longrightarrow \mathit{proj}\ b\cdot(\mathit{proj}\ a\cdot k) = \mathit{proj}\ a\cdot k$
by (*metis proj-inf lattice-meet-absorb2*)

Joins of lists in complete lattices.

lemma *fold-join-absorb*: $(\bigwedge x.\ x \in \mathit{set}\ xs \Longrightarrow x \sqsubseteq b) \Longrightarrow \mathit{fold}\ (\sqcup)\ xs\ b = (b::'a::\mathit{lattice\text{-}complete})$
by (*induct xs*) (*simp-all add*: *lattice-join-absorb2*)

lemma *fold-join-bots*: $\mathit{fold}\ (\sqcup)\ (\mathit{map}\ (\lambda\text{-}.\ \bot)\ xs)\ (\bot::'a::\mathit{lattice\text{-}complete}) = \bot$
by (*induct xs*) (*simp-all add*: *lattice-join-bot-left*)

lemma *fold-join-least*:
$(\bigwedge x.\ x \in \mathit{set}\ xs \Longrightarrow x \sqsubseteq y) \Longrightarrow b \sqsubseteq y \Longrightarrow \mathit{fold}\ (\sqcup)\ xs\ b \sqsubseteq (y::'a::\mathit{lattice\text{-}complete})$
by (*induct xs arbitrary*: *b*) (*simp-all add*: *lattice-join-least*)

lemma *fold-join-base*: $b \sqsubseteq \mathit{fold}\ (\sqcup)\ xs\ (b::'a::\mathit{lattice\text{-}complete})$
proof (*induct xs arbitrary*: *b*)
case (*Cons x xs*)
have $b \sqsubseteq x \sqcup b$ **by** (*rule lattice-join-ge2*)
also have $\ldots \sqsubseteq \mathit{fold}\ (\sqcup)\ xs\ (x \sqcup b)$ **by** (*rule Cons*)
finally show *?case* **by** *simp*
qed *simp*

35.5 The laws

A *semilawful* expression has supported entries, its minimal LOD is the join of their lower parts, and its presence and values are its content semantics of its contents, at every key of its key LOD and every conforming dataset (the content law). A *lawful* one is moreover *saturated* (each lower part covers the part of the minimal LOD its upper part keeps, so groups depend on the frame LOD only through the key LOD) and has entries only at the frame's data keys (the key law).

context *dataspace*
begin

abbreviation *kd* :: ('*k*, '*f*, '*r*, '*v*) *expr* ⇒ ('*k semi-factor*, '*f*, '*r*) *frame* ⇒ '*k semi-factor* **where**
kd E F ≡ *emind E* ⊔ *flod F*

definition *semilawful* :: ('*k*, '*f*, '*r*, '*v*) *expr* ⇒ *bool* **where**
semilawful E ⟷ *emind E* = *fold* (⊔) (*map elower* (*edims E*)) ⊥ ∧ (∀ *e*∈*set* (*edims E*). *entry-wf e*) ∧
(∀ *data F k*. *conforms data* ⟶ *proj* (*kd E F*)·*k* = *k* ⟶ *esem E data F k* = *csem E* (*contents data* (*edims E*) *F k*))

definition *lawful* :: ('*k*, '*f*, '*r*, '*v*) *expr* ⇒ *bool* **where**
lawful E ⟷ *semilawful E* ∧ (∀ *e*∈*set* (*edims E*). (*eupper e* ⊓ *emind E*) ⊑ *elower e*) ∧
(∀ *data F k*. *conforms data* ⟶ *esem E data F k* ≠ *None* ⟶ *keyimg data F* (*kd E F*) *k*)

lemma *semilawfulI*:
assumes *emind E* = *fold* (⊔) (*map elower* (*edims E*)) ⊥
and ⋀*e*. *e* ∈ *set* (*edims E*) ⟹ *entry-wf e*
and ⋀*data F k*. ⟦*conforms data*; *proj* (*emind E* ⊔ *flod F*)·*k* = *k*⟧ ⟹
esem E data F k = *csem E* (*contents data* (*edims E*) *F k*)
shows *semilawful E*
unfolding *semilawful-def* **using** *assms* **by** *blast*

lemma *semilawful-mind-lub*: *semilawful E* ⟹ *emind E* = *fold* (⊔) (*map elower* (*edims E*)) ⊥
unfolding *semilawful-def* **by** *blast*

lemma *semilawful-wf*: *semilawful E* ⟹ *e* ∈ *set* (*edims E*) ⟹ *entry-wf e*
unfolding *semilawful-def* **by** *blast*

lemma *semilawful-keyed*: *semilawful E* ⟹ *e* ∈ *set* (*edims E*) ⟹ *keyed-view* (*eview e*)
and *semilawful-guard-supported*: *semilawful E* ⟹ *e* ∈ *set* (*edims E*) ⟹ *supported* (*eguard e*)
using *semilawful-wf* **unfolding** *entry-wf-def* **by** *blast*+

lemma *semilawful-csem*:
⟦*semilawful E*; *conforms data*; *proj* (*kd E F*)·*k* = *k*⟧ ⟹ *esem E data F k* = *csem E* (*contents data* (*edims E*) *F k*)
unfolding *semilawful-def* **by** *blast*

Equal contents give equal values, across datasets and frames.

lemma *semilawful-content-law*:
⟦*semilawful E*; *conforms data*; *conforms data′*; *proj* (*kd E F*)·*k* = *k*; *proj* (*kd E F′*)·*k′* = *k′*;
contents data (*edims E*) *F k* = *contents data′* (*edims E*) *F′ k′*⟧ ⟹ *esem E data F k* = *esem E data′ F′ k′*
by (*simp only*: *semilawful-csem*)

lemma *lawful-semiI*:
assumes *semilawful E*
and ⋀*e*. *e* ∈ *set* (*edims E*) ⟹ (*eupper e* ⊓ *emind E*) ⊑ *elower e*
and ⋀*data F k*. *conforms data* ⟹ *esem E data F k* ≠ *None* ⟹ *keyimg data F* (*emind E* ⊔ *flod F*) *k*
shows *lawful E*
unfolding *lawful-def* **using** *assms* **by** *blast*

lemma *lawfulI*:
assumes ⋀*e*. *e* ∈ *set* (*edims E*) ⟹ (*eupper e* ⊓ *emind E*) ⊑ *elower e*
and *emind E* = *fold* (⊔) (*map elower* (*edims E*)) ⊥
and ⋀*e*. *e* ∈ *set* (*edims E*) ⟹ *entry-wf e*
and ⋀*data F k*. ⟦*conforms data*; *proj* (*emind E* ⊔ *flod F*)·*k* = *k*⟧ ⟹
esem E data F k = *csem E* (*contents data* (*edims E*) *F k*)
and ⋀*data F k*. *conforms data* ⟹ *esem E data F k* ≠ *None* ⟹ *keyimg data F* (*emind E* ⊔ *flod F*) *k*
shows *lawful E*
by (*rule lawful-semiI*[*OF semilawfulI*[*OF assms*(2−4)] *assms*(1,5)])

lemma *lawful-semilawful*: *lawful E* ⟹ *semilawful E*
unfolding *lawful-def* **by** *blast*

lemma *lawful-saturated*: *lawful E* ⟹ *e* ∈ *set* (*edims E*) ⟹ (*eupper e* ⊓ *emind E*) ⊑ *elower e*
unfolding *lawful-def* **by** *blast*

lemma *lawful-key-law*: *lawful E* ⟹ *conforms data* ⟹ *esem E data F k* ≠ *None* ⟹ *keyimg data F* (*kd E F*) *k*
unfolding *lawful-def* **by** *blast*

lemma *lawful-keyed*: *lawful E* ⟹ *e* ∈ *set* (*edims E*) ⟹ *keyed-view* (*eview e*)
by (*rule semilawful-keyed*[*OF lawful-semilawful*])

lemma *lower-below-mind*:
assumes *E*: *semilawful E* **and** *e*: *e* ∈ *set* (*edims E*)

shows *elower e* ⊑ *emind E*
proof −
have *elower e* ∈ *set* (*map elower* (*edims E*)) **using** *e* **by** *force*
then show *?thesis* **unfolding** *semilawful-mind-lub*[*OF E*] **by** (*rule fold-join-upper*)
qed

lemma *group-below-kd*:
semilawful E ⟹ *e* ∈ *set* (*edims E*) ⟹ *elower e* ⊔ (*eupper e* ⊓ *flod F*) ⊑ *kd E F*
by (*rule join-belowI*[*OF below-trans*[*OF lower-below-mind lattice-join-ge1*]
below-trans[*OF truncate-below lattice-join-ge2*]])

Contents agree when the groups do.

lemma *content-group-cong*:
group data (*elower e*) (*eupper e*) *F k* = *group data′* (*elower e*) (*eupper e*) *F′ k′* ⟹
content data e F k = *content data′ e F′ k′*
by (*simp add*: *content-def cgraph-def efiber-def*)

Contents only depend on the point section at a section covering the frame, the key LOD and the sections of the entries' views.

definition *entry-reads* :: *′r semi-factor* ⇒ (*′k*, *′r*) *entry* ⇒ *bool* **where**
entry-reads c e ⟷ *fst* (*vsec* (*eview e*)) ⊑ *c* ∧ *fst* (*vsec* (*oview* (*eguard e*))) ⊑ *c*

lemma *contents-section*:
assumes *E*: *semilawful E* **and** *F*: *frame-reads c F* **and** *K*: *csupp* (*kd E F*) ⊑ *c*
and *cols*: ⋀*e*. *e* ∈ *set* (*edims E*) ⟹ *entry-reads c e*
and *sec*: *pts-section c data* = *pts-section c data′*
shows *contents data* (*edims E*) *F k* = *contents data′* (*edims E*) *F k*
proof (*rule map-cong*[*OF refl*])
fix *e* **assume** *e*: *e* ∈ *set* (*edims E*)
have *G*: *csupp* (*elower e* ⊔ (*eupper e* ⊓ *flod F*)) ⊑ *c*
by (*rule below-trans*[*OF csupp-mono*[*OF group-below-kd*[*OF E e*]] *K*])
have *r*: *fst* (*vsec* (*eview e*)) ⊑ *c fst* (*vsec* (*oview* (*eguard e*))) ⊑ *c*
using *cols*[*OF e*] **by** (*simp-all add*: *entry-reads-def*)
show *content data e F k* = *content data′ e F k*
unfolding *content-section*[*OF F G r*] *sec* **..**
qed

The *grain* of an expression at a frame joins the group LODs of its entries. Contents, and so values, depend on the key only through its part at the grain: an expression emits at its grain, and its values at finer keys are broadcast.

definition *grain* :: (*′k*, *′f*, *′r*, *′v*) *expr* ⇒ (*′k semi-factor*, *′f*, *′r*) *frame* ⇒ *′k semi-factor* **where**
grain E F = *fold* (⊔) (*map* (λ*e*. *elower e* ⊔ (*eupper e* ⊓ *flod F*)) (*edims E*)) ⊥

lemma *group-below-grain*: *e* ∈ *set* (*edims E*) ⟹ *elower e* ⊔ (*eupper e* ⊓ *flod F*) ⊑ *grain E F*

unfolding *grain-def* **by** (*rule fold-join-upper*) *force*

lemma *grain-below-kd*: *semilawful E* $\Longrightarrow$ *grain E F* $\sqsubseteq$ *kd E F*
unfolding *grain-def* **by** (*rule fold-join-least*) (*auto intro*: *group-below-kd*)

theorem *sem-grain*:
assumes *E*: *semilawful E* **and** *d*: *conforms data* **and** *k*: *proj* (*kd E F*)·*k* = *k*
shows *esem E data F k* = *esem E data F* (*proj* (*grain E F*)·*k*)
proof (*rule semilawful-content-law*[*OF E d d k*])
show *proj* (*kd E F*)·(*proj* (*grain E F*)·*k*) = *proj* (*grain E F*)·*k*
by (*rule proj-absorb*[*OF grain-below-kd*[*OF E*]])
show *contents data* (*edims E*) *F k* = *contents data* (*edims E*) *F* (*proj* (*grain E F*)·*k*)
proof (*rule map-cong*[*OF refl*])
fix *e* **assume** *e*: *e* ∈ *set* (*edims E*)
have *proj* (*elower e* ⊔ (*eupper e* ⊓ *flod F*))·(*proj* (*grain E F*)·*k*) = *proj* (*elower e* ⊔ (*eupper e* ⊓ *flod F*))·*k*
by (*rule proj-le*[*OF group-below-grain*[*OF e*]])
then show *content data e F k* = *content data e F* (*proj* (*grain E F*)·*k*)
by (*intro content-group-cong*) (*simp add*: *group-def*)
qed
qed

For lawful expressions the frame LOD inside the minimal LOD does not matter: groups only see the key LOD.

lemma *group-kd*:
lawful E $\Longrightarrow$ *e* ∈ *set* (*edims E*) $\Longrightarrow$ *elower e* ⊔ (*eupper e* ⊓ *flod F*) = *elower e* ⊔ (*eupper e* ⊓ *kd E F*)
by (*rule sat-absorb*[*symmetric*, *OF lawful-saturated*])

lemma *contents-kd*:
assumes *E*: *lawful E* **and** *F*: *amb F* = *amb F′ slice F* = *slice F′ kd E F* = *kd E F′*
shows *contents data* (*edims E*) *F k* = *contents data* (*edims E*) *F′ k*
proof (*rule map-cong*[*OF refl*])
fix *e* **assume** *e*: *e* ∈ *set* (*edims E*)
have *group data* (*elower e*) (*eupper e*) *F k* = *group data* (*elower e*) (*eupper e*) *F′ k*
by (*rule group-cong*[*OF F*(*1*,*2*)]) (*simp-all only*: *group-kd*[*OF E e*] *F*(*3*))
then show *content data e F k* = *content data e F′ k* **by** (*rule content-group-cong*)
qed

Pegging the frame to a key does not change that key's contents: the pin's demand within a mask is part of the group already, by saturation.

lemma *peg-group*:
assumes *E*: *lawful E* **and** *e*: *e* ∈ *set* (*edims E*) **and** *cK*: *compact K* **and** *K*: *proj* (*kd E F*)·*K* = *K*
shows *group data* (*elower e*) (*eupper e*) (*peg K F*) *K* = *group data* (*elower e*) (*eupper e*) *F K*

proof −
let *?G* = *elower e* ⊔ (*eupper e* ⊓ *flod F*) **and** *?T* = (*eupper e* ⊓ *kd E F*)
have *le*: *?T* ⊑ *?G* **unfolding** *group-kd*[*OF E e*] **by** (*rule lattice-join-ge2*)
have *pin*: *holds* (*coarsen* (*eupper e*) (*spin K*)) (*rkey*·*y*) **if** *y*: *rproj ?G y* = *proj ?G*·*K* **for** *y*
proof −
have *proj* (*eupper e*)·*K* = *proj ?T*·*K* **by** (*simp only*: *proj-inf K*)
also have ... = *proj ?T*·(*proj ?G*·*K*) **by** (*rule proj-le*[*OF le*, *symmetric*])
also have ... = *proj ?T*·(*rproj ?G y*) **by** (*simp only*: *y*)
also have ... = *rproj ?T y* **by** (*rule proj-le*[*OF le*])
finally have *proj* (*eupper e*)·*K* ⊑ *rkey*·*y* **using** *proj-below*[*of ?T rkey*·*y*] **by** *simp*
then show *?thesis* **by** (*rule holds-coarsen-spin*[*OF cK*])
qed
show *?thesis* **unfolding** *group-def* **using** *pin* **by** (*auto simp*: *coarsen-conj*)
qed

lemma *peg-contents*:
assumes *E*: *lawful E* **and** *cK*: *compact K* **and** *K*: *proj* (*kd E F*)·*K* = *K*
shows *contents data* (*edims E*) (*peg K F*) *K* = *contents data* (*edims E*) *F K*
proof (*rule map-cong*[*OF refl*])
fix *e* **assume** *e*: *e* ∈ *set* (*edims E*)
show *content data e* (*peg K F*) *K* = *content data e F K*
by (*rule content-group-cong*[*OF peg-group*[*OF E e cK K*]])
qed

lemma *key-lives*: *lawful E* ⟹ *conforms data* ⟹ *esem E data F k* ≠ *None* ⟹ *proj* (*kd E F*)·*k* = *k*
by (*auto dest*!: *lawful-key-law simp*: *keyimg-def*)

The effective-LOD law: the frame LOD matters only through the key LOD.

theorem *eff-law*:
assumes *E*: *lawful E* **and** *d*: *conforms data* **and** *F*: *amb F* = *amb F′ slice F* = *slice F′ kd E F* = *kd E F′*
shows *esem E data F* = *esem E data F′*
proof
fix *k*
show *esem E data F k* = *esem E data F′ k*
proof (*cases proj* (*kd E F*)·*k* = *k*)
case *True*
then have *proj* (*kd E F′*)·*k* = *k* **by** (*simp add*: *F*(*3*))
then show *?thesis*
by (*rule semilawful-content-law*[*OF lawful-semilawful*[*OF E*] *d d True - contents-kd*[*OF E F*]])
next
case *False*
then have *proj* (*kd E F′*)·*k* ≠ *k* **by** (*simp add*: *F*(*3*))
then have *esem E data F′ k* = *None* **using** *key-lives*[*OF E d*, *of F′ k*] **by** *blast*
moreover have *esem E data F k* = *None* **using** *False key-lives*[*OF E d*, *of F*

k] **by** *blast*
  ultimately show *?thesis* **by** *simp*
 qed
qed

The peg law: pegging the frame to a key does not change the value there.

theorem *peg-law*:
 assumes *E*: *lawful E* **and** *d*: *conforms data* **and** *cK*: *compact K*
 shows *esem E data* (*peg K F*) *K* = *esem E data F K*
proof (*cases proj* (*kd E F*)·*K* = *K*)
 case *True*
 have *proj* (*kd E* (*peg K F*))·*K* = *K* **using** *True* **by** *simp*
 then show *?thesis*
  by (*rule semilawful-content-law*[*OF lawful-semilawful*[*OF E*] *d d - True peg-contents*[*OF E cK True*]])
next
 case *False*
 then have *proj* (*kd E* (*peg K F*))·*K* ≠ *K* **by** *simp*
 then have *esem E data* (*peg K F*) *K* = *None* **using** *key-lives*[*OF E d*, *of peg K F K*] **by** *blast*
 moreover have *esem E data F K* = *None* **using** *False key-lives*[*OF E d*, *of F K*] **by** *blast*
 ultimately show *?thesis* **by** *simp*
qed

Locality: the semantics at a frame depends on the dataset only through its point section at a section covering the frame, the key LOD and the entries' columns.

theorem *sem-section*:
 assumes *E*: *lawful E* **and** *d*: *conforms data* **and** *d′*: *conforms data′*
  and *F*: *frame-reads c F* **and** *K*: *csupp* (*kd E F*) ⊑ *c*
  and *cols*: ⋀*e*. *e* ∈ *set* (*edims E*) ⟹ *entry-reads c e*
  and *sec*: *pts-section c data* = *pts-section c data′*
 shows *esem E data F k* = *esem E data′ F k*
proof (*cases proj* (*kd E F*)·*k* = *k*)
 case *True*
 show *?thesis*
  by (*rule semilawful-content-law*[*OF lawful-semilawful*[*OF E*] *d d′ True True*
      contents-section[*OF lawful-semilawful*[*OF E*] *F K cols sec*]])
next
 case *False*
 then have *esem E data F k* = *None esem E data′ F k* = *None*
  using *key-lives*[*OF E d*] *key-lives*[*OF E d′*] **by** *blast+*
 then show *?thesis* **by** *simp*
qed

end

35.6 Lemmas for the combinators

lemma *concat-map-blocks*:
assumes *concat* (*map f xs*) = *concat* (*map g xs*) **and** ⋀*x*. *x* ∈ *set xs* ⟹ *length* (*f x*) = *length* (*g x*)
and *x* ∈ *set xs*
shows *f x* = *g x*
using *assms*
proof (*induct xs*)
case (*Cons y ys*)
have *l*: *length* (*f y*) = *length* (*g y*) **by** (*rule Cons.prems*(*2*)) *simp*
have *f y* = *g y* ∧ *concat* (*map f ys*) = *concat* (*map g ys*)
using *Cons.prems*(*1*) **by** (*simp add*: *append-eq-append-conv*[*OF disjI1*[*OF l*]])
then show *?case* **using** *Cons* **by** *auto*
qed *simp*

lemma *proj-sup-eq*:
proj (*a* ⊔ *b*)·*x* = *proj* (*a* ⊔ *b*)·*z* ⟷ *proj a*·*x* = *proj a*·*z* ∧ *proj b*·*x* = *proj b*·*z*
proof
assume *e*: *proj* (*a* ⊔ *b*)·*x* = *proj* (*a* ⊔ *b*)·*z*
show *proj a*·*x* = *proj a*·*z* ∧ *proj b*·*x* = *proj b*·*z*
using *arg-cong*[*OF e*, *of Rep-cfun* (*proj a*)] *arg-cong*[*OF e*, *of Rep-cfun* (*proj b*)]
by (*simp add*: *proj-le lattice-join-ge1 lattice-join-ge2*)
qed (*simp add*: *proj-sup*)

context *dataspace*
begin

The group of the full mask on top of an LOD is the key's own group.

lemma *group-top*:
assumes *proj* (*c* ⊔ *flod F*)·*k* = *k*
shows *group data c lattice-top F k* = {*y* ∈ *lower-maximal data*. *frows F y* ∧ *rproj* (*c* ⊔ *flod F*) *y* = *k*}
using *assms* **by** (*auto simp*: *group-def truncate-top frows-def*)

lemma *keyimg-group-top*:
assumes *proj* (*c* ⊔ *flod F*)·*k* = *k*
shows *keyimg data F* (*c* ⊔ *flod F*) *k* ⟷ *group data c lattice-top F k* ≠ {}
by (*auto simp*: *group-top*[*OF assms*] *keyimg-def*)

end

end
theory *Expr-Combinators*
imports *Content-Law*
begin

36 Combinators

Each combinator transforms the entries of its inner expressions' dims along with their semantics, and is lawful on lawful inner expressions. Collecting a list of expressions into a tuple, followed by a map, expresses every operator of any arity.

36.1 Tuples

A collected entry is present where every component is; its value is a tuple that is lazy in the components, so an undefined component does not make the tuple undefined. No order classes are needed for values; they will be once tuples are key values.

domain (*'a*::*domain*) *ltuple* = *LNil* | *LCons* (**lazy** *'a*) *'a ltuple*

fun *ltuple-of* :: *'a*::*domain list* ⇒ *'a ltuple* **where**
  ltuple-of [] = *LNil*
| *ltuple-of* (*x* # *xs*) = *LCons*·*x*·(*ltuple-of xs*)

lemma *ltuple-of-not-bottom*: *ltuple-of xs* ≠ ⊥
  by (*induct xs*) *simp-all*

lemma *those-not-None-iff*: *those xs* ≠ *None* ⟷ (∀ *x*∈*set xs*. *x* ≠ *None*)
  by (*induct xs*) (*auto split*: *option.splits*)

context *dataspace*
begin

36.2 Keys

The key at an LOD *c*. Its only entry is the group of rows with that key part, the frame's filters coarsened to *c*, keyed there and reading no columns, so its content is that key part; it is present where that group is nonempty, so it is gated at its own row LOD, not at the frame's keys.

lemma *join-meet-self*: *c* ⊔ (*c* ⊓ *L*) = *c* **for** *c* :: *'k semi-factor*
  by (*rule lattice-join-absorb1*[*OF lattice-meet-le1*])

definition *key-at* :: *'k semi-factor* ⇒ (*'k*, *'f*, *'r*, *'k*) *expr* **where**
  key-at c = ⦇*emind* = *c*, *edims* = [*mk-entry* (*sel c c*) *gtrue* (⊥, *c*)],
    esem = λ*data F k*. *if group data c c F k* = {} *then None else Some* (*proj c*·*k*),
    csem = λ*Xs*. *if hd Xs* = *Map.empty then None else Some* (*the-elem* (*dom* (*hd Xs*)))⦈

lemma *key-at-simps* [*simp*]:
  emind (*key-at c*) = *c edims* (*key-at c*) = [*mk-entry* (*sel c c*) *gtrue* (⊥, *c*)]
  esem (*key-at c*) *data F k* = (*if group data c c F k* = {} *then None else Some* (*proj c*·*k*))

csem (*key-at c*) *Xs* = (*if hd Xs* = *Map.empty then None else Some* (*the-elem* (*dom* (*hd Xs*))))
by (*simp-all add*: *key-at-def*)

lemma *group-own*: *y* ∈ *group data c c F k* ⟹ *rproj c y* = *proj c·k*
by (*simp add*: *group-def join-meet-self*)

lemma *content-key-at*:
content data (*mk-entry* (*sel c c*) *gtrue* (⊥, *c*)) *F k* = (*if group data c c F k* = {} *then Map.empty else* [*proj c·k* ↦ ⊥])
proof −
have *cgraph data* (*mk-entry* (*sel c c*) *gtrue* (⊥, *c*)) *F k* = (λ-. (*proj c·k*, ⊥)) ‘ *group data c c F k*
unfolding *cgraph-def efiber-def cpair-def* **by** (*rule image-cong*) (*simp-all add*: *group-own semi-factor-p-bot*)
then show *?thesis* **unfolding** *content-def* **by** (*simp add*: *image-constant-conv*)
qed

theorem *semilawful-key-at*: *semilawful* (*key-at c*)
proof (*rule semilawfulI*, *simp-all only*: *key-at-simps*)
show *entry-wf e* **if** *e* ∈ *set* [*mk-entry* (*sel c c*) *gtrue* (⊥, *c*)] **for** *e*
using *that* **by** (*simp add*: *entry-wf-view keyed-view-key supported-gtrue*)
show *c* = *fold* (⊔) (*map elower* [*mk-entry* (*sel c c*) *gtrue* (⊥, *c*)]) ⊥ **by** (*simp add*: *lattice-join-bot-right*)
show (*if group data c c F k* = {} *then None else Some* (*proj c·k*))
= (*if hd* (*contents data* [*mk-entry* (*sel c c*) *gtrue* (⊥, *c*)] *F k*) = *Map.empty then None*
else Some (*the-elem* (*dom* (*hd* (*contents data* [*mk-entry* (*sel c c*) *gtrue* (⊥, *c*)] *F k*))))) **for** *data F k*
by (*simp only*: *list.map list.sel content-key-at*) *simp*
qed

36.3 Table columns

A table column reads a section of the row. An external oracle names the key part that determines it in every conforming dataset; then the rows of a key's group carry one section, and the column's value is the column function of it. Its view is that section, keyed at the oracle's key part.

definition *column* :: *'r semi-factor* ⇒ *'k semi-factor* ⇒ (*'r* → *'w*::*cpo*) ⇒ (*'k*, *'f*, *'r*, *'w*) *expr* **where**
column c a v = (|*emind* = *a*, *edims* = [*mk-entry* (*sel a a*) *gtrue* (*c*, *a*)],
esem = λ*data F k*. *if group data a a F k* = {} *then None*
else Some (*THE w*. *w* ∈ (λ*y*. *v*·(*y* ⌊ *c*)) ‘ *group data a a F k*),
csem = λ*Xs*. *if hd Xs* = *Map.empty then None else Some* (*v*·(*the* (*hd Xs* (*the-elem* (*dom* (*hd Xs*)))))))|)

lemma *column-simps* [*simp*]:
emind (*column c a v*) = *a edims* (*column c a v*) = [*mk-entry* (*sel a a*) *gtrue* (*c*,

```
a)]
  esem (column c a v) data F k = (if group data a a F k = {} then None
      else Some (THE w. w ∈ (λy. v·(y ⌊ c)) ‘ group data a a F k))
  csem (column c a v) Xs = (if hd Xs = Map.empty then None else Some (v·(the
(hd Xs (the-elem (dom (hd Xs)))))))
  by (simp-all add: column-def)
```

The rows of a column's group carry one section.

```
lemma column-group-section:
  assumes fd: universally (λP. vfd P (⊥, a) (c, ⊥))
    and d: conforms data and x: x ∈ group data a a F k and y: y ∈ group data a
a F k
  shows y ⌊ c = x ⌊ c
proof −
  have y ⌊ fst (c, a) = x ⌊ fst (c, a)
  proof (rule view-agree[OF keyed-view-columns[OF fd] d])
    show x ∈ lower-maximal data y ∈ lower-maximal data using x y by (simp-all
add: group-def)
    show rproj a y = rproj a x using x y by (simp add: group-own)
  qed simp
  then show ?thesis by simp
qed

theorem semilawful-column:
  assumes fd: universally (λP. vfd P (⊥, a) (c, ⊥))
  shows semilawful (column c a v)
proof (rule semilawfulI, simp-all only: column-simps)
  show entry-wf e if e ∈ set [mk-entry (sel a a) gtrue (c, a)] for e
   using that by (simp add: entry-wf-view keyed-view-columns[OF fd] supported-gtrue)
  show a = fold (⊔) (map elower [mk-entry (sel a a) gtrue (c, a)]) ⊥
    by (simp add: lattice-join-bot-right)
  show (if group data a a F k = {} then None else Some (THE w. w ∈ (λy. v·(y
⌊ c)) ‘ group data a a F k))
      = (if hd (contents data [mk-entry (sel a a) gtrue (c, a)] F k) = Map.empty
then None
        else Some (v·(the (hd (contents data [mk-entry (sel a a) gtrue (c, a)] F k)
          (the-elem (dom (hd (contents data [mk-entry (sel a a) gtrue (c, a)] F
k)))))))))
    if d: conforms data for data F k
  proof (cases group data a a F k = {})
    case False
    then obtain x where x: x ∈ group data a a F k by blast
    have sec: (λy. y ⌊ c) ‘ group data a a F k = {x ⌊ c}
      using column-group-section[OF fd d x] x by blast
    have cgraph data (mk-entry (sel a a) gtrue (c, a)) F k = (λ-. (proj a·k, x ⌊ c))
‘ group data a a F k
      unfolding cgraph-def efiber-def cpair-def
      by (rule image-cong) (simp-all add: group-own column-group-section[OF fd d
x])
```

then have *C*: *content data* (*mk-entry* (*sel a a*) *gtrue* (*c*, *a*)) *F k* = [*proj a*·*k* ↦ *x* ⌊ *c*]
unfolding *content-def* **using** *False* **by** (*simp add*: *image-constant-conv*)
have *V*: (λ*y*. *v*·(*y* ⌊ *c*)) ‘ *group data a a F k* = {*v*·(*x* ⌊ *c*)}
using *sec* **by** (*simp only*: *image-image*[*of Rep-cfun v* λ*y*. *y* ⌊ *c*, *symmetric*, *unfolded comp-def*]) *simp*
show *?thesis* **using** *False* **by** (*simp only*: *list.map list.sel C V*) *simp*
next
case *True*
have *cgraph data* (*mk-entry* (*sel a a*) *gtrue* (*c*, *a*)) *F k* = {} **using** *True* **by** (*simp add*: *cgraph-def efiber-def*)
then show *?thesis* **using** *True* **by** (*simp add*: *content-def*)
qed
qed

36.4 Maps

definition *smap* :: (*'v*::*cpo* → *'w*::*cpo*) ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* ⇒ (*'k*, *'f*, *'r*, *'w*) *expr*
where
smap g E = ⦇*emind* = *emind E*, *edims* = *edims E*, *esem* = λ*data F k*. *map-option* (*Rep-cfun g*) (*esem E data F k*),
csem = λ*Xs*. *map-option* (*Rep-cfun g*) (*csem E Xs*)⦈

lemma *smap-simps* [*simp*]:
emind (*smap g E*) = *emind E edims* (*smap g E*) = *edims E*
esem (*smap g E*) *data F k* = *map-option* (*Rep-cfun g*) (*esem E data F k*)
csem (*smap g E*) *Xs* = *map-option* (*Rep-cfun g*) (*csem E Xs*)
by (*simp-all add*: *smap-def*)

theorem *semilawful-smap*: **assumes** *semilawful E* **shows** *semilawful* (*smap g E*)
proof −
have *ES*: *semilawful E* **by** (*fact assms*)
show *?thesis*
proof (*rule semilawfulI*, *simp-all only*: *smap-simps*)
show *entry-wf e* **if** *e* ∈ *set* (*edims E*) **for** *e* **by** (*rule semilawful-wf*[*OF ES that*])
show *emind E* = *fold* (⊔) (*map elower* (*edims E*)) ⊥ **by** (*rule semilawful-mind-lub*[*OF ES*])
show *map-option* (*Rep-cfun g*) (*esem E data F k*) = *map-option* (*Rep-cfun g*) (*csem E* (*contents data* (*edims E*) *F k*))
if *conforms data proj* (*emind E* ⊔ *flod F*)·*k* = *k* **for** *data F k*
by (*simp only*: *semilawful-csem*[*OF ES that*])
qed
qed

theorem *lawful-smap*: **assumes** *lawful E* **shows** *lawful* (*smap g E*)
proof −
have *EL*: *lawful E* **by** (*fact assms*)
show *?thesis*
proof (*rule lawful-semiI*[*OF semilawful-smap*[*OF lawful-semilawful*[*OF assms*]]],

simp-all only: *smap-simps*)
  show (*eupper e* ⊓ *emind E*) ⊑ *elower e* **if** *e* ∈ *set* (*edims E*) **for** *e*
    by (*rule lawful-saturated*[*OF EL that*])
  show *keyimg data F* (*emind E* ⊔ *flod F*) *k* **if** *d*: *conforms data* **and** *map-option* (*Rep-cfun g*) (*esem E data F k*) ≠ *None* **for** *data F k*
    using *that*(*2*) **by** (*intro lawful-key-law*[*OF EL d*]) *simp*
 qed
qed

A function of the key at an LOD.

definition *value-at* :: *'k semi-factor* ⇒ (*'k* → *'v*::*cpo*) ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* **where**
 value-at c g = *smap g* (*key-at c*)

lemma *value-at-simps* [*simp*]:
 emind (*value-at c g*) = *c edims* (*value-at c g*) = [*mk-entry* (*sel c c*) *gtrue* (⊥, *c*)]
 esem (*value-at c g*) *data F k* = (*if group data c c F k* = {} *then None else Some* (*g*·(*proj c*·*k*)))
 csem (*value-at c g*) *Xs* = (*if hd Xs* = *Map.empty then None else Some* (*g*·(*the-elem* (*dom* (*hd Xs*)))))
 by (*simp-all add*: *value-at-def*)

theorem *semilawful-value-at*: *semilawful* (*value-at c g*)
 unfolding *value-at-def* **by** (*rule semilawful-smap*[*OF semilawful-key-at*])

36.5 *RESTRICT*

Restricting conjoins the rows a test passes to the ambient filter: inner fibers are the outer fibers' rows passing it, so the test is conjoined to each entry's guard.

definition *restrict-entry* :: (*'r*, *'k*) *guard* ⇒ (*'k*, *'r*) *entry* ⇒ (*'k*, *'r*) *entry* **where**
 restrict-entry γ *e* = *e*(|*eguard* := *gand* γ (*eguard e*)|)

lemma *restrict-entry-simps* [*simp*]:
 esel (*restrict-entry* γ *e*) = *esel e*
 eguard (*restrict-entry* γ *e*) = *gand* γ (*eguard e*) *eview* (*restrict-entry* γ *e*) = *eview e*
 by (*simp-all add*: *restrict-entry-def*)

definition *restrict* :: (*'r*, *'k*) *guard* ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* **where**
 restrict γ *E* = (|*emind* = *emind E*, *edims* = *map* (*restrict-entry* γ) (*edims E*),
   esem = λ*data F*. *esem E data* (*frestrict* (*gsel* γ) *F*), *csem* = *csem E*|)

lemma *restrict-simps* [*simp*]:
 emind (*restrict* γ *E*) = *emind E*
 edims (*restrict* γ *E*) = *map* (*restrict-entry* γ) (*edims E*)
 esem (*restrict* γ *E*) *data F* = *esem E data* (*frestrict* (*gsel* γ) *F*)
 csem (*restrict* γ *E*) = *csem E*

by (*simp-all add*: *restrict-def*)

lemma *group-frestrict*:
group data a ϱ (*frestrict φ F*) *k* = {*y* ∈ *group data a ϱ F k*. *selects φ y*}
by (*auto simp*: *group-def*)

lemma *content-frestrict*:
content data e (*frestrict* (*gsel γ*) *F*) *k* = *content data* (*restrict-entry γ e*) *F k*
unfolding *content-def cgraph-def efiber-def cpair-def* **by** (*auto simp*: *group-frestrict*)

theorem *semilawful-restrict*:
assumes *g*: *supported γ* **and** *ES*: *semilawful E*
shows *semilawful* (*restrict γ E*)
proof (*rule semilawfulI*, *simp-all only*: *restrict-simps*)
show *entry-wf e* **if** *e* ∈ *set* (*map* (*restrict-entry γ*) (*edims E*)) **for** *e*
using *that semilawful-wf*[*OF ES*] *entry-wf-guard*[*OF - g*] **by** (*auto simp*: *restrict-entry-def*)
show *emind E* = *fold* (⊔) (*map elower* (*map* (*restrict-entry γ*) (*edims E*))) ⊥
by (*simp add*: *comp-def semilawful-mind-lub*[*OF ES*])
show *esem E data* (*frestrict* (*gsel γ*) *F*) *k* = *csem E* (*contents data* (*map* (*restrict-entry γ*) (*edims E*)) *F k*)
if *d*: *conforms data* **and** *k*: *proj* (*emind E* ⊔ *flod F*)·*k* = *k* **for** *data F k*
proof −
have *proj* (*emind E* ⊔ *flod* (*frestrict* (*gsel γ*) *F*))·*k* = *k* **using** *k* **by** *simp*
then show *?thesis* **by** (*simp add*: *semilawful-csem*[*OF ES d*] *content-frestrict comp-def*)
qed
qed

theorem *lawful-restrict*:
assumes *g*: *supported γ* **and** *EL*: *lawful E*
shows *lawful* (*restrict γ E*)
proof (*rule lawful-semiI*[*OF semilawful-restrict*[*OF g lawful-semilawful*[*OF EL*]]], *simp-all only*: *restrict-simps*)
show (*eupper e* ⊓ *emind E*) ⊑ *elower e* **if** *e* ∈ *set* (*map* (*restrict-entry γ*) (*edims E*)) **for** *e*
using *that* **by** (*auto intro*: *lawful-saturated*[*OF EL*])
show *keyimg data F* (*emind E* ⊔ *flod F*) *k* **if** *d*: *conforms data* **and** *esem E data* (*frestrict* (*gsel γ*) *F*) *k* ≠ *None* **for** *data F k*
proof −
have *keyimg data* (*frestrict* (*gsel γ*) *F*) (*emind E* ⊔ *flod F*) *k*
using *lawful-key-law*[*OF EL that*] **by** *simp*
then show *?thesis* **by** (*rule keyimg-mono*[*rotated*]) (*simp add*: *frows-frestrict*)
qed
qed

36.6 *FOR* and *FIXED*

E FOR τ G: the inner expression under the frame with LOD and slice truncated to τ and G conjoined, at the truncated key. Its upper parts meet τ; each entry's guard is conjoined with G coarsened to the entry's upper part. It is not gated: the result is broadcast over the dropped detail. *FIXED D* is *FOR D* without a filter.

definition *for-entry* :: *'k semi-factor* ⇒ *'f* ⇒ (*'k*, *'r*) *entry* ⇒ (*'k*, *'r*) *entry* **where**
for-entry τ G e = *e*(|*esel* := *sel* (*elower e*) (*eupper e* ⊓ *τ*), *eguard* := *gand* (*guard-holds* (*coarsen* (*eupper e*) *G*)) (*eguard e*)|)

lemma *for-entry-simps* [*simp*]:
elower (*for-entry τ G e*) = *elower e* *eupper* (*for-entry τ G e*) = *eupper e* ⊓ *τ*
eguard (*for-entry τ G e*) = *gand* (*guard-holds* (*coarsen* (*eupper e*) *G*)) (*eguard e*)
eview (*for-entry τ G e*) = *eview e*
by (*simp-all add*: *for-entry-def*)

definition *filtfor* :: *'k semi-factor* ⇒ *'f* ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* **where**
filtfor τ G E = (|*emind* = *emind E*, *edims* = *map* (*for-entry τ G*) (*edims E*),
esem = *λdata F k*. *esem E data* (*ffor τ G F*) (*proj* (*emind E* ⊔ *flod* (*ffor τ G F*))·*k*), *csem* = *csem E*|)

abbreviation *fixed* :: *'k semi-factor* ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* **where**
fixed D ≡ *filtfor D* **1**

lemma *filtfor-simps* [*simp*]:
emind (*filtfor τ G E*) = *emind E*
edims (*filtfor τ G E*) = *map* (*for-entry τ G*) (*edims E*)
esem (*filtfor τ G E*) *data F k* = *esem E data* (*ffor τ G F*) (*proj* (*emind E* ⊔ *flod* (*ffor τ G F*))·*k*)
csem (*filtfor τ G E*) = *csem E*
by (*simp-all add*: *filtfor-def*)

Inside the truncated frame, at the truncated key, a group is the outer group through the composite mask, cut down by the filter coarsened to the inner upper part.

lemma *group-ffor*:
assumes *am*: *a* ⊑ *m*
shows *group data a ϱ* (*ffor τ G F*) (*proj* (*m* ⊔ *flod* (*ffor τ G F*))·*k*)
= {*y* ∈ *group data a* (*ϱ* ⊓ *τ*) *F k*. *holds* (*coarsen ϱ G*) (*rkey*·*y*)}
proof −
let *?X* = *a* ⊔ ((*ϱ* ⊓ *τ*) ⊓ *flod F*)
have *lod*: *a* ⊔ (*ϱ* ⊓ *flod* (*ffor τ G F*)) = *?X* **by** (*simp add*: *truncate-inf-comp*)
have *le*: *?X* ⊑ *m* ⊔ *flod* (*ffor τ G F*)
unfolding *flod-ffor truncate-inf-comp* **by** (*rule lattice-join-mono*[*OF am truncate-below*])

have *key*: *proj ?X·(proj (m ⊔ flod (ffor τ G F))·k) = proj ?X·k* **by** (*rule proj-le*[*OF le*])
have *sl*: *holds* (*coarsen ϱ* (*slice* (*ffor τ G F*))) *k′* ⟷
holds (*coarsen* (*ϱ* ⊓ *τ*) (*slice F*)) *k′* ∧ *holds* (*coarsen ϱ G*) *k′* **for** *k′*
by (*simp add*: *coarsen-conj coarsen-coarsen*)
show *?thesis* **unfolding** *group-def lod key sl* **by** *auto*
qed

lemma *content-ffor*:
assumes *elower e* ⊑ *m*
shows *content data e* (*ffor τ G F*) (*proj* (*m* ⊔ *flod* (*ffor τ G F*))·*k*) = *content data* (*for-entry τ G e*) *F k*
unfolding *content-def cgraph-def efiber-def cpair-def group-ffor*[*OF assms*] **by** *auto*

lemma *content-const-top*:
assumes *proj* (*c* ⊔ *flod F*)·*k* = *k*
shows *content data* (*mk-entry* (*sel c lattice-top*) *gtrue* ⊥) *F k*
= (*if keyimg data F* (*c* ⊔ *flod F*) *k then* [⊥ ↦ ⊥] *else Map.empty*)
proof −
have *cgraph data* (*mk-entry* (*sel c lattice-top*) *gtrue* ⊥) *F k* = (λ-. (⊥, ⊥)) ‘ *group data c lattice-top F k*
unfolding *cgraph-def efiber-def cpair-def* **by** (*simp add*: *proj-bot semi-factor-p-bot*)
moreover have *gmap* {⊥ :: *′k* × *′r*} = [⊥ ↦ ⊥] **using** *gmap-singleton*[*of* ⊥::*′k* ⊥::*′r*] **by** *simp*
ultimately show *?thesis* **unfolding** *content-def* **by** (*simp add*: *image-constant-conv keyimg-group-top*[*OF assms*])
qed

theorem *semilawful-filtfor*: **assumes** *semilawful E* **shows** *semilawful* (*filtfor τ G E*)
proof −
have *ES*: *semilawful E* **by** (*fact assms*)
let *?m* = *emind E*
have *inner*: *contents data* (*edims E*) (*ffor τ G H*) (*proj* (*?m* ⊔ *flod* (*ffor τ G H*))·*κ*)
= *contents data* (*map* (*for-entry τ G*) (*edims E*)) *H κ* **for** *data H κ*
unfolding *map-map comp-def* **by** (*rule map-cong*[*OF refl*], *rule content-ffor*[*OF lower-below-mind*[*OF ES*]])
show *?thesis*
proof (*rule semilawfulI*, *simp-all only*: *filtfor-simps*)
show *entry-wf e* **if** *e* ∈ *set* (*map* (*for-entry τ G*) (*edims E*)) **for** *e*
using *that semilawful-wf*[*OF ES*]
by (*auto simp*: *entry-wf-def supported-gand supported-guard-holds*)
show *?m* = *fold* (⊔) (*map elower* (*map* (*for-entry τ G*) (*edims E*))) ⊥
by (*simp add*: *comp-def semilawful-mind-lub*[*OF ES*])
show *esem E data* (*ffor τ G F*) (*proj* (*?m* ⊔ *flod* (*ffor τ G F*))·*k*)
= *csem E* (*contents data* (*map* (*for-entry τ G*) (*edims E*)) *F k*)
if *d*: *conforms data* **for** *data F k*

by (*simp only*: *semilawful-csem*[*OF ES d proj-idem*] *inner*)
qed
qed

36.7 Including dimensions

Evaluating under the frame raised by M is the same as raising each entry's lower part by its masked part of M: the entry-level form of *finclude*.

definition *include-entry* :: $'k$ *semi-factor* $\Rightarrow$ ($'k$, $'r$) *entry* $\Rightarrow$ ($'k$, $'r$) *entry* **where**
include-entry M e = *e*(|*esel* := *sel* (*elower e* $\sqcup$ (*eupper e* $\sqcap$ *M*)) (*eupper e*)|)

lemma *include-entry-simps* [*simp*]:
elower (*include-entry M e*) = *elower e* $\sqcup$ (*eupper e* $\sqcap$ *M*) *eupper* (*include-entry M e*) = *eupper e*
eguard (*include-entry M e*) = *eguard e eview* (*include-entry M e*) = *eview e*
by (*simp-all add*: *include-entry-def*)

lemma *content-include-entry*: *content data* (*include-entry M e*) *F k* = *content data e* (*finclude M F*) *k*
proof −
have *lod*: (*elower e* $\sqcup$ (*eupper e* $\sqcap$ *M*)) $\sqcup$ (*eupper e* $\sqcap$ *flod F*) = *elower e* $\sqcup$ (*eupper e* $\sqcap$ *flod* (*finclude M F*))
by (*simp add*: *flod-finclude truncate-sup lattice-join-ac*)
have *group data* (*elower e* $\sqcup$ (*eupper e* $\sqcap$ *M*)) (*eupper e*) *F k* = *group data* (*elower e*) (*eupper e*) (*finclude M F*) *k*
by (*rule group-cong*) (*simp-all only*: *amb-finclude slice-finclude lod*)
then show *?thesis* **unfolding** *content-def cgraph-def efiber-def cpair-def* **by** *simp*
qed

lemma *entry-wf-include-entry*: *entry-wf e* $\Longrightarrow$ *entry-wf* (*include-entry M e*)
unfolding *entry-wf-def include-entry-simps* **.**

36.8 The gate

The gate turns a semilawful expression into a lawful one: it evaluates under the frame including the minimal LOD, which saturates the entries (their lower parts take in the masked minimal LOD), and it is present exactly at the frame's data keys, which a presence entry at the full key records.

definition *gate* :: ($'k$, $'f$, $'r$, $'v$) *expr* $\Rightarrow$ ($'k$, $'f$, $'r$, $'v$) *expr* **where**
gate E = (|*emind* = *emind E*,
edims = *mk-entry* (*sel* (*emind E*) *lattice-top*) *gtrue* $\bot$ # *map* (*include-entry* (*emind E*)) (*edims E*),
esem = λ*data F k*. *if keyimg data F* (*emind E* $\sqcup$ *flod F*) *k*
then esem E data (*finclude* (*emind E*) *F*) *k else None*,
csem = λ*Xs*. *if hd Xs* = *Map.empty then None else csem E* (*tl Xs*)|)

lemma *gate-simps* [*simp*]:
emind (*gate E*) = *emind E*

edims (*gate E*) = *mk-entry* (*sel* (*emind E*) *lattice-top*) *gtrue* ⊥ # *map* (*include-entry* (*emind E*)) (*edims E*)
esem (*gate E*) *data F k* = (*if keyimg data F* (*emind E* ⊔ *flod F*) *k*
then esem E data (*finclude* (*emind E*) *F*) *k else None*)
csem (*gate E*) *Xs* = (*if hd Xs* = *Map.empty then None else csem E* (*tl Xs*))
by (*simp-all add*: *gate-def*)

theorem *lawful-gate*: **assumes** *semilawful E* **shows** *lawful* (*gate E*)
proof −
have *ES*: *semilawful E* **by** (*fact assms*)
let *?m* = *emind E*
let *?h* = *mk-entry* (*sel ?m lattice-top*) *gtrue* ⊥ :: ('*k*, '*r*) *entry*
have *kd*: *?m* ⊔ *flod* (*finclude ?m F*) = *?m* ⊔ *flod F* **for** *F*
by (*simp add*: *flod-finclude lattice-join-ac*)
show *?thesis*
proof (*rule lawfulI*, *simp-all only*: *gate-simps*)
show *entry-wf e* **if** *e* ∈ *set* (*?h* # *map* (*include-entry ?m*) (*edims E*)) **for** *e*
using *that* **by** (*auto intro*: *entry-wf-view keyed-view-bot-view supported-gtrue entry-wf-include-entry semilawful-wf*[*OF ES*])
show (*eupper e* ⊓ *?m*) ⊑ *elower e* **if** *e* ∈ *set* (*?h* # *map* (*include-entry ?m*) (*edims E*)) **for** *e*
using *that* **by** (*auto simp*: *truncate-top intro*: *lattice-join-ge2*)
show *?m* = *fold* (⊔) (*map elower* (*?h* # *map* (*include-entry ?m*) (*edims E*))) ⊥
proof −
have *fold* (⊔) (*map elower* (*map* (*include-entry ?m*) (*edims E*))) *?m* = *?m*
by (*rule fold-join-absorb*)
(*auto intro*: *lattice-join-least*[*OF lower-below-mind*[*OF ES*] *truncate-below*])
then show *?thesis* **by** (*simp add*: *lattice-join-bot-right*)
qed
show *keyimg data F* (*?m* ⊔ *flod F*) *k*
if *conforms data* **and** (*if keyimg data F* (*?m* ⊔ *flod F*) *k then esem E data* (*finclude ?m F*) *k else None*) ≠ *None*
for *data F k*
using *that* **by** (*simp split*: *if-splits*)
show (*if keyimg data F* (*?m* ⊔ *flod F*) *k then esem E data* (*finclude ?m F*) *k else None*)
= (*if hd* (*contents data* (*?h* # *map* (*include-entry ?m*) (*edims E*)) *F k*) = *Map.empty then None*
else csem E (*tl* (*contents data* (*?h* # *map* (*include-entry ?m*) (*edims E*)) *F k*)))
if *d*: *conforms data* **and** *k*: *proj* (*?m* ⊔ *flod F*)·*k* = *k* **for** *data F k*
proof −
have *h*: *hd* (*contents data* (*?h* # *map* (*include-entry ?m*) (*edims E*)) *F k*) = *Map.empty*
⟷ ¬ *keyimg data F* (*?m* ⊔ *flod F*) *k*
by (*simp only*: *list.map list.sel content-const-top*[*OF k*]) *simp*
have *t*: *tl* (*contents data* (*?h* # *map* (*include-entry ?m*) (*edims E*)) *F k*)
= *contents data* (*edims E*) (*finclude ?m F*) *k*

```
      by (simp add: comp-def content-include-entry)
    have proj (?m ⊔ flod (finclude ?m F))·k = k by (simp only: kd k)
    then show ?thesis by (simp only: h t semilawful-csem[OF ES d]) simp
  qed
 qed
qed
```

36.9 Collect

The components are evaluated under the frame itself, each at its own key; their entries are the collected expression's entries, unchanged. Collecting is semilawful: it is not gated, and a component sees nothing of the others' minimal LODs. The gate widens it back to the full key.

Cutting a list into consecutive blocks of given lengths.

```
fun blocks :: nat list ⇒ 'a list ⇒ 'a list list where
  blocks [] xs = []
| blocks (n # ns) xs = take n xs # blocks ns (drop n xs)

lemma blocks-concat: blocks (map (λx. length (f x)) xs) (concat (map f xs)) = map f xs
  by (induct xs) simp-all

definition collect :: ('k, 'f, 'r, 'a::domain) expr list ⇒ ('k, 'f, 'r, 'a ltuple) expr where
  collect Es = (|emind = fold (⊔) (map emind Es) ⊥, edims = concat (map edims Es),
    esem = λdata F k. map-option ltuple-of (those (map (λE. esem E data F (proj (kd E F)·k)) Es)),
    csem = λXs. map-option ltuple-of (those (map (λ(E, Ys). csem E Ys) (zip Es (blocks (map (λE. length (edims E)) Es) Xs))))|)

lemma collect-simps [simp]:
  emind (collect Es) = fold (⊔) (map emind Es) ⊥
  edims (collect Es) = concat (map edims Es)
  esem (collect Es) data F k = map-option ltuple-of (those (map (λE. esem E data F (proj (kd E F)·k)) Es))
  csem (collect Es) Xs = map-option ltuple-of (those (map (λ(E, Ys). csem E Ys) (zip Es (blocks (map (λE. length (edims E)) Es) Xs))))
  by (simp-all add: collect-def)
```

The contents of a semilawful expression only see the key at its own key LOD.

```
lemma contents-own-key:
  assumes E: semilawful E
  shows contents data (edims E) F (proj (kd E F)·k) = contents data (edims E) F k
proof (rule map-cong[OF refl])
  fix e assume e: e ∈ set (edims E)
```

have *proj* (*elower e* ⊔ (*eupper e* ⊓ *flod F*))·(*proj* (*kd E F*)·*k*) = *proj* (*elower e* ⊔ (*eupper e* ⊓ *flod F*))·*k*
by (*rule proj-le*[*OF group-below-kd*[*OF E e*]])
then show *content data e F* (*proj* (*kd E F*)·*k*) = *content data e F k*
by (*intro content-group-cong*) (*simp add*: *group-def*)
qed

theorem *semilawful-collect*:
assumes *law*: ⋀*E*. *E* ∈ *set Es* ⟹ *semilawful E*
shows *semilawful* (*collect Es*)
proof (*rule semilawfulI*, *simp-all only*: *collect-simps*)
show *entry-wf e* **if** *e* ∈ *set* (*concat* (*map edims Es*)) **for** *e*
using *that* **by** (*auto intro*: *semilawful-wf*[*OF law*])
show *fold* (⊔) (*map emind Es*) ⊥ = *fold* (⊔) (*map elower* (*concat* (*map edims Es*))) ⊥
proof (*rule below-antisym*)
let *?R* = *fold* (⊔) (*map elower* (*concat* (*map edims Es*))) ⊥
show *fold* (⊔) (*map emind Es*) ⊥ ⊑ *?R*
proof (*rule fold-join-least*)
fix *x* **assume** *x* ∈ *set* (*map emind Es*)
then obtain *E* **where** *E*: *E* ∈ *set Es* **and** *x*: *x* = *emind E* **by** *auto*
show *x* ⊑ *?R* **unfolding** *x semilawful-mind-lub*[*OF law*[*OF E*]]
proof (*rule fold-join-least*)
fix *a* **assume** *a* ∈ *set* (*map elower* (*edims E*))
then show *a* ⊑ *?R* **by** (*intro fold-join-upper*) (*use E* **in** *force*)
qed *simp*
qed *simp*
show *?R* ⊑ *fold* (⊔) (*map emind Es*) ⊥
proof (*rule fold-join-least*)
fix *a* **assume** *a* ∈ *set* (*map elower* (*concat* (*map edims Es*)))
then obtain *E e* **where** *E*: *E* ∈ *set Es* **and** *e*: *e* ∈ *set* (*edims E*) **and** *a*: *a* = *elower e* **by** *auto*
show *a* ⊑ *fold* (⊔) (*map emind Es*) ⊥
unfolding *a* **by** (*rule below-trans*[*OF lower-below-mind*[*OF law*[*OF E*] *e*] *fold-join-upper*]) (*simp add*: *E*)
qed *simp*
qed
show *map-option ltuple-of* (*those* (*map* (λ*E*. *esem E data F* (*proj* (*kd E F*)·*k*)) *Es*))
= *map-option ltuple-of* (*those* (*map* (λ(*E*, *Ys*). *csem E Ys*)
(*zip Es* (*blocks* (*map* (λ*E*. *length* (*edims E*)) *Es*) (*contents data* (*concat* (*map edims Es*)) *F k*)))))
if *d*: *conforms data* **for** *data F k*
proof −
have *c*: *contents data* (*concat* (*map edims Es*)) *F k* = *concat* (*map* (λ*E*. *contents data* (*edims E*) *F k*) *Es*)
by (*simp add*: *map-concat comp-def*)
have *b*: *blocks* (*map* (λ*E*. *length* (*edims E*)) *Es*) (*contents data* (*concat* (*map edims Es*)) *F k*)

```
      = map (λE. contents data (edims E) F k) Es
    using blocks-concat[of λE. contents data (edims E) F k Es] by (simp only: c
length-map)
   have z: map (λ(E, Ys). csem E Ys) (zip Es (map (λE. contents data (edims
E) F k) Es))
      = map (λE. csem E (contents data (edims E) F k)) Es
    by (induct Es) simp-all
   have esem E data F (proj (kd E F)·k) = csem E (contents data (edims E) F
k) if E: E ∈ set Es for E
   by (simp only: semilawful-csem[OF law[OF E] d proj-idem] contents-own-key[OF
law[OF E]])
   then show ?thesis by (simp only: b z cong: map-cong)
 qed
qed

end

end
theory Key-Reduction
 imports Expr-Combinators
begin
```

37 Evaluating components at their own keys

Collecting evaluates every component under the frame raised by all the components' minimal LODs, at the full key. A component may instead be evaluated under the plain frame at its own key when raising the frame does not refine its groups. Raising by M refines the group of an entry with selector (a, ϱ) from $a \sqcup \varrho \sqcap L$ to $a \sqcup \varrho \sqcap (L \sqcup M)$. Two cases leave the content alone, as long as the full key is a data key: a row entry, whose coarse content has at most one element; and an entry whose coarse group LOD determines the masked part of M in every conforming dataset.

```
context dataspace
begin

definition reducible :: 'k semi-factor ⇒ ('k, 'f, 'r, 'v) expr ⇒ ('k semi-factor, 'f,
'r) frame ⇒ bool where
 reducible M E F ⟷ (∀ e∈set (edims E).
    row-entry e ∨ universally (λP. vfd P (⊥, elower e ⊔ (eupper e ⊓ flod F)) (⊥,
(eupper e ⊓ M))))

end

context dataspace
begin

lemma content-reduce:
```

assumes *ES*: *semilawful E* **and** *d*: *conforms data* **and** *M*: *emind E* $\sqsubseteq$ *M* **and** *e*: *e* $\in$ *set* (*edims E*)
and *red*: *row-entry e* $\vee$ *universally* (λ*P*. *vfd P* ($\bot$, *elower e* $\sqcup$ (*eupper e* $\sqcap$ *flod F*)) ($\bot$, (*eupper e* $\sqcap$ *M*)))
and *ki*: *keyimg data F* (*M* $\sqcup$ *flod F*) *k*
shows *content data e* (*finclude M F*) *k* = *content data e F* (*proj* (*kd E F*)·*k*)
proof −
let *?a* = *elower e* **and** *?ϱ* = *eupper e*
let *?Gc* = *?a* $\sqcup$ (*?ϱ* $\sqcap$ *flod F*) **and** *?T* = (*?ϱ* $\sqcap$ *M*)
let *?Gf* = *?a* $\sqcup$ (*?ϱ* $\sqcap$ (*flod F* $\sqcup$ *M*)) **and** *?W* = *M* $\sqcup$ *flod F*
have *Gf*: *?Gf* = *?Gc* $\sqcup$ *?T* **by** (*simp add*: *truncate-sup lattice-join-assoc*)
have *aM*: *?a* $\sqsubseteq$ *M* **by** (*rule below-trans*[*OF lower-below-mind*[*OF ES e*] *M*])
have *Gc-kd*: *?Gc* $\sqsubseteq$ *kd E F* **by** (*rule group-below-kd*[*OF ES e*])
have (*?ϱ* $\sqcap$ (*flod F* $\sqcup$ *M*)) $\sqsubseteq$ *?W*
using *truncate-below*[*of ?ϱ flod F* $\sqcup$ *M*] **by** (*simp only*: *lattice-join-commute*)
then have *Gf-W*: *?Gf* $\sqsubseteq$ *?W* **by** (*rule join-belowI*[*OF below-trans*[*OF aM lattice-join-ge1*]])
have *Gc-W*: *?Gc* $\sqsubseteq$ *?W*
by (*rule below-trans*[*OF - Gf-W*]) (*simp add*: *Gf lattice-join-ge1*)
obtain *x* **where** *x*: *x* $\in$ *lower-maximal data frows F x* **and** *kx*: *k* = *rproj ?W x*
using *ki* **by** (*auto simp*: *keyimg-def*)
have *pk*: *proj G*·*k* = *rproj G x* **if** *G* $\sqsubseteq$ *?W* **for** *G*
unfolding *kx* **by** (*rule proj-le*[*OF that*])
have *pkd*: *proj ?Gc*·(*proj* (*kd E F*)·*k*) = *proj ?Gc*·*k* **by** (*rule proj-le*[*OF Gc-kd*])
let *?P* = λ*y*. *y* $\in$ *lower-maximal data* $\wedge$ *selects* (*amb F*) *y* $\wedge$ *holds* (*coarsen ?ϱ* (*slice F*)) (*rkey*·*y*)
have *fine*: *group data ?a ?ϱ* (*finclude M F*) *k* = {*y*. *?P y* $\wedge$ *rproj ?Gf y* = *rproj ?Gf x*}
by (*auto simp*: *group-def flod-finclude pk*[*OF Gf-W*])
have *coarse*: *group data ?a ?ϱ F* (*proj* (*kd E F*)·*k*) = {*y*. *?P y* $\wedge$ *rproj ?Gc y* = *rproj ?Gc x*}
by (*auto simp*: *group-def pkd pk*[*OF Gc-W*])
have *fine-eq*: *rproj ?Gf y* = *rproj ?Gf x* $\longleftrightarrow$ *rproj ?Gc y* = *rproj ?Gc x* $\wedge$ *rproj ?T y* = *rproj ?T x* **for** *y*
unfolding *Gf* **by** (*rule rproj-sup-eq*)
have *Px*: *?P x* **using** *x* **by** (*auto simp*: *frows-def intro*: *holds-coarsen*)
have *det*: *determines* (*lower-maximal data*) *v g* **if** *universally* (λ*P*. *determines P v g*) **for** *v* **and** *g* :: *'r* $\to$ *'b*::*cpo*
using *universallyD*[*OF that*] *d* **.**
show *?thesis*
using *red*
proof
assume *row*: *row-entry e*
let *?g* = λ*y*. *ofun* (*eguard e*)·*y*
have *img*: *cpair e* ‘ {*y* $\in$ *Y*. *?g y* = *TT*} = (*if ?g x* = *TT then* {*cpair e x*} *else* {})
if *Y*: *Y* $\subseteq$ {*y*. *?P y* $\wedge$ *rproj ?Gc y* = *rproj ?Gc x*} **and** *xY*: *x* $\in$ *Y* **for** *Y*
by (*rule row-entry-image*[*OF semilawful-wf*[*OF ES e*] *row d - xY lattice-join-ge1*]) (*use Y* **in** *blast*)+

```
    have cgraph data e (finclude M F) k = cpair e ‘ {y ∈ {y. ?P y ∧ rproj ?Gf y
= rproj ?Gf x}. ?g y = TT}
      unfolding cgraph-def efiber-def fine ..
    also have ... = (if ?g x = TT then {cpair e x} else {})
      by (rule img) (use Px in ‹auto simp: fine-eq›)
    also have ... = cpair e ‘ {y ∈ {y. ?P y ∧ rproj ?Gc y = rproj ?Gc x}. ?g y =
TT}
      by (rule img[symmetric]) (use Px in auto)
    also have ... = cgraph data e F (proj (kd E F)·k)
      unfolding cgraph-def efiber-def coarse ..
    finally show ?thesis unfolding content-def by simp
  next
    assume fd: universally (λP. vfd P (⊥, ?Gc) (⊥, ?T))
    have rproj ?T y = rproj ?T x if y: ?P y rproj ?Gc y = rproj ?Gc x for y
    proof −
     have vpart (⊥, ?Gc) y = vpart (⊥, ?Gc) x using y(2) by (simp add: proj-bot)
      then have vpart (⊥, ?T) y = vpart (⊥, ?T) x
        using vfdD[OF det[OF fd]] y(1) x(1) by blast
      then show ?thesis by simp
    qed
    then have {y. ?P y ∧ rproj ?Gf y = rproj ?Gf x} = {y. ?P y ∧ rproj ?Gc y
= rproj ?Gc x}
      unfolding fine-eq by blast
    then show ?thesis by (intro content-group-cong) (simp only: fine coarse)
  qed
qed

theorem sem-reduce:
  assumes ES: semilawful E and d: conforms data and M: emind E ⊑ M and
red: reducible M E F
    and ki: keyimg data F (M ⊔ flod F) k
  shows esem E data (finclude M F) k = esem E data F (proj (kd E F)·k)
proof (rule semilawful-content-law[OF ES d d])
  have W: kd E (finclude M F) = M ⊔ flod F
  proof −
    have emind E ⊔ (flod F ⊔ M) = (emind E ⊔ M) ⊔ flod F by (simp only:
lattice-join-ac)
    then show ?thesis by (simp only: flod-finclude lattice-join-absorb2[OF M])
  qed
  obtain x where k = rproj (M ⊔ flod F) x using ki by (auto simp: keyimg-def)
  then show proj (kd E (finclude M F))·k = k unfolding W by simp
  show proj (kd E F)·(proj (kd E F)·k) = proj (kd E F)·k by simp
  show contents data (edims E) (finclude M F) k = contents data (edims E) F
(proj (kd E F)·k)
  proof (rule map-cong[OF refl])
    fix e assume e: e ∈ set (edims E)
    have r: row-entry e ∨ universally (λP. vfd P (⊥, elower e ⊔ (eupper e ⊓ flod
F)) (⊥, (eupper e ⊓ M)))
      using red e by (auto simp: reducible-def)
```

show *content data e* (*finclude M F*) *k* = *content data e F* (*proj* (*kd E F*)·*k*)
by (*rule content-reduce*[*OF ES d M e r ki*])
qed
qed

end

The gate without widening: under reducibility by its own minimal LOD, the gate evaluates the expression under the plain frame, at its own key.

context *dataspace*
begin

theorem *gate-reduce*:
assumes *E*: *semilawful E* **and** *d*: *conforms data* **and** *red*: *reducible* (*emind E*) *E F*
shows *esem* (*gate E*) *data F k* = (*if keyimg data F* (*emind E* ⊔ *flod F*) *k*
then esem E data F (*proj* (*emind E* ⊔ *flod F*)·*k*) *else None*)
proof −
have *ES*: *semilawful E* **by** (*fact E*)
show *?thesis* **using** *sem-reduce*[*OF ES d below-refl red*] **by** *simp*
qed

end

A collection is reducible when its components are; then *gate-reduce* evaluates the gated collection under the plain frame, each component at its own key.

context *dataspace*
begin

lemma *reducible-collect*: *reducible M* (*collect Es*) *F* ⟷ (∀ *E*∈*set Es*. *reducible M E F*)
unfolding *reducible-def* **by** *auto*

end

end
theory *Aggregate*
imports *Content-Law*
begin

38 Aggregation

An aggregate folds the entries of an inner expression over each fiber of the frame's LOD with a commutative semigroup operation. The fold adjoins a unit for the empty fiber, which gives no entry; it is not ⊥: a monotone operation with unit ⊥ would be inflationary ($x = x \oplus \bot \sqsubseteq x \oplus y$), which rules out sums on flat values.

definition *opt-op* :: (*'v*::*pcpo* → *'v* → *'v*) ⇒ *'v option* ⇒ *'v option* ⇒ *'v option*
where
opt-op f a b = (*case a of None* ⇒ *b* | *Some x* ⇒ *Some* (*case b of None* ⇒ *x* | *Some y* ⇒ *f*·*x*·*y*))

lemma *comm-monoid-opt-op*:
assumes *abel-semigroup* (λ*a b*. *f*·*a*·*b*)
shows *comm-monoid* (*opt-op f*) *None*
proof −
interpret *abel-semigroup* λ*a b*. *f*·*a*·*b* **by** (*fact assms*)
show *?thesis*
by *standard* (*auto simp*: *opt-op-def assoc commute left-commute split*: *option.splits*)
qed

definition *agg-fold* :: (*'v*::*pcpo* → *'v* → *'v*) ⇒ (*'a* ⇀ *'v*) ⇒ *'a set* ⇒ *'v option*
where
agg-fold f v A = *comm-monoid-set.F* (*opt-op f*) *None v A*

context
fixes *f* :: *'v*::*pcpo* → *'v* → *'v*
assumes *f*: *abel-semigroup* (λ*a b*. *f*·*a*·*b*)
begin

interpretation *A*: *comm-monoid-set opt-op f None*
unfolding *comm-monoid-set-def* **by** (*rule comm-monoid-opt-op*[*OF f*])

lemma *agg-fold-empty*: *agg-fold f v* {} = *None*
by (*simp add*: *agg-fold-def*)

lemma *agg-fold-singleton*: *agg-fold f v* {*a*} = *v a*
by (*simp add*: *agg-fold-def opt-op-def split*: *option.splits*)

lemma *agg-fold-nonempty*: *agg-fold f v A* ≠ *None* ⟹ *A* ≠ {}
by (*auto simp*: *agg-fold-empty*)

lemma *agg-fold-present*: *finite A* ⟹ *a* ∈ *A* ⟹ *v a* ≠ *None* ⟹ *agg-fold f v A* ≠ *None*
unfolding *agg-fold-def* **by** (*subst A.remove*) (*auto simp*: *opt-op-def*)

lemma *agg-fold-absent*: (⋀*a*. *a* ∈ *A* ⟹ *v a* = *None*) ⟹ *agg-fold f v A* = *None*
unfolding *agg-fold-def* **by** (*rule A.neutral*) *simp*

lemma *agg-fold-cong*: *A* = *B* ⟹ (⋀*a*. *a* ∈ *B* ⟹ *v a* = *w a*) ⟹ *agg-fold f v A* = *agg-fold f w B*
unfolding *agg-fold-def* **by** (*simp cong*: *A.cong*)

Folding over two images of one set of rows, identified alike, with the same entries.

lemma *agg-fold-image-cong*:
assumes $\bigwedge y\ z.\ y \in Z \Longrightarrow z \in Z \Longrightarrow g\ y = g\ z \longleftrightarrow g'\ y = g'\ z$
and $\bigwedge y.\ y \in Z \Longrightarrow v\ (g\ y) = w\ (g'\ y)$
shows *agg-fold f v* (*g* ‘ *Z*) = *agg-fold f w* (*g'* ‘ *Z*)
unfolding *agg-fold-def*
proof (*rule A.reindex-bij-witness*[**where** $i = \lambda b.\ g\ (SOME\ y.\ y \in Z \wedge g'\ y = b)$
and $j = \lambda a.\ g'\ (SOME\ y.\ y \in Z \wedge g\ y = a)$])
fix *a* **assume** *a* ∈ *g* ‘ *Z*
then obtain *y* **where** *y*: *y* ∈ *Z g y* = *a* **by** *blast*
let *?y* = *SOME y*. *y* ∈ *Z* ∧ *g y* = *a*
have *y'*: *?y* ∈ *Z g ?y* = *a* **using** *someI*[*of* λ*y*. *y* ∈ *Z* ∧ *g y* = *a*, *OF conjI*[*OF y*]] **by** *simp-all*
let *?z* = *SOME z*. *z* ∈ *Z* ∧ *g'* *z* = *g'* *?y*
have *z*: *?z* ∈ *Z g'* *?z* = *g'* *?y* **using** *someI*[*of* λ*z*. *z* ∈ *Z* ∧ *g'* *z* = *g'* *?y* *?y*] *y'*(*1*) **by** *simp-all*
show *g ?z* = *a* **using** *assms*(*1*)[*OF z*(*1*) *y'*(*1*)] *z*(*2*) *y'*(*2*) **by** *simp*
show *g'* *?y* ∈ *g'* ‘ *Z* **using** *y'*(*1*) **by** *blast*
show *w* (*g'* *?y*) = *v a* **using** *assms*(*2*)[*OF y'*(*1*)] *y'*(*2*) **by** *simp*
next
fix *b* **assume** *b* ∈ *g'* ‘ *Z*
then obtain *y* **where** *y*: *y* ∈ *Z g'* *y* = *b* **by** *blast*
let *?y* = *SOME y*. *y* ∈ *Z* ∧ *g'* *y* = *b*
have *y'*: *?y* ∈ *Z g'* *?y* = *b* **using** *someI*[*of* λ*y*. *y* ∈ *Z* ∧ *g'* *y* = *b*, *OF conjI*[*OF y*]] **by** *simp-all*
let *?z* = *SOME z*. *z* ∈ *Z* ∧ *g z* = *g ?y*
have *z*: *?z* ∈ *Z g ?z* = *g ?y* **using** *someI*[*of* λ*z*. *z* ∈ *Z* ∧ *g z* = *g ?y ?y*] *y'*(*1*) **by** *simp-all*
show *g'* *?z* = *b* **using** *assms*(*1*)[*OF z*(*1*) *y'*(*1*)] *z*(*2*) *y'*(*2*) **by** *simp*
show *g ?y* ∈ *g* ‘ *Z* **using** *y'*(*1*) **by** *blast*
qed

end

lemma *agg-fold-bij*:
assumes *f*: *abel-semigroup* (λ*a b*. *f*·*a*·*b*) **and** *h*: *bij-betw h A B*
shows *agg-fold f* (*v* ∘ *h*) *A* = *agg-fold f v B*
proof −
interpret *A*: *comm-monoid-set opt-op f None*
unfolding *comm-monoid-set-def* **by** (*rule comm-monoid-opt-op*[*OF f*])
show *?thesis* **unfolding** *agg-fold-def comp-def* **by** (*rule A.reindex-bij-betw*[*OF h*])
qed

context *dataspace*
begin

The rows of a frame in the fiber of a key at an LOD.

definition *afiber* :: *'r lower-pd* ⇒ (*'k semi-factor*, *'f*, *'r*) *frame* ⇒ *'k semi-factor* ⇒ *'k* ⇒ *'r set* **where**

afiber data F c k = {*y* ∈ *lower-maximal data*. *frows F y* ∧ *rproj c y* = *k*}

aggregate R f E folds the inner values over the inner keys below each key of the frame, the inner expression being evaluated under the frame refined by the extra requested dimensions *R* (a grouping inside the aggregate). The inner keys are those where the inner expression is present: its own domain, so for a lawful inner expression they are data keys. Its keys are the frame's, so it decides no detail: the lower parts of the inner entries move into the views' key parts, and the fold reads the fiber's classes at *emind E* ⊔ *R*.

definition *inner-keys* :: *'k semi-factor* ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* ⇒ *'r lower-pd* ⇒ (*'k semi-factor*, *'f*, *'r*) *frame* ⇒ *'k* ⇒ *'k set* **where**
inner-keys R E data F k = {*κ*. *proj* (*kd E* (*finclude R F*))·*κ* = *κ* ∧ *proj* (*flod F*)·*κ* = *k* ∧
esem E data (*finclude R F*) *κ* ≠ *None*}

For a lawful inner expression at a conforming dataset, its keys below *k* are the defined ones among the inner keys of the fiber of *k*.

lemma *inner-keys-fiber*:
assumes *EL*: *lawful E* **and** *d*: *conforms data*
shows *inner-keys R E data F k* = {*κ* ∈ *rproj* (*kd E* (*finclude R F*)) ‘ *afiber data F* (*flod F*) *k*.
esem E data (*finclude R F*) *κ* ≠ *None*}
proof −
let *?X* = *kd E* (*finclude R F*)
have *le*: *flod F* ⊑ *?X* **by** (*simp add*: *flod-finclude below-trans*[*OF lattice-join-ge1 lattice-join-ge2*])
show *?thesis*
proof (*rule set-eqI*, *rule iffI*)
fix *κ* **assume** *κ* ∈ *inner-keys R E data F k*
then have *kfix*: *proj ?X*·*κ* = *κ* **and** *below*: *proj* (*flod F*)·*κ* = *k* **and** *kdef*: *esem E data* (*finclude R F*) *κ* ≠ *None*
by (*simp-all add*: *inner-keys-def*)
obtain *x* **where** *x*: *x* ∈ *lower-maximal data frows F x* **and** *kx*: *κ* = *rproj ?X x*
using *lawful-key-law*[*OF EL d kdef*] **by** (*auto simp*: *keyimg-def*)
have *rproj* (*flod F*) *x* = *k* **using** *below* **unfolding** *kx* **by** (*simp only*: *proj-le*[*OF le*])
then show *κ* ∈ {*κ* ∈ *rproj ?X* ‘ *afiber data F* (*flod F*) *k*. *esem E data* (*finclude R F*) *κ* ≠ *None*}
using *x kx kdef* **by** (*auto simp*: *afiber-def*)
next
fix *κ* **assume** *κ* ∈ {*κ* ∈ *rproj ?X* ‘ *afiber data F* (*flod F*) *k*. *esem E data* (*finclude R F*) *κ* ≠ *None*}
then obtain *x* **where** *x*: *x* ∈ *afiber data F* (*flod F*) *k* **and** *kx*: *κ* = *rproj ?X x* **and** *kdef*: *esem E data* (*finclude R F*) *κ* ≠ *None* **by** *blast*
have *rproj* (*flod F*) *x* = *k* **using** *x* **by** (*simp add*: *afiber-def*)
then have *proj* (*flod F*)·*κ* = *k* **unfolding** *kx* **by** (*simp only*: *proj-le*[*OF le*])
then show *κ* ∈ *inner-keys R E data F k* **using** *kdef* **by** (*simp add*: *inner-keys-def kx*)

qed
qed

An inner entry, seen from outside the aggregate: its lower part and its requested part move into its view's key part, so the outer content is keyed by the inner keys.

definition *agg-entry* :: *'k semi-factor* ⇒ (*'k*, *'r*) *entry* ⇒ (*'k*, *'r*) *entry* **where**
agg-entry R e = *e*(|*esel* := *sel* ⊥ (*eupper e*), *eview* := (*ecols e*, *ekeys e* ⊔ (*elower e* ⊔ (*eupper e* ⊓ *R*)))|)

lemma *agg-entry-simps* [*simp*]:
elower (*agg-entry R e*) = ⊥ *eupper* (*agg-entry R e*) = *eupper e*
eguard (*agg-entry R e*) = *eguard e eview* (*agg-entry R e*) = (*ecols e*, *ekeys e* ⊔ (*elower e* ⊔ (*eupper e* ⊓ *R*)))
by (*simp-all add*: *agg-entry-def*)

lemma *keyed-view-raise*: *keyed-view* (*c*, *a*) ⟹ *keyed-view* (*c*, *a* ⊔ *W*)
proof −
assume *keyed-view* (*c*, *a*)
then have *keyed-view* ((*c*, *a*) ⊔ (⊥, *W*)) **by** (*rule keyed-view-sup*[*OF - keyed-view-key*])
then show *?thesis* **by** (*simp only*: *lattice-join-prod fst-conv snd-conv lattice-join-bot-right*)
qed

lemma *entry-wf-agg-entry*: *entry-wf e* ⟹ *entry-wf* (*agg-entry R e*)
unfolding *entry-wf-def agg-entry-simps* **using** *keyed-view-raise*[*of ecols e ekeys e*] **by** *simp*

The content semantics folds the inner content semantics over the fold entry's keys, reading each inner entry's content at a key off the outer content: the pairs whose key has the inner key's part at the entry's lower and requested parts, cut back to the inner key part.

definition *agg-read* :: *'k semi-factor* ⇒ (*'k*, *'r*) *entry* ⇒ *'k* ⇒ (*'k* × *'r*) *set* ⇒ (*'k* × *'r*) *set* **where**
agg-read R e p X = (λ*x*. (*proj* (*ekeys e*)·(*fst x*), *snd x*)) ‘
{*x* ∈ *X*. *proj* (*elower e* ⊔ (*eupper e* ⊓ *R*))·(*fst x*) = *proj* (*elower e* ⊔ (*eupper e* ⊓ *R*))·*p*}

On maps the read picks, at each inner key, the one outer key of the slice that projects to it, and drops the inner key when there is none or more than one. Which keys collide is decided by the domain alone, so the read is monotone on all maps; on contents nothing collides.

definition *agg-src* :: *'k semi-factor* ⇒ (*'k*, *'r*) *entry* ⇒ *'k* ⇒ (*'k* ⇒ *'r option*) ⇒ *'k* ⇒ *'k* ⇒ *bool* **where**
agg-src R e p X q q0 ⟷ *q0* ∈ *dom X* ∧
proj (*elower e* ⊔ (*eupper e* ⊓ *R*))·*q0* = *proj* (*elower e* ⊔ (*eupper e* ⊓ *R*))·*p* ∧ *proj* (*ekeys e*)·*q0* = *q*

definition *agg-pick* :: *'k semi-factor* ⇒ (*'k*, *'r*) *entry* ⇒ *'k* ⇒ (*'k* ⇒ *'r option*) ⇒ *'k* ⇒ *'r option* **where**
agg-pick R e p X q = (*if* ∃!*q0*. *agg-src R e p X q q0 then X* (*THE q0*. *agg-src R e p X q q0*) *else None*)

definition *agg-inner* :: *'k semi-factor* ⇒ (*'k*, *'r*) *entry list* ⇒ (*'k* ⇒ *'r option*) *list* ⇒ *'k* ⇒ (*'k* ⇒ *'r option*) *list* **where**
agg-inner R es Xs p = *map2* (λ*e X*. *agg-pick R e p X*) *es Xs*

A graph is *shaped* for an entry when it is single-valued with keys at the entry's key part. Outer keys at the outer key part are determined by their parts at the inner key part and at the lower and requested parts, so on shaped outer graphs the pick agrees with the graph read.

definition *gshaped* :: (*'k*, *'r*) *entry* ⇒ (*'k* × *'r*) *set* ⇒ *bool* **where**
gshaped e X ⟷ *single-valued X* ∧ (∀ *x*∈*X*. *proj* (*ekeys e*)·(*fst x*) = *fst x*)

lemma *cgraph-gshaped*: *entry-wf e* ⟹ *conforms data* ⟹ *gshaped e* (*cgraph data e F k*)
unfolding *gshaped-def* **using** *cgraph-single-valued*[*of e data F k*] **by** (*auto simp*: *cgraph-def cpair-def*)

lemma *agg-read-key-eq*:
assumes *X*: *gshaped* (*agg-entry R e*) *X* **and** *Y*: *gshaped* (*agg-entry R e*) *Y*
and *q*: (*q*, *v*) ∈ *X* **and** *q'*: (*q'*, *w*) ∈ *Y*
and *A*: *proj* (*elower e* ⊔ (*eupper e* ⊓ *R*))·*q* = *proj* (*elower e* ⊔ (*eupper e* ⊓ *R*))·*q'*
and *s*: *proj* (*ekeys e*)·*q* = *proj* (*ekeys e*)·*q'*
shows *q* = *q'*
proof −
let *?S* = *ekeys e* ⊔ (*elower e* ⊔ (*eupper e* ⊓ *R*))
have *q* = *proj ?S*·*q q'* = *proj ?S*·*q'* **using** *X Y q q'* **by** (*force simp*: *gshaped-def*)+
moreover have *proj ?S*·*q* = *proj ?S*·*q'* **using** *A s* **by** (*simp only*: *proj-sup-eq*)
ultimately show *?thesis* **by** *simp*
qed

lemma *agg-read-single-valued*:
assumes *X*: *gshaped* (*agg-entry R e*) *X*
shows *single-valued* (*agg-read R e p X*)
proof (*rule single-valuedI*)
fix *q u u'* **assume** *u*: (*q*, *u*) ∈ *agg-read R e p X* **and** *u'*: (*q*, *u'*) ∈ *agg-read R e p X*
obtain *q1* **where** *1*: (*q1*, *u*) ∈ *X q* = *proj* (*ekeys e*)·*q1*
proj (*elower e* ⊔ (*eupper e* ⊓ *R*))·*q1* = *proj* (*elower e* ⊔ (*eupper e* ⊓ *R*))·*p*
using *u* **unfolding** *agg-read-def* **by** *force*
obtain *q2* **where** *2*: (*q2*, *u'*) ∈ *X q* = *proj* (*ekeys e*)·*q2*
proj (*elower e* ⊔ (*eupper e* ⊓ *R*))·*q2* = *proj* (*elower e* ⊔ (*eupper e* ⊓ *R*))·*p*
using *u'* **unfolding** *agg-read-def* **by** *force*
have *q1* = *q2* **by** (*rule agg-read-key-eq*[*OF X X 1*(*1*) *2*(*1*)]) (*use 1*(*2*,*3*) *2*(*2*,*3*) **in** *simp-all*)

then show $u = u'$ **using** *single-valuedD*[*of X*] *X 1*(*1*) *2*(*1*) **by** (*simp add*: *gshaped-def*)
qed

lemma *agg-pick-graph*:
assumes *X*: *gshaped* (*agg-entry R e*) (*graph X*)
shows *agg-pick R e p X q* = *gmap* (*agg-read R e p* (*graph X*)) *q*
proof −
let *?A* = *elower e* ⊔ (*eupper e* ⊓ *R*) **and** *?P* = *agg-src R e p X q*
have *uniq*: *q1* = *q2* **if** *1*: *?P q1* **and** *2*: *?P q2* **for** *q1 q2*
proof −
obtain *v w* **where** *v*: *X q1* = *Some v* **and** *w*: *X q2* = *Some w* **using** *1 2* **by** (*auto simp*: *agg-src-def*)
show *?thesis*
by (*rule agg-read-key-eq*[*OF X X*, *of q1 v q2 w*]) (*use 1 2 v w* **in** ‹*simp-all add*: *graph-def agg-src-def*›)
qed
show *?thesis*
proof (*cases* ∃ *q0*. *?P q0*)
case *True*
then obtain *q0* **where** *q0*: *?P q0* **by** *blast*
then obtain *v* **where** *v*: *X q0* = *Some v* **by** (*auto simp*: *agg-src-def*)
have *ex1*: ∃!*q0*. *?P q0* **using** *q0 uniq* **by** *blast*
have *th*: (*THE q0*. *?P q0*) = *q0* **using** *q0 uniq* **by** *blast*
have (*q*, *v*) ∈ *agg-read R e p* (*graph X*)
unfolding *agg-read-def* **using** *q0 v* **by** (*force simp*: *graph-def agg-src-def intro*: *rev-image-eqI*)
then have *gmap* (*agg-read R e p* (*graph X*)) *q* = *Some v* **by** (*rule gmap-apply*[*OF agg-read-single-valued*[*OF X*]])
then show *?thesis* **unfolding** *agg-pick-def* **using** *ex1 th v* **by** *simp*
next
case *False*
then have *q* ∉ *fst* ‘ *agg-read R e p* (*graph X*) **unfolding** *agg-read-def* **by** (*force simp*: *graph-def agg-src-def*)
then show *?thesis* **unfolding** *agg-pick-def gmap-def* **using** *False* **by** *auto*
qed
qed

definition *agg-csem* :: *'k semi-factor* ⇒ (*'v*::*pcpo* → *'v* → *'v*) ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* ⇒ (*'k* ⇒ *'r option*) *list* ⇒ *'v option* **where**
agg-csem R f E Xs = *agg-fold f* (λ*p*. *csem E* (*agg-inner R* (*edims E*) (*tl Xs*) *p*))
{*p* ∈ *dom* (*hd Xs*). *csem E* (*agg-inner R* (*edims E*) (*tl Xs*) *p*) ≠ *None*}

lemma *map2-map*: *map2 g xs* (*map h xs*) = *map* (λ*x*. *g x* (*h x*)) *xs*
by (*induct xs*) *simp-all*

definition *aggregate* :: *'k semi-factor* ⇒ (*'v*::*pcpo* → *'v* → *'v*) ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* ⇒ (*'k*, *'f*, *'r*, *'v*) *expr* **where**
aggregate R f E = (|*emind* = ⊥,

edims = *mk-entry* (*sel* ⊥ *lattice-top*) *gtrue* (⊥, *emind E* ⊔ *R*) # *map* (*agg-entry R*) (*edims E*),
esem = λ*data F k*. *agg-fold f* (*esem E data* (*finclude R F*)) (*inner-keys R E data F k*),
csem = *agg-csem R f E*⦈

lemma *aggregate-simps* [*simp*]:
emind (*aggregate R f E*) = ⊥
edims (*aggregate R f E*) = *mk-entry* (*sel* ⊥ *lattice-top*) *gtrue* (⊥, *emind E* ⊔ *R*) # *map* (*agg-entry R*) (*edims E*)
esem (*aggregate R f E*) *data F k* = *agg-fold f* (*esem E data* (*finclude R F*)) (*inner-keys R E data F k*)
csem (*aggregate R f E*) = *agg-csem R f E*
by (*simp-all add*: *aggregate-def*)

The fold entry reads the fiber.

lemma *agg-fiber*: *proj* (*flod F*)·*k* = *k* ⟹ *group data* ⊥ *lattice-top F k* = *afiber data F* (*flod F*) *k*
by (*simp add*: *group-top*[*of* ⊥, *simplified lattice-join-bot-left*] *afiber-def lattice-join-bot-left*)

An inner group is the outer group through the same upper part, cut down by the inner key's lower part and requested part.

lemma *agg-group*:
assumes *y*: *rproj* (*flod F*) *y* = *k* **and** *am*: *a* ⊑ *m*
shows *group data a* ϱ (*finclude R F*) (*rproj* (*m* ⊔ *flod* (*finclude R F*)) *y*)
= {*y*′ ∈ *group data* ⊥ ϱ *F k*. *rproj* (*a* ⊔ (ϱ ⊓ *R*)) *y*′ = *rproj* (*a* ⊔ (ϱ ⊓ *R*)) *y*}
proof −
let *?X* = *m* ⊔ *flod* (*finclude R F*) **and** *?A* = *a* ⊔ (ϱ ⊓ *R*) **and** *?L* = (ϱ ⊓ *flod F*)
have *lod*: *a* ⊔ (ϱ ⊓ *flod* (*finclude R F*)) = *?A* ⊔ *?L* **by** (*simp add*: *flod-finclude truncate-sup lattice-join-ac*)
have *mR*: *m* ⊔ *R* ⊑ *?X* **by** (*simp add*: *flod-finclude lattice-join-mono lattice-join-ge2*)
have *AX*: *?A* ⊑ *?X* **by** (*rule below-trans*[*OF lattice-join-mono*[*OF am truncate-below*] *mR*])
have *LX*: *?L* ⊑ *?X*
by (*rule below-trans*[*OF truncate-below*]) (*simp add*: *flod-finclude below-trans*[*OF lattice-join-ge1 lattice-join-ge2*])
have *kA*: *proj ?A*·(*rproj ?X y*) = *rproj ?A y* **by** (*rule proj-le*[*OF AX*])
have *kL*: *proj ?L*·(*rproj ?X y*) = *proj ?L*·*k*
proof −
have *proj ?L*·(*rproj ?X y*) = *rproj ?L y* **by** (*rule proj-le*[*OF LX*])
also have ... = *proj ?L*·(*rproj* (*flod F*) *y*) **by** (*rule proj-le*[*OF truncate-below*, *symmetric*])
finally show *?thesis* **by** (*simp only*: *y*)
qed
show *?thesis* **unfolding** *group-def lod proj-sup-eq kA kL* **by** (*auto simp*: *lattice-join-bot-left proj-bot*)
qed

Inner keys of a fiber correspond to their parts at $m \sqcup R$.

lemma *agg-keys-bij*:
bij-betw (*Rep-cfun* (*proj* ($m \sqcup R$))) (*rproj* ($m \sqcup$ *flod* (*finclude R F*)) ‘ *afiber data F* (*flod F*) *k*)
(*rproj* ($m \sqcup R$) ‘ *afiber data F* (*flod F*) *k*)
proof −
let *?X* = $m \sqcup$ *flod* (*finclude R F*) **and** *?Z* = *afiber data F* (*flod F*) *k*
have *le*: $m \sqcup R \sqsubseteq$ *?X* **by** (*simp add*: *flod-finclude lattice-join-mono lattice-join-ge2*)
have *pr*: *proj* ($m \sqcup R$)·(*rproj ?X y*) = *rproj* ($m \sqcup R$) *y* **for** *y* **by** (*rule proj-le*[*OF le*])
show *?thesis*
proof (*rule bij-betw-imageI*)
show *inj-on* (*Rep-cfun* (*proj* ($m \sqcup R$))) (*rproj ?X* ‘ *?Z*)
proof (*rule inj-onI*)
fix $\kappa 1$ $\kappa 2$ **assume** *k1*: $\kappa 1 \in$ *rproj ?X* ‘ *?Z* **and** *k2*: $\kappa 2 \in$ *rproj ?X* ‘ *?Z*
and *e*: *proj* ($m \sqcup R$)·$\kappa 1$ = *proj* ($m \sqcup R$)·$\kappa 2$
obtain *y1* **where** *y1*: *y1* ∈ *?Z* $\kappa 1$ = *rproj ?X y1* **using** *k1* **by** *blast*
obtain *y2* **where** *y2*: *y2* ∈ *?Z* $\kappa 2$ = *rproj ?X y2* **using** *k2* **by** *blast*
have *rproj* ($m \sqcup R$) *y1* = *rproj* ($m \sqcup R$) *y2* **using** *e* **unfolding** *y1*(*2*) *y2*(*2*) *pr* **.**
moreover have *rproj* (*flod F*) *y1* = *rproj* (*flod F*) *y2* **using** *y1*(*1*) *y2*(*1*) **by** (*simp add*: *afiber-def*)
ultimately have *rproj* (($m \sqcup R$) $\sqcup$ *flod F*) *y1* = *rproj* (($m \sqcup R$) $\sqcup$ *flod F*) *y2*
by (*simp only*: *rproj-sup-eq*)
moreover have ($m \sqcup R$) $\sqcup$ *flod F* = *?X* **by** (*simp add*: *flod-finclude lattice-join-ac*)
ultimately show $\kappa 1 = \kappa 2$ **by** (*simp add*: *y1*(*2*) *y2*(*2*))
qed
show *Rep-cfun* (*proj* ($m \sqcup R$)) ‘ *rproj ?X* ‘ *?Z* = *rproj* ($m \sqcup R$) ‘ *?Z*
by (*simp only*: *image-image pr*)
qed
qed

The content graph of an inner entry at an inner key is read off the outer entry's: the pairs whose key has the inner key's part at the entry's lower and requested parts.

lemma *agg-content-inner*:
assumes *ES*: *semilawful E* **and** *e*: *e* ∈ *set* (*edims E*) **and** *yZ*: *y* ∈ *afiber data G* (*flod G*) *kG*
and κ: κ = *rproj* (*emind E* $\sqcup$ *flod* (*finclude R G*)) *y* **and** *py*: *proj* (*emind E* $\sqcup$ *R*)·κ = *p*
shows *cgraph data e* (*finclude R G*) κ = *agg-read R e p* (*cgraph data* (*agg-entry R e*) *G kG*)
proof −
let *?m* = *emind E* **and** *?a* = *elower e* **and** *?ϱ* = *eupper e* **and** *?s* = *ekeys e*
have *am*: *?a* $\sqsubseteq$ *?m* **by** (*rule lower-below-mind*[*OF ES e*])
let *?A* = *?a* $\sqcup$ (*?ϱ* $\sqcap$ *R*)
have *AmR*: *?A* $\sqsubseteq$ *?m* $\sqcup$ *R* **by** (*rule lattice-join-mono*[*OF am truncate-below*])

have *mR*: *?m* ⊔ *R* ⊑ *?m* ⊔ *flod* (*finclude R G*) **by** (*simp add*: *flod-finclude lattice-join-mono lattice-join-ge2*)
have *yA*: *rproj ?A y* = *proj ?A*·*p*
proof −
have *proj ?A*·*p* = *proj ?A*·*κ* **unfolding** *py*[*symmetric*] **by** (*rule proj-le*[*OF AmR*])
also have ... = *rproj ?A y* **unfolding** *κ* **by** (*rule proj-le*[*OF below-trans*[*OF AmR mR*]])
finally show *?thesis* **by** *simp*
qed
have *gy*: *rproj* (*flod G*) *y* = *kG* **using** *yZ* **by** (*simp add*: *afiber-def*)
have *grp*: *group data ?a ?ϱ* (*finclude R G*) *κ* = {*y*′ ∈ *group data* ⊥ *?ϱ G kG*. *rproj ?A y*′ = *proj ?A*·*p*}
unfolding *κ agg-group*[*OF gy am*] *yA* **..**
have *pA*: *proj ?A*·(*rproj* (*?s* ⊔ *?A*) *y*′) = *rproj ?A y*′ **for** *y*′ **by** (*rule proj-le*[*OF lattice-join-ge2*])
have *ps*: *proj ?s*·(*rproj* (*?s* ⊔ *?A*) *y*′) = *rproj ?s y*′ **for** *y*′ **by** (*rule proj-le*[*OF lattice-join-ge1*])
let *?h* = *λy*′. (*rproj* (*?s* ⊔ *?A*) *y*′, *y*′ ⌊ *ecols e*)
let *?Y* = *efiber data* (*agg-entry R e*) *G kG*
have {*x* ∈ *?h* ‘ *?Y*. *proj ?A*·(*fst x*) = *proj ?A*·*p*} = *?h* ‘ {*y*′ ∈ *?Y*. *rproj ?A y*′ = *proj ?A*·*p*}
by (*auto simp*: *pA*)
then have *agg-read R e p* (*cgraph data* (*agg-entry R e*) *G kG*)
= *cpair e* ‘ {*y*′ ∈ *?Y*. *rproj ?A y*′ = *proj ?A*·*p*}
unfolding *agg-read-def cgraph-def cpair-def agg-entry-simps fst-conv snd-conv* **by** (*simp add*: *image-image ps*)
moreover have {*y*′ ∈ *?Y*. *rproj ?A y*′ = *proj ?A*·*p*} = *efiber data e* (*finclude R G*) *κ*
unfolding *efiber-def grp* **by** *auto*
ultimately show *?thesis* **unfolding** *cgraph-def* **by** *simp*
qed

theorem *lawful-aggregate*:
assumes *EL*: *lawful E* **and** *f*: *abel-semigroup* (*λa b*. *f*·*a*·*b*)
shows *lawful* (*aggregate R f E*)
proof −
have *ES*: *semilawful E* **by** (*rule lawful-semilawful*[*OF EL*])
let *?m* = *emind E*
let *?h* = *mk-entry* (*sel* ⊥ *lattice-top*) *gtrue* (⊥, *?m* ⊔ *R*) :: (′*k*, ′*r*) *entry*
have *bj*: ⊥ ⊔ *X* = *X* **for** *X* :: ′*k semi-factor* **by** (*rule lattice-join-bot-left*)
note *fiber* = *agg-fiber*
note *bij* = *agg-keys-bij*[*of ?m R*]
show *?thesis*
proof (*rule lawfulI*, *simp-all only*: *aggregate-simps*)
show *entry-wf e* **if** *e* ∈ *set* (*?h* # *map* (*agg-entry R*) (*edims E*)) **for** *e*
using *that* **by** (*auto intro*: *entry-wf-view keyed-view-key supported-gtrue entry-wf-agg-entry semilawful-wf*[*OF ES*])
show (*eupper e* ⊓ ⊥) ⊑ *elower e* **if** *e* ∈ *set* (*?h* # *map* (*agg-entry R*) (*edims*

E)) **for** *e*
using *that* **by** (*auto simp*: *lattice-meet-bot-right*)
show (⊥::'*k semi-factor*) = *fold* (⊔) (*map elower* (*?h* # *map* (*agg-entry R*) (*edims E*))) ⊥
proof −
have *l*: *map elower* (*?h* # *map* (*agg-entry R*) (*edims E*)) = *map* (λ-. ⊥) (*?h* # *map* (*agg-entry R*) (*edims E*))
by *simp*
show *?thesis* **by** (*simp only*: *l fold-join-bots*)
qed
show *keyimg data F* (⊥ ⊔ *flod F*) *k*
if *d*: *conforms data* **and** *agg-fold f* (*esem E data* (*finclude R F*)) (*inner-keys R E data F k*) ≠ *None*
for *data F k*
proof −
have *inner-keys R E data F k* ≠ {} **using** *agg-fold-nonempty*[*OF f that*(*2*)] **by** *auto*
then have *afiber data F* (*flod F*) *k* ≠ {} **by** (*auto simp*: *inner-keys-fiber*[*OF EL d*])
then show *?thesis* **by** (*auto simp*: *afiber-def keyimg-def bj*)
qed
show *agg-fold f* (*esem E data* (*finclude R F*)) (*inner-keys R E data F k*)
= *agg-csem R f E* (*contents data* (*?h* # *map* (*agg-entry R*) (*edims E*)) *F k*)
if *d*: *conforms data* **and** *k*: *proj* (⊥ ⊔ *flod F*)·*k* = *k* **for** *data F k*
proof −
have *k1*: *proj* (*flod F*)·*k* = *k* **using** *k* **by** (*simp only*: *bj*)
let *?Z* = *afiber data F* (*flod F*) *k*
let *?K* = *rproj* (*?m* ⊔ *flod* (*finclude R F*)) ‘ *?Z*
let *?P* = *rproj* (*?m* ⊔ *R*) ‘ *?Z* **and** *?π* = *Rep-cfun* (*proj* (*?m* ⊔ *R*))
let *?v* = *esem E data* (*finclude R F*)
let *?X* = *contents data* (*map* (*agg-entry R*) (*edims E*)) *F k*
let *?Φ* = λ*p*. *csem E* (*agg-inner R* (*edims E*) *?X p*)
have *hd*: *dom* (*hd* (*contents data* (*?h* # *map* (*agg-entry R*) (*edims E*)) *F k*)) = *?P*
by (*simp add*: *content-def cgraph-def cpair-def efiber-def fiber*[*OF k1*] *image-image*)
have *tl*: *tl* (*contents data* (*?h* # *map* (*agg-entry R*) (*edims E*)) *F k*) = *?X*
by (*simp only*: *list.map*(*2*) *list.sel*(*3*))
define ψ **where** ψ = *inv-into ?K ?π*
have *b*: *bij-betw* ψ *?P ?K* **unfolding** ψ-*def* **by** (*rule bij-betw-inv-into*[*OF bij*])
have *sP*: *?π* ‘ *?K* = *?P* **by** (*rule bij-betw-imp-surj-on*[*OF bij*])
have *pt*: *?Φ p* = *?v* (ψ *p*) **if** *p*: *p* ∈ *?P* **for** *p*
proof −
have *pK*: ψ *p* ∈ *?K* **unfolding** ψ-*def* **by** (*rule inv-into-into*) (*simp only*: *sP p*)
obtain *y* **where** *y*: *y* ∈ *?Z* ψ *p* = *rproj* (*?m* ⊔ *flod* (*finclude R F*)) *y* **using** *pK* **by** *blast*
have *py*: *proj* (*?m* ⊔ *R*)·(ψ *p*) = *p* **unfolding** ψ-*def* **by** (*rule f-inv-into-f*) (*simp only*: *sP p*)

have *agg-inner R (edims E) ?X p = map (λe. agg-pick R e p (content data (agg-entry R e) F k)) (edims E)*
unfolding *agg-inner-def map-map map2-map* **by** (*simp add*: *comp-def*)
also have *... = contents data (edims E) (finclude R F) (ψ p)*
proof (*rule map-cong*[*OF refl*])
fix *e* **assume** *e*: *e ∈ set (edims E)*
have *wf*: *entry-wf (agg-entry R e)* **by** (*rule entry-wf-agg-entry*[*OF semi-lawful-wf*[*OF ES e*]])
have *g*: *graph (content data (agg-entry R e) F k) = cgraph data (agg-entry R e) F k*
by (*rule graph-content*[*OF wf d*])
have *sh*: *gshaped (agg-entry R e) (graph (content data (agg-entry R e) F k))*
unfolding *g* **by** (*rule cgraph-gshaped*[*OF wf d*])
show *agg-pick R e p (content data (agg-entry R e) F k) = content data e (finclude R F) (ψ p)*
proof
fix *q*
have *agg-pick R e p (content data (agg-entry R e) F k) q = gmap (agg-read R e p (cgraph data (agg-entry R e) F k)) q*
by (*simp only*: *agg-pick-graph*[*OF sh*] *g*)
then show *agg-pick R e p (content data (agg-entry R e) F k) q = content data e (finclude R F) (ψ p) q*
by (*simp only*: *content-def agg-content-inner*[*OF ES e y*(*1*) *y*(*2*) *py*])
qed
qed
finally have *c*: *agg-inner R (edims E) ?X p = contents data (edims E) (finclude R F) (ψ p)* **.**
have *proj (kd E (finclude R F))·(ψ p) = ψ p* **by** (*simp only*: *y*(*2*) *proj-idem*)
then show *?thesis* **by** (*simp only*: *c semilawful-csem*[*OF ES d*])
qed
let *?Q = {p ∈ ?P. ?v (ψ p) ≠ None}*
have *D*: *inner-keys R E data F k = ψ ‘ ?Q*
unfolding *inner-keys-fiber*[*OF EL d*] *bij-betw-imp-surj-on*[*OF b*, *symmetric*] **by** *auto*
have *T*: *{p ∈ ?P. ?Φ p ≠ None} = ?Q* **using** *pt* **by** *auto*
have *bQ*: *bij-betw ψ ?Q (ψ ‘ ?Q)*
by (*rule inj-on-imp-bij-betw*[*OF inj-on-subset*[*OF bij-betw-imp-inj-on*[*OF b*]]]) *blast*
have *agg-fold f ?v (ψ ‘ ?Q) = agg-fold f (?v ∘ ψ) ?Q*
by (*rule agg-fold-bij*[*OF f bQ*, *symmetric*])
also have *... = agg-fold f ?Φ ?Q*
proof (*rule agg-fold-cong*[*OF f refl*])
fix *p* **assume** *p ∈ ?Q*
then show *(?v ∘ ψ) p = ?Φ p* **by** (*simp only*: *comp-apply pt mem-Collect-eq*)
qed
finally show *?thesis* **unfolding** *agg-csem-def hd tl T D* **.**
qed
qed

qed

end

end
theory *Ast*
  imports *Expr-Combinators Value*
begin

39 Expression syntax

The abstract syntax of the expression language, after name resolution: references to dimensions and to table columns, literals, calls of function symbols, aggregations, requested grouping (*GROUP BY*), explicit cuts (*FOR*) and filters (*WHERE*). Values are concrete: literals and expressions are data (*scalar datum*), dimensions are keys (*scalar with-na*). The conditions of cuts are boolean expressions on dimensions, which serve as slices; filters are row-level boolean expressions, which may read columns. Not yet covered: *DISTINCT*, *ISOLATE*, *TOTAL*, and the dimension kind (which needs staging).

Function symbols: the connectives, equality and the *NA* test are known to the language, for normal forms, coarsening and pins; other symbols are interpreted by a table.

datatype *'fn fsym = FAnd | FOr | FNot | FEq | FIsNA | FOther 'fn*

datatype *('d, 'c, 'fn, 'ag) ast =*
  Dim 'd
| Col 'c
| Lit scalar datum
| Call 'fn fsym ('d, 'c, 'fn, 'ag) ast list
| Aggr 'ag ('d, 'c, 'fn, 'ag) ast
| GroupBy 'd list ('d, 'c, 'fn, 'ag) ast
| For 'd list ('d, 'c, 'fn, 'ag) ast ('d, 'c, 'fn, 'ag) ast
| Where ('d, 'c, 'fn, 'ag) ast ('d, 'c, 'fn, 'ag) ast

40 The known function symbols

Booleans as data, and the n-ary connectives: strict, so every missing argument is infectious. The empty conjunction is true, the empty disjunction false.

definition *d-bool* :: *tr* → *scalar datum* **where**
  d-bool = na-emb oo Known oo SBool

fixrec *d-ands* :: *scalar datum ltuple* → *scalar datum* **where**

```
  d-ands·LNil = d-bool·TT
| xs ≠ ⊥ ⟹ d-ands·(LCons·x·xs) = dlift2·sand·x·(d-ands·xs)

fixrec d-ors :: scalar datum ltuple → scalar datum where
  d-ors·LNil = d-bool·FF
| xs ≠ ⊥ ⟹ d-ors·(LCons·x·xs) = dlift2·sor·x·(d-ors·xs)

fixrec arg1 :: ('a::domain → 'b::domain) → 'a ltuple → 'b where
  arg1·f·(LCons·x·LNil) = f·x

fixrec arg2 :: ('a::domain → 'a → 'b::domain) → 'a ltuple → 'b where
  arg2·f·(LCons·x·(LCons·y·LNil)) = f·x·y

definition d-eq :: scalar datum → scalar datum → scalar datum where
  d-eq = dlift2·(Λ x y. SBool·(scalar-eq·x·y))

definition d-is-na :: scalar datum → scalar datum where
  d-is-na = (Λ x. d-bool·(is-na·x))

primrec fsym-sem :: ('fn ⇒ scalar datum ltuple → scalar datum) ⇒ 'fn fsym ⇒
scalar datum ltuple → scalar datum where
  fsym-sem other FAnd = d-ands
| fsym-sem other FOr = d-ors
| fsym-sem other FNot = arg1·(dlift1·snot)
| fsym-sem other FEq = arg2·d-eq
| fsym-sem other FIsNA = arg1·d-is-na
| fsym-sem other (FOther fn) = other fn

primrec fsym-arity :: ('fn ⇒ nat ⇒ bool) ⇒ 'fn fsym ⇒ nat ⇒ bool where
  fsym-arity other FAnd n ⟷ True
| fsym-arity other FOr n ⟷ True
| fsym-arity other FNot n ⟷ n = 1
| fsym-arity other FEq n ⟷ n = 2
| fsym-arity other FIsNA n ⟷ n = 1
| fsym-arity other (FOther fn) n ⟷ other fn n

lemma truth-d-bool [simp]: truth-d·(d-bool·t) = t
  by (cases t rule: trE) (simp-all add: d-bool-def truth-d-def)
```

Truth of the connectives: a conjunction is true where all conjuncts are; a disjunction where all disjuncts are defined and one is true. Both are defined where all arguments are.

```
lemma tr-and-TT-iff: tr-and·x·y = TT ⟷ x = TT ∧ y = TT
  by (cases x rule: trE; cases y rule: trE) (simp-all add: TT-def FF-def)

lemma tr-and-bot-iff: tr-and·x·y = ⊥ ⟷ x = ⊥ ∨ y = ⊥
  by (cases x rule: trE; cases y rule: trE) (simp-all add: TT-def FF-def)

lemma tr-or-TT-iff: tr-or·x·y = TT ⟷ x ≠ ⊥ ∧ y ≠ ⊥ ∧ (x = TT ∨ y = TT)
```

by (*cases x rule*: *trE*; *cases y rule*: *trE*) (*simp-all add*: *TT-def FF-def*)

lemma *tr-or-bot-iff*: *tr-or·x·y* = ⊥ ⟷ *x* = ⊥ ∨ *y* = ⊥
by (*cases x rule*: *trE*; *cases y rule*: *trE*) (*simp-all add*: *TT-def FF-def*)

lemma *truth-d-ands*:
truth-d·(*d-ands·*(*ltuple-of xs*)) = *TT* ⟷ (∀ *x*∈*set xs*. *truth-d·x* = *TT*)
truth-d·(*d-ands·*(*ltuple-of xs*)) ≠ ⊥ ⟷ (∀ *x*∈*set xs*. *truth-d·x* ≠ ⊥)
by (*induct xs*) (*simp-all add*: *ltuple-of-not-bottom truth-d-and*[*simplified*] *tr-and-TT-iff tr-and-bot-iff*)

lemma *truth-d-ors*:
truth-d·(*d-ors·*(*ltuple-of xs*)) = *TT* ⟷ (∀ *x*∈*set xs*. *truth-d·x* ≠ ⊥) ∧ (∃ *x*∈*set xs*. *truth-d·x* = *TT*)
truth-d·(*d-ors·*(*ltuple-of xs*)) ≠ ⊥ ⟷ (∀ *x*∈*set xs*. *truth-d·x* ≠ ⊥)
by (*induct xs*) (*auto simp*: *ltuple-of-not-bottom truth-d-or*[*simplified*] *tr-or-TT-iff tr-or-bot-iff*)

41 Dimensions and the key reading

Dimensions name LODs of the keys, each with a value read off the key there. A dimension's value reads only its LOD, and at its LOD it is the whole information: the value orders the key's part at the LOD. Nothing ties dimensions to each other: hierarchies are inclusions of their LODs. Every compact key lists dimensions it lives at, possibly more than it needs (a year key may list the day). Expressions on dimensions alone, with the known and the tabled function symbols, read keys directly.

primrec *tuple-cfun* :: (*'r*::*cpo* → *'v*::*domain*) *list* ⇒ *'r* → *'v ltuple* **where**
tuple-cfun [] = (Λ *y*. *LNil*)
| *tuple-cfun* (*g* # *gs*) = (Λ *y*. *LCons·*(*g·y*)·(*tuple-cfun gs·y*))

lemma *tuple-cfun-apply*: *tuple-cfun gs·y* = *ltuple-of* (*map* (λ*g*. *g·y*) *gs*)
by (*induct gs*) *simp-all*

definition *key-datum* :: *scalar with-na* → *scalar datum* **where**
key-datum = *na-bind·*(*Val oo Known*)

lemma *key-datum-simps* [*simp*]:
key-datum·⊥ = ⊥ *key-datum·NA* = *NA key-datum·*(*Val·x*) = *Val·*(*Known·x*)
by (*simp-all add*: *key-datum-def*)

locale *key-language* =
fixes *dim-lod* :: *'d* ⇒ *'k*::*lod-domain semi-factor*
and *dim-val* :: *'d* ⇒ *'k* → *scalar with-na*
and *key-dims* :: *'k* ⇒ *'d list*
and *fn-other* :: *'fn* ⇒ *scalar datum ltuple* → *scalar datum*
assumes *dim-compact*: *compact* (*dim-lod d*)

and *dim-reads*: *dim-val d·(proj (dim-lod d)·k) = dim-val d·k*
and *dim-reflects*: *dim-val d·k ⊑ dim-val d·k′ ⟹ proj (dim-lod d)·k ⊑ proj (dim-lod d)·k′*
and *key-dims-lives*: *compact K ⟹ proj (fold (⊔) (map dim-lod (key-dims K)) ⊥)·K = K*
begin

definition *lod* :: *′d list ⇒ ′k semi-factor* **where**
lod ds = fold (⊔) (map dim-lod ds) ⊥

lemma *dim-below-iff*: *dim-val d·k ⊑ dim-val d·k′ ⟷ proj (dim-lod d)·k ⊑ proj (dim-lod d)·k′*
by (*metis dim-reflects dim-reads monofun-cfun-arg*)

lemma *dim-eq-iff*: *dim-val d·k = dim-val d·k′ ⟷ proj (dim-lod d)·k = proj (dim-lod d)·k′*
by (*simp add*: *po-eq-conv dim-below-iff*)

lemma *fold-join-least*:
b ⊑ c ⟹ (⋀x. x ∈ set xs ⟹ x ⊑ c) ⟹ fold (⊔) xs b ⊑ (c :: ′a::lattice-complete)
by (*induct xs arbitrary*: *b*) (*simp-all add*: *join-belowI*)

lemma *lod-below-iff*: *lod ds ⊑ c ⟷ (∀ d∈set ds. dim-lod d ⊑ c)*
proof
assume *lod ds ⊑ c*
moreover have *dim-lod d ⊑ lod ds* **if** *d ∈ set ds* **for** *d*
unfolding *lod-def* **by** (*rule fold-join-upper*) (*simp add*: *that*)
ultimately show *∀ d∈set ds. dim-lod d ⊑ c* **by** (*blast intro*: *below-trans*)
qed (*unfold lod-def*, *rule fold-join-least*, *auto*)

lemma *compact-lod*: *compact* (*lod ds*)
unfolding *lod-def* **by** (*rule compact-fold-sup*) (*auto simp*: *dim-compact*)

A key is below another as soon as each of its listed dimensions' values is.

lemma *proj-fold-below*:
assumes *proj b·K ⊑ k* **and** *⋀d. d ∈ set ds ⟹ proj (dim-lod d)·K ⊑ k*
shows *proj (fold (⊔) (map dim-lod ds) b)·K ⊑ k*
using *assms* **by** (*induct ds arbitrary*: *b*) (*simp-all add*: *proj-sup join-belowI*)

lemma *key-dims-below*:
assumes *K*: *compact K* **and** *le*: *⋀d. d ∈ set (key-dims K) ⟹ dim-val d·K ⊑ dim-val d·k*
shows *K ⊑ k*
proof −
have *proj (fold (⊔) (map dim-lod (key-dims K)) ⊥)·K ⊑ k*
proof (*rule proj-fold-below*)
show *proj ⊥·K ⊑ k* **by** (*simp add*: *proj-bot*)
show *proj (dim-lod d)·K ⊑ k* **if** *d ∈ set (key-dims K)* **for** *d*
using *dim-reflects*[*OF le*[*OF that*]] *proj-below* **by** (*rule below-trans*)

qed
then show *?thesis* **by** (*simp only*: *key-dims-lives*[*OF K*])
qed

41.1 The key reading

fun *kdims* :: ('*d*, '*c*, '*fn*, '*ag*) *ast* ⇒ '*d list* **where**
kdims (*Dim d*) = [*d*]
| *kdims* (*Call fn es*) = *concat* (*map kdims es*)
| *kdims* - = []

fun *kval* :: ('*d*, '*c*, '*fn*, '*ag*) *ast* ⇒ '*k* → *scalar datum* **where**
kval (*Dim d*) = *key-datum oo dim-val d*
| *kval* (*Lit v*) = (Λ -. *v*)
| *kval* (*Call fn es*) = *fsym-sem fn-other fn oo tuple-cfun* (*map kval es*)
| *kval* - = ⊥

definition *reads-in* :: '*k semi-factor* ⇒ ('*d*, '*c*, '*fn*, '*ag*) *ast* ⇒ *bool* **where**
reads-in A e ⟷ (∀ *d*∈*set* (*kdims e*). *dim-lod d* ⊑ *A*)

lemma *kval-reads*: *reads-in A e* ⟹ *kval e*·(*proj A*·*k*) = *kval e*·*k*
proof (*induct e rule*: *kval.induct*)
case (*1 d*)
have *dim-val d*·(*proj A*·*k*) = *dim-val d*·(*proj* (*dim-lod d*)·(*proj A*·*k*)) **by** (*simp only*: *dim-reads*)
also have . . . = *dim-val d*·*k* **using** *1* **by** (*simp add*: *reads-in-def proj-le dim-reads*)
finally show *?case* **by** *simp*
next
case (*3 fn es*)
then have *map* (λ*e*. *kval e*·(*proj A*·*k*)) *es* = *map* (λ*e*. *kval e*·*k*) *es* **by** (*auto simp*: *reads-in-def*)
then show *?case* **by** (*simp only*: *kval.simps cfcomp2 tuple-cfun-apply map-map comp-def*)
qed *simp-all*

definition *kholds* :: ('*d*, '*c*, '*fn*, '*ag*) *ast* ⇒ '*k* ⇒ *bool* **where**
kholds e k ⟷ *truth-d*·(*kval e*·*k*) = *TT*

lemma *kholds-up*: *kholds e k* ⟹ *k* ⊑ *k'* ⟹ *kholds e k'*
unfolding *kholds-def* **by** (*metis TT-below-iff monofun-cfun-arg*)

lemma *kholds-open*: *adm* (λ*k*. ¬ *kholds e k*)
unfolding *kholds-def* **by** (*simp add*: *compact-TT eq-commute*[*of* - *TT*])

lemma *reads-in-meet*: *reads-in* (*A* ⊓ *B*) *e* ⟷ *reads-in A e* ∧ *reads-in B e*
unfolding *reads-in-def*
by (*blast intro*: *lattice-meet-greatest below-trans*[*OF* - *lattice-meet-le1*] *below-trans*[*OF* - *lattice-meet-le2*])

41.2 Coarsening conditions

Coarsening a condition to an LOD drops what reads outside it, as in the source: conjunctions keep their surviving conjuncts (and vanish when none survives), disjunctions vanish as soon as one disjunct does, anything else survives whole or not at all. Dropping only ever replaces a part in positive position by truth, so it weakens: what was true stays true, what was defined stays defined. Negation is not looked into; normal forms push it to the literals first.

fun *crs* :: *'k semi-factor* ⇒ (*'d*, *'c*, *'fn*, *'ag*) *ast* ⇒ (*'d*, *'c*, *'fn*, *'ag*) *ast option*
where
  crs A (*Call FAnd es*) = (*if es* ≠ [] ∧ *filter* (λ*r*. *r* ≠ *None*) (*map* (*crs A*) *es*) = []
then None
    else Some (*Call FAnd* (*map the* (*filter* (λ*r*. *r* ≠ *None*) (*map* (*crs A*) *es*)))))
| *crs A* (*Call FOr es*) = (*if None* ∈ *set* (*map* (*crs A*) *es*) *then None*
    else Some (*Call FOr* (*map the* (*map* (*crs A*) *es*))))
| *crs A e* = (*if reads-in A e then Some e else None*)

lemma *set-kept*: *set* (*map the* (*filter* (λ*r*. *r* ≠ *None*) (*map f xs*))) = {*y*. ∃ *x*∈*set xs*. *f x* = *Some y*}
  by (*induct xs*) *auto*

lemma *set-all-some*:
  None ∉ *set* (*map f xs*) ⟹ *set* (*map the* (*map f xs*)) = {*y*. ∃ *x*∈*set xs*. *f x* = *Some y*}
  by (*induct xs*) *auto*

lemma *crs-inside*: *reads-in A e* ⟹ *crs A e* = *Some e*
proof (*induct A e rule*: *crs.induct*)
  case (*1 A es*)
  have *map* (*crs A*) *es* = *map Some es* **using** *1*(*1*) *1*(*3*) **by** (*simp add*: *reads-in-def*)
  then show *?case* **by** (*simp add*: *comp-def filter-map cong*: *map-cong*)
next
  case (*2 A es*)
  have *map* (*crs A*) *es* = *map Some es* **using** *2*(*1*) *2*(*3*) **by** (*simp add*: *reads-in-def*)
  then show *?case* **by** (*simp add*: *comp-def cong*: *map-cong*)
qed *simp-all*

lemma *crs-reads*: *crs A e* = *Some e'* ⟹ *reads-in A e'* ∧ *set* (*kdims e'*) ⊆ *set* (*kdims e*)
proof (*induct A e arbitrary*: *e' rule*: *crs.induct*)
  case (*1 A es*)
  have *IH*: *reads-in A c'* ∧ *set* (*kdims c'*) ⊆ *set* (*kdims c*) **if** *c* ∈ *set es crs A c* = *Some c'* **for** *c c'*
    using *1*(*1*)[*OF that*] **.**
  obtain *es'* **where** *e'*: *e'* = *Call FAnd es'* **and** *es'*: *set es'* = {*c'*. ∃ *c*∈*set es*. *crs A c* = *Some c'*}
    using *1*(*3*) *set-kept*[*of crs A es*] **by** (*auto split*: *if-splits*)

show *?case* **unfolding** e' **using** es' *IH* **by** (*fastforce simp*: *reads-in-def*)
next
case (*2 A es*)
have *IH*: *reads-in A c'* $\wedge$ *set* (*kdims c'*) $\subseteq$ *set* (*kdims c*) **if** $c \in$ *set es crs A c* = *Some c'* **for** *c c'*
using *2*(*1*)[*OF that*] **.**
obtain es' **where** e': $e' =$ *Call FOr es'* **and** es': *set es'* = {*c'*. $\exists$ *c*$\in$*set es*. *crs A c* = *Some c'*}
using *2*(*3*) *set-all-some*[*of crs A es*] **by** (*auto split*: *if-splits*)
show *?case* **unfolding** e' **using** es' *IH* **by** (*fastforce simp*: *reads-in-def*)
qed (*auto split*: *if-splits*)

lemma *kept-kept*:
map the (*filter* (λ*r*. *r* $\neq$ *None*) (*map g* (*map the* (*filter* (λ*r*. *r* $\neq$ *None*) (*map f xs*)))))
= *map the* (*filter* (λ*r*. *r* $\neq$ *None*) (*map* (λ*x*. *Option.bind* (*f x*) *g*) *xs*))
by (*induct xs*) (*auto split*: *option.split*)

lemma *all-some-bind*:
None $\notin$ *set* (*map f xs*) $\Longrightarrow$ *map g* (*map the* (*map f xs*)) = *map* (λ*x*. *Option.bind* (*f x*) *g*) *xs*
by (*induct xs*) (*auto*, *metis option.exhaust-sel bind.simps*(*2*))

lemma *crs-bind*: *Option.bind* (*crs B e*) (*crs A*) = *crs* (*A* $\sqcap$ *B*) *e*
proof (*induct B e arbitrary*: *A rule*: *crs.induct*)
case (*1 B es*)
have *IH*: *map* (*crs* (*A* $\sqcap$ *B*)) *es* = *map* (λ*c*. *Option.bind* (*crs B c*) (*crs A*)) *es*
using *1*(*1*) **by** *simp*
show *?case*
proof (*cases es* $\neq$ [] $\wedge$ *filter* (λ*r*. *r* $\neq$ *None*) (*map* (*crs B*) *es*) = [])
case *True*
then have *filter* (λ*r*. *r* $\neq$ *None*) (*map* (*crs* (*A* $\sqcap$ *B*)) *es*) = []
unfolding *IH* **by** (*auto simp*: *filter-empty-conv*)
then show *?thesis* **using** *True* **by** *simp*
next
case *False*
have *kk*: *map the* (*filter* (λ*r*. *r* $\neq$ *None*) (*map* (*crs A*) (*map the* (*filter* (λ*r*. *r* $\neq$ *None*) (*map* (*crs B*) *es*)))))
= *map the* (*filter* (λ*r*. *r* $\neq$ *None*) (*map* (*crs* (*A* $\sqcap$ *B*)) *es*)) **unfolding** *kept-kept IH* **..**
have *nil*: *map the xs* = [] $\longleftrightarrow$ *xs* = [] **for** *xs* :: *'x option list* **by** *simp*
show *?thesis*
unfolding *crs.simps*(*1*)[*of B*] *if-not-P*[*OF False*] *bind.simps*(*2*) *crs.simps*(*1*)[*of A*] *crs.simps*(*1*)[*of A* $\sqcap$ *B*] *kk*
apply (*rule if-cong*[*OF - refl refl*])
using *False arg-cong*[*OF kk*, *of* λ*xs*. *xs* = []] **unfolding** *nil* **by** (*cases es*) *auto*
qed
next
case (*2 B es*)

have *IH*: *map* (*crs* (*A* ⊓ *B*)) *es* = *map* (λ*c*. *Option.bind* (*crs B c*) (*crs A*)) *es*
using *2*(*1*) **by** *simp*
show *?case*
proof (*cases None* ∈ *set* (*map* (*crs B*) *es*))
case *True*
then have *None* ∈ *set* (*map* (*crs* (*A* ⊓ *B*)) *es*) **unfolding** *IH* **by** *force*
then show *?thesis* **using** *True* **by** *simp*
next
case *False*
have *map* (*crs A*) (*map the* (*map* (*crs B*) *es*)) = *map* (*crs* (*A* ⊓ *B*)) *es*
unfolding *IH* **by** (*rule all-some-bind*[*OF False*])
then show *?thesis*
unfolding *crs.simps*(*2*)[*of B*] *if-not-P*[*OF False*] *bind.simps*(*2*) *crs.simps*(*2*)[*of A*] *crs.simps*(*2*)[*of A* ⊓ *B*]
by *simp*
qed
qed (*auto simp*: *reads-in-meet*)

lemma *kval-call* [*simp*]: *kval* (*Call fn es*)·*k* = *fsym-sem fn-other fn*·(*ltuple-of* (*map* (λ*e*. *kval e*·*k*) *es*))
by (*simp add*: *tuple-cfun-apply comp-def*)

Coarsening weakens: true stays true, defined stays defined.

lemma *crs-weakens*:
crs A e = *Some e*′ ⟹ (*truth-d*·(*kval e*·*k*) = *TT* ⟶ *truth-d*·(*kval e*′·*k*) = *TT*) ∧
(*truth-d*·(*kval e*·*k*) ≠ ⊥ ⟶ *truth-d*·(*kval e*′·*k*) ≠ ⊥)
proof (*induct A e arbitrary*: *e*′ *rule*: *crs.induct*)
case (*1 A es*)
have *IH*: (*truth-d*·(*kval c*·*k*) = *TT* ⟶ *truth-d*·(*kval c*′·*k*) = *TT*) ∧
(*truth-d*·(*kval c*·*k*) ≠ ⊥ ⟶ *truth-d*·(*kval c*′·*k*) ≠ ⊥) **if** *c* ∈ *set es crs A c* = *Some c*′ **for** *c c*′
using *1*(*1*)[*OF that*] **.**
obtain *es*′ **where** *e*′: *e*′ = *Call FAnd es*′ **and** *es*′: *set es*′ = {*c*′. ∃ *c*∈*set es*. *crs A c* = *Some c*′}
using *1*(*3*) *set-kept*[*of crs A es*] **by** (*auto split*: *if-splits*)
show *?case* **unfolding** *e*′ *kval-call fsym-sem.simps* **using** *es*′ *IH* **by** (*auto simp*: *truth-d-ands*)
next
case (*2 A es*)
have *IH*: (*truth-d*·(*kval c*·*k*) = *TT* ⟶ *truth-d*·(*kval c*′·*k*) = *TT*) ∧
(*truth-d*·(*kval c*·*k*) ≠ ⊥ ⟶ *truth-d*·(*kval c*′·*k*) ≠ ⊥) **if** *c* ∈ *set es crs A c* = *Some c*′ **for** *c c*′
using *2*(*1*)[*OF that*] **.**
have *all*: ∃ *c*′. *crs A c* = *Some c*′ **if** *c* ∈ *set es* **for** *c*
using *2*(*3*) *that* **by** (*auto split*: *if-splits*) (*metis image-eqI option.exhaust*)
obtain *es*′ **where** *e*′: *e*′ = *Call FOr es*′ **and** *es*′: *set es*′ = {*c*′. ∃ *c*∈*set es*. *crs A c* = *Some c*′}
using *2*(*3*) *set-all-some*[*of crs A es*] **by** (*auto split*: *if-splits*)

have *defd*: $\forall\, c'{\in}set\ es'.\ truth\text{-}d{\cdot}(kval\ c'{\cdot}k) \neq \bot$ **if** $\forall\, c{\in}set\ es.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) \neq \bot$
using *that IH* **unfolding** *es'* **by** *blast*
have *T*: $truth\text{-}d{\cdot}(kval\ (Call\ FOr\ xs){\cdot}k) = TT \longleftrightarrow$
$(\forall\, c{\in}set\ xs.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) \neq \bot) \wedge (\exists\, c{\in}set\ xs.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) = TT)$
and *D*: $truth\text{-}d{\cdot}(kval\ (Call\ FOr\ xs){\cdot}k) \neq \bot \longleftrightarrow (\forall\, c{\in}set\ xs.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) \neq \bot)$ **for** *xs*
unfolding *kval-call fsym-sem.simps* **by** (*simp-all add*: *truth-d-ors*)
show *?case* **unfolding** *e' T D*
proof (*rule conjI*; *rule impI*)
assume $(\forall\, c{\in}set\ es.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) \neq \bot) \wedge (\exists\, c{\in}set\ es.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) = TT)$
then have *d*: $\forall\, c{\in}set\ es.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) \neq \bot$ **and** $\exists\, c{\in}set\ es.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) = TT$ **by** *blast+*
then obtain *c c'* **where** *c*: $c \in set\ es\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) = TT$ **and** *c'*: $crs\ A\ c = Some\ c'$ **using** *all* **by** *blast*
have $c' \in set\ es'$ **using** *c(1) c'* **unfolding** *es'* **by** *blast*
then show $(\forall\, c{\in}set\ es'.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) \neq \bot) \wedge (\exists\, c{\in}set\ es'.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) = TT)$
using *defd*[*OF d*] *IH*[*OF c(1) c'*] *c(2)* **by** *blast*
next
show $\forall\, c{\in}set\ es'.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) \neq \bot$ **if** $\forall\, c{\in}set\ es.\ truth\text{-}d{\cdot}(kval\ c{\cdot}k) \neq \bot$
by (*rule defd*[*OF that*])
qed
qed (*auto split*: *if-splits*)

lemma *reads-in-mono*: $reads\text{-}in\ A\ e \Longrightarrow A \sqsubseteq B \Longrightarrow reads\text{-}in\ B\ e$
unfolding *reads-in-def* **by** (*blast intro*: *below-trans*)

lemma *reads-in-lod*: $reads\text{-}in\ (lod\ (kdims\ e))\ e$
unfolding *reads-in-def* **using** *lod-below-iff*[*of kdims e lod* (*kdims e*)] **by** *simp*

end

42 Conditions as slices

A slice is a finite set of conditions, read as their conjunction; it is written at the join of their dimensions' LODs. Pins conjoin a test per dimension the key lists: *IS NA*, *NOT IS NA*, or equality with the key's value, as the value is *NA*, the not-null root, or a scalar.

type-synonym $('d, 'c, 'fn, 'ag)\ slice = ('d, 'c, 'fn, 'ag)\ ast\ fset$

lemma *lattice-lub-empty*: $lub\ \{\} = (\bot :: 'a{::}lattice\text{-}complete)$
by (*rule bottomI*) (*rule lattice-lub-least*, *simp*)

lemma *lattice-lub-Un*: $lub\ (A \cup B) = lub\ A \sqcup (lub\ B :: 'a{::}lattice\text{-}complete)$
proof (*rule below-antisym*)
show $lub\ (A \cup B) \sqsubseteq lub\ A \sqcup lub\ B$

by (*rule lattice-lub-least*, *rule is-ubI*)
(*auto intro*: *below-trans*[*OF lattice-lub-upper lattice-join-ge1*] *below-trans*[*OF lattice-lub-upper lattice-join-ge2*])
show *lub A* ⊔ *lub B* ⊑ *lub* (*A* ∪ *B*)
by (*rule join-belowI*; *rule lattice-lub-least*, *rule is-ubI*, *rule lattice-lub-upper*) *simp-all*
qed

lemma *lattice-lub-compact*:
finite S ⟹ (⋀*x*. *x* ∈ *S* ⟹ *compact x*) ⟹ *compact* (*lub S* :: *'k*::{*bifinite*, *distributive-cpo*} *semi-factor*)
proof (*induct rule*: *finite-induct*)
case *empty*
then show *?case* **by** (*simp add*: *lattice-lub-empty*)
next
case (*insert x S*)
have *lub* (*insert x S*) = *lub* {*x*} ⊔ *lub S* **using** *lattice-lub-Un*[*of* {*x*} *S*] **by** *simp*
moreover have *lub* {*x*} = *x* **by** (*rule lub-singleton*)
ultimately show *?case* **using** *insert* **by** (*simp add*: *compact-semi-factor-sup*)
qed

lemma *adm-not-ball-finite*:
finite S ⟹ (⋀*e*. *e* ∈ *S* ⟹ *adm* (λ*k*. ¬ *P e k*)) ⟹ *adm* (λ*k*. ¬ (∀ *e*∈*S*. *P e k*))
by (*induct rule*: *finite-induct*) (*simp-all add*: *adm-disj*)

context *key-language*
begin

definition *slice-lod* :: (*'d*, *'c*, *'fn*, *'ag*) *slice* ⇒ *'k semi-factor* **where**
slice-lod S = *lub* ((λ*e*. *lod* (*kdims e*)) ‘ *fset S*)

definition *slice-open* :: (*'d*, *'c*, *'fn*, *'ag*) *slice* ⇒ *'k sel* **where**
slice-open S = *Abs-sel* (λ*k*. ∀ *e*∈*fset S*. *kholds e k*)

lemma *selects-slice-open* [*simp*]: *selects* (*slice-open S*) *k* ⟷ (∀ *e*∈*fset S*. *kholds e k*)
proof −
have *up*: ∀ *x y*. (∀ *e*∈*fset S*. *kholds e x*) ⟶ *x* ⊑ *y* ⟶ (∀ *e*∈*fset S*. *kholds e y*)
by (*blast intro*: *kholds-up*)
have *adm*: *adm* (λ*k*. ¬ (∀ *e*∈*fset S*. *kholds e k*))
by (*rule adm-not-ball-finite*[*OF finite-fset kholds-open*])
have (λ*k*. ∀ *e*∈*fset S*. *kholds e k*) ∈ {*P*. (∀ *x y*. *P x* ⟶ *x* ⊑ *y* ⟶ *P y*) ∧ *adm* (λ*x*. ¬ *P x*)}
using *up adm* **by** (*simp only*: *mem-Collect-eq*)
then show *?thesis* **by** (*simp only*: *slice-open-def Abs-sel-inverse*)
qed

definition *slice-coarsen* :: *'k semi-factor* ⇒ (*'d*, *'c*, *'fn*, *'ag*) *slice* ⇒ (*'d*, *'c*, *'fn*, *'ag*) *slice* **where**

slice-coarsen A S = Abs-fset {e'. ∃ e∈fset S. crs A e = Some e'}

lemma *fset-slice-coarsen*: *fset (slice-coarsen A S) = {e'. ∃ e∈fset S. crs A e = Some e'}*
proof −
have *finite {e'. ∃ e∈fset S. crs A e = Some e'}*
by (*rule finite-subset*[*of* - (λe. *the* (*crs A e*)) ' *fset S*]) (*auto intro*: *rev-image-eqI*)
then show *?thesis* **unfolding** *slice-coarsen-def* **by** (*rule Abs-fset-inverse*[*simplified*])
qed

definition *pin-test* :: *'k ⇒ 'd ⇒* (*'d*, *'c*, *'fn*, *'ag*) *ast list* **where**
pin-test K d = (if dim-val d·K = ⊥ then []
else if dim-val d·K = NA then [*Call FIsNA* [*Dim d*]]
else if dim-val d·K = Val·⊥ then [*Call FNot* [*Call FIsNA* [*Dim d*]]]
else [*Call FEq* [*Dim d*, *Lit* (*key-datum·*(*dim-val d·K*))]])

definition *slice-pin* :: *'k ⇒* (*'d*, *'c*, *'fn*, *'ag*) *slice* **where**
slice-pin K = fset-of-list (*concat* (*map* (*pin-test K*) (*key-dims K*)))

definition *ast-slicing* :: ((*'d*, *'c*, *'fn*, *'ag*) *slice*, *'k*) *slicer-ops* **where**
ast-slicing = (|sl-conj = (|∪|), sl-unit = {||}, sl-lod = slice-lod, sl-open = slice-open,
sl-coarsen = slice-coarsen, sl-pin = slice-pin|)

42.1 The slicer laws

lemma *slice-lod-union*: *slice-lod* (*S* |∪| *T*) = *slice-lod S* ⊔ *slice-lod T*
by (*simp add*: *slice-lod-def image-Un lattice-lub-Un*)

lemma *slice-lod-empty*: *slice-lod* {||} = ⊥
by (*simp add*: *slice-lod-def lattice-lub-empty*)

lemma *slice-open-union*: *slice-open* (*S* |∪| *T*) = *inf* (*slice-open S*) (*slice-open T*)
by (*rule sel-eqI*) *auto*

lemma *slice-open-empty*: *slice-open* {||} = *top*
by (*rule sel-eqI*) *simp*

lemma *lod-below-slice-lod*: *e* |∈| *S* ⟹ *lod* (*kdims e*) ⊑ *slice-lod S*
unfolding *slice-lod-def* **by** (*rule lattice-lub-upper*) *simp*

lemma *reads-in-slice-lod*: *e* |∈| *S* ⟹ *reads-in* (*slice-lod S*) *e*
by (*rule reads-in-mono*[*OF reads-in-lod lod-below-slice-lod*])

lemma *slice-open-lod*: *selects* (*slice-open S*) (*proj* (*slice-lod S*)·*k*) ⟷ *selects* (*slice-open S*) *k*
by (*simp add*: *kholds-def kval-reads*[*OF reads-in-slice-lod*])

lemma *compact-slice-lod*: *compact* (*slice-lod S*)
unfolding *slice-lod-def* **by** (*rule lattice-lub-compact*) (*auto simp*: *compact-lod*)

lemma *slice-coarsen-union*: *slice-coarsen A* (*S* |∪| *T*) = *slice-coarsen A S* |∪| *slice-coarsen A T*
by (*subst fset-inject*[*symmetric*]) (*auto simp*: *fset-slice-coarsen*)

lemma *slice-coarsen-inside*: *slice-lod S* ⊑ *A* ⟹ *slice-coarsen A S* = *S*
proof (*subst fset-inject*[*symmetric*])
assume *A*: *slice-lod S* ⊑ *A*
have *crs A e* = *Some e* **if** *e* |∈| *S* **for** *e*
by (*rule crs-inside*[*OF reads-in-mono*[*OF reads-in-slice-lod*[*OF that*] *A*]])
then show *fset* (*slice-coarsen A S*) = *fset S* **by** (*auto simp*: *fset-slice-coarsen*)
qed

lemma *slice-lod-coarsen*: *slice-lod* (*slice-coarsen A S*) ⊑ *slice-lod S* ⊓ *A*
unfolding *slice-lod-def*
proof (*rule lattice-lub-least*, *rule is-ubI*)
fix *x* **assume** *x* ∈ (λ*e*. *lod* (*kdims e*)) ‘ *fset* (*slice-coarsen A S*)
then obtain *e e*′ **where** *e*: *e* |∈| *S* **and** *c*: *crs A e* = *Some e*′ **and** *x*: *x* = *lod* (*kdims e*′)
by (*auto simp*: *fset-slice-coarsen*)
have *r*: *reads-in A e*′ **and** *sub*: *set* (*kdims e*′) ⊆ *set* (*kdims e*) **using** *crs-reads*[*OF c*] **by** *simp-all*
have *x* ⊑ *lod* (*kdims e*) **unfolding** *x lod-below-iff* **using** *sub lod-below-iff*[*of kdims e lod* (*kdims e*)] **by** *auto*
then have *x* ⊑ *lub* ((λ*e*. *lod* (*kdims e*)) ‘ *fset S*)
using *lod-below-slice-lod*[*OF e*] **unfolding** *slice-lod-def* **by** (*rule below-trans*)
moreover have *x* ⊑ *A* **unfolding** *x lod-below-iff* **using** *r* **by** (*simp add*: *reads-in-def*)
ultimately show *x* ⊑ *lub* ((λ*e*. *lod* (*kdims e*)) ‘ *fset S*) ⊓ *A* **by** (*rule lattice-meet-greatest*)
qed

lemma *slice-coarsen-cut*:
selects (*slice-open S*) *k* ⟹ *selects* (*slice-open* (*slice-coarsen A S*)) (*proj A*·*k*)
proof (*simp only*: *selects-slice-open fset-slice-coarsen*, *intro ballI*)
fix *e*′ **assume** *S*: ∀ *e*∈*fset S*. *kholds e k* **and** *e*′ ∈ {*e*′. ∃ *e*∈*fset S*. *crs A e* = *Some e*′}
then obtain *e* **where** *e*: *e* |∈| *S* **and** *c*: *crs A e* = *Some e*′ **by** *auto*
have *kholds e*′ *k* **using** *S e crs-weakens*[*OF c*, *of k*] **by** (*simp add*: *kholds-def*)
then show *kholds e*′ (*proj A*·*k*) **using** *crs-reads*[*OF c*] **by** (*simp add*: *kholds-def kval-reads*)
qed

lemma *slice-coarsen-slod*: *slice-coarsen A S* = *slice-coarsen* (*A* ⊓ *slice-lod S*) *S*
proof (*subst fset-inject*[*symmetric*])
have *crs* (*A* ⊓ *slice-lod S*) *e* = *crs A e* **if** *e* |∈| *S* **for** *e*
using *crs-bind*[*of slice-lod S e A*] *crs-inside*[*OF reads-in-slice-lod*[*OF that*]] **by** *simp*
then show *fset* (*slice-coarsen A S*) = *fset* (*slice-coarsen* (*A* ⊓ *slice-lod S*) *S*)
by (*auto simp*: *fset-slice-coarsen*)

qed

lemma *slice-coarsen-coarsen*: *slice-coarsen A* (*slice-coarsen B S*) = *slice-coarsen* (*A* ⊓ *B*) *S*
proof (*subst fset-inject*[*symmetric*])
have *crs* (*A* ⊓ *B*) *e* = *Some e*″ ⟷ (∃ *e*′. *crs B e* = *Some e*′ ∧ *crs A e*′ = *Some e*″) **for** *e e*″
by (*simp only*: *crs-bind*[*symmetric*] *bind-eq-Some-conv*)
then show *fset* (*slice-coarsen A* (*slice-coarsen B S*)) = *fset* (*slice-coarsen* (*A* ⊓ *B*) *S*)
unfolding *fset-slice-coarsen* **by** *blast*
qed

end

42.2 Pins

Scalars are flat, so a defined scalar is maximal; so are *NA* and a key value *Val*·*s* for a defined scalar *s*. The upsets of key values are therefore exactly what the three tests select.

lemma *scalar-maximal*: *s* ≠ ⊥ ⟹ *s* ⊑ *x* ⟹ *x* = (*s* :: *scalar*)
by (*cases s*; *cases x*) (*auto simp*: *flat-below-iff*)

lemma *truth-d-is-na*: *truth-d*·(*d-is-na*·(*key-datum*·*w*)) = *is-na*·*w*
by (*cases w*) (*simp-all add*: *d-is-na-def*)

lemma *truth-d-eq-key*:
assumes *s* ≠ ⊥
shows *truth-d*·(*d-eq*·(*key-datum*·*w*)·(*key-datum*·(*Val*·*s*))) = *TT* ⟷ *w* = *Val*·*s*
proof −
have *strict*: (Λ *x y*. *SBool*·(*scalar-eq*·*x*·*y*))·⊥·*y* = ⊥ (Λ *x y*. *SBool*·(*scalar-eq*·*x*·*y*))·*x*·⊥ = ⊥ **for** *x y*
by *simp-all*
have *truth-d*·(*d-eq*·*a*·*b*) = *scalar-eq*·(*plain*·*a*)·(*plain*·*b*) **for** *a b*
unfolding *d-eq-def truth-d-def* **by** (*simp add*: *plain-dlift2*[*OF strict*])
moreover have *scalar-eq*·(*plain*·(*key-datum*·*w*))·*s* = *TT* ⟷ *w* = *Val*·*s*
proof (*cases w*)
case (*Val x*)
then show *?thesis* **using** *assms* **by** (*cases x* = ⊥) (*simp-all add*: *scalar-eq-defined TT-def*)
qed *simp-all*
ultimately show *?thesis* **by** *simp*
qed

context *key-language*
begin

lemma *kholds-pin-test*:
(∀ *e*∈*set* (*pin-test K d*). *kholds e k*) ⟷ *dim-val d*·*K* ⊑ *dim-val d*·*k*

proof −
let *?v* = *dim-val d·K* **and** *?w* = *dim-val d·k*
show *?thesis*
proof (*cases ?v*)
case *bottom*
then show *?thesis* **by** (*simp add*: *pin-test-def*)
next
case *NA*
have *is-na·?w* = *TT* ⟷ *?w* = *NA* **by** (*cases ?w*) *simp-all*
moreover have *NA* ⊑ *?w* ⟷ *?w* = *NA* **by** (*cases ?w*) *simp-all*
ultimately show *?thesis* **using** *NA* **by** (*simp add*: *pin-test-def kholds-def truth-d-is-na*)
next
case (*Val s*)
show *?thesis*
proof (*cases s* = ⊥)
case *True*
have *neg·*(*is-na·?w*) = *TT* ⟷ *Val·*⊥ ⊑ *?w* **by** (*cases ?w*) *simp-all*
then show *?thesis* **using** *Val True*
by (*simp add*: *pin-test-def kholds-def truth-d-is-na truth-d-not*[*simplified*])
next
case *False*
have *Val·s* ⊑ *?w* ⟷ *?w* = *Val·s*
by (*cases ?w*) (*auto dest*: *scalar-maximal*[*OF False*])
then show *?thesis* **using** *Val False*
by (*simp add*: *pin-test-def kholds-def truth-d-eq-key*[*OF False*, *simplified*])
qed
qed
qed

lemma *selects-slice-pin*:
compact K ⟹ *selects* (*slice-open* (*slice-pin K* :: ('*d*, '*c*, '*fn*, '*ag*) *slice*)) *k* ⟷ *K* ⊑ *k*
proof
assume *K*: *compact K* **and** *selects* (*slice-open* (*slice-pin K* :: ('*d*, '*c*, '*fn*, '*ag*) *slice*)) *k*
then have ∀ *d*∈*set* (*key-dims K*). ∀ *e*∈*set* (*pin-test K d* :: ('*d*, '*c*, '*fn*, '*ag*) *ast list*). *kholds e k*
by (*simp add*: *slice-pin-def fset-of-list.rep-eq*)
then show *K* ⊑ *k* **by** (*intro key-dims-below*[*OF K*]) (*simp add*: *kholds-pin-test*)
next
assume *K* ⊑ *k*
then have *dim-val d·K* ⊑ *dim-val d·k* **for** *d* **by** (*rule monofun-cfun-arg*)
then have ∀ *e*∈*set* (*pin-test K d* :: ('*d*, '*c*, '*fn*, '*ag*) *ast list*). *kholds e k* **for** *d*
by (*simp only*: *kholds-pin-test*)
then show *selects* (*slice-open* (*slice-pin K* :: ('*d*, '*c*, '*fn*, '*ag*) *slice*)) *k*
by (*auto simp*: *slice-pin-def fset-of-list.rep-eq*)
qed

theorem *slicer-ast-slicing*: *slicer* (*ast-slicing* :: (('*d*, '*c*, '*fn*, '*ag*) *slice*, '*k*) *slicer-ops*)
by *unfold-locales*
(*simp-all del*: *selects-slice-open add*: *ast-slicing-def funion-ac lattice-join-ac slice-lod-union slice-lod-empty slice-open-union*
slice-open-empty slice-open-lod compact-slice-lod slice-coarsen-union slice-coarsen-inside
slice-lod-coarsen slice-coarsen-cut slice-coarsen-slod[*symmetric*] *slice-coarsen-coarsen selects-slice-pin*)

end

end
theory *Ast-Semantics*
imports *Ast Aggregate*
begin

43 Levels of expressions

Two levels are read off the dims. A *row-level* expression (an attribute) reads every content at most at its lower part, so all rows of a group agree on the section and each content has at most one key. A *total-level* expression (an aggregation) has no minimal LOD. Constants are both; the remaining expressions are aggregations with explicit grouping. Row level is preserved by maps and by collecting, since collecting only widens lower parts.

context *dataspace*
begin

definition *row-level* :: ('*k*, '*f*, '*r*, '*v*) *expr* ⇒ *bool* **where**
row-level E ⟷ (∀ *e*∈*set* (*edims E*). *row-entry e*)

definition *total-level* :: ('*k*, '*f*, '*r*, '*v*) *expr* ⇒ *bool* **where**
total-level E ⟷ *emind E* = ⊥

lemma *row-level-value-at*: *row-level* (*value-at c g*)
by (*simp add*: *row-level-def row-entry-def*)

lemma *row-level-column*: *row-level* (*column c a v*)
by (*simp add*: *row-level-def row-entry-def*)

lemma *total-level-value-at-bot*: *total-level* (*value-at* ⊥ *g*)
by (*simp add*: *total-level-def*)

lemma *row-level-smap*: *row-level* (*smap g E*) ⟷ *row-level E*
by (*simp add*: *row-level-def*)

lemma *row-level-collect*: (⋀*E*. *E* ∈ *set Es* ⟹ *row-level E*) ⟹ *row-level* (*collect Es*)
unfolding *row-level-def collect-simps* **by** *fastforce*

lemma *total-level-aggregate*: *total-level* (*aggregate M f E*)
by (*simp add*: *total-level-def*)

The content graphs of a semilawful row-level expression have at most one element: the rows of a group agree on the key at the lower part, which determines the view.

lemma *row-level-content-single*:
assumes *E*: *semilawful E* **and** *r*: *row-level E* **and** *d*: *conforms data* **and** *e*: $e \in$ *set* (*edims E*)
and *u*: $u \in$ *cgraph data e F k* **and** *u'*: $u' \in$ *cgraph data e F k*
shows $u = u'$
proof −
let *?G* = *elower e* ⊔ (*eupper e* ⊓ *flod F*)
obtain *x* **where** *x*: $x \in$ *group data* (*elower e*) (*eupper e*) *F k* **using** *u* **by** (*auto simp*: *cgraph-def efiber-def*)
have *cgraph data e F k* = (*if ofun* (*eguard e*)·*x* = *TT then* {*cpair e x*} *else* {})
unfolding *cgraph-def efiber-def*
proof (*rule row-entry-image*[*OF semilawful-wf*[*OF E e*] - *d* - *x lattice-join-ge1*[*of elower e eupper e* ⊓ *flod F*]])
show *row-entry e* **using** *r e* **by** (*simp add*: *row-level-def*)
show *group data* (*elower e*) (*eupper e*) *F k* ⊆ *lower-maximal data* **by** (*auto simp*: *group-def*)
show *rproj ?G y* = *rproj ?G x* **if** $y \in$ *group data* (*elower e*) (*eupper e*) *F k* **for** *y*
using *that x* **by** (*simp add*: *group-def*)
qed
then show *?thesis* **using** *u u'* **by** (*simp split*: *if-splits*)
qed

end

44 Semantics

The symbol tables: dimensions and tabled function symbols as in the key language; each table column reads a section of the row, and an oracle names the dimension determining it in every conforming dataset; tabled function symbols admit some numbers of arguments; aggregation symbols are commutative and associative. The slicer is the one of conditions; a cut's condition is a slice of one conjunct.

locale *ast-space* = *key-language dim-lod dim-val key-dims fn-other* + *keyed-schema rules keying*
for *dim-lod* :: *'d* ⇒ *'k*::{*domain*, *lod-domain*} *semi-factor*
and *dim-val* :: *'d* ⇒ *'k* → *scalar with-na* **and** *key-dims* :: *'k* ⇒ *'d list*
and *fn-other* :: *'fn* ⇒ *scalar datum ltuple* → *scalar datum*
and *rules* :: *'r*::{*domain*, *lod-domain*} *schema-rules* **and** *keying* :: (*'r*, *'k*) *keying* +

```
  fixes col-sec :: 'c ⇒ 'r semi-factor and col-dim :: 'c ⇒ 'd and col-val :: 'c ⇒ 'r
→ scalar datum
    and fn-arity :: 'fn ⇒ nat ⇒ bool
    and ag-sem :: 'ag ⇒ scalar datum → scalar datum → scalar datum
  assumes col-reads: col-val c·(x ⌊ col-sec c) = col-val c·x
    and col-fd: universally (λP. vfd P (⊥, dim-lod (col-dim c)) (col-sec c, ⊥))
    and ag-abel: abel-semigroup (λa b. ag-sem g·a·b)
begin

sublocale dataspace ast-slicing :: (('d, 'c, 'fn, 'ag) slice, 'k) slicer-ops rules keying
  by (simp add: dataspace-def slicer-ast-slicing keyed-schema-axioms)
```

The *requested* dimensions of an expression: *GROUP BY* raises them, an aggregation consumes those of its argument and requests none itself; cuts and filters leave them alone.

```
fun requested :: ('d, 'c, 'fn, 'ag) ast ⇒ 'k semi-factor where
  requested (Dim d) = ⊥
| requested (Col c) = ⊥
| requested (Lit v) = ⊥
| requested (Call fn es) = fold (⊔) (map requested es) ⊥
| requested (Aggr g e) = ⊥
| requested (GroupBy ds e) = lod ds ⊔ requested e
| requested (For ds C e) = requested e
| requested (Where C e) = requested e
```

Row-level expressions read the parts of their views directly: dimensions the key part, columns their sections, calls their arguments' parts. Their view joins the views of the dimensions and columns they read. (A reading of row-level conditions apart from the expression semantics, until the latter gives continuous functions of the view parts itself.)

```
fun pval :: ('d, 'c, 'fn, 'ag) ast ⇒ 'r × 'k → scalar datum where
  pval (Dim d) = key-datum oo dim-val d oo csnd
| pval (Col c) = col-val c oo cfst
| pval (Lit v) = (Λ -. v)
| pval (Call fn es) = fsym-sem fn-other fn oo tuple-cfun (map pval es)
| pval - = ⊥

fun rview :: ('d, 'c, 'fn, 'ag) ast ⇒ ('r, 'k) view where
  rview (Dim d) = (⊥, dim-lod d)
| rview (Col c) = (col-sec c, dim-lod (col-dim c))
| rview (Lit v) = ⊥
| rview (Call fn es) = fold (⊔) (map rview es) ⊥
| rview - = ⊥

fun row-ast :: ('d, 'c, 'fn, 'ag) ast ⇒ bool where
  row-ast (Dim d) ⟷ True
| row-ast (Col c) ⟷ True
| row-ast (Lit v) ⟷ True
```

| *row-ast* (*Call fn es*) ⟷ *fsym-arity fn-arity fn* (*length es*) ∧ (∀ *e*∈*set es*. *row-ast e*)
| *row-ast* - ⟷ *False*

A row-level condition is a guard through its truth value.

definition *row-guard* :: (*'d*, *'c*, *'fn*, *'ag*) *ast* ⇒ (*'r*, *'k*) *guard* **where**
  row-guard C = (|*vfun* = *truth-d oo pval C*, *oview* = *rview C*|)

lemma *row-guard-view* [*simp*]: *oview* (*row-guard C*) = *rview C*
  by (*simp add*: *row-guard-def*)

The views of row-level expressions are keyed.

lemma *keyed-view-bot*: *keyed-view* (⊥ :: (*'r*, *'k*) *view*)
  unfolding *inst-prod-pcpo* **by** (*rule keyed-view-key*)

lemma *keyed-view-fold*:
  (⋀*v*. *v* ∈ *set vs* ⟹ *keyed-view v*) ⟹ *keyed-view b* ⟹ *keyed-view* (*fold* (⊔) *vs b*)
  by (*induct vs arbitrary*: *b*) (*simp-all add*: *keyed-view-sup*)

lemma *keyed-view-rview*: *row-ast e* ⟹ *keyed-view* (*rview e*)
proof (*induct e rule*: *row-ast.induct*)
  case (*4 fn es*)
 then show *?case* **unfolding** *rview.simps* **by** (*intro keyed-view-fold keyed-view-bot*) *auto*
qed (*simp-all add*: *keyed-view-key keyed-view-columns*[*OF col-fd*] *keyed-view-bot*)

lemma *supported-row-guard*: *row-ast C* ⟹ *supported* (*row-guard C*)
  by (*simp add*: *supported-def keyed-view-rview*)

fun *asem* :: (*'d*, *'c*, *'fn*, *'ag*) *ast* ⇒ (*'k*, (*'d*, *'c*, *'fn*, *'ag*) *slice*, *'r*, *scalar datum*) *expr* **where**
  asem (*Dim d*) = *value-at* (*dim-lod d*) (*key-datum oo dim-val d*)
| *asem* (*Col c*) = *column* (*col-sec c*) (*dim-lod* (*col-dim c*)) (*col-val c*)
| *asem* (*Lit v*) = *value-at* ⊥ (Λ -. *v*)
| *asem* (*Call fn es*) = *smap* (*fsym-sem fn-other fn*) (*collect* (*map asem es*))
| *asem* (*Aggr g e*) = *aggregate* (*requested e*) (*ag-sem g*) (*gate* (*asem e*))
| *asem* (*GroupBy ds e*) = *asem e*
| *asem* (*For ds C e*) = *filtfor* (*lod ds*) {|*C*|} (*asem e*)
| *asem* (*Where C e*) = *restrict* (*row-guard C*) (*asem e*)

Well-formedness: every call has a number of arguments its symbol admits.

fun *wf* :: (*'d*, *'c*, *'fn*, *'ag*) *ast* ⇒ *bool* **where**
  wf (*Dim d*) ⟷ *True*
| *wf* (*Col c*) ⟷ *True*
| *wf* (*Lit v*) ⟷ *True*
| *wf* (*Call fn es*) ⟷ *fsym-arity fn-arity fn* (*length es*) ∧ (∀ *e*∈*set es*. *wf e*)
| *wf* (*Aggr g e*) ⟷ *wf e*
| *wf* (*GroupBy ds e*) ⟷ *wf e*

| *wf* (*For ds C e*) ⟷ *wf e*
| *wf* (*Where C e*) ⟷ *row-ast C* ∧ *wf e*

The semantics is semilawful; gating it at the top gives the lawful expression.

theorem *semilawful-asem*: *wf e* ⟹ *semilawful* (*asem e*)
proof (*induct e rule*: *wf.induct*)
  case (*4 fn es*)
  then show *?case* **by** (*auto intro*!: *semilawful-smap semilawful-collect*)
next
  case (*5 g e*)
  then show *?case* **by** (*simp add*: *lawful-semilawful lawful-aggregate*[*OF lawful-gate ag-abel*])
next
  case (*8 C e*)
  then show *?case* **by** (*simp add*: *semilawful-restrict supported-row-guard*)
qed (*simp-all add*: *semilawful-value-at semilawful-column*[*OF col-fd*] *semilawful-filtfor*)

corollary *lawful-gate-asem*: *wf e* ⟹ *lawful* (*gate* (*asem e*))
  by (*rule lawful-gate*[*OF semilawful-asem*])

References to dimensions and columns and literals are row level, literals and aggregations total level; calls of row-level arguments are row level.

lemma *row-level-Dim*: *row-level* (*asem* (*Dim d*)) **and** *row-level-Col*: *row-level* (*asem* (*Col c*))
  and *row-level-Lit*: *row-level* (*asem* (*Lit v*))
  and *total-level-Lit*: *total-level* (*asem* (*Lit v*)) **and** *total-level-Aggr*: *total-level* (*asem* (*Aggr g e*))
  by (*simp-all add*: *row-level-value-at row-level-column total-level-value-at-bot total-level-aggregate*)

lemma *row-level-Call*: (⋀*e*. *e* ∈ *set es* ⟹ *row-level* (*asem e*)) ⟹ *row-level* (*asem* (*Call fn es*))
  unfolding *asem.simps row-level-smap* **by** (*rule row-level-collect*) *auto*

end

end
theory *Bag-Join*
  imports *HOLCF−extra.BagPD LOD*
begin

default-sort *bifinite*

45 Outer joins of bags

A binary outer join of two bags of rows. The sides keep their own row types; a pair of rows is combined by a monotone c (the pair, or the join of two disjoint blocks of a universal row), and an orphan is combined with bottom.

Rows meet through key maps into a common key order: a pair is a *decided match* when the keys are equal and live, which makes them maximal. Other keys never match: partial keys, and maximal keys that are not live such as a decided NULL, which pass through as orphans.

A decided match of $n{\cdot}x$ and $m{\cdot}y$ gives n m joined rows. Besides, each row x of multiplicity n keeps n $N(x)$ orphan copies, where $N(x)$ bounds the partners that refinements can still bring:

- ∞ on an *Open* side (the 1-side of an N:1 join: more referencing rows may join);
- on a *Closed* side with a decided match, the undecided partner mass compatible with its key;
- on a *Closed* side without a decided match, the largest partner mass that a live key above its key can collect, and at least one copy. Refinement only merges rows, so partner masses only shrink and the join is monotone in both arguments. No deduplication is involved: duplicate rows simply multiply.

45.1 Sides and indices

datatype *join-side* = *Open* | *Closed*

The rows of a join on the basis are indexed by joined pairs and by the orphans of either side.

datatype ('*a*, '*b*) *join-index* =
  Joined '*a compact-basis* '*b compact-basis*
| *Left-Orphan* '*a compact-basis*
| *Right-Orphan* '*b compact-basis*

definition *join-indices* :: '*a bag-basis* ⇒ '*b bag-basis* ⇒ ('*a*, '*b*) *join-index set*
  where *join-indices t u* =
  case-prod Joined ‘ (*bsupp t* × *bsupp u*) ∪ (*Left-Orphan* ‘ *bsupp t* ∪ *Right-Orphan* ‘ *bsupp u*)

lemma *join-indices-iff* [*simp*]:
  Joined x y ∈ *join-indices t u* ⟷ *x* ∈ *bsupp t* ∧ *y* ∈ *bsupp u*
  Left-Orphan x ∈ *join-indices t u* ⟷ *x* ∈ *bsupp t*
  Right-Orphan y ∈ *join-indices t u* ⟷ *y* ∈ *bsupp u*
  by (*auto simp*: *join-indices-def*)

lemma *finite-join-indices* [*simp*]: *finite* (*join-indices t u*)
  by (*simp add*: *join-indices-def*)

lemma *sum-join-indices*:

sum g (join-indices t u) = (∑ x∈bsupp t. ∑ y∈bsupp u. g (Joined x y)) +
((∑ x∈bsupp t. g (Left-Orphan x)) + (∑ y∈bsupp u. g (Right-Orphan y)))
proof −
have *sum g (join-indices t u) = sum g (case-prod Joined ‘ (bsupp t × bsupp u))*
+
(sum g (Left-Orphan ‘ bsupp t) + sum g (Right-Orphan ‘ bsupp u))
unfolding *join-indices-def* **by** (*subst sum.union-disjoint*; *auto*)+
then show *?thesis*
by (*simp add*: *sum.reindex inj-on-def sum.cartesian-product prod.case-distrib*)
qed

primrec *join-label* ::
(*'a compact-basis ⇒ 'b compact-basis ⇒ 'c compact-basis*) ⇒ (*'a*, *'b*) *join-index*
⇒ *'c compact-basis*
where
join-label c (Joined x y) = c x y
| *join-label c (Left-Orphan x) = c x compact-bot*
| *join-label c (Right-Orphan y) = c compact-bot y*

primrec *join-swap* :: (*'a*, *'b*) *join-index* ⇒ (*'b*, *'a*) *join-index*
where
join-swap (Joined x y) = Joined y x
| *join-swap (Left-Orphan x) = Right-Orphan x*
| *join-swap (Right-Orphan y) = Left-Orphan y*

lemma *join-swap-swap* [*simp*]: *join-swap (join-swap i) = i*
by (*cases i*) *simp-all*

lemma *join-swap-indices*: *i ∈ join-indices t u ⟹ join-swap i ∈ join-indices u t*
by (*cases i*) *simp-all*

Joined rows as pairs, and keys of basis elements.

definition *pair-basis* :: *'a compact-basis ⇒ 'b compact-basis ⇒ ('a × 'b) compact-basis*
where *pair-basis x y = Abs-compact-basis* (*Rep-compact-basis x*, *Rep-compact-basis y*)

lemma *Rep-pair-basis* [*simp*]:
Rep-compact-basis (pair-basis x y) = (Rep-compact-basis x, Rep-compact-basis y)
by (*simp add*: *pair-basis-def*)

lemma *pair-basis-mono*:
assumes *x ⊑ x'* **and** *y ⊑ y'*
shows *pair-basis x y ⊑ pair-basis x' y'*
proof (*rule compact-below-iff* [*THEN iffD2*])
show *Rep-compact-basis (pair-basis x y) ⊑ Rep-compact-basis (pair-basis x' y')*
using *assms* [*THEN compact-below-iff* [*THEN iffD1*]] **by** (*simp only*: *Rep-pair-basis Pair-below-iff*)
qed

abbreviation *on-basis* :: $('a \to 'k{::}cpo) \Rightarrow 'a$ *compact-basis* $\Rightarrow 'k$
where *on-basis* $f\ x \equiv f{\cdot}(Rep\text{-}compact\text{-}basis\ x)$

lemma *monofun-on-basis*: *monofun* (*on-basis* f)
by (*rule monofunI*, *rule monofun-cfun-arg*, *erule compact-below-iff* [*THEN iffD1*])

45.2 Partner masses

Keys range over *Keys*; the live keys are maximal there.

locale *bag-join-keys* =
fixes *Live Keys* :: $'k{::}cpo\ set$
assumes *live-maximal*: $k \in Live \Longrightarrow k \sqsubseteq k' \Longrightarrow k' \in Keys \Longrightarrow k' = k$
begin

definition *decided-match* :: $'k \Rightarrow 'k \Rightarrow bool$
where *decided-match* $k\ k' \longleftrightarrow k \in Live \wedge k' = k$

lemma *decided-match-sym*: *decided-match* $k\ k' \longleftrightarrow$ *decided-match* $k'\ k$
by (*auto simp*: *decided-match-def*)

For a row with key k against the bag u on the other side, whose rows have keys *key*: the decided partner mass, the undecided partner mass below k, and the largest mass that a live key above k can collect.

definition *decided-mass* :: $('b$ *compact-basis* $\Rightarrow 'k) \Rightarrow 'b$ *bag-basis* $\Rightarrow 'k \Rightarrow$ *enat*
where *decided-mass key u k* = *bag-mass u* (λy. *decided-match k* (*key y*))

definition *undecided-mass* :: $('b$ *compact-basis* $\Rightarrow 'k) \Rightarrow 'b$ *bag-basis* $\Rightarrow 'k \Rightarrow$ *enat*
where *undecided-mass key u k* = *bag-mass u* (λy. *key y* $\sqsubseteq$ *k* $\wedge$ *key y* $\neq$ *k*)

definition *reachable-mass* :: $('b$ *compact-basis* $\Rightarrow 'k) \Rightarrow 'b$ *bag-basis* $\Rightarrow 'k \Rightarrow$ *enat*
where *reachable-mass key u k* = (*SUP* $k' \in \{k' \in Live.\ k \sqsubseteq k'\}$. *bag-mass u* ($\lambda y$. *key y* $\sqsubseteq k'$))

fun *orphan-count* :: *join-side* $\Rightarrow$ ($'b$ *compact-basis* $\Rightarrow 'k$) $\Rightarrow 'b$ *bag-basis* $\Rightarrow 'k \Rightarrow$ *enat*
where
orphan-count Open key u k = ∞
| *orphan-count Closed key u k* =
(*if decided-mass key u k* = *0 then max 1* (*reachable-mass key u k*) *else undecided-mass key u k*)

lemma *decided-mass-live*: *decided-mass key u k* $\neq$ *0* $\Longrightarrow$ *k* $\in$ *Live*
by (*auto simp*: *decided-mass-def bag-mass-nonzero-iff decided-match-def*)

lemma *orphan-count-0*: *orphan-count w key u k* = *0* $\Longrightarrow$ *w* = *Closed* $\wedge$ *decided-mass key u k* $\neq$ *0*
by (*cases w*) (*auto simp*: *max-def split*: *if-split-asm*)

lemma *decided-undecided-mass*:
k ∈ *Live* ⟹ *decided-mass key u k* + *undecided-mass key u k* = *bag-mass u* (λ*y*. *key y* ⊑ *k*)
unfolding *decided-mass-def undecided-mass-def*
by (*subst bag-mass-disj*[*symmetric*]) (*auto simp*: *decided-match-def intro*: *bag-mass-cong*)

lemma *reachable-mass-upper*:
k′ ∈ *Live* ⟹ *k* ⊑ *k*′ ⟹ *bag-mass u* (λ*y*. *key y* ⊑ *k*′) ≤ *reachable-mass key u k*
unfolding *reachable-mass-def* **by** (*rule SUP-upper*) *simp*

lemma *reachable-mass-key-anti*: *k* ⊑ *k1* ⟹ *reachable-mass key u k1* ≤ *reachable-mass key u k*
unfolding *reachable-mass-def* **by** (*rule SUP-subset-mono*) (*auto intro*: *below-trans*)

A row without a decided match holds enough copies for whatever a row above it emits.

lemma *orphan-count-step*:
assumes *k* ⊑ *k1* **and** *decided-mass key u k* = *0*
shows *decided-mass key u k1* + *orphan-count w key u k1* ≤ *orphan-count w key u k*
proof (*cases w*)
case *Open*
then show *?thesis*
by *simp*
next
case *Closed*
show *?thesis*
proof (*cases decided-mass key u k1* = *0*)
case *True*
have *max 1* (*reachable-mass key u k1*) ≤ *max 1* (*reachable-mass key u k*)
using *reachable-mass-key-anti*[*OF assms*(*1*)] **by** (*rule max.mono*[*OF order-refl*])
then show *?thesis*
using *Closed True assms*(*2*) **by** *simp*
next
case *nz*: *False*
then have *live*: *k1* ∈ *Live*
by (*rule decided-mass-live*)
have *decided-mass key u k1* + *orphan-count w key u k1* = *bag-mass u* (λ*y*. *key y* ⊑ *k1*)
using *Closed nz decided-undecided-mass*[*OF live*] **by** *simp*
also have … ≤ *reachable-mass key u k*
by (*rule reachable-mass-upper*[*OF live assms*(*1*)])
also have … ≤ *orphan-count w key u k*
using *Closed assms*(*2*) **by** *simp*
finally show *?thesis* **.**
qed
qed

Refining the other side only lowers the counts.

lemma *decided-mass-up*:
assumes $t \leq_b t'$ **and** *monofun key* **and** *keys*: $\bigwedge x.$ *key* $x \in$ *Keys*
and *decided-mass key t k* $\neq$ *0*
shows *decided-mass key t' k* $\neq$ *0*
proof −
obtain x **where** x: $x \in$ *bsupp t decided-match k* (*key x*)
using *assms*(*4*) **by** (*auto simp*: *decided-mass-def bag-mass-nonzero-iff*)
obtain x' **where** $x' \in$ *bsupp t' x* $\sqsubseteq$ x'
using *bag-le-lower*[*OF assms*(*1*) *x*(*1*)] **by** *blast*
moreover have *decided-match k* (*key x'*)
using *x*(*2*) *monofunE*[*OF assms*(*2*) ‹$x \sqsubseteq x'$›] *live-maximal keys* **by** (*auto simp*: *decided-match-def*)
ultimately show *?thesis*
by (*auto simp*: *decided-mass-def bag-mass-nonzero-iff*)
qed

lemma *undecided-mass-anti*:
assumes $t \leq_b t'$ **and** *monofun key* **and** *keys*: $\bigwedge x.$ *key* $x \in$ *Keys* **and** $k \in$ *Live*
shows *undecided-mass key t' k* $\leq$ *undecided-mass key t k*
unfolding *undecided-mass-def*
proof (*rule bag-le-mass*[*OF assms*(*1*)])
fix x x'
assume $x \sqsubseteq x'$ **and** x': *key* $x' \sqsubseteq k \wedge$ *key* $x' \neq k$
have *key* $x \sqsubseteq$ *key* x'
using *assms*(*2*) ‹$x \sqsubseteq x'$› **by** (*rule monofunE*)
show *key* $x \sqsubseteq k \wedge$ *key* $x \neq k$
proof
show *key* $x \sqsubseteq k$
using ‹*key* $x \sqsubseteq$ *key* x'› x' **by** (*blast intro*: *below-trans*)
show *key* $x \neq k$
using ‹*key* $x \sqsubseteq$ *key* x'› x' *live-maximal*[*OF assms*(*4*)] *keys* **by** *blast*
qed
qed

lemma *reachable-mass-anti*:
assumes $t \leq_b t'$ **and** *monofun key*
shows *reachable-mass key t' k* $\leq$ *reachable-mass key t k*
unfolding *reachable-mass-def*
by (*rule SUP-subset-mono*[*OF order-refl*], *rule bag-le-mass*[*OF assms*(*1*)])
(*metis assms*(*2*) *monofunE below-trans*)

lemma *orphan-count-anti*:
assumes $t \leq_b t'$ **and** *monofun key* **and** $\bigwedge x.$ *key* $x \in$ *Keys*
shows *orphan-count w key t' k* $\leq$ *orphan-count w key t k*
proof (*cases w*)
case *Open*
then show *?thesis*
by *simp*

```
next
  case Closed
  show ?thesis
  proof (cases decided-mass key t k = 0)
    case nz: False
    then show ?thesis
      using Closed undecided-mass-anti[OF assms decided-mass-live[OF nz]] de-
cided-mass-up[OF assms nz]
      by simp
  next
    case True
    show ?thesis
    proof (cases decided-mass key t' k = 0)
      case True
      have max 1 (reachable-mass key t' k) ≤ max 1 (reachable-mass key t k)
          using reachable-mass-anti[OF assms(1,2)] by (rule max.mono[OF or-
der-refl])
      then show ?thesis
        using Closed True ‹decided-mass key t k = 0› by simp
    next
      case nz: False
      then have live: k ∈ Live
        by (rule decided-mass-live)
      have undecided-mass key t' k ≤ undecided-mass key t k
        by (rule undecided-mass-anti[OF assms live])
      also have ... ≤ bag-mass t (λx. key x ⊑ k)
      using decided-undecided-mass[OF live, of key t] by (metis le-iff-add add.commute)
      also have ... ≤ reachable-mass key t k
        by (rule reachable-mass-upper[OF live below-refl])
      also have ... ≤ max 1 (reachable-mass key t k)
        by simp
      finally show ?thesis
        using Closed nz True by simp
    qed
  qed
qed
```

45.3 The join on the basis

```
primrec join-mult ::
    join-side ⇒ join-side ⇒ ('a compact-basis ⇒ 'k) ⇒ ('b compact-basis ⇒ 'k) ⇒
      'a bag-basis ⇒ 'b bag-basis ⇒ ('a, 'b) join-index ⇒ enat
  where
    join-mult wl wr lkey rkey t u (Joined x y) =
      (if decided-match (lkey x) (rkey y) then bmult t x * bmult u y else 0)
  | join-mult wl wr lkey rkey t u (Left-Orphan x) = bmult t x * orphan-count wl
rkey u (lkey x)
  | join-mult wl wr lkey rkey t u (Right-Orphan y) = bmult u y * orphan-count wr
lkey t (rkey y)
```

lemma *join-mult-nonzero*: $\exists\, i{\in}$*join-indices t u. join-mult wl wr lkey rkey t u i* $\neq$ *0*
proof −
  obtain *x* **where** *x*: *x* ∈ *bsupp t*
    using *bsupp-nonempty* **by** *blast*
  show *?thesis*
  proof (*cases orphan-count wl rkey u* (*lkey x*) = *0*)
    case *True*
    then obtain *y* **where** *y* ∈ *bsupp u decided-match* (*lkey x*) (*rkey y*)
      by (*auto dest*!: *orphan-count-0 simp*: *decided-mass-def bag-mass-nonzero-iff*)
    then show *?thesis*
      using *x* **by** (*intro bexI*[*of - Joined x y*]) (*simp-all add*: *bsupp-iff*)
  next
    case *False*
    then show *?thesis*
      using *x* **by** (*intro bexI*[*of - Left-Orphan x*]) (*simp-all add*: *bsupp-iff*)
  qed
qed

definition *BJoin* ::
    (′*a compact-basis* ⇒ ′*b compact-basis* ⇒ ′*c compact-basis*) ⇒
      join-side ⇒ *join-side* ⇒ (′*a compact-basis* ⇒ ′*k*) ⇒ (′*b compact-basis* ⇒ ′*k*)
⇒
      ′*a bag-basis* ⇒ ′*b bag-basis* ⇒ ′*c bag-basis*
  where *BJoin c wl wr lkey rkey t u* =
    BFamily (*join-indices t u*) (*join-label c*) (*join-mult wl wr lkey rkey t u*)

The join is symmetric up to swapping the sides.

lemma *BJoin-swap*: *BJoin c wl wr lkey rkey t u* = *BJoin* (λ*y x. c x y*) *wr wl rkey lkey u t*
proof −
  have *bij*: *bij-betw join-swap* (*join-indices t u*) (*join-indices u t*)
   by (*rule bij-betw-byWitness*[**where** *f* ′ = *join-swap*]) (*auto intro*: *join-swap-indices*)
  have *join-label* (λ*y x. c x y*) (*join-swap i*) = *join-label c i* **for** *i*
    by (*cases i*) *simp-all*
  moreover have *join-mult wr wl rkey lkey u t* (*join-swap i*) = *join-mult wl wr lkey rkey t u i* **for** *i*
    by (*cases i*) (*simp-all add*: *decided-match-sym mult.commute*)
  ultimately show *?thesis*
    unfolding *BJoin-def*
    using *BFamily-reindex*[*OF bij*, *of join-label* (λ*y x. c x y*) *join-mult wr wl rkey lkey u t*]
    by *simp*
qed

45.4 Monotonicity

A plan P from t to t' lifts to the joins with u. A pair follows its left row. A left orphan without a decided match sends copies to the pairs that its

refined row now matches, and the rest to the refined orphan. Right orphans stay where they are.

```
context
  fixes P :: 'a compact-basis ⇒ 'a compact-basis ⇒ enat
    and wl wr :: join-side
    and lkey :: 'a compact-basis ⇒ 'k and rkey :: 'b compact-basis ⇒ 'k
    and t' :: 'a bag-basis and u :: 'b bag-basis
begin

fun lift-plan :: ('a, 'b) join-index ⇒ ('a, 'b) join-index ⇒ enat
  where
    lift-plan (Joined x y) (Joined x' y') =
      (if y' = y then if decided-match (lkey x) (rkey y) then P x x' * bmult u y else
0 else 0)
  | lift-plan (Left-Orphan x) (Joined x' y) =
      (if decided-mass rkey u (lkey x) = 0 ∧ decided-match (lkey x') (rkey y)
       then P x x' * bmult u y else 0)
  | lift-plan (Left-Orphan x) (Left-Orphan x') =
      P x x' * orphan-count wl rkey u (lkey x')
  | lift-plan (Right-Orphan y) (Right-Orphan y') =
      (if y' = y then bmult u y * orphan-count wr lkey t' (rkey y) else 0)
  | lift-plan i j = 0

end

lemma BJoin-mono-left:
  assumes tt: t ≤b t' and lkey: monofun lkey and keys: ⋀x. lkey x ∈ Keys
    and c: ⋀x x' y y'. x ⊑ x' ⟹ y ⊑ y' ⟹ c x y ⊑ c x' y'
  shows BJoin c wl wr lkey rkey t u ≤b BJoin c wl wr lkey rkey t' u
proof −
  obtain P where P: bag-plan t t' P
    using tt unfolding bag-le-def by blast
  have Pkey: lkey x ⊑ lkey x' if P x x' ≠ 0 for x x'
    using lkey bag-plan-below[OF P that] by (rule monofunE)
  — a decided key does not move
  have Pfix: lkey x' = lkey x if P x x' ≠ 0 decided-mass rkey u (lkey x) ≠ 0 for x
x'
     using live-maximal decided-mass-live[OF that(2)] Pkey[OF that(1)] keys by
blast
  let ?m = join-mult wl wr lkey rkey t u and ?n = join-mult wl wr lkey rkey t' u
  let ?p = lift-plan P wl wr lkey rkey t' u
  have sum-decided-mass:
    (∑ y∈bsupp u. if decided-match k (rkey y) then e * bmult u y else 0) = e *
decided-mass rkey u k
    for k e
   by (simp add: sum.inter-filter[symmetric] decided-mass-def bag-mass-def sum-distrib-left
if-distrib)
  show ?thesis
    unfolding BJoin-def
```

```
  proof (rule BFamily-le[where p = ?p])
    show ∃ i∈join-indices t u. ?m i ≠ 0
      by (rule join-mult-nonzero)
  next
    fix i j
    assume i ∈ join-indices t u j ∈ join-indices t' u ?p i j ≠ 0
    then show join-label c i ⊑ join-label c j
      by (cases i; cases j) (auto split: if-split-asm intro!: c dest: bag-plan-below[OF
P])
  next
    fix i
    assume i: i ∈ join-indices t u
    show sum (?p i) (join-indices t' u) ≤ ?m i
    proof (cases i)
      case (Joined x y)
      then have y: y ∈ bsupp u
        using i by simp
      have sum (?p i) (join-indices t' u) =
          (if decided-match (lkey x) (rkey y) then sum (P x) (bsupp t') * bmult u y
else 0)
        using y by (simp add: Joined sum-join-indices sum-distrib-right)
      then show ?thesis
        by (simp add: Joined bag-plan-row[OF P])
    next
      case (Left-Orphan x)
      have bound: (∑ y∈bsupp u. ?p i (Joined x' y)) + ?p i (Left-Orphan x')
          ≤ P x x' * orphan-count wl rkey u (lkey x) for x'
      proof (cases P x x' = 0)
        case False
        show ?thesis
        proof (cases decided-mass rkey u (lkey x) = 0)
          case True
           have P x x' * decided-mass rkey u (lkey x') + P x x' * orphan-count wl
rkey u (lkey x')
              ≤ P x x' * orphan-count wl rkey u (lkey x)
            using orphan-count-step[OF Pkey[OF False] True]
            by (simp add: distrib-left[symmetric] mult-left-mono)
          then show ?thesis
           using True by (simp add: Left-Orphan sum-decided-mass mult.commute)
        next
          case nz: False
          then show ?thesis
            using Pfix[OF False nz] by (simp add: Left-Orphan)
        qed
      qed (simp add: Left-Orphan)
      have sum (?p i) (join-indices t' u) =
          (∑ x'∈bsupp t'. (∑ y∈bsupp u. ?p i (Joined x' y)) + ?p i (Left-Orphan
x'))
        by (simp add: Left-Orphan sum-join-indices sum.distrib)
```

```
    also have ... ≤ (∑ x'∈bsupp t'. P x x' * orphan-count wl rkey u (lkey x))
      by (intro sum-mono bound)
    also have ... = ?m i
      by (simp add: Left-Orphan sum-distrib-right[symmetric] bag-plan-row[OF
P])
    finally show ?thesis .
  next
    case (Right-Orphan y)
    then have y: y ∈ bsupp u
      using i by simp
     have sum (?p i) (join-indices t' u) = bmult u y * orphan-count wr lkey t'
(rkey y)
      using y by (simp add: Right-Orphan sum-join-indices)
    also have ... ≤ ?m i
       using orphan-count-anti[OF tt lkey keys] by (simp add: Right-Orphan
mult-left-mono)
    finally show ?thesis .
  qed
 next
  fix j
  assume j: j ∈ join-indices t' u
  show ?n j ≤ (∑ i∈join-indices t u. ?p i j)
  proof (cases j)
    case (Joined x' y)
    then have y: y ∈ bsupp u
      using j by simp
    show ?thesis
    proof (cases decided-match (lkey x') (rkey y))
      case True
      have cover: P x x' * bmult u y ≤ (∑ y'∈bsupp u. ?p (Joined x y') j) + ?p
(Left-Orphan x) j
        for x
      proof (cases P x x' = 0 ∨ decided-mass rkey u (lkey x) = 0)
        case False
        then have decided-match (lkey x) (rkey y)
          using True Pfix by auto
        then show ?thesis
          using y by (simp add: Joined)
      qed (auto simp: Joined True)
      have ?n j ≤ (∑ x∈bsupp t. P x x') * bmult u y
      using True bag-plan-col[OF P, of x'] by (simp add: Joined mult-right-mono)
       also have ... ≤ (∑ x∈bsupp t. (∑ y'∈bsupp u. ?p (Joined x y') j) + ?p
(Left-Orphan x) j)
        unfolding sum-distrib-right by (intro sum-mono cover)
      also have ... = (∑ i∈join-indices t u. ?p i j)
        by (simp add: Joined sum-join-indices sum.distrib)
      finally show ?thesis .
    qed (simp add: Joined)
  next
```

```
    case (Left-Orphan x')
    have ?n j ≤ (∑ x∈bsupp t. P x x') * orphan-count wl rkey u (lkey x')
     using bag-plan-col[OF P, of x'] by (simp add: Left-Orphan mult-right-mono)
    also have ... = (∑ i∈join-indices t u. ?p i j)
      by (simp add: Left-Orphan sum-join-indices sum-distrib-right)
    finally show ?thesis .
  next
    case (Right-Orphan y)
    then have y ∈ bsupp u
      using j by simp
    then show ?thesis
      by (simp add: Right-Orphan sum-join-indices)
  qed
 next
  fix i
  assume i: i ∈ join-indices t u and ?m i ≠ 0
  show ∃j∈join-indices t' u. ?n j ≠ 0 ∧ join-label c i ⊑ join-label c j
  proof (cases i)
    case (Joined x y)
    with i ‹?m i ≠ 0› have x: x ∈ bsupp t and y: y ∈ bsupp u
      and d: decided-match (lkey x) (rkey y)
      by (simp-all split: if-split-asm)
    obtain x' where x': x' ∈ bsupp t' P x x' ≠ 0
      using bag-plan-row-ex[OF P x] .
    have decided-mass rkey u (lkey x) ≠ 0
      using y d by (auto simp: decided-mass-def bag-mass-nonzero-iff)
    then have decided-match (lkey x') (rkey y)
      using d Pfix[OF x'(2)] by simp
    then show ?thesis
      using x' y bag-plan-below[OF P x'(2)]
      by (intro bexI[of - Joined x' y]) (auto simp: Joined bsupp-iff intro: c)
  next
    case (Left-Orphan x)
    with i have x: x ∈ bsupp t
      by simp
    obtain x' where x': x' ∈ bsupp t' P x x' ≠ 0
      using bag-plan-row-ex[OF P x] .
    note xx' = bag-plan-below[OF P x'(2)]
    show ?thesis
    proof (cases orphan-count wl rkey u (lkey x') = 0)
      case True
      then obtain y where y ∈ bsupp u decided-match (lkey x') (rkey y)
      by (auto dest!: orphan-count-0 simp: decided-mass-def bag-mass-nonzero-iff)
      then show ?thesis
          using x' xx' by (intro bexI[of - Joined x' y]) (auto simp: Left-Orphan
bsupp-iff intro: c)
    next
      case False
      then show ?thesis
```

using x' xx' **by** (*intro bexI*[*of* - *Left-Orphan* x']) (*auto simp*: *Left-Orphan bsupp-iff intro*: *c*)
qed
next
case (*Right-Orphan y*)
with *i* **have** *y*: $y \in$ *bsupp u*
by *simp*
show *?thesis*
proof (*cases orphan-count wr lkey t'* (*rkey y*) = *0*)
case *True*
then obtain x' **where** $x' \in$ *bsupp t' decided-match* (*rkey y*) (*lkey x'*)
by (*auto dest*!: *orphan-count-0 simp*: *decided-mass-def bag-mass-nonzero-iff*)
then show *?thesis*
using *y*
by (*intro bexI*[*of* - *Joined x' y*]) (*auto simp*: *Right-Orphan bsupp-iff decided-match-sym intro*: *c*)
next
case *False*
then show *?thesis*
using *y* **by** (*intro bexI*[*of* - *Right-Orphan y*]) (*auto simp*: *Right-Orphan bsupp-iff*)
qed
qed
qed *simp-all*
qed

lemma *BJoin-mono-right*:
assumes $u \leq_b u'$ **and** *monofun rkey* **and** $\bigwedge y.$ *rkey y* $\in$ *Keys*
and *c*: $\bigwedge x\ x'\ y\ y'.\ x \sqsubseteq x' \Longrightarrow y \sqsubseteq y' \Longrightarrow c\ x\ y \sqsubseteq c\ x'\ y'$
shows *BJoin c wl wr lkey rkey t u* $\leq_b$ *BJoin c wl wr lkey rkey t u'*
unfolding *BJoin-swap*[*of c wl wr lkey rkey t*] **by** (*rule BJoin-mono-left*[*OF assms*(*1*−*3*)]) (*rule c*)

45.5 The join of bags into pairs

definition *bag-pair-join* ::
join-side ⇒ *join-side* ⇒ ('*a* → '*k*) ⇒ ('*b* → '*k*) ⇒ '*a bag-pd* → '*b bag-pd* → ('*a* × '*b*) *bag-pd*
where *bag-pair-join wl wr lkey rkey* = *bag-pd.extension* (λ*t*. *bag-pd.extension* (λ*u*.
bag-principal (*BJoin pair-basis wl wr* (*on-basis lkey*) (*on-basis rkey*) *t u*)))

lemma *bag-pair-join-principal*:
assumes $\bigwedge x.$ *lkey*·*x* $\in$ *Keys* **and** $\bigwedge y.$ *rkey*·*y* $\in$ *Keys*
shows *bag-pair-join wl wr lkey rkey*·(*bag-principal t*)·(*bag-principal u*) =
bag-principal (*BJoin pair-basis wl wr* (*on-basis lkey*) (*on-basis rkey*) *t u*)
unfolding *bag-pair-join-def*
by (*simp add*: *bag-pd.extension-principal bag-pd.extension-mono BJoin-mono-left BJoin-mono-right*

monofun-on-basis pair-basis-mono assms)

Sanity check, an N:1 join of two rows with the same decided key: the referencing row is consumed by the match, while the referenced row keeps an open-ended number of orphan copies.

lemma *bag-pair-join-unit-match*:
fixes *lkey* :: *'a* → *'k* **and** *rkey* :: *'b* → *'k*
assumes *a*: *Cpo.compact a* **and** *b*: *Cpo.compact b* **and** *live*: *lkey·a* ∈ *Live*
and *k*: *rkey·b* = *lkey·a* **and** *keys*: ⋀*x*. *lkey·x* ∈ *Keys* ⋀*y*. *rkey·y* ∈ *Keys*
shows *bag-pair-join Closed Open lkey rkey·*(*bag-unit·a*)*·*(*bag-unit·b*) =
bag-unit·(*a*, *b*) ∪*b* *bag-omega·*(*bag-unit·*(⊥, *b*))
proof −
define *a' b'* **where** *a'* = *Abs-compact-basis a* **and** *b'* = *Abs-compact-basis b*
have *ra*: *Rep-compact-basis a'* = *a* **and** *rb*: *Rep-compact-basis b'* = *b*
using *a b* **by** (*simp-all add*: *a'-def b'-def*)
have *d*: *decided-match* (*on-basis lkey a'*) (*on-basis rkey b'*)
using *live k* **by** (*simp add*: *ra rb decided-match-def*)
have *l*: *orphan-count Closed* (*on-basis rkey*) (*BUnit b'*) (*on-basis lkey a'*) = *0*
using *d* **by** (*simp add*: *decided-mass-def undecided-mass-def bag-mass-BUnit ra rb k*)
have *BJoin pair-basis Closed Open* (*on-basis lkey*) (*on-basis rkey*) (*BUnit a'*) (*BUnit b'*) =
BPlus (*BUnit* (*pair-basis a' b'*)) (*BOmega* (*BUnit* (*pair-basis compact-bot b'*)))
by (*rule bag-basis-eqI*)
(*simp add*: *BJoin-def bmult-BFamily-sum join-mult-nonzero sum-join-indices d l bmult-BUnit*
bmult-BOmega eq-commute[*of* - *pair-basis* - -] *del*: *orphan-count.simps*(*2*))
moreover have *bag-unit·a* = *bag-principal* (*BUnit a'*) *bag-unit·b* = *bag-principal* (*BUnit b'*)
bag-unit·(*a*, *b*) = *bag-principal* (*BUnit* (*pair-basis a' b'*))
bag-unit·(⊥::*'a*, *b*) = *bag-principal* (*BUnit* (*pair-basis compact-bot b'*))
using *bag-unit-Rep-compact-basis* [*of a'*] *bag-unit-Rep-compact-basis* [*of b'*]
bag-unit-Rep-compact-basis [*of pair-basis a' b'*]
bag-unit-Rep-compact-basis [*of pair-basis* (*compact-bot*::*'a compact-basis*) *b'*]
by (*simp-all add*: *ra rb*)
ultimately show *?thesis*
by (*simp only*: *bag-pair-join-principal*[*OF keys*] *bag-omega-principal bag-plus-principal*)
qed

end

45.6 Joins on one row domain

Both sides live in one row domain *'r*, in disjoint blocks A and B. The left key columns S and the right key columns T are different columns, related by the transports φ and ψ (the column correspondence of the join); the live keys are the maximal points of T. A joined pair is the join of its two blocks, which exists because the blocks are disjoint, so no compatibility condition

on pairs is needed.

The join of two continuous functions whose values are always joinable is continuous.

lemma *cont-join-joinable*:
assumes *f*: *cont f* **and** *g*: *cont g* **and** *j*: $\bigwedge x.\ f\ x \Uparrow g\ x$
shows *cont* $(\lambda x.\ f\ x \sqcup g\ x)$
proof (*rule contI2*)
show *monofun* $(\lambda x.\ f\ x \sqcup g\ x)$
proof (*rule monofunI*)
fix *x y* :: *'a*
assume $x \sqsubseteq y$
then have $f\ x \sqsubseteq f\ y\ g\ x \sqsubseteq g\ y$
using *f g* **by** (*simp-all add*: *cont2monofunE*)
then show $f\ x \sqcup g\ x \sqsubseteq f\ y \sqcup g\ y$
using *joinableD*[*OF j*[*of y*]] **by** (*blast intro*: *join-belowI below-trans*)
qed
fix *Y* :: *nat* ⇒ *'a*
assume *Y*: *chain Y* **and** *ch*: *chain* $(\lambda i.\ f\ (Y\ i) \sqcup g\ (Y\ i))$
have $f\ (\bigsqcup i.\ Y\ i) \sqsubseteq (\bigsqcup i.\ f\ (Y\ i) \sqcup g\ (Y\ i))$
unfolding *cont2contlubE*[*OF f Y*] **by** (*rule lub-mono*[*OF ch2ch-cont*[*OF f Y*] *ch joinableD*(*1*)[*OF j*]])
moreover have $g\ (\bigsqcup i.\ Y\ i) \sqsubseteq (\bigsqcup i.\ f\ (Y\ i) \sqcup g\ (Y\ i))$
unfolding *cont2contlubE*[*OF g Y*] **by** (*rule lub-mono*[*OF ch2ch-cont*[*OF g Y*] *ch joinableD*(*2*)[*OF j*]])
ultimately show $f\ (\bigsqcup i.\ Y\ i) \sqcup g\ (\bigsqcup i.\ Y\ i) \sqsubseteq (\bigsqcup i.\ f\ (Y\ i) \sqcup g\ (Y\ i))$
by (*rule join-belowI*)
qed

locale *bag-join-columns* =
fixes *A B S T* :: *'r*::{*bifinite*, *distributive-cpo*} *semi-factor* **and** $\varphi\ \psi$ :: $'r \to 'r$
assumes *disjoint*: $A \sqcap B = \bot$
and $\psi\varphi$: $\psi\ oo\ \varphi = proj\ S$ **and** $\varphi\psi$: $\varphi\ oo\ \psi = proj\ T$
and φ-*T*: *proj T oo* $\varphi = \varphi$ **and** ψ-*S*: *proj S oo* $\psi = \psi$
begin

lemma $\psi\varphi$-*app* [*simp*]: $\psi{\cdot}(\varphi{\cdot}x) = proj\ S{\cdot}x$
using $\psi\varphi$ **by** (*simp add*: *cfun-eq-iff*)

lemma $\varphi\psi$-*app* [*simp*]: $\varphi{\cdot}(\psi{\cdot}x) = proj\ T{\cdot}x$
using $\varphi\psi$ **by** (*simp add*: *cfun-eq-iff*)

lemma φ-*T*-*app* [*simp*]: *proj T*$\cdot(\varphi{\cdot}x) = \varphi{\cdot}x$
using φ-*T* **by** (*simp add*: *cfun-eq-iff*)

lemma ψ-*S*-*app* [*simp*]: *proj S*$\cdot(\psi{\cdot}x) = \psi{\cdot}x$
using ψ-*S* **by** (*simp add*: *cfun-eq-iff*)

lemma φ-*S*-*app* [*simp*]: $\varphi{\cdot}(proj\ S{\cdot}x) = \varphi{\cdot}x$

by (*metis* φ-*T-app* $\varphi\psi$-*app* $\psi\varphi$-*app*)

lemma ψ-*T-app* [*simp*]: $\psi{\cdot}(\mathit{proj}\ T{\cdot}x) = \psi{\cdot}x$
by (*metis* ψ-*S-app* $\varphi\psi$-*app* $\psi\varphi$-*app*)

A decided match reads the same from either side.

lemma *match-transport*:
(*maximal-at* T $(\varphi{\cdot}x) \wedge \mathit{proj}\ T{\cdot}y = \varphi{\cdot}x) \longleftrightarrow (\mathit{maximal\text{-}at}\ S\ (\mathit{proj}\ S{\cdot}x) \wedge \mathit{proj}\ S{\cdot}x = \psi{\cdot}y)$
proof
assume *m*: *maximal-at* T $(\varphi{\cdot}x) \wedge \mathit{proj}\ T{\cdot}y = \varphi{\cdot}x$
have $\mathit{proj}\ S{\cdot}x = \psi{\cdot}y$
by (*metis m* $\psi\varphi$-*app* ψ-*T-app*)
moreover have *maximal-at* S $(\mathit{proj}\ S{\cdot}x)$
unfolding *maximal-at-def*
proof (*intro conjI allI impI*)
fix t
assume *t*: $\mathit{proj}\ S{\cdot}t = t$ **and** *le*: $\mathit{proj}\ S{\cdot}x \sqsubseteq t$
have $\varphi{\cdot}t = \varphi{\cdot}x$
using *maximal-at-eq*[*OF m*[*THEN conjunct1*] φ-*T-app*] *monofun-cfun-arg*[*OF le*, *of* φ] **by** *simp*
then show $t = \mathit{proj}\ S{\cdot}x$
by (*metis t* $\psi\varphi$-*app*)
qed *simp*
ultimately show *maximal-at* S $(\mathit{proj}\ S{\cdot}x) \wedge \mathit{proj}\ S{\cdot}x = \psi{\cdot}y$
by *blast*
next
assume *m*: *maximal-at* S $(\mathit{proj}\ S{\cdot}x) \wedge \mathit{proj}\ S{\cdot}x = \psi{\cdot}y$
have $\mathit{proj}\ T{\cdot}y = \varphi{\cdot}x$
by (*metis m* $\varphi\psi$-*app* φ-*S-app*)
moreover have *maximal-at* T $(\varphi{\cdot}x)$
unfolding *maximal-at-def*
proof (*intro conjI allI impI*)
fix t
assume *t*: $\mathit{proj}\ T{\cdot}t = t$ **and** *le*: $\varphi{\cdot}x \sqsubseteq t$
have $\psi{\cdot}t = \mathit{proj}\ S{\cdot}x$
using *maximal-at-eq*[*OF m*[*THEN conjunct1*] ψ-*S-app*] *monofun-cfun-arg*[*OF le*, *of* ψ] **by** *simp*
then show $t = \varphi{\cdot}x$
by (*metis t* $\varphi\psi$-*app* φ-*S-app*)
qed *simp*
ultimately show *maximal-at* T $(\varphi{\cdot}x) \wedge \mathit{proj}\ T{\cdot}y = \varphi{\cdot}x$
by *blast*
qed

lemma *blocks-joinable*: $\mathit{proj}\ A{\cdot}x \Uparrow \mathit{proj}\ B{\cdot}y$
proof (*rule proj-slide*)
show $\mathit{proj}\ B{\cdot}(\mathit{proj}\ B{\cdot}y) = \mathit{proj}\ B{\cdot}y$
by *simp*

have *proj B·(proj A·x) = ⊥*
using *proj-inf[of B A x] disjoint* **by** (*simp add*: *lattice-meet-commute proj-bot*)
then show *proj B·(proj A·x) ⊑ proj B·y*
by *simp*
qed

definition *block-join* :: *'r × 'r → 'r*
where *block-join* = (Λ(*x*, *y*). *proj A·x ⊔ proj B·y*)

lemma *block-join-beta* [*simp*]: *block-join·(x, y) = proj A·x ⊔ proj B·y*
unfolding *block-join-def* **by** (*simp add*: *split-beta' cont-join-joinable blocks-joinable*)

sublocale *keys*: *bag-join-keys* {*k*. *maximal-at T k*} {*k*. *proj T·k = k*}
by *unfold-locales* (*simp add*: *maximal-at-eq*)

definition *bag-join* :: *join-side ⇒ join-side ⇒ 'r bag-pd → 'r bag-pd → 'r bag-pd*
where *bag-join wl wr* = (Λ *X Y*. *bag-map·block-join·*(*keys.bag-pair-join wl wr φ* (*proj T*)·*X*·*Y*))

lemma *bag-join-eq*: *bag-join wl wr·X·Y = bag-map·block-join·*(*keys.bag-pair-join wl wr φ* (*proj T*)·*X*·*Y*)
by (*simp add*: *bag-join-def*)

Sanity check: an N:1 join of an A-row and a B-row with matching decided keys gives the joined row once and the referenced row open-endedly often.

lemma *bag-join-unit-match*:
assumes *a*: *Cpo.compact a proj A·a = a* **and** *b*: *Cpo.compact b proj B·b = b*
and *m*: *maximal-at T* (*φ·a*) *proj T·b = φ·a*
shows *bag-join Closed Open·*(*bag-unit·a*)·(*bag-unit·b*) =
bag-unit·(*a ⊔ b*) ∪$_b$ *bag-omega·*(*bag-unit·b*)
proof −
have *keys.bag-pair-join Closed Open φ* (*proj T*)·(*bag-unit·a*)·(*bag-unit·b*) =
bag-unit·(*a*, *b*) ∪$_b$ *bag-omega·*(*bag-unit·*(⊥, *b*))
using *a b m* **by** (*intro keys.bag-pair-join-unit-match*) *simp-all*
moreover have *block-join·*(*a*, *b*) = *a ⊔ b block-join·*(⊥, *b*) = *b*
using *a b deflation.below*[*OF proj-defl*, *of A* ⊥] **by** (*simp-all add*: *join-absorbI*)
ultimately show *?thesis*
by (*simp add*: *bag-join-eq*)
qed

end

end

References

[1] P. Buneman, A. Jung, and A. Ohori. Using powerdomains to generalize relational databases. *Theoretical Computer Science*, 91:23–55, 1991.